

Intelligenza Artificiale

[Appunti di Piai Luca]

Academic Year 2023-2024

Indice

1	Introduzione	7
1.1	Che cos'è una IA?	7
1.2	Agenti razionali	7
2	Intelligent agents	8
2.1	Agents and environments	8
2.2	Task environment e descrizioni PEAS	9
2.2.1	Proprietà task environment	11
2.3	Tipologie di agenti	12
2.3.1	L'architettura di un agente	13
3	Solving problems by searching	15
3.1	Problem-solving agents	15
3.2	Formulazione di un single-state problem	16
3.2.1	Esempi	16
3.3	Ricerca una soluzione	17
3.3.1	Misurare le prestazioni delle strategie di ricerca	19
3.4	Uninformed Search Strategies	20
3.4.1	Breadth-first search	20
3.4.2	Uniform-cost search	20
3.4.3	Depth-first search	21
3.4.4	Iterative deepening depth-first search	21
3.4.5	Bidirectional search	22
3.5	Informed (heuristic) search strategies	23
3.5.1	Greedy best-first search	23
3.5.2	A* search: minimizzare il costo totale stimato della soluzione	24
3.6	Funzioni euristiche	27
3.6.1	Effective branching factor	27
3.6.2	Learning heuristics from experience	28
3.7	Algoritmi di ricerca meno classici	29
3.7.1	Local search and optimization problems	29
3.7.2	Hill-climbing search (or gradient ascent/descent)	29
3.7.3	Simulating annealing	30
3.7.4	Local beam search	30
3.7.5	Genetic algorithms	30
4	Logical agents	31
4.1	Knowledge-Based Agents	31
4.2	The Wumpus world	31
4.3	La logica in generale	33
4.4	Logica proposizionale	34

4.4.1	Proposizioni nel Wumpus World	35
4.4.2	Inferenza per risoluzione, CNF e dimostrazioni per assurdo	35
4.4.3	Vantaggi e svantaggi della logica proposizionale	36
4.5	First order logic	37
4.5.1	Terms and atomic sentences	37
4.5.2	Complex sentences	38
4.5.3	Quantifiers	38
4.5.4	Knowledge engineering in First-Order Logic	39
4.6	Inferenza nella First-Order Logic	39
4.6.1	Substitutions	39
4.6.2	Riduzione all'inferenza proposizionale	39
4.6.3	Unificazione	41
4.6.4	Inferenza per risoluzione in First-Order Logic	41
4.6.5	Generalized Modus Ponens (GMP)	43
5	Quantificare l'incertezza	45
5.1	Decision-Theoretic agent	45
5.2	Inferenza e distribuzione congiunta di probabilità	45
5.2.1	Il problema con la distribuzione di probabilità congiunta	46
5.3	La regola di Bayes e il suo utilizzo	47
6	Probabilistic reasoning	48
6.1	Bayesian networks	48
6.1.1	Da distribuzione di probabilità congiunta a Bayesian network	49
6.1.2	Conditional probability tables e i vantaggi della product decomposition	50
6.1.3	Esempio: burglary alarm	51
6.2	Inferenza esatta nelle Bayesian networks	51
6.2.1	Inference by enumeration	51
6.2.2	Variable elimination	53
6.3	Inferenza approssimata nelle Bayesian networks	53
6.3.1	Direct sampling	53
6.3.2	Inference by Markov chain simulation	56
7	Probabilistic reasoning over time	58
7.1	Tempo ed incertezza	58
7.1.1	Transition model e sensor model	58
7.2	Inferenza nei temporal models	59
7.2.1	Filtraggio e predizione	60
7.2.2	Smoothing	63
7.2.3	Trovare la sequenza più probabile	64
7.3	Hidden Markov models	66
7.3.1	Algoritmi semplificati basati su matrici	66
7.3.2	Esempio di HMM: robot localization	67
7.4	Esempi	69
7.4.1	Markov model: Grades example	69
7.4.2	HMM: baby example (sequence of observations)	70
8	Inferenza causale	74
8.1	Introduzione	74
8.1.1	Correlazione e causalità	74
8.1.2	Il paradosso di Simpson	75
8.1.3	Il problema delle probabilità congiunte	75

8.2	Il modello causale	76
8.2.1	Modello causale strutturale (SCM)	76
8.2.2	Causal networks	77
8.2.3	Esempio: sprinkler problem	77
8.2.4	D-separation	78
8.3	Interventi e osservazioni	79
8.3.1	Intervento tramite chirurgia	80
8.4	Calcolare gli interventi basandosi sulle osservazioni: adjustment formula . . .	80
8.4.1	Effetto causale medio	81
8.5	Unobserved confounders	81
8.5.1	Backdoor path	82
8.5.2	Backdoor adjustment	83
8.5.3	Esempi backdoor path	83
8.5.4	Verificare la correttezza dei modelli usando il criterio di backdoor . .	84
8.5.5	Frontdoor path e frontdoor adjustment	84
8.6	Causal discovery	86
8.6.1	Algoritmo PC	87
8.6.2	Algoritmo FCI	89
8.7	Mediation	89
8.8	Linear SCM	90
8.8.1	Linear Gaussian models	91
8.8.2	Total causal effect	92
9	Making complex decisions	93
9.1	Introduzione	93
9.1.1	Expected utility	93
9.1.2	Modello di transizione	94
9.1.3	Problemi decisionali sequenziali (MDP)	95
9.1.4	Utilità nel tempo	96
9.2	Equazione di Bellman	97
9.2.1	Value iteration algorithm	98
9.2.2	Policy iteration algorithm	99
9.3	MDP parzialmente osservabili (POMDP)	100
9.3.1	Definizione di un POMDP	100
9.3.2	Stima del belief state	101
9.3.3	Il ciclo di decisione in POMDP	101
9.3.4	Modello di transizione e funzione di ricompensa nei POMDP	102
9.3.5	Policy ottima	102
10	Machine learning	104
10.1	Forme di apprendimento	104
10.2	Supervised learning	105
10.2.1	Bias e underfitting	105
10.2.2	Varianza e overfitting	105
10.2.3	Compromesso bias-varianza e scelta dell'ipotesi ottima	106
10.2.4	Caso di studio: restaurant example	106
10.3	Decision tree	107
10.3.1	Importanza degli attributi	107
10.3.2	Learning decision trees	108
10.3.3	La scelta dell'attributo e definizione di importanza	109
10.3.4	Generalizzazione e sovradattamento	111

10.4	Selezione del modello e ottimizzazioni	111
10.4.1	Da tasso di errore a loss function	112
10.4.2	La migliore ipotesi	113
10.4.3	Gestire la complessità: regolarizzazione	114
10.5	Regressione lineare	114
10.5.1	Regressione lineare multipla	116
10.6	Classificazione lineare	116
10.6.1	Support vector machines	116
11	Deep learning	119
11.1	Principi base	119
11.1.1	Separabilità lineare e non	120
11.2	Reti feedforward semplici	122
11.2.1	Activation functions più utilizzate	123
11.2.2	Apprendere i pesi della rete	124
11.3	Codifica degli input	125
11.4	Output layers e loss functions	125
11.4.1	Output layers per classificazione e regressione	125
11.5	Hidden layers	126
11.6	Reti convoluzionali	127
11.6.1	Receptive field	128
11.6.2	Pooling e downsampling	129
11.6.3	Operazioni tensoriali in CNN	129
11.7	Algoritmi di apprendimento	130
11.7.1	Calcolo di gradienti nei grafi computazionali	130
11.8	Generalizzazione	132
11.8.1	Dropout	132
11.9	Reti neurali ricorrenti	133
11.9.1	Addestramento di una RNN	134
11.9.2	RNN con LSTM	134
11.10	Apprendimento non supervisionato	135
11.10.1	Autoencoders	135
11.10.2	Variational autoencoder	136
11.10.3	VAEs vs AEs	137
11.10.4	Generative adversarial networks	137
11.10.5	Transfer learning	138
11.11	Language models	139
11.11.1	Word encoding and embedding	139
11.11.2	Token prediction	139
11.11.3	Sequence-to-sequence models	140

Capitolo 1

Introduzione

1.1 Che cos'è una IA?

Da quando si è iniziato a parlare di intelligenza artificiale sono state date molte definizioni, queste possono essere raggruppate in quattro categorie.

<i>Thinking humanly</i>	<i>Thinking rationally</i>
<i>Acting humanly</i>	<i>Acting rationally</i>

Le definizioni che appartengono alla prima riga riguardano il ragionamento mentre quelle della seconda riga riguardano l'agire. Le definizioni della prima colonna misurano il successo paragonando le caratteristiche dell'intelligenza artificiale a quelle dell'uomo; le definizioni della seconda colonna misurano le prestazioni rispetto ad una misura ideale detta **razionalità**. Ognuna delle quattro categorie ha permesso di studiare l'argomento seguendo diversi approcci.

L'approccio che studieremo in questo corso è l'*Acting rationally*. Agire in maniera razionale significa "fare le cose giuste". Ci si aspetta che l'intelligenza artificiale faccia la cosa giusta cercando in qualche modo di massimizzare il raggiungimento dell'obiettivo, avendo a disposizione delle informazioni fornite da noi.

Non necessariamente "agire in maniera razionale" implica un ragionamento (Thinking), per esempio sbattere le palpebre o respirare sono due azioni che facciamo senza pensare; tuttavia il ragionamento dovrebbe essere al servizio dell'agire in maniera razionale.

1.2 Agenti razionali

Definiamo **agente** un'entità che *percepisce* ed *agisce*. Il nostro obiettivo all'interno del corso è quello di progettare l'**agente razionale** (o la classe di agenti) che ha le migliori prestazioni per un particolare compito in un particolare ambiente.

Capitolo 2

Intelligent agents

2.1 Agents and environments

Un agente intelligente razionale viene definito come un'entità che osserva l'ambiente in cui si trova ad operare tramite dei sensori (input) e attraverso degli attuatori agisce, ovvero effettua delle azioni (output). Le **percezioni** dell'agente sono gli input, le **azioni** sono gli output, inoltre definiamo **sequenza di percezioni** la lista di tutte le percezioni che l'agente ha percepito da quando è stato reso operativo.

I dispositivi utilizzati dall'agente dipendono dal contesto in cui si trova ad operare, mentre le azioni da svolgere sono definite dal programmatore. Il concetto di agente intelligente che abbiamo appena definito in maniera semplice e generica è ancora in uso al giorno d'oggi.

Matematicamente un agente può essere rappresentato da una funzione che mappa le percezioni nell'azione:

$$f : P^* \rightarrow A$$

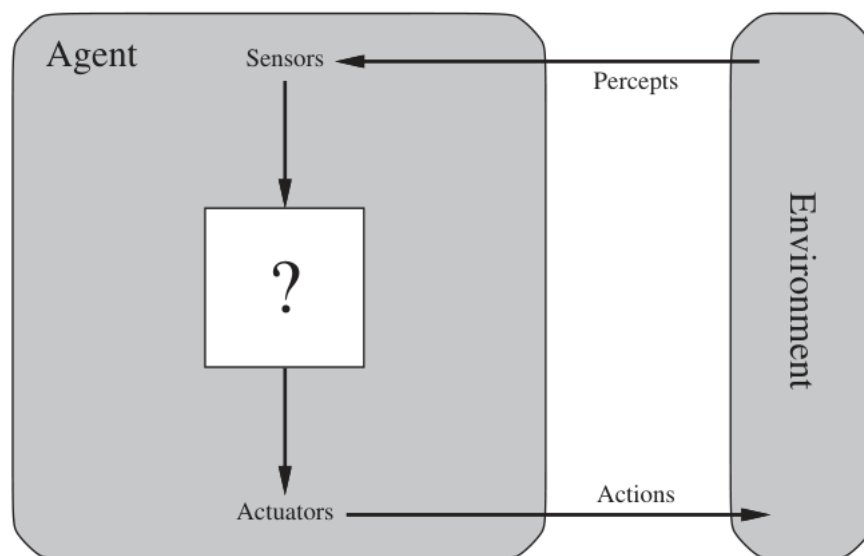


Figura 2.1: Agente intelligente che comunica con l'ambiente tramite sensori e attuatori.

Un esempio di agente intelligente è il vacuum-world agent: il suo scopo è quello di pulire le stanze di una casa. Consideriamo un caso semplice in cui ci sono solamente due stanze e l'agente può muoversi dall'una all'altra in cerca dello sporco da pulire.

Se non c'è modo di pulire la stanza? Come reagisce l'agente? Se l'agente viene realizzato in modo da reagire basandosi solo sull'ultima percezione, allora in questo caso potrebbe

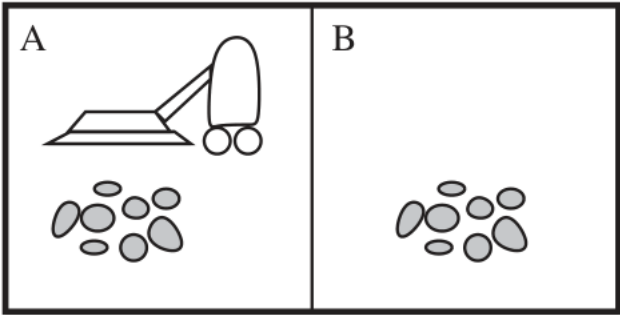


Figura 2.2: Un vacuum-world cleaner con due stanze.

Ambiente	stanza A, stanza B.
Percezioni	sensore che individua lo sporco.
Azioni	pulire, spostarsi in A, spostarsi in B.

Tabella 2.1: Ambiente, percezioni ed azioni dell'agente.

Sequenza percezioni	Azione
(A, Pulito)	Spostati in B
(A, Sporco)	Aspira
(B, Pulito)	Spostati in A
(B, Sporco)	Aspira
(A, Pulito) (A, Pulito)	Spostati in B
(A, Pulito) (A, Sporco)	Aspira
⋮	⋮
(A, Pulito) (A, Pulito) (A, Pulito)	Spostati in B
(A, Pulito) (A, Pulito) (A, Sporco)	Aspira
⋮	⋮

Tabella 2.2: Storico percezioni e azioni associate.

rimanere bloccato all'infinito in quella stanza. Un modo più intelligente di risolvere il problema è quello di far sì che l'agente agisca considerando anche le percezioni precedenti.

2.2 Task environment e descrizioni PEAS

Un agente razionale è tale se fa la cosa giusta, cioè se esegue l'azione che ci si aspetta che faccia in un particolare caso. Ma cosa significa fare la cosa giusta? Un agente che opera in un ambiente genera una sequenza di azioni che si basano sulle percezioni che riceve in input. Queste azioni modificano lo stato dell'ambiente, l'ambiente quindi passa attraverso una sequenza di stati. Se la sequenza ottenuta è desiderabile, allora l'agente si è comportato correttamente. La nozione di desiderabilità viene definita dalla misura delle performance, la quale valuta tutte le sequenze di stati dell'ambiente.

Uno dei passi fondamentali nella progettazione di un agente è quello di specificare il più dettagliatamente possibile il **task environment**. Un task environment è in sostanza un problema che l'agente razionale tenta di risolvere fornendo una soluzione. Ogni task ha associata una descrizione **PEAS** (*Performance, Environment, Actions, Sensors*). La descrizione PEAS comprende:

- la misura delle performance;
- la conoscenza a priori dell'ambiente: si tratta della conoscenza fornita da noi e non quella che apprende da solo;
- le possibili azioni che può svolgere;
- sensori che permettono di percepire l'ambiente in cui opera.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits	Roads, other traffic, pedestrians, customers	Steering, accelerator, brake, signal, horn, display	Cameras, sonar, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard

Figura 2.3: Descrizione PEAS nel task environment di un taxi a guida autonoma.

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Figura 2.4: Esempi di agenti e delle loro descrizioni PEAS.

2.2.1 Proprietà task environment

Completamente o parzialmente osservabile I sensori dell'agente possono fornire la completa conoscenza dell'ambiente all'agente; in tal caso si parla di task environment **completamente osservabile**. In questo caso non è necessario che l'agente tenga in memoria dati riguardanti l'ambiente circostante.

Il task environment può essere **parzialmente osservabile** se sono presenti errori o rumori nei dati registrati dai sensori. Potrebbe essere parzialmente osservabile anche a causa di limiti fisici.

Deterministico o stocastico Un ambiente di lavoro è **deterministico** quando è determinato dallo stato corrente e dalle azioni dell'agente. Lo stato dell'ambiente è sempre noto anche dopo alle azioni intraprese dell'agente. Un esempio di ambiente di lavoro deterministico è quello in cui può lavorare un agente applicato alle catene di montaggio.

Viceversa è **stocastico** quando non è certo quale sia lo stato dell'ambiente, cioè lo stato non è completamente predicibile. Un ambiente deterministico che è parzialmente osservabile appare all'agente come un ambiente non deterministico. Nella maggior parte dei casi reali gli ambienti sono non deterministici, per esempio il taxi a guida autonoma lavora in un ambiente non deterministico, infatti non può predire con assoluta certezza il comportamento del traffico. In questo tipo di ambienti entra in gioco la **teoria della probabilità**.

Episodico o sequenziale Se il task environment è **episodico**, allora possiamo dividere le esperienze dell'agente in episodi atomici e associare ciascuno ad una singola percezione e all'azione corrispondente. L'episodio al tempo T non dipende dalle azioni compiute al tempo $t < T$. Fanno parte di questa categoria tutti i task di classificazione, come per esempio il riconoscimento delle facce in una foto.

Se l'ambiente di lavoro è **sequenziale**, allora le decisioni passate influenzano quelle future. Per esempio nel caso del taxi a guida autonoma, scegliere una strada piuttosto che un'altra influenza le future decisioni; l'agente deve considerare le conseguenze che comportano le sue decisioni.

Statico o dinamico Un ambiente di lavoro **dinamico** cambia mentre l'agente sta prendendo le decisioni. Il taxi a guida autonoma si trova in questo tipo di ambiente, infatti sia il taxi che le altre macchine sono in continuo movimento.

Viceversa in un ambiente di lavoro **statico** l'ambiente non cambia, o meglio, le proprietà rilevanti dell'ambiente non cambiano mentre l'agente prende le decisioni. In questi casi il tempo non è importante e non è fondamentale la reattività della risposta. Ambienti statici sono per esempio i giochi da tavolo. Se però l'ambiente non cambia ma le prestazioni dell'agente influiscono sul punteggio allora l'ambiente è **semi-dinamico**.

Discreto o continuo In un ambiente di lavoro **discreto**: stati, percezioni e azioni cambiano ad intervalli di tempo costanti. Tipicamente si ha un numero finito di stati, percezioni e azioni. Il vacuum cleaner automatico lavora in un task environment discreto.

Viceversa, nel caso **continuo**, uno o più aspetti cambiano continuamente nel tempo. Il taxi a guida autonoma rientra in questa categoria.

Singolo o multi agente **Singolo** se un singolo agente è rilevante nell'ambiente. In un task environment singolo possono esserci più agenti ma i loro obiettivi non devono essere ostacolati o influenzati dagli obiettivi degli altri agenti presenti. Un esempio è un agente che risolve un puzzle.

Nel caso **multi agente** sono presenti due o più agenti che competono o collaborano per raggiungere un obiettivo. Nel primo caso si parla di **competitive** multi-agent environment, nel secondo caso si parla di **cooperative** multi-agent environment.

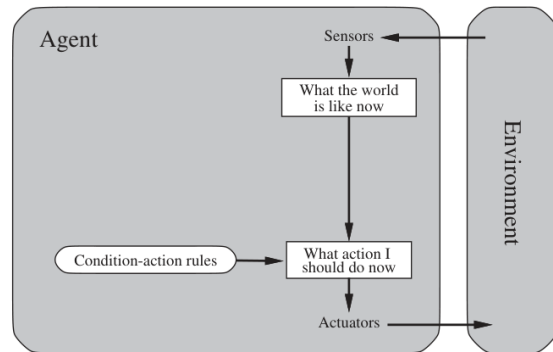
Conosciuto o sconosciuto Non riferito all'ambiente in cui opera l'agente ma al suo stato di conoscenza delle "leggi della fisica" dell'ambiente. Se l'agente conosce tali leggi, allora i risultati (probabilità) di ogni azione sono dati. Un ambiente può essere conosciuto ma parzialmente osservabile. Per esempio in solitario le regole sono conosciute ma ci sono delle carte girate.

Nel caso in cui l'agente non conosca le leggi dell'ambiente, allora deve impararle. In questo caso i risultati delle azioni non sono noti a priori. Un ambiente sconosciuto può essere completamente osservabile. Per esempio nel caso di un videogioco, lo schermo può mostrarmi interamente lo stato attuale del gioco ma non conosco cosa fanno i tasti del controller finché non li testo.

2.3 Tipologie di agenti

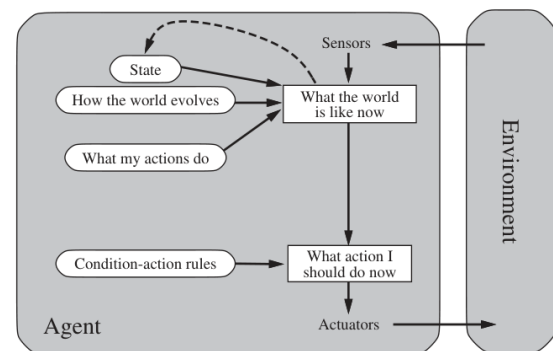
Agente semplice reattivo Si tratta del tipo di agente più semplice di tutti. Ogni percezione viene associata ad una azione, non avviene nessun ragionamento complesso per la scelta dell'azione e non viene considerato lo storico delle percezioni. Diciamo che il comportamento dell'agente è basato su delle semplici **condition-action** (if-then) rules. Questa tipologia richiede che l'ambiente sia completamente osservabile.

Le prime versioni di robot cleaners che sono state realizzate, sono state implementate con un comportamento di questo tipo. Il comportamento che seguivano era del tipo: "muoviti finché non sbatti addosso a qualcosa, poi ruota di x gradi e riparti".



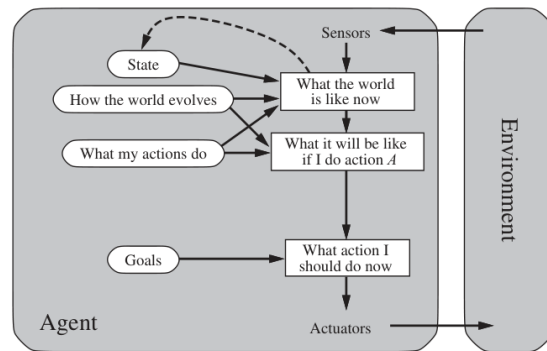
Model-based reflex agent Rispetto alla tipologia precedente, questo mantiene in memoria uno stato (per esempio la posizione precedente). Necessita di un **transition model** per conoscere i cambiamenti dell'ambiente in conseguenza alle sue azioni e un **sensor model** per anticipare quelle che potrebbero essere le prossime percezioni.

Un esempio di questo tipo di agente è il *Kalman filter-based localization*: il transition model viene utilizzato per predire la posizione del robot (il suo stato); il sensor model permette di predire le percezioni e di confrontarle con quelle reali per correggere lo stato. In questo modo l'agente può muoversi attraverso una stanza.



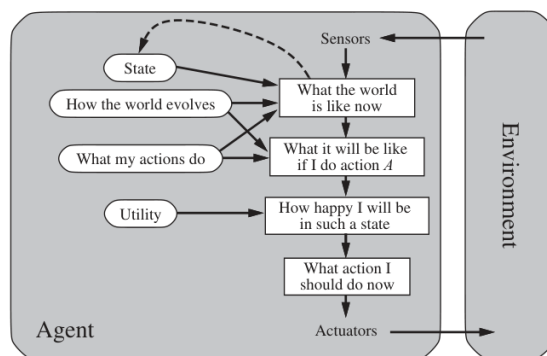
Goal-based agent In molti casi conoscere l'ambiente non è sufficiente. L'azione da compiere deve essere scelta in base a quali sono i suoi effetti immediati e per portare a termine un obiettivo a lungo termine.

Un esempio di agente di questo tipo sono le macchine a guida autonoma, dove l'agente pianifica la miglior sequenza di azioni per sorpassare un veicolo (goal), oppure per parcheggiare (goal). In entrambi i casi sfrutta la conoscenza dell'ambiente per trovare la soluzione migliore possibile per raggiungere il goal.



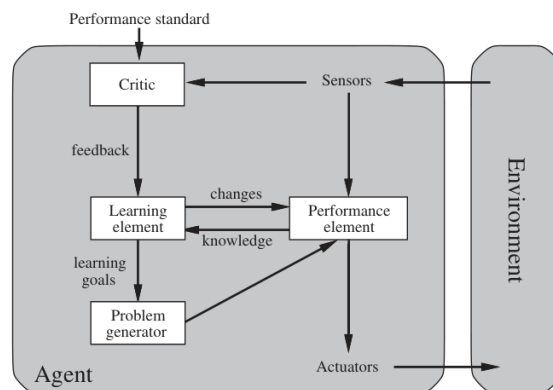
Utility-based agents L'agente di questa tipologia non solo cerca la sequenza di azioni per raggiungere il proprio obiettivo ma utilizza una **utility function** per massimizzare le performance. Gli agenti razionali cercano di massimizzare l'**utilità attesa**.

Un esempio è il taxi a guida autonoma. Il suo obiettivo è quello di raggiungere una certa destinazione e nel raggiungimento dell'obiettivo deve considerare la benzina, la strada più corta e il tempo necessario.



Learning agent Questa tipologia di agenti impara dalle proprie esperienze; si adatta agli ambienti sconosciuti o a quelli dinamici. Sono caratterizzati da quattro elementi principali:

1. *learning*: per fare miglioramenti;
2. *performance*: per selezionare le azioni;
3. *critic*: per valutare i risultati;
4. *problem generator*: per suggerire nuove azioni al fine di esplorare l'ambiente.



2.3.1 L'architettura di un agente

Le componenti che abbiamo utilizzato per rappresentare la logica che segue l'agente di una data tipologia, possono essere rappresentate in maniera più o meno dettagliata. Consideriamo il caso della componente "What my actions do", la quale descrive i possibili cambiamenti nell'ambiente causati dalle azioni dell'agente, ovvero i possibili stati in cui si può trovare. Possiamo individuare tre livelli di complessità crescente per rappresentare gli stati e le transizioni che portano l'uno all'altro:

- **Atomic:** gli stati sono delle black box, senza una struttura interna. Per esempio un grafo dove ogni nodo rappresenta una destinazione.
- **Factored:** ad ogni stato vengono aggiunti degli attributi. Per esempio una posizione GPS, informazioni sul traffico, etc.
- **Structured:** vengono aggiunte delle relazioni tra attributi diversi. Per esempio viene messo in relazione il traffico con l'orario e con il tempo ("all'orario di punta con mal tempo c'è molto traffico").

Il livello di dettaglio dipende dall'applicazione che vogliamo realizzare e dal goal scelto.

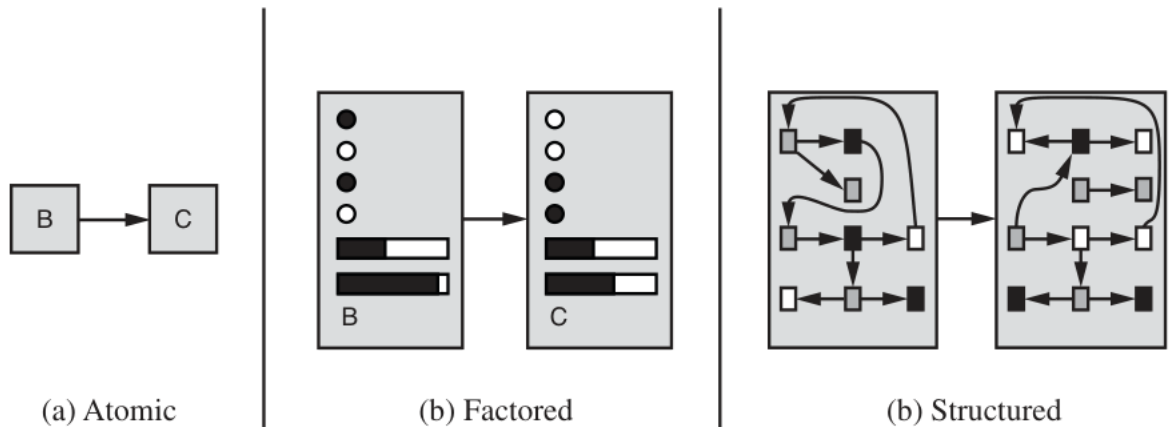


Figura 2.5: Tre livelli di complessità per rappresentare gli stati e le transizioni.

Capitolo 3

Solving problems by searching

3.1 Problem-solving agents

Un problem-solving agent è un goal-based agent che pianifica una sequenza per raggiungere il goal prefissato. Utilizza la rappresentazione atomica degli stati e cerca la soluzione considerando le sequenze di stati che permettono di raggiungere il goal. Solitamente la ricerca della sequenza può essere rappresentata da un grafo.

Gli algoritmi di ricerca possono essere divisi in due categorie principali:

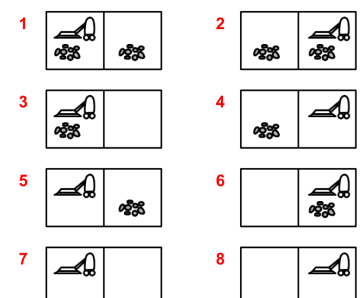
- **Uninformed:** l'agente non sa quando "dista" il suo obiettivo.
- **Informed:** l'agente ha una stima della distanza del suo obiettivo.

A seconda delle caratteristiche del task environment, possiamo definire quattro categorie di problemi:

1. **Single-state problem** se il task environment è deterministico e completamente osservabile. In questo caso l'agente sa esattamente in che stato si troverà e la soluzione del problema è una sequenza di stati.
2. **Conformant problem** se il task environment è deterministico e non osservabile. L'agente potrebbe non sapere dove si trova, la soluzione del problema è una sequenza (se ne esiste una).
3. **Contingency problem** se il task environment è non deterministico e/o parzialmente osservabile. In questo caso l'agente ricava informazioni basandosi sulle percezioni del suo stato attuale. La soluzione a questo tipo di problemi è una **policy**: questa fornisce la migliore soluzione basandosi sullo stato in cui l'agente crede di essere.
4. **Exploration model** se il non conosciamo i possibili stati.

Esempio: vacuum world

- *Single-state:* start in (5);
soluzione [Destra, Aspira].
- *Conformant:* start in (1), ..., (8);
soluzione [Destra, Aspira, Sinistra, Aspira].
- *Contingency:* start in (5);
soluzione [Destra, if sporco then aspira].



3.2 Formulazione di un single-state problem

Per formulare correttamente un problema dobbiamo definire:

- **Insieme di stati possibili.**
- **Stato iniziale** (es: $ln(Arad)$).
- **Uno o più obiettivi (stati):** questi possono essere espliciti (es: $ln(Bucharest)$) o impliciti (una proprietà astratta, es: *vincere la partita*).
- **Insieme delle azioni:** descritte da una funzione $ACTIONS(s)$ che ritorna un insieme di azioni possibili nello stato s (es: dallo stato $ln(Arad)$ le azioni tornate sono $Go(Sibiu)$, $Go(Timisoara)$, $Go(Zerind)$).
- **Transition model:** una descrizione di cosa fa ciascuna azione; viene specificata una funzione $RESULT(s, a)$ che ritorna lo stato risultante dell'azione a fatta nello stato s (es: $RESULT[ln(Arad), Go(Zerind)] = ln(Zerind)$);
- **Funzione dei costi e costo associato alla sequenza:** $C(x, a, y)$ è il costo associato all'esecuzione dell'azione a dallo stato x per raggiungere lo stato y . Si assume che il costo sia positivo. Il costo della sequenza può essere dato dalla somma delle distanze, dal numero di azioni eseguite, etc.

3.2.1 Esempi

Vacuum world Definiamo:

- Stati:
 - *Sporco* (ignoriamo la quantità di sporco).
 - *Posizione* del robot.
 - Totale stati = 2 posizioni del robot · 4 combinazioni di posizioni dello sporco;
 - Stato iniziale qualsiasi.
- Azioni: $\{Left, Right, Suck, NoOp\}$.
- Transition model: vedi Figura 3.1.
- Obiettivo: nessuna stanza sporca.
- Costo sequenza: 1 per azione (0 per *Noop*).

8-puzzle Definiamo:

- Stati: posizioni delle tessere.
- Azioni: muovi la tessera nera *Left, Right, Up, Down*.
- Transition model: vedi Figura 3.2.
- Obiettivo: ottenere una data configurazione delle tessere.
- Costo sequenza: 1 per mossa.

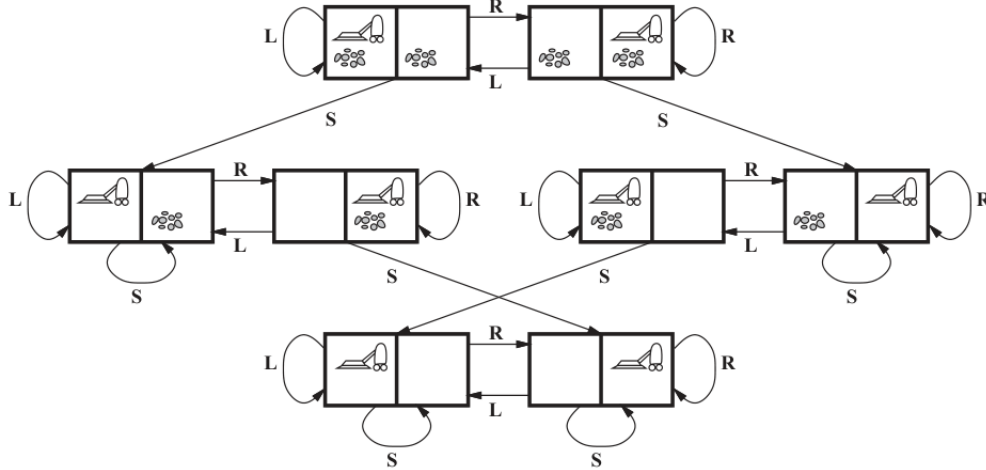


Figura 3.1: Tutti i possibili stati del vacuum world. L = Left, R = Right, S = Suck.

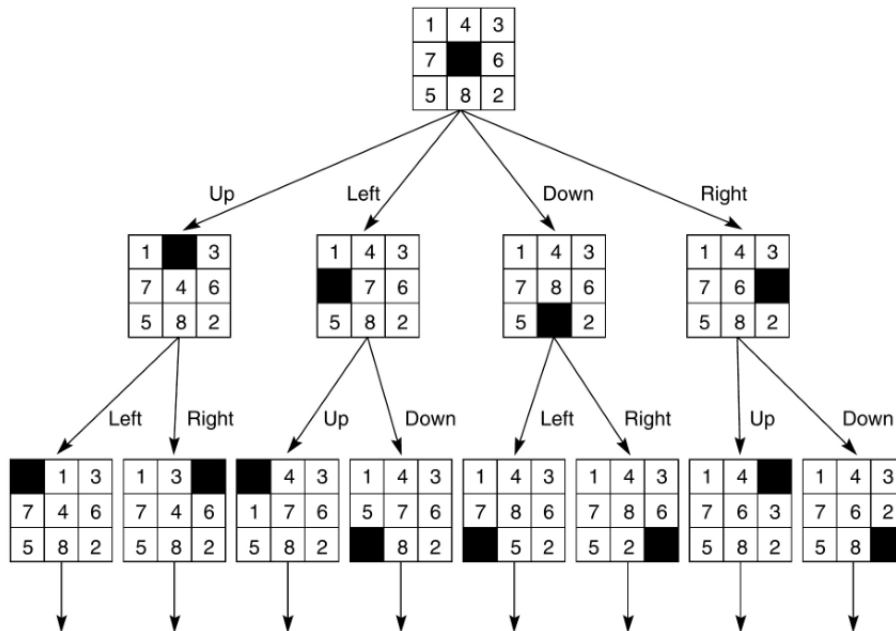


Figura 3.2: Possibili stati dell'8-puzzle.

3.3 Ricercare una soluzione

Dopo aver formulato il problema possiamo passare a risolverlo. Come abbiamo anticipato, risolvere un problema significa trovare una sequenza di azioni. Gli algoritmi di ricerca permettono di trovare una soluzione considerando diverse possibili sequenze di azioni.

Il nodo radice di un **albero di ricerca** rappresenta lo stato iniziale, le azioni vengono rappresentate dai rami mentre i **nodi** corrispondono a degli stati appartenenti ai possibili stati del problema che stiamo considerando.

Consideriamo di trovarci nello stato iniziale *In(Arad)* di Figura 3.5(a). Per poter considerare una nuova azione, dobbiamo **espandere** lo stato corrente. Così facendo andiamo a **generare** un nuovo insieme di stati. Nell'immagine Figura 3.5(b): espandendo il nodo

genitore $In(Arad)$ abbiamo aggiunto tre rami che portano a tre nodi figli. Ora si tratta di capire quale strada intraprendere, ovvero quale stato scegliere tra i tre generati.

Definiamo **frontiera** l'insieme nei nodi dell'albero che non hanno figli (foglie) e che possono essere espansi. Nella Figura 3.5(b) la frontiera è $\{Sibiu, Timisoara, Zerind\}$. Il processo di espansione dei nodi della frontiera continua finché non viene trovata una soluzione oppure non ci sono più nodi da espandere.

```

1 # Descrizione informale di un algoritmo di ricerca ad albero generico
2 def function Tree-Search(problem, strategy) returns a solution, or failure {
3   initialize the search tree using the initial state of problem
4   loop do {
5     if there are no candidates for expansion then return failure;
6     choose a leaf node for expansion according to strategy;
7     if the node contains a goal state then return the corresponding solution;
8     else expand the node and add the resulting nodes to the search tree;
9   }
10 }
```

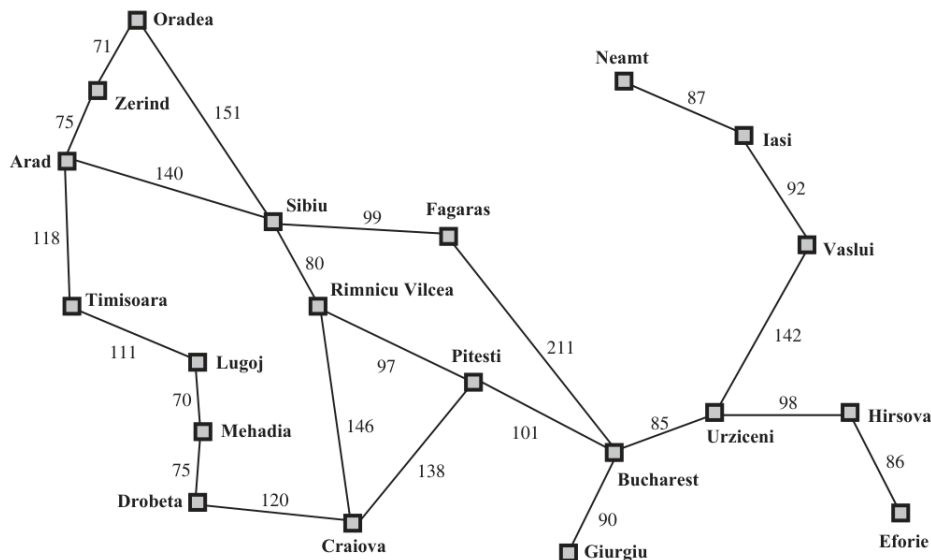


Figura 3.3: Problema: raggiungere Bucharest partendo da Arad.

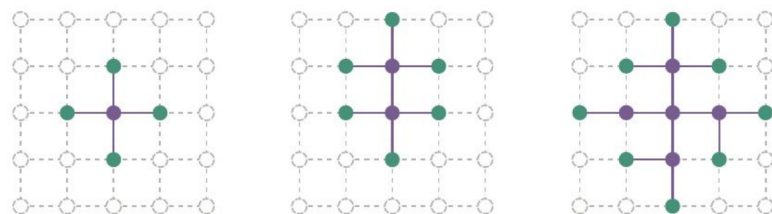


Figura 3.4: Rappresentazione di: nodi non espansi (grigi); nodi espansi (viola); nodi della frontiera (verde).

Ogni nodo dell'albero è una struttura dati che contiene quattro componenti: lo stato; un puntatore al genitore; un puntatore figli (se ne ha); la profondità; il costo per raggiungerlo.

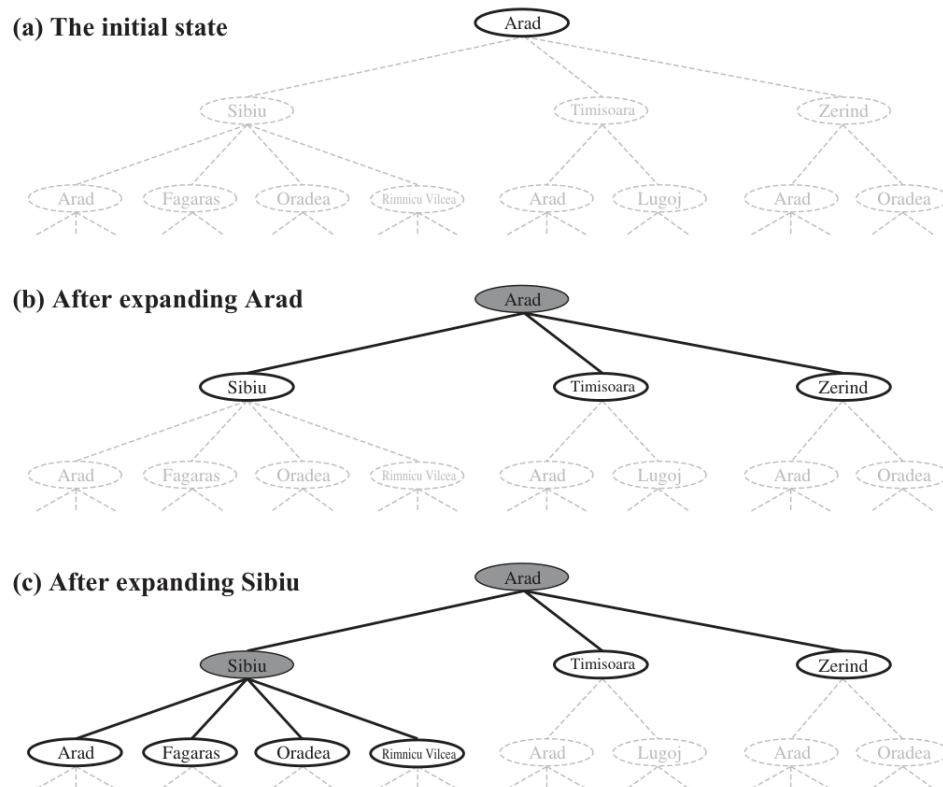


Figura 3.5: Esempio di albero di ricerca. Obiettivo trovare un percorso che porta da Arad a Bucharest.

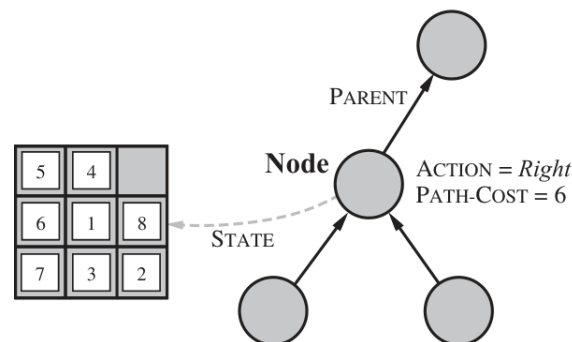


Figura 3.6: Struttura di un nodo dell'albero

Uno stato non è un nodo, è una rappresentazione di una configurazione fisica. Nota inoltre che due nodi possono contenere lo stesso stato.

3.3.1 Misurare le prestazioni delle strategie di ricerca

Le strategie di ricerca vengono confrontate basandosi su quattro parametri:

- **completezza:** se esiste una soluzione, questa viene sempre trovata?
- **complessità temporale:** proporzionale al numero di nodi generati/espansi;
- **complessità spaziale:** massimo numero di nodi contemporaneamente in memoria;

- **ottimalità**: trova sempre la soluzione avente il costo minimo?

Definiamo inoltre:

- b = massimo branching factor (numero di nodi per figlio interno) dell'albero di ricerca;
- d = profondità della soluzione con costo minimo;
- m = massima profondità dello spazio degli stati (potrebbe essere infinita).

3.4 Uninformed Search Strategies

Le ricerche non informate (o **blind search**) sfruttano soltanto le informazioni fornite dalla definizione del problema. La caratteristica che differenzia una strategia da un'altra è l'ordine con cui vengono espansi i nodi dell'albero di ricerca.

3.4.1 Breadth-first search

La BFS espande il nodo non espanso che si trova alla profondità minore. La frontiera è una coda FIFO. Nella Figura 3.7: $Frontier = \{A\} \rightarrow Frontier = \{B, C\} \rightarrow Frontier = \{C, D, E\} \rightarrow Frontier = \{D, E, F, G\}$.

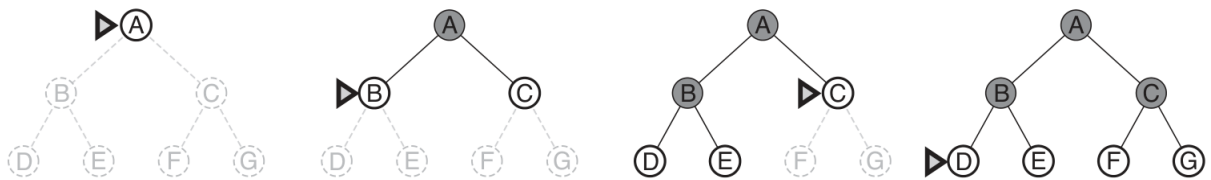


Figura 3.7: Ordine di espansione della ricerca BFS.

Complete	Sì (se il branching factor è finito)
Time	$O(b^d)$
Space	$O(b^d)$
Optimal	Sì se il costo = 1; no in generale

Tabella 3.1: Caratteristiche del BFS.

Il problema principale del BFS è l'occupazione di spazio in memoria.

3.4.2 Uniform-cost search

La uniform-cost search (algoritmo di Dijkstra) espande il nodo non espanso con costo minimo. La frontiera è una coda ordinata in maniera decrescente in base al costo del nodo. Se i nodi hanno tutti stesso costo, allora diventa un BFS.

Complete	Sì, se il costo dei rami $\geq \epsilon$
Time	peggio di $O(b^d)$ se costi dei rami molto piccoli
Space	peggio di $O(b^d)$ se costi dei rami molto piccoli
Optimal	Sì

Tabella 3.2: Caratteristiche del uniform-cost search.

3.4.3 Depth-first search

Il DFS espande il noto non espanso che ha profondità maggiore. La frontiera è una coda LIFO. Nella Figura 3.8: $Frontier = \{A\} \rightarrow Frontier = \{B, C\} \rightarrow Frontier = \{D, E, C\} \rightarrow \dots$
 $Frontier = \{H, I, E, C\} \rightarrow \text{etc.}$

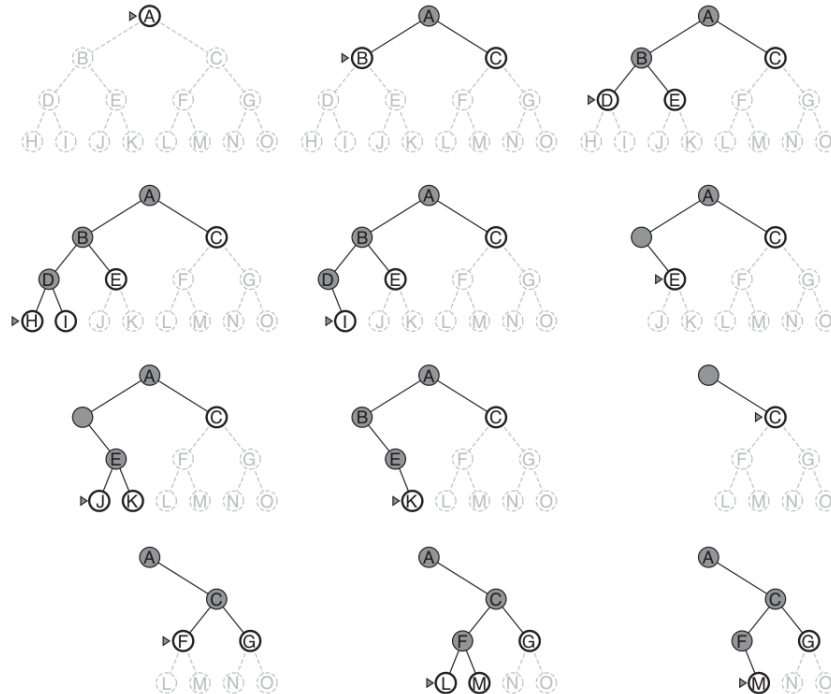


Figura 3.8: Ordine di espansione della ricerca DFS.

Complete	No: fallisce se profondità infinita o presenza di loop
Time	$O(b^m)$
Space	$O(bm)$
Optimal	No

Tabella 3.3: Caratteristiche del uniform-cost search.

Introducendo un vincolo sulla profondità massima (**depth-limited search**) si può ottenere la completezza. Il problema principale svantaggio del DFS è la complessità temporale nel caso pessimo, mentre il suo vantaggio è l'occupazione in memoria lineare.

3.4.4 Iterative deepening depth-first search

Si tratta di una depth-limited search in cui il limite di profondità massima viene incrementato iterativamente.

```

1 def function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
2   for depth = 0 to  $\infty$  do {
3     result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem,depth)
4     if result  $\neq$  cutoff then return result
5   }
```

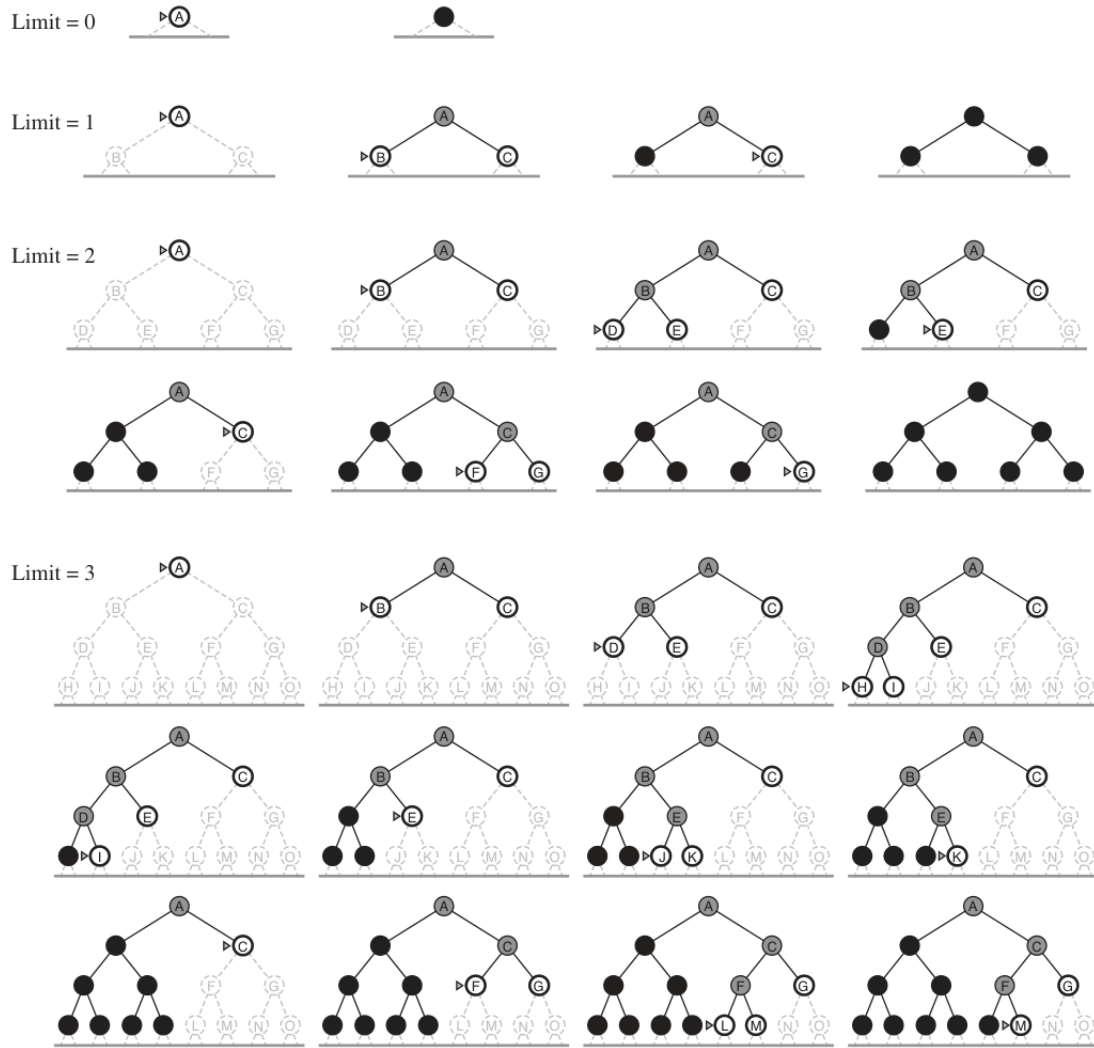


Figura 3.9: Esecuzione della deeping DFS.

Complete	Si
Time	$O(b^d)$ asintoticamente come BFS
Space	$O(bd)$
Optimal	Si, se costo = 1 (con delle modifiche funziona su uniform-cost tree)

Tabella 3.4: Caratteristiche del uniform-cost search.

3.4.5 Bidirectional search

L'idea è quella di sfruttare due ricerche contemporaneamente: una che parte dallo stato iniziale per raggiungere l'obiettivo; l'altra che parte dall'obiettivo per raggiungere lo stato iniziale. La ricerca termina quando le frontiere delle due ricerche lanciate si incontrano. Può essere implementata sfruttando la BFS o la uniform-cost search. Il vantaggio principale è che $O(b^{d/2}) + O(b^{d/2}) \ll O(b^d)$.

Complete	Si se b e lo spazio degli stati sono finiti
Time	$O(b^{d/2})$
Space	$O(b^{d/2})$
Optimal	Si se i rami hanno tutti stesso costo

Tabella 3.5: Caratteristiche del uniform-cost search.

3.5 Informed (heuristic) search strategies

Le strategie di ricerca informate hanno a disposizione una stima della distanza dell'obiettivo. Gli algoritmi che realizzano questo tipo di ricerca possono trovare soluzioni in maniera più efficiente degli algoritmi che realizzano ricerche non informate.

L'approccio generale che consideriamo viene chiamato **best-first search**. La best-first search è un'istanza dell'algoritmo generale TREE-SEARCH in cui la scelta del prossimo nodo da espandere viene fatta basandosi su una **evaluation function** $f(n)$. La evaluation function fornisce in output il costo stimato, il nodo che viene espanso è quello che ha il costo stimato minore.

La definizione della funzione f differenzia gli algoritmi che implementano questo tipo di ricerca. La maggior parte degli algoritmi definisce, come componente di f , una **funzione euristica** $h(n)$:

$$h(n) = \text{costo stimato del percorso a costo minimo partendo} \\ \text{dallo stato del nodo } n \text{ allo stato obiettivo}$$

dunque se n è l'obiettivo, allora $h(n) = 0$.

3.5.1 Greedy best-first search

Greedy best-first search punta ad espandere il nodo che si trova più vicino al goal, in questo modo si dovrebbe raggiungere velocemente ad una soluzione. In pratica prende la decisione sul nodo da espandere basandosi solamente sulla funzione euristica, quindi si ha che $f(n) = h(n)$.

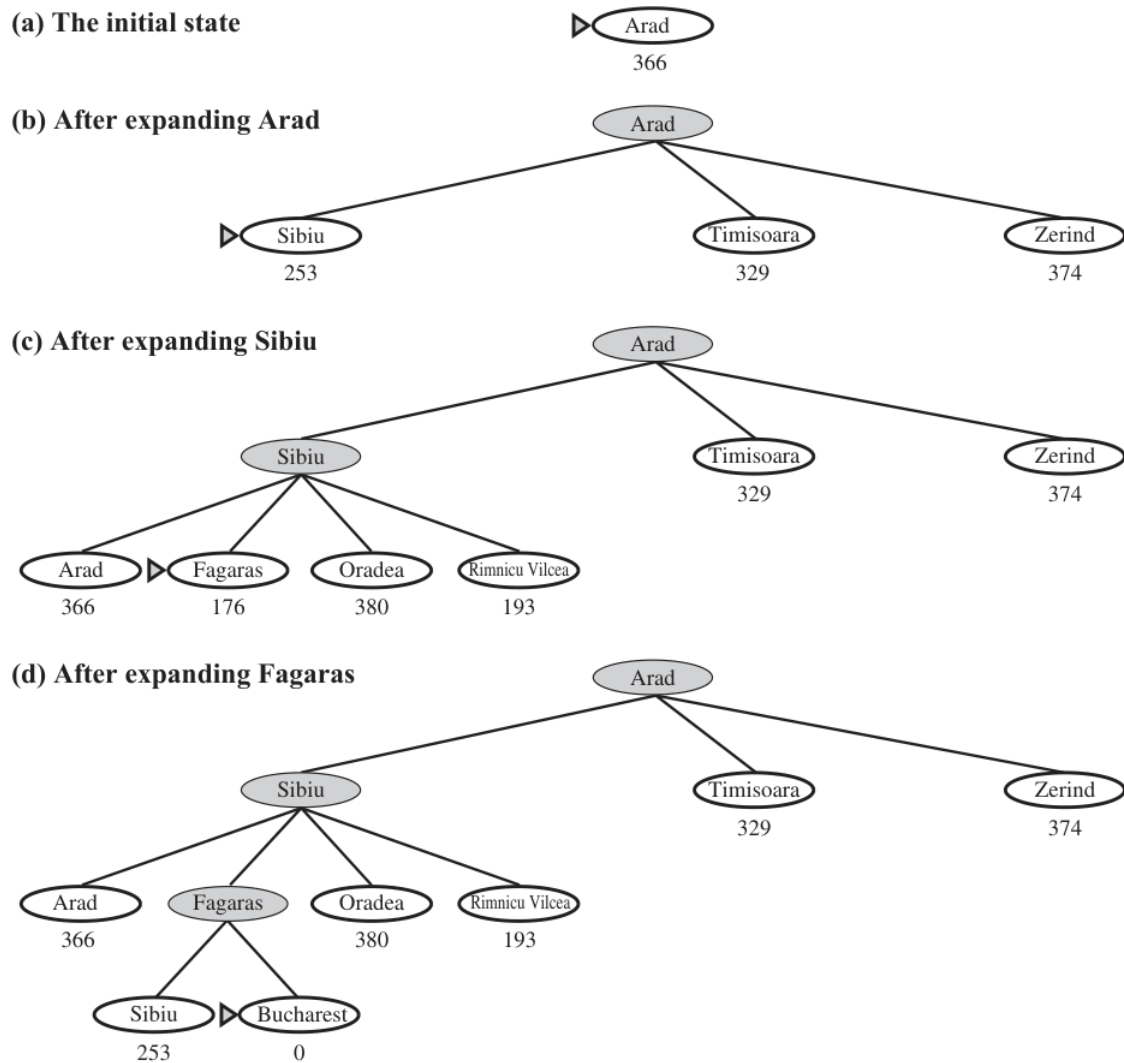
Nell'esempio di Figura 3.3, l'obiettivo è quello di raggiungere Bucharest partendo da Arad. Possiamo utilizzare la **straight-line distance heuristic** che chiamiamo semplicemente h_{SLD} . La h_{SLD} fornisce la distanza in linea d'aria dal luogo in cui ci troviamo fino a Bucharest. Queste distanze non possono essere ricavate dalla descrizione del problema, devono essere fornite (Figura 3.10).

Nella Figura 3.11 mostra alcuni passi dell'esecuzione del greedy best-first search usando h_{SLD} per trovare un percorso da Arad a Bucharest. Il primo nodo che viene espanso è *Sibu* in quanto ha la distanza in linea d'aria associata minore rispetto a *Timisoara* e *Zerind*. Successivamente viene espanso *Fagras* e infine *Bucharest*. La ricerca termina avendo espanso solo nodi appartenenti alla soluzione, tuttavia non ha trovato il percorso con costo minimo.

Complete	No, può rimanere bloccato in loop
Time	$O(b^m)$, la scelta euristica giusta può migliorarlo
Space	$O(b^m)$, mantiene tutti i nodi in memoria
Optimal	No

Tabella 3.6: Caratteristiche del Greedy best-first search.

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

 Figura 3.10: Valori di h_{SLD} .

 Figura 3.11: Passi della Greedy best-first search usando h_{SLD} .

3.5.2 A* search: minimizzare il costo totale stimato della soluzione

La A* search (pronunciata come A-star search) utilizza la seguente evaluation function:

$$f(n) = g(n) + h(n)$$

dove $g(n)$ torna il costo per raggiungere il nodo n dallo start e $h(n)$ il costo per raggiungere il goal dal nodo n . In questo modo $f(n)$ fornisce il costo stimato della soluzione a costo minimo passando per il nodo n . In sostanza la A* search è identica alla uniform-cost search ad eccezione per il fatto che usa $g + h$ invece che g solamente.

Condizione per l'ottimalità (per versione tree-search) La condizione per l'ottimalità della A* search è che $h(n)$ sia una **euristica ammissibile**. Una euristica ammissibile non fornisce mai una sovrastima del costo per raggiungere il goal, per natura sono ottimistiche perché pensano che il costo della soluzione del problema sia più piccolo di quello che è realmente. Nel paragrafo precedente abbiamo introdotto la funzione euristica h_{SLD} , questa era una euristica ammissibile, infatti considerare la distanza in linea d'aria è una sottostima. Riassumendo:

$$h(n) \geq 0, \quad h(G) = 0 \text{ per ogni goal } G$$

$$h(n) \leq h'(n) \text{ dove } h'(n) \text{ è il vero costo da } n \text{ al goal}$$

se sono verificate tali proprietà, allora l'algoritmo di ricerca è ottimo.

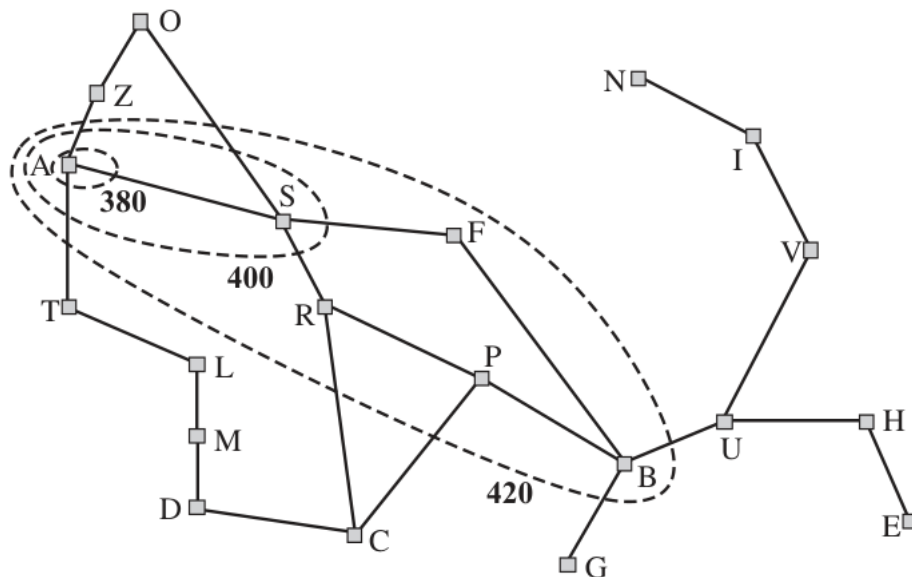
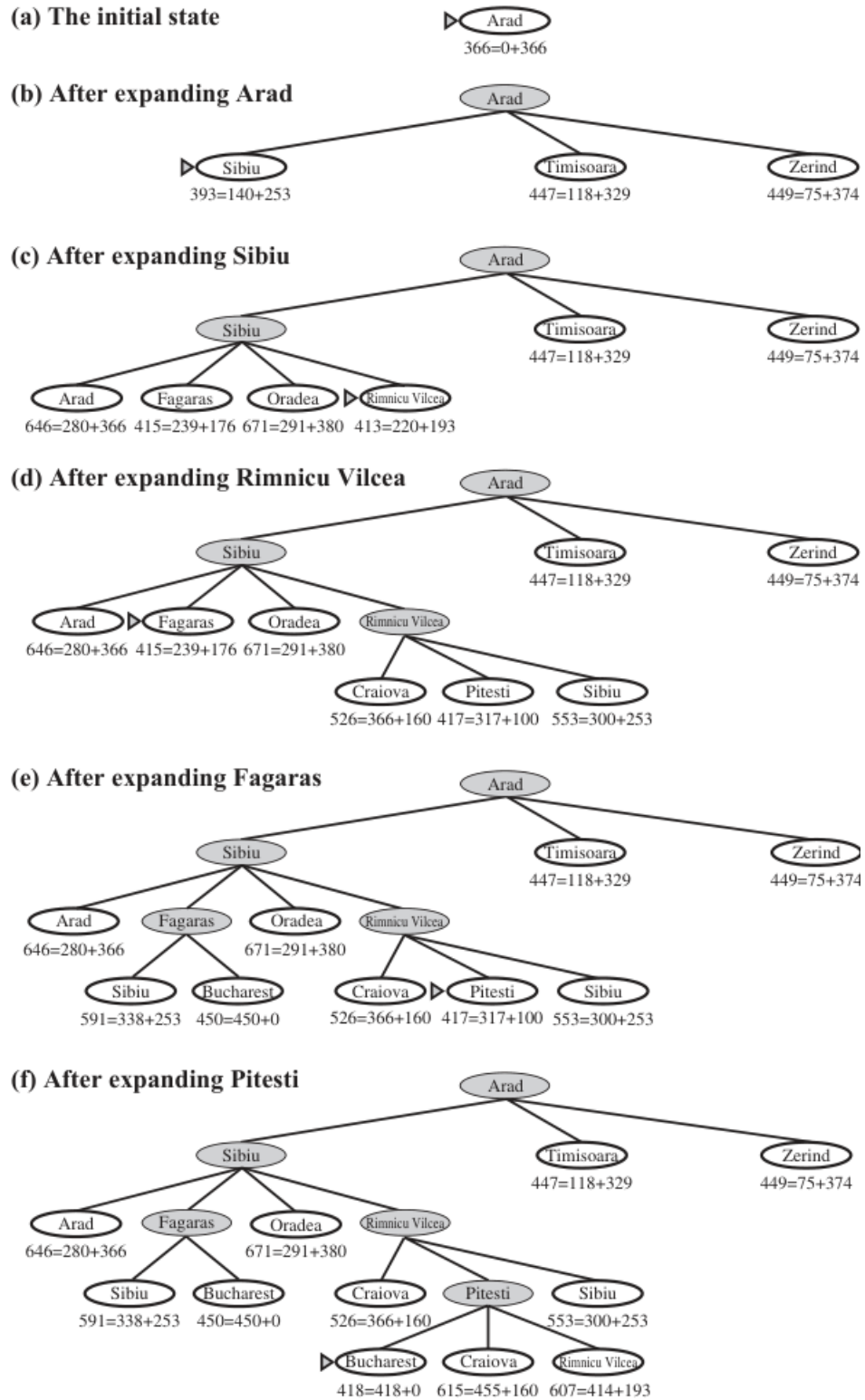


Figura 3.12: Mappa della Romania con le aree tratteggiate che indicano lo stato di avanzamento quando $f = 380, 400, 420$. I nodi dentro alle aree hanno un costo dato da f minore di quello associato all'area. A differenza della BFS, i contorni della A* search "tendono" verso il goal.

Complete	No, se non ci sono infiniti nodi con $f \leq f(G)$
Time	$O(b^d)$ al caso peggiore, dipende da quanto è buona h
Space	$O(b^d)$, mantiene tutti i nodi in memoria
Optimal	Sì, se h è euristica ammissibile

Tabella 3.7: Caratteristiche del A* search.


 Figura 3.13: Passi della A* search usando $f = h + g$, dove $h = h_{SLD}$.

Weighted A* Si tratta di una variante in cui la funzione euristica fornisce delle sovrastime, dunque non rispetta il vincolo di ammissibilità. In questo modo la ricerca non è ottimale, tuttavia può essere più efficiente in quanto il numero totale di nodi espansi può essere più

piccolo rispetto ad un normale ricerca A^* . La evaluation function è

$$f(n) = g(n) + W \cdot h(n) \quad (1 < W < \infty)$$

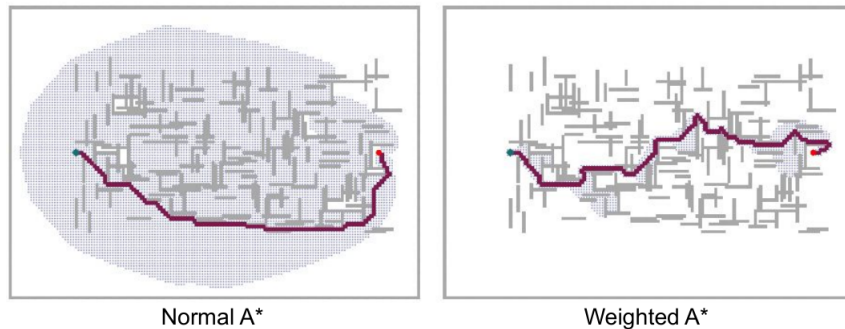


Figura 3.14: La ricerca A^* normale visita tutti i nodi intermedi (punti grigi) prima di arrivare all'obiettivo. La ricerca A^* pesata con $W = 2$ riduce lo spazio di ricerca di un fattore 7 e incrementa il costo finale del 5%.

La ricerca A^* pesata può essere vista come una generalizzazione delle altre:

- Weighted A^* search: $f(n) = g(n) + W \cdot h(n) \quad (1 < W < \infty)$.
- A^* search: $f(n) = g(n) + h(n) \quad (W = 1)$.
- Uniform-cost (Dijkstra) search: $f(n) = g(n) \quad (W = 0)$.
- Greedy best-first search: $f(n) = h(n) \quad W \rightarrow \infty$.

3.6 Funzioni euristiche

3.6.1 Effective branching factor

Un modo per definire la qualità di una funzione euristica è considerare **l'effective branching factor** b^* . Se il numero totale dei nodi generati da A^* per un particolare problema è N e la soluzione si trova a profondità d , allora b^* è il branching factor che un albero uniforme con profondità d dovrebbe avere per contenere $N + 1$ nodi. Quindi:

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d.$$

Le misurazioni sperimentali fatte su b^* su un gruppo di problemi mostrano che il parametro fornisce delle ottime indicazioni sull'utilità di una funzione euristica. In generale, una funzione euristica progettata correttamente ha un valore di b^* molto vicino a 1, in questo modo problemi complicati possono essere risolti in tempi computazionali ragionevoli.

Prendiamo come esempio il gioco 8-puzzle. L'obiettivo del gioco è quello di spostare le tessere verticalmente e orizzontalmente sfruttando la cella vuota, per ottenere una data configurazione. Risolvere il problema usando una ricerca ad albero esaustiva richiede troppo tempo. Possiamo risolvere il problema usando la ricerca A^* , per farlo dobbiamo definire una funzione euristica ammissibile. Le due funzioni candidate più comuni sono:

- h_1 = numero di tessere che non si trovano al posto giusto. Nell'immagine, tutte e 8 sono nel posto sbagliato, quindi $h_1 = 8$. h_1 è euristica ammissibile in quanto è una sottostima dire che tutte le celle che non sono al loro posto vanno mosse almeno una volta.

- h_2 = somma delle distanze delle tessere dalla loro posizione al goal. Spesso viene chiamata *Manhattan distance*. h_2 è euristica ammissibile in quanto fornisce una sottostima, infatti si tratta del numero di mosse minime per ottenere la configurazione obiettivo. Dall'immagine abbiamo che $h_2 = 18$.

Entrambe forniscono una sottostima, infatti il numero di mosse per ottenere la soluzione è 26.

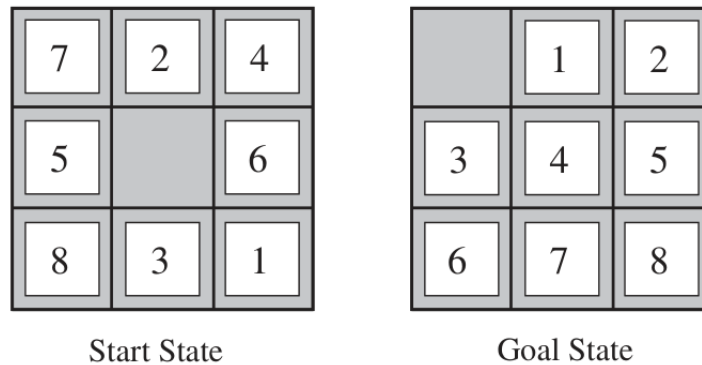


Figura 3.15: 8-puzzle game, la soluzione si trova a 26 passi di distanza.

d	Search Cost (nodes generated)			Effective Branching Factor		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Data averaged over 100 puzzles for each solution length d

3.6.2 Learning heuristics from experience

L'idea è quella di ricavare la funzione euristica basandosi su dei dati ricavati direttamente sul campo. Un approccio comune è quello di definire due funzioni x_1, x_2 e utilizzare una combinazione lineare di queste per predire h :

$$h(n) = c_1 x_1(n) + c_2 x_2(n)$$

dove c_1, c_2 sono costanti che vengono aggiornate in base ai dati ricavati dalle simulazioni.

3.7 Algoritmi di ricerca meno classici

3.7.1 Local search and optimization problems

In molti problemi di ottimizzazione, non è importante il percorso che ci porta alla soluzione ma solamente raggiungerla: raggiungere lo stato obiettivo è la soluzione. Per questa tipologia di problemi si utilizzano algoritmi che effettuano miglioramenti iterativi. Viene mantenuto memorizzato un singolo "stato corrente" e da questo effettuano miglioramenti al fine di raggiungere lo stato obiettivo. Partendo da uno stato iniziale, la **ricerca locale** opera guardando gli stati vicini senza tenere traccia del percorso seguito o degli stati raggiunti. Il problema principale è che possono esserci delle parti dello spazio di ricerca che non vengono mai esplorate e lo stato obiettivo potrebbe non essere mai raggiunto.

Nelle prossime sottosezioni analizziamo alcuni algoritmi che implementano delle ricerche locali.

3.7.2 Hill-climbing search (or gradient ascent/descent)

Semplicemente un loop che si muove nella direzione in cui i crescono i valori. L'algoritmo termina quando raggiunge un picco, ovvero quando le direzioni in cui può muoversi hanno valori che decrescono.

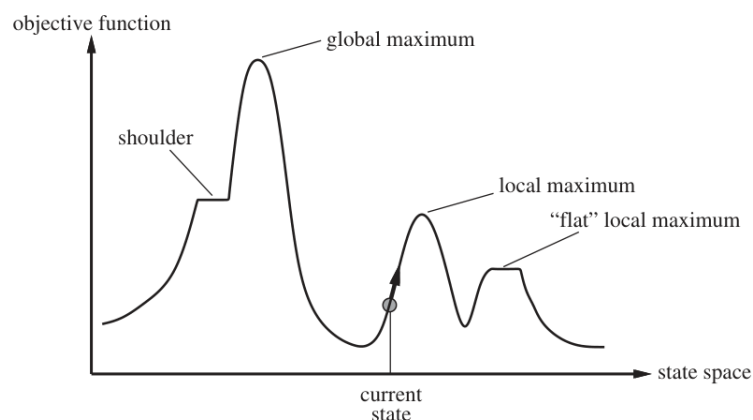


Figura 3.16: Hill-climbing search.

```

1 def function Hill-Climbing(problem) returns a state that is a local maximum {
2   current = Make-Node(problem.Initial-State)
3   loop do {
4     neighbor = a highest-valued successor of current
5     if neighbor.Value <= current.Value then return current.State
6     current = neighbor
7   }
8 }
```

Per risolvere il problema dei massimi locali si può utilizzare la variante **random-restart hill hill-climbing**. Esiste anche la versione **random sideways moves** in cui l'algoritmo si muove in direzioni casuali.

3.7.3 Simulating annealing

Un algoritmo hill-climbing che non effettua mai delle "cattive scelte" è per forza di cose incompleto in quanto rimarrà sicuramente bloccato in un massimo/minimo locale. L'idea dell'algoritmo simulating annealing è proprio quella di effettuare ogni tanto una cattiva scelta. Funziona in maniera simile all'algoritmo hill-climbing ma si muove in maniera casuale e accetta *bad moves* con una probabilità p .

$$p(x) \propto e^{-\Delta E(x)/T}$$

```

1 def function Simulated-Annealing( problem, schedule) returns a solution state {
2
3   Inputs: problem, a problem
4         schedule, a mapping from time to "temperature"
5   local variables: current, a node
6                   next, a node
7                   T, a "temperature" controlling prob. of downward steps
8
9   current = Make-Node(Initial-State[problem])
10  for t = 1 to  $\infty$  do {
11    T = schedule[t]
12    if T = 0 then return current
13    next = a randomly selected successor of current
14     $\Delta E$  = Value[next] - Value[current]
15    if  $\Delta E > 0$  then current = next
16    else current = next only with probability  $e^{-\Delta E/T}$ 
17  }
18 }
```

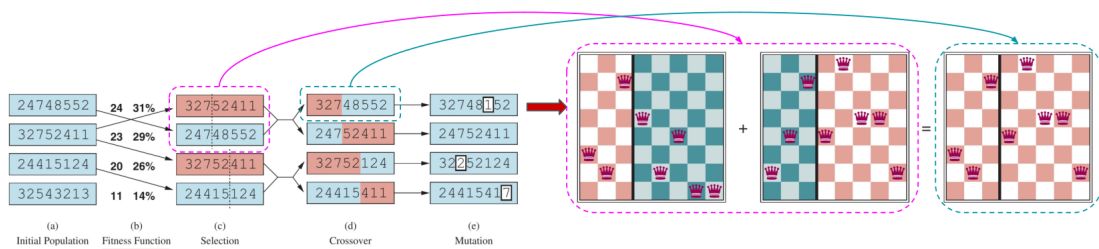
3.7.4 Local beam search

Local beam search mantiene in memoria più stati, non solo quello corrente. Inizia da k stati generati randomicamente. Ad ogni step vengono generati tutti i k successori dei k stati generati: se tra di questi si trova lo stato goal, allora l'algoritmo termina; altrimenti seleziona i migliori k successori dalla lista completa e ripete il processo.

Il problema è che accade spesso che tutti gli stati prima o poi si blocchino sullo stesso massimo/minimo locale. Una soluzione è quella di utilizzare la **stochastic beam search**: invece di scegliere i migliori k successori, sceglie in maniera casuale.

3.7.5 Genetic algorithms

Si tratta di una variante della stochastic beam search in cui gli stati successori vengono generati da due stati genitori invece che modificando un singolo stato.



Capitolo 4

Logical agents

4.1 Knowledge-Based Agents

La **knowledge base (KB)** è un insieme di proposizioni scritte in un qualche linguaggio formale. Attraverso le proposizioni forniamo (tell) all'agente una conoscenza di base, il resto riesce a ricavarcelo interrogandosi e rispondendosi basandosi sulla KB. Questo approccio di creazione di un agente viene detto **dichiarativo** (declarative).

L'agente può essere analizzato prendendo in considerazione due aspetti: **knowledge level**; **implementation level**. Il livello di conoscenza permette di analizzare ciò che l'agente sa, indipendentemente da come ciò è stato implementato. Il livello implementativo si focalizza sulle strutture dati che realizzano la KB e gli algoritmi che permettono di manipolarla.

Il tipo più semplice di knowledge-base agent deve essere in grado di:

- rappresentare stati, azioni, etc;
- incorporare nuove percezioni;
- aggiornare la sua rappresentazione interna del mondo (il modello interno);
- dedurre proprietà del mondo che ancora non conosce;
- dedurre la corretta azione da compiere.

```
1 def function KB-AGENT(percept) returns an action {  
2  
3     persistent: KB, a knowledge base  
4         t, a counter, initially 0, indicating time  
5  
6     TELL(KB, MAKE-PERCEPT-SENTENCE(percept,t)) # salvo la percezione in KB  
7     action = ASK(KB, MAKE-ACTION-QUERY(t)) # chiedo quale azione svolgere  
8     TELL(KB, MAKE-ACTION-SENTENCE(action,t)) # salvo l'azione in KB  
9     t = t + 1  
10    return action  
11 }
```

4.2 The Wumpus world

Consideriamo un esempio in cui il knowledge-base agent mostra le sue potenzialità.

Descrizione PEAS del problema:

- Misura delle performance: trovare l'oro +1000, morire -1000; -1 per ogni passo e -10 se viene utilizzato l'arco.
- Ambiente:
 - nelle caselle adiacenti al wumpus si percepisce della puzza;
 - nelle caselle adiacenti alle pozze si percepisce della brezza;
 - nelle caselle adiacenti alle pozze si percepisce della brezza;
 - si nota un luccichio se e solo se ci si trova nella casella in cui c'è l'oro;
 - sparando con l'arco nella direzione corretta si uccide il wumpus;
 - è possibile raccogliere l'oro e depositarlo in un'altra casella.
- Attuatori: svolta a destra, svolta a sinistra, vai dritto, raccogli, rilascia, spara.
- Sensori: sensore della brezza, del luccichio, dell'odore.

L'obiettivo è quello di massimizzare il costo complessivo.

completamente osservabile	No, ci si basa sulle percezioni
Deterministico	Sì, si conosce il risultato di ogni azione
Episodico	No, sequenziale a livello di azioni
Statico	Sì, il wumpus e le pozze non si muovono
Discreto	Sì
Single-agent	Sì (il wumpus non è un agente)

Tabella 4.1: Caratteristiche dell'ambiente.

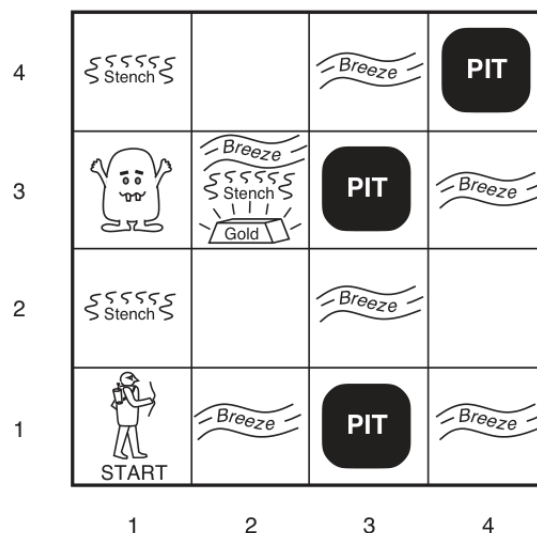


Figura 4.1: Wumpus world. L'agente si trova nell'angolo in basso a sinistra.

Ora che abbiamo descritto il problema e l'ambiente, l'agente può iniziare ad operare. Dalla Figura 4.2(a): l'agente si trova in uno stato sicuro, non percepisce nulla e conclude che le celle [1,2] e [2,1] sono anch'esse sicure. Decide quindi di muoversi in [2, 1] (Figura 4.2(2)). Nella casella corrente percepisce della brezza e di conseguenza ipotizza che in [2, 2] o in [3, 1] ci sia una pozza.

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1 A OK	2,1 OK	3,1	4,1

A = Agent
 B = Breeze
 G = Glitter, Gold
 OK = Safe square
 P = Pit
 S = Stench
 V = Visited
 W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2 P?	3,2	4,2
OK			
1,1 V OK	2,1 A B OK	3,1 P?	4,1

(a)
(b)

Figura 4.2: La prima scelta dell'agente.

L'agente decide (Figura 4.3) di tornare in [1, 1] e successivamente in [1, 2] (entrambe caselle sicure). Nella posizione corrente non percepisce della brezza, dunque deduce che in [2, 2] non può esserci una pozza e quindi la pozza è in [3, 1]. In [1, 2] percepisce della puzza e, siccome in [2, 1] non aveva percepito della puzza, deduce che il wumpus si deve trovare in [1, 3].

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

Figura 4.3: Seconda e terza mossa dell'agente.

4.3 La logica in generale

Possiamo definire la **logica** come un linguaggio formale che permette di rappresentare delle informazioni e di trarre delle conclusioni. Come ogni linguaggio formale, ha una **sintassi** che ci permette di costruire delle proposizioni e una **semantica** che definisce il senso della proposizione. Per esempio in aritmetica $x + 2 \geq y$ è una proposizione, mentre $x^2 + y >$ non

lo è perché la sintassi non è corretta. $x + 2 \geq y$ è vero in un modello in cui $x = 7, y = 1$; è falso in un modello in cui $x = 0, y = 6$.

Un altro concetto fondamentale è la **conseguenza logica (entailment)** (o implicazione logica).

$$KB \models \alpha$$

La knowledge-base ha come conseguenza logica la proposizione α se e solo se α è vera in tutti i modelli in cui KB è vera. Per esempio se la KB contiene "*John ha vinto*" e "*Mary ha vinto*" allora una conseguenza logica è "*Uno tra John e Mary ha vinto*". Oppure ancora, $x + y = 4$ ha come conseguenza logica che $4 = x + y$.

4.4 Logica proposizionale

La logica proposizionale è il tipo di logica più semplice. Definiamo **proposizioni atomiche** le componenti di base che non possono essere decomposte ulteriormente. Costruiamo proposizioni complesse legando più proposizioni atomiche con dei connettivi logici. Date due proposizioni S_1, S_2 definiamo:

Negazione	$\neg S_1$
Congiunzione	$S_1 \wedge S_2$
Disgiunzione	$S_1 \vee S_2$
Implicazione	$S_1 \Rightarrow S_2$
Doppia implicazione	$S_1 \Leftrightarrow S_2$

Tabella 4.2: I connettivi logici.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Figura 4.4: Tabelle di verità dei connettivi logici.

$$\begin{aligned}
 (\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
 (\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
 ((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
 ((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
 \neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
 (\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
 (\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
 (\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
 \neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
 \neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
 (\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
 (\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
 \end{aligned}$$

Figura 4.5: Equivalenze logiche standard.

4.4.1 Proposizioni nel Wumpus World

Ora che abbiamo definito la semantica della logica proposizionale, possiamo costruire un knowledge-base agent da utilizzare nel problema del wumpus. Poniamo:

- $P_{i,j}$ vera se c'è una pozza in $[i, j]$
- $B_{i,j}$ vera se c'è della brezza in $[i, j]$
- $W_{i,j}$ vera se c'è il wumpus in $[i, j]$
- $S_{i,j}$ vera se c'è del cattivo odore in $[i, j]$

Possiamo riscrivere formalmente le proposizioni che abbiamo descritto nella sezione 4.2:

- "Non c'è una pozza in $[1, 1]$ " diventa

$$R_1 : \neg P_{1,1}$$

(R_i indica il numero della proposizione).

- "Si percepisce brezza in una cella se e solo la cella è adiacente ad una pozza" diventa (andrebbe fatto per ogni cella, ci limitiamo a due)

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{1,2})$$

$$R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1}).$$

- Includiamo le deduzioni tratte dalle percezioni dell'agente: "non c'è brezza in $[1, 1]$ "; "non c'è brezza in $[2, 1]$ ".

$$R_4 : \neg B_{1,1}$$

$$R_5 : B_{2,1}.$$

4.4.2 Inferenza per risoluzione, CNF e dimostrazioni per assurdo

Utilizziamo la **dimostrazione per assurdo** per dimostrare che $KB \models \alpha$ dimostrando che il negato di $KB \models \alpha$ è impossibile. Il negato dell'implicazione logica $KB \models \alpha$ è $\neg(\neg KB \vee \alpha) = KB \wedge \neg\alpha$.

Definizione 4.1 (Clausola). *Un tipo particolare di formula logica è la clausola (clause), cioè una disgiunzione (or) di literals (proposizioni atomiche o loro negazioni). Per esempio:*

$$x_1 \vee \neg x_2 \vee x_3$$

Definizione 4.2 (Conjunctive normal form). *Una formula è detta in forma congiuntiva normale (Conjunctive Normal Form, CNF) se è una congiunzione (and) di clausole.*

Vediamo come convertire la formula logica

$$R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

in CFN procedendo per step:

1. da $\alpha \Leftrightarrow \beta$ a $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$:

$$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$$

2. da $\alpha \Rightarrow \beta$ a $\neg\alpha \vee \beta$:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$$

3. usando De Morgan:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$$

4. applicando la distributività:

$$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$$

La regola di risoluzione (**resolution**) è una regola di inferenza logica. Questa deriva una nuova clausola partendo da due clausole generatrici che contengono una literal e l'altra la sua negazione. La nuova clausola derivata viene detta risolvente (resolvent). Per esempio:

$$\begin{array}{lcl} x_1 \vee x_2 \vee x_3 & & \\ \neg x_1 \vee x_2 \vee \neg x_4 & \Rightarrow & x_2 \vee x_3 \vee \neg x_4 \text{ (risolvente)} \end{array}$$

il risolvente è stato ottenuto ragionando per casi: se x_1 è falsa, allora deve essere vera $x_2 \vee x_3$ (prima clausola); se x_1 è vera, allora deve essere vera $x_2 \vee \neg x_4$ (seconda clausola).

Vediamo un'applicazione dell'inferenza per risoluzione nel caso del wumpus. Se l'agente si trova in [1, 1] e non percepisce della brezza, possiamo inferire che non c'è una pozza in [1, 2]? Definiamo:

$$KB = (B_{1,1} \Leftrightarrow (P_{1,2} \wedge P_{2,1})) \vee \neg B_{1,2}, \quad \alpha = \neg P_{1,2}$$

Per provare che $KB \models \alpha$, dimostriamo per assurdo che $KB \wedge \neg\alpha$ è impossibile.

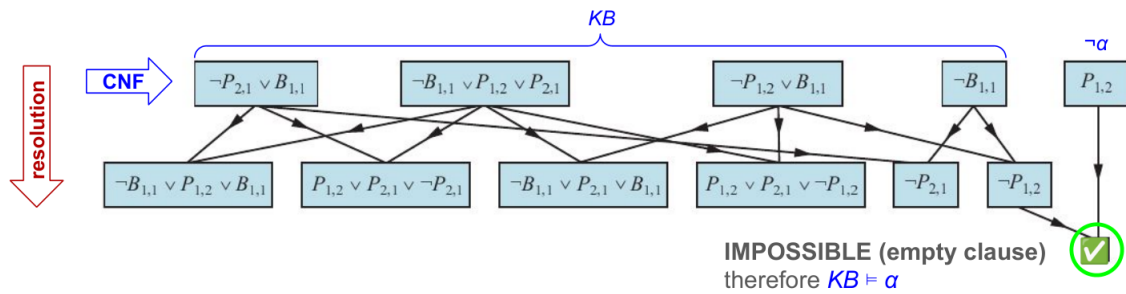


Figura 4.6: Esempio resolution per il problema del wumpus.

Come mostra la Figura 4.6 abbiamo ottenuto la clausola $\neg P_{1,2}$ e la clausola $P_{1,2}$. Siccome non è possibile assegnare un valore per renderle entrambe vere, abbiamo in questo modo dimostrato che $KB \wedge \neg\alpha$ è impossibile e di conseguenza l'implicazione logica $KB \models \alpha$ è verificata.

4.4.3 Vantaggi e svantaggi della logica proposizionale

La logica proposizionale è un formalismo chiaro la cui sintassi permette di modellare diverse tipologie di problemi. Permette inoltre di considerare informazioni negate, parziali o disgiuntive (a differenza di molti database) e permette di costruire proposizioni che sono la combinazione di altre proposizioni. Un altro vantaggio della logica proposizionale è il fatto di essere indipendente dal contesto in cui viene applicata (a differenza del linguaggio naturale). Tuttavia ha un potere espressivo molto limitato (a differenza del linguaggio naturale): non è applicabile in ogni contesto e per ogni problema.

4.5 First order logic

Abbiamo visto che nel mondo della logica proposizionale esistono solo proposizioni. La logica dei predicati del primo ordine combina il meglio dei linguaggi formali e del linguaggio naturale. Nel mondo della first order logic troviamo:

- **Oggetti:** *persone, case, numeri, teorie, colori, guerre, secoli*, etc.
- **Relazioni** o **proprietà:** *rosso, rotondo, primo, fratello di, più grande di, dentro a, parte di, si trova dopo a, possiede, si trova tra*, etc.
- **Funzioni** (ritornano uno specifico valore): *padre di, migliore amico, seconda metà di, uno in più di, la fine di*, etc.

Per esempio:

- *"One plus two equals three."*
Oggetti: one, two, three, one plus two; Relazioni: equals; Funzioni: plus.
- *"Squares neighboring the wumpus are smelly."*
Oggetti: wumpus, squares; Proprietà: smelly; Relazioni: neighboring.
- *"Evil King John ruled England in 1200."*
Oggetti: John, England, 1200; Relazioni: ruled; Proprietà: evil, king.

La sintassi di base che viene utilizzata:

- **Costanti:** *KingJohn, 2, Rome*, etc.
- **Predicati:** *Brother, i*, etc.
- **Funzioni:** *Sqrt, LeftLegOf*, etc.
- **Variabili:** *x, y, a, b,*, etc.
- **Connettivi:** $\wedge, \vee, \neg, \Rightarrow, \Leftrightarrow$, etc.
- **Uguaglianza:** $=$.
- **Quantificatori:** $\forall, \exists$.

4.5.1 Terms and atomic sentences

Un termine (**terms**) è un'espressione logica che si riferisce ad un oggetto. I termini possono essere costanti o variabili. Nel caso generale un termine complesso è formato da una *funzione simbolo* che prende come argomento una lista di termini (es: *LeftLeg(John)*). Nota che questi termini complicati sono solo dei nomi complicati che assegniamo, non sono funzioni che vengono implementate (non viene implementata la funzione *LeftLeg* che prende come argomento il nome di una persona e ritorna una gamba). Per riferirsi alle relazioni tra gli oggetti vengono utilizzati i predicati (**predicate**).

Combinando predicati e termini si vanno a formare le proposizioni atomiche (**atomic sentences**) le quali espongono i fatti. Una proposizione atomica è formata dal simbolo di un predicato seguito da una lista di termini messa tra parentesi (es: *Brother(Richard, John)*) e ritorna vero o falso. Le proposizioni atomiche possono anche avere termini complessi come argomenti (es: *Married(Father(Richard), Mother(John))*).

4.5.2 Complex sentences

Possiamo creare delle proposizioni complesse combinando proposizioni atomiche attraverso i connettivi $\wedge, \vee, \neg, \Rightarrow, \Leftrightarrow$ in maniera analoga a come abbiamo fatto nella logica proposizionale. Per esempio: $Sibling(KingJohn, Richard) \Rightarrow Sibling(Richard, KingJohn)$; $\neg King(Richard) \Rightarrow King(John)$.

4.5.3 Quantifiers

Attraverso i quantificatori possiamo esprimere in maniera semplice una proprietà che riguarda a una collezione di oggetti senza dover elencarli uno ad uno. I quantificatori che vengono utilizzati sono: $\forall, \exists$.

Per esempio (**universal quantification**): "everyone at Oxford is smart" scritto in logica dei predicati del primo ordine diventa

$$\forall x At(x, Oxford) \Rightarrow Smart(x).$$

Nota che l'intera proposizione è vera se e solo se è vera per ogni oggetto della collezione, ovvero possiamo riscriverla come

$$\begin{aligned} &(At(A, Oxford) \Rightarrow Smart(A)) \wedge \\ &(At(B, Oxford) \Rightarrow Smart(B)) \wedge \\ &(At(C, Oxford) \Rightarrow Smart(C)) \wedge \text{etc.} \end{aligned}$$

Un altro esempio è il seguente (**existential quantification**): "someone who is not at Oxford is smart" in first order logic diventa

$$\exists x \neg At(x, Oxford) \wedge Smart(x).$$

In questo caso la proposizione può essere riscritta come

$$\begin{aligned} &(\neg At(A, Oxford) \wedge Smart(A)) \vee \\ &(\neg At(B, Oxford) \wedge Smart(B)) \vee \\ &(\neg At(C, Oxford) \wedge Smart(C)) \vee \text{etc.} \end{aligned}$$

Proprietà dei quantificatori

- L'ordine non cambia se i quantificatori sono dello stesso tipo:

$$\forall x \forall y \text{ è equivalente a scrivere } \forall y \forall x$$

$$\exists x \exists y \text{ è equivalente a scrivere } \exists y \exists x$$

- L'ordine cambia se i quantificatori non sono dello stesso tipo:

$$\exists x \forall y \text{ è diverso da scrivere } \forall y \exists x.$$

Per esempio:

$$\exists x \forall y Loves(x, y) \rightarrow \text{"There is a person who loves everyone in the world"}$$

$$\forall y \exists x Loves(x, y) \rightarrow \text{"Everyone in the world is loved by at least one person"}$$

4.5.4 Knowledge engineering in First-Order Logic

Il knowledge engineer è colui che studia un problema appartenente ad un certo dominio; riesce a ricavare quali sono i concetti importanti e riesce a creare una rappresentazione formale degli oggetti e delle relazioni appartenenti al dominio.

I passi da seguire quando si affronta un problema di knowledge engineering sono i seguenti:

1. Identificare le domande.
2. Definire la conoscenza di base del problema.
3. Definire il vocabolario di predicati, funzioni e costanti.
4. Codificare la conoscenza generale del dominio.
5. Codificare una descrizione dell'istanza del problema da risolvere.
6. Porre domande alla procedura d'inferenza e ottenere le risposte.
7. Debuggare e valutare la Knowledge Based.

La parte più complessa e dispendiosa del processo si trova ai punti 3, 4, 5, infatti si tratta dei passi che ci permettono di modellare il problema che stiamo considerando. Una volta modellato il problema si utilizzano algoritmi di risoluzione standard per risolverlo.

4.6 Inferenza nella First-Order Logic

Abbiamo visto come sfruttare l'inferenza nella logica proposizionale. In questa sezione vediamo quali tecniche di inferenza possiamo sfruttare per la first order logic.

4.6.1 Substitutions

Data una proposizione S e una sostituzione σ , la nuova proposizione S_σ si ottiene applicando σ a S . Per esempio:

$$\begin{aligned} S &= \text{Smarter}(x,y) \quad (\text{proposizione originale}) \\ \sigma &= \{ x / \text{Hilary}, y / \text{Bill} \} \quad (\text{sostituzione}) \\ S_\sigma &= \text{Smarter}(\text{Hilary}, \text{Bill}) \quad (\text{proposizione dopo la sostituzione}) \end{aligned}$$

Si può scrivere:

$$S_\sigma \equiv \mathbf{Subst}(\{ x / \text{Hilary}, y / \text{Bill} \}, \text{Smarter}(x,y))$$

La funzione **Ask**(KB, S) ritorna alcune o tutte le sostituzioni σ tali che soddisfino $KB \models S_\sigma$.

4.6.2 Riduzione all'inferenza proposizionale

L'inferenza in FOL può essere applicata convertendo le proposizioni in logica proposizionale e successivamente applicando l'inferenza per risoluzione che abbiamo studiato nella sezione precedente.

Il primo passo è quello di rimuovere il quantificatore $\forall$ seguendo la regola della **Universal Instantiation (UI)**:

$$\frac{\forall x \alpha}{\text{Subst}(\{x, g\}, \alpha)}$$

per ogni variabile x e **ground term** g . In sostanza dice di sostituire la variabile x con tutti i possibili oggetti g . Per esempio:

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$$

diventa

$$\begin{aligned} &\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John}) \\ &\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John})) \\ &\vdots \\ &\text{tutti i possibili casi} \end{aligned}$$

Per rimuovere il quantificatore $\exists$ si segue la regola della **Existential instantiation (EI)**:

$$\frac{\exists x \alpha}{\text{Subst}(\{x/k\}, \alpha)}$$

Si tratta di aggiungere una costante k (detta **costante di Skolem**) la quale non appare nel knowledge based, è di fatto un "oggetto fittizio". Per esempio:

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

diventa

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

dove C_1 è una nuova costante che non appartiene al KB.

Nota che sfruttando la regola **UI** aggiungiamo nel KB una serie di nuove proposizioni che non aggiungono nuovi significati al KB: il "nuovo" KB è a livello logico equivalente a quello "vecchio". Mentre applicando la regola **EI** ottengo un "nuovo" KB che non è equivalente a quello "vecchio", tuttavia non vengono alterati gli output: un'implicazione logica è soddisfacibile se e solo se lo era anche nel "vecchio" KB.

Esempio di riduzione all'inferenza proposizionale Supponiamo che il KB contenga solamente i seguenti fatti:

$$\begin{aligned} &\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x) \\ &\text{King}(\text{John}) \\ &\text{Greedy}(\text{John}) \end{aligned}$$

e gli unici oggetti sono *John* e *Richard*. Applicando la **proposizionalizzazione** otteniamo:

$$\begin{aligned} &\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John}) \\ &\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard}) \\ &\text{King}(\text{John}) \\ &\text{Greedy}(\text{John}) \end{aligned}$$

I simboli di proposizione ($\text{King}(\text{John})$, $\text{Greedy}(\text{John})$, $\text{Evil}(\text{John})$, ...), possono essere riscritti come proposizioni (KingJohn , GreedyJohn , EvilJohn , ...).

Il **problema principale** della proposizionalizzazione è che può generare molte proposizioni che potrebbero essere irrilevanti per il quesito che abbiamo posto. Esistono altre tecniche che permettono di evitare questo problema.

4.6.3 Unificazione

L'unificazione permette di trovare una sostituzione in modo tale che due espressioni logiche differenti risultino equivalenti. Si tratta di una componente fondamentale per l'inferenza in FOL che permette di risolvere il problema della proposizionalizzazione.

L'algoritmo *Unify* prende in input due proposizioni e torna un unificatore per queste due (se esiste):

$$\text{Unify}(\alpha, \beta) = \Theta \text{ se } \text{Subst}(\Theta, \alpha) = \text{Subst}(\Theta, \beta)$$

α	β	Θ
$\text{Knows}(\text{John}, x)$	$\text{Knows}(\text{John}, \text{Jane})$	$\{x / \text{Jane}\}$
$\text{Knows}(\text{John}, x)$	$\text{Knows}(y, \text{Mary})$	$\{x / \text{Mary}, y / \text{John}\}$
$\text{Knows}(\text{John}, x)$	$\text{Knows}(y, \text{Mother}(y))$	$\{y / \text{John}, x / \text{Mother}(\text{John})\}$
$\text{Knows}(\text{John}, x)$	$\text{Knows}(x, \text{Mary})$	<i>fail due to variable overlap*</i>

Tabella 4.3: Esempi di applicazione di $\text{Unify}(\alpha, \beta) = \Theta$. * Per eliminare questo tipo di problema si cambiano i nomi delle variabili, per esempio: $\text{Knows}(z, \text{Mary})$ e quindi si ha $\Theta = \{x / \text{Mary}, z / \text{John}\}$.

4.6.4 Inferenza per risoluzione in First-Order Logic

Nella FOL si sfrutta l'unificazione. Vediamo direttamente un esempio:

$$\frac{\neg \text{Rich}(x) \wedge \text{Unhappy}(x)}{\text{Rich}(\text{Ken})} \Rightarrow \text{Unhappy}(\text{Ken}) \quad \text{con } \theta = \{x / \text{Ken}\}$$

dove $\Theta = \text{Unify}(l_i, \neg m_j) = \text{Unify}(\text{Rich}(\text{Ken}), \neg \text{Rich}(x)) = \{x / \text{Ken}\}$.

Similmente a come abbiamo visto per la logica proposizionale, applichiamo la regola di risoluzione alla forma congiuntiva normale $\text{CNF}(KB \wedge \neg \alpha)$ e se questa risulta essere insoddisfacibile, allora abbiamo dimostrato (per assurdo) che $KB \models \alpha$.

Vediamo come convertire la proposizione

$$\forall x [\forall y \text{Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{Loves}(y, x)]$$

in CNF procedendo per step:

1. Rimozione di $\Leftrightarrow$ e $\Rightarrow$:

$$\forall x [\neg \forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{Loves}(y, x)]$$

2. Porto all'interno le negazioni:

$$\forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists y \text{Loves}(y, x)]$$

3. Standardizzo le variabili: ogni quantificatore ne deve utilizzare una diversa per evitare possibili conflitti

$$\forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{Loves}(z, x)]$$

4. Elimino $\exists$ con una forma generale di *EI* usando le **Skolem functions**:

$$\forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee [\text{Loves}(G(x), x)]$$

non possiamo utilizzare delle semplice variabili in quanto queste non esprimono la dipendenza con la variabile x .

5. Elimino $\forall$:

$$[\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee [\text{Loves}(G(x), x)]$$

in questo caso non perdiamo nessuna informazione rimuovendo solo $\forall x$.

6. Applico la distributività ($\wedge$ over $\vee$):

$$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)] \wedge [\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)]$$

abbiamo ottenuto una congiunzione di due clausole.

Esempio applicazione di inferenza per risoluzione in FOL La legge dice che vendere armi alle nazioni nemiche è un crimine. Lo stato di *Nono* è nemico dell'America e ha dei missili. Questi missili sono stati venduti dal colonnello West il quale è americano. Vogliamo provare che il colonnello West è un criminale. Possiamo definire:

- Domanda: $KB \models \text{Criminal}(\text{West})$?
- Definiamo il KB:
 - $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$
rimuovendo $\Rightarrow$ diventa
 $\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$
 - $\text{Enemy}(\text{Nono}, \text{America})$
 - $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$
rimuovendo $\Rightarrow$ diventa
 $\neg \text{Enemy}(x, \text{America}) \vee \text{Hostile}(x)$
 - $\exists x \text{ Owns}(\text{Nono}, x) \wedge \text{Missile}(x)$
applicando **EI** diventa
 $\text{Owns}(\text{Nono}, M_1) \wedge \text{Missile}(M_1)$ dove M_1 è una costante di Skolem
 - $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$
rimuovendo $\Rightarrow$ diventa
 $\neg \text{Missile}(x) \vee \text{Weapon}(x)$
 - $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
rimuovendo $\Rightarrow$ diventa
 $\neg \text{Missile}(x) \vee \neg \text{Owns}(\text{Nono}, x) \vee \text{Sells}(\text{West}, x, \text{Nono})$
 - $\text{American}(\text{West})$

Si tratta ora di verificare se $KB \models \text{Criminal}(\text{West})$ provando per assurdo che non è possibile verificare $KB \wedge \neg \text{Criminal}(\text{West})$.

Abbiamo dimostrato (Figura 4.7) per assurdo che $KB \wedge \neg \text{Criminal}(\text{West})$ è impossibile e dunque possiamo concludere che $KB \models \text{Criminal}(\text{West})$ è verificato.

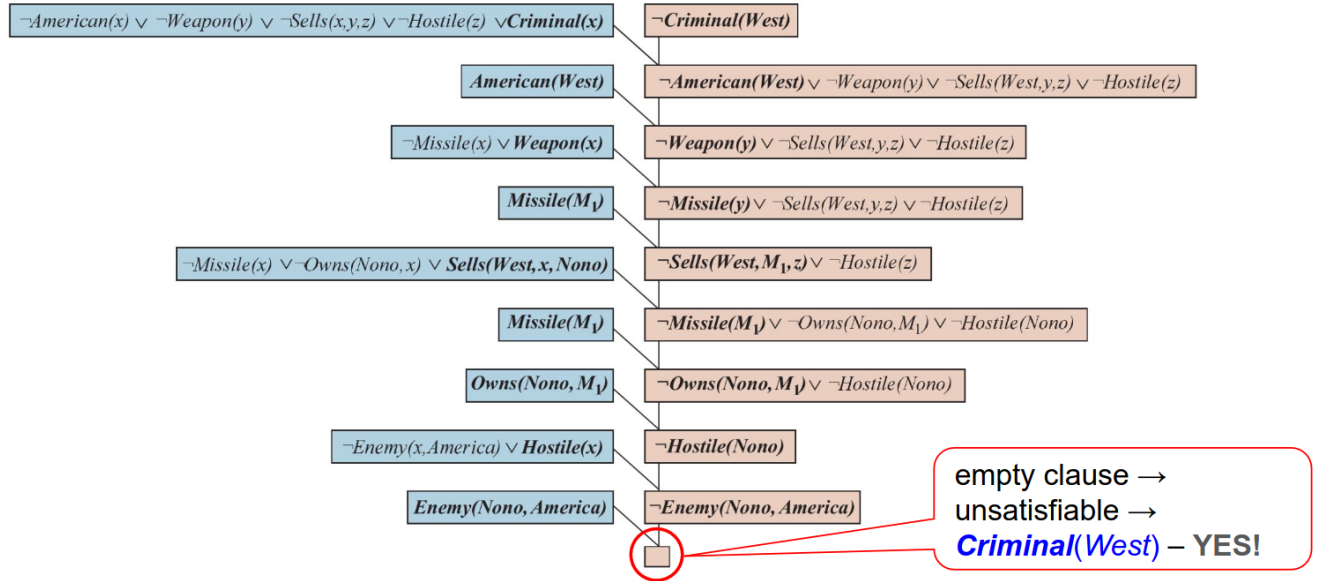


Figura 4.7: Applicazione del resolution.

4.6.5 Generalized Modus Ponens (GMP)

Per p_i, p'_i e q proposizioni atomiche per cui esiste una sostituzione Θ tale che

$$Subst(\Theta, p'_i) = Subst(\Theta, p_i),$$

per ogni i ,

$$\frac{p'_1, \dots, p'_n \quad (p_1 \wedge \dots \wedge p_n \Rightarrow q)}{Subst(\Theta, q)}$$

Per esempio, poniamo di avere il seguente KB

$$\begin{aligned} &\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x) \\ &\text{King}(\text{John}) \\ &\forall y \text{ Greedy}(y) \end{aligned}$$

otteniamo

$$\begin{aligned} p'_1 &\text{ è } \text{King}(\text{John}) & p_1 &\text{ è } \text{King}(x) \\ p'_2 &\text{ è } \text{Greedy}(\text{John}) & p_2 &\text{ è } \text{Greedy}(x) \\ \Theta &\text{ è } \{x / \text{John}, y / \text{John}\} & q &\text{ è } \text{Evil}(x) \\ Subst(\Theta, q) &\text{ è } \text{Evil}(\text{John}) \end{aligned}$$

Forward chaining Il **forward chaining (FC)** è un algoritmo che sfrutta GMP sul KB in maniera iterativa. In particolare sfrutta le **definite clauses**¹ (disgiunzioni in cui è presente un solo literal positivo, cioè non negato) per generare nuove informazioni da aggiungere al KB.

L'algoritmo non ammette che ci siano quantificatori di esistenza ($\exists$) e assume che i quantificatori universali ($\forall$) siano stati eliminati con la regola della Universal Instantiation. Nelle applicazioni pratiche viene eseguito l'algoritmo ogni volta che viene aggiunta una nuova formula al KB, in questo modo si ricavano tutte le nuove conseguenze che la nuova informazione porta con sé.

¹Anche chiamate clausole di Horn.

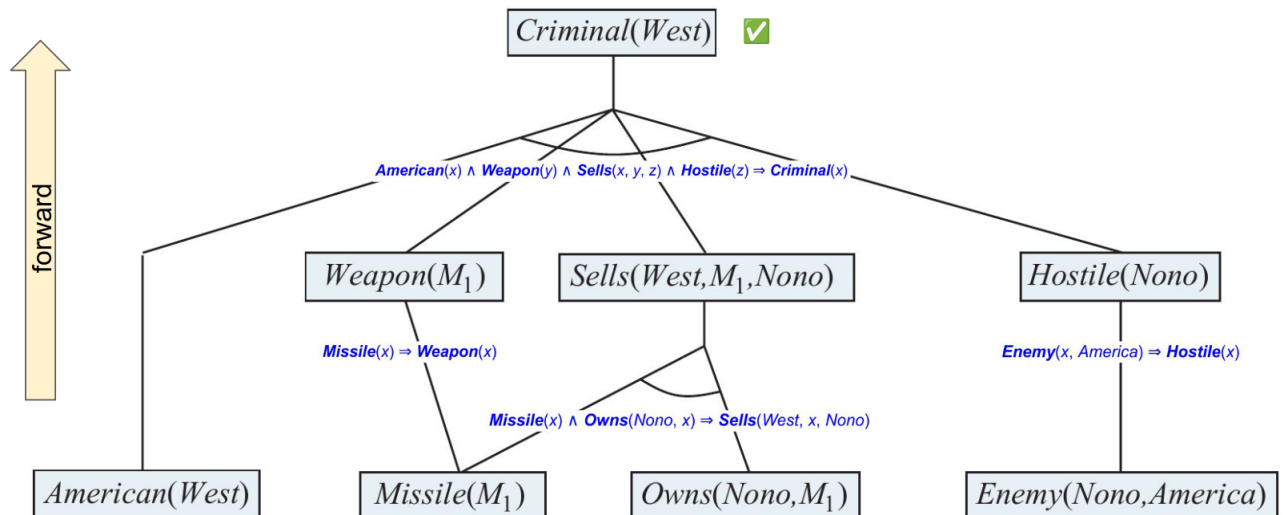


Figura 4.8: Esecuzione del Forward Chaining nel caso dell'esempio del crimine. I fatti iniziali si trovano al livello più basso, i fatti inferiti (GMP) nella prima iterazione sono nel livello intermedio, i fatti inferiti nella seconda iterazione sono nel livello più alto.

Backward chaining Il **backward chaining (BC)** è un algoritmo che funziona in maniera analoga al forward chaining, solo che invece di partire dai fatti e applicare GMP finché non raggiunge una risposta, fa l'esatto opposto: parte dalla risposta alla domanda che abbiamo posto e applica GMP per ricavare le premesse che mi porterebbero ad una tale risposta. Ogni sostituzione effettuata per unificare una frase atomica con la conseguenza di un'implicazione deve essere propagata a ciascuna premessa.

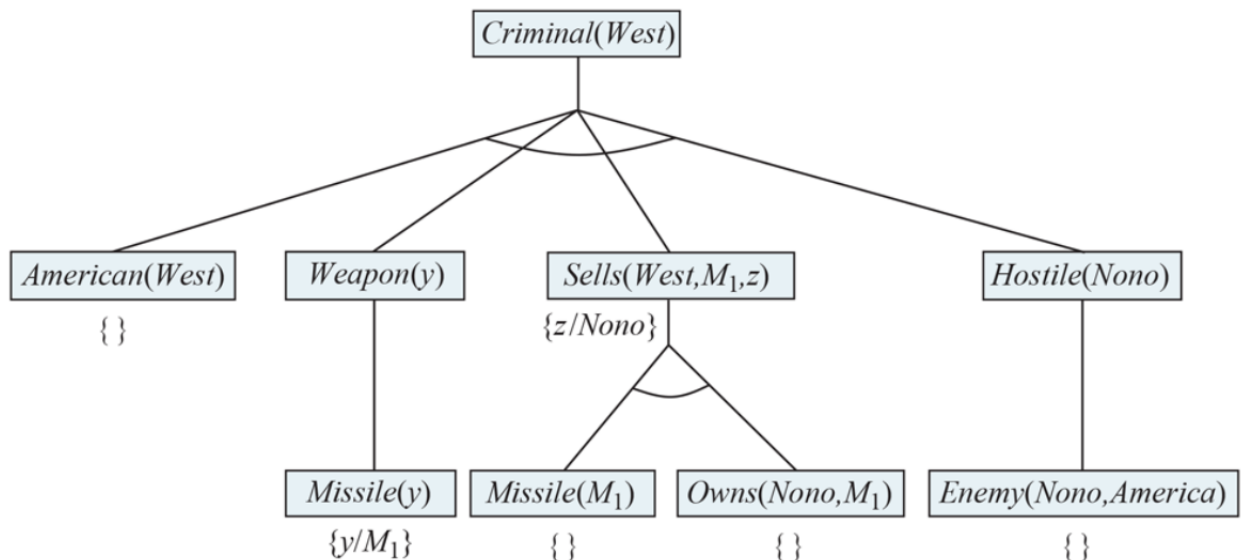


Figura 4.9: Esecuzione del Backward Chaining nel caso dell'esempio del crimine. L'ordine di visita è quello di un algoritmo DFS. Per provare che *Criminal(West)*, dobbiamo provare le quattro congiunzioni del livello sottostante. Alcune di queste sono nel KB, altre richiedono altre iterazioni dell'algoritmo.

Capitolo 5

Quantificare l'incertezza

5.1 Decision-Theoretic agent

Il nostro obiettivo è quello di progettare degli agenti intelligenti che prendano decisioni razionali basate sull'obiettivo che devono raggiungere. La **teoria della probabilità** è uno strumento ci permette di rappresentare l'incertezza del mondo in cui opera l'agente. Un agente razionale sceglie la prossima azione da eseguire basandosi sulla media di tutti i possibili risultati, pesati dalla probabilità.

Nei problemi del mondo reale l'incertezza può essere causata dal fatto che l'ambiente è parzialmente osservabile, non deterministico oppure sono presenti più agenti. Gli agenti che operano in questo tipo di ambienti, sfruttano la **teoria dell'utilità**: ogni stato viene associato ad un grado di utilità/inutilità; l'agente sceglie l'azioni basandosi sullo stato che ha maggiore utilità. La combinazione di teoria dell'utilità e teoria della probabilità forma la **teoria della decisione**. Un agente razionale sceglie l'azione che ha l'aspettazione dell'utilità massima, mediata su tutti i possibili risultati dell'azione.

```
1 def function DT-AGENT(percept) returns an action {  
2  
3     persistent: belief state, probabilistic beliefs about the current state of  
4         the world  
5         action, the agent's action  
6  
7     update belief state based on action and percept  
8     calculate outcome probabilities for actions,  
9         given action descriptions and current belief state  
10    select action with highest expected utility  
11        given probabilities of outcomes and utility information  
12    return action  
}
```

5.2 Inferenza e distribuzione congiunta di probabilità

Vediamo come alcuni metodi per applicare l'inferenza probabilistica.

Dalla tabella di Figura 5.1, possiamo sfruttare la distribuzione congiunta di probabilità per ricavare le **probabilità marginali**. Per esempio:

$$P(\text{cavity}) = \sum_{z \in \{\text{Catch}, \text{Toothache}\}} P(\text{Cavity}, z) = 0.108 + 0.012 + 0.072 + 0.008 = 0.2.$$

	<i>toothache</i>		$\neg$ <i>toothache</i>	
	<i>catch</i>	$\neg$ <i>catch</i>	<i>catch</i>	$\neg$ <i>catch</i>
<i>cavity</i>	0.108	0.012	0.072	0.008
$\neg$ <i>cavity</i>	0.016	0.064	0.144	0.576

Figura 5.1: Distribuzione congiunta di tre variabili aleatorie.

dove *Cavity* è una variabile aleatoria che può assumere i valori $\{cavity, \neg cavity\}$. Oppure possiamo calcolare la probabilità di una certa proposizione, come per esempio

$$P(cavity \vee toothache) = 0.108 + 0.012 + 0.072 + 0.008 + 0.016 + 0.064 = 0.28.$$

$$P(\neg cavity | toothache) \stackrel{\text{def}}{=} \frac{P(\neg cavity \wedge toothache)}{P(toothache)} = \frac{0.016 + 0.064}{0.108 + 0.012 + 0.016 + 0.064} = 0.4$$

$$P(cavity | toothache) \stackrel{\text{def}}{=} \frac{P(cavity \wedge toothache)}{P(toothache)} = \frac{0.108 + 0.012}{0.108 + 0.012 + 0.016 + 0.064} = 0.6$$

Notiamo che come ci si aspetta, la somma degli ultimi due risultati dà 1. Notiamo inoltre che in questi due calcoli, il termine $\alpha = 1/P(toothache)$ rimane costante indipendentemente dal valore che assume *Cavity*. Questo fattore può essere visto come una costante di **normalizzazione** α per la distribuzione $P(Cavity | toothache)$: assicura che la somma dia 1. Sfruttando questa notazione:

$$\begin{aligned} P(Cavity | toothache) &= \alpha P(Cavity | toothache) \\ &= \alpha [P(Cavity, toothache, catch) + P(Cavity, toothache, \neg catch)] \\ &= \alpha [\langle 0.108, 0.016 \rangle + \langle 0.012, 0.064 \rangle] \\ &= \alpha \langle 0.12, 0.08 \rangle = \langle 0.6, 0.4 \rangle \end{aligned}$$

In questo modo possiamo calcolare $P(Cavity | toothache)$ anche senza conoscere $P(toothache)$. Arrivati all'ultimo passaggio possiamo ricavare $\alpha = 1/(0.12 + 0.08)$ per far sì di ottenere $\langle 0.6, 0.4 \rangle$. Attraverso la normalizzazione abbiamo ricavato un'informazione che non era stata fornita ($P(toothache)$).

Da questo esempio possiamo trarre una tecnica di inferenza generale. Definiamo una query che riguarda una singola variabile X (*Cavity* nell'esempio precedente), una lista di variabili E note (dette anche **evidenze**, nell'esempio c'è solo *Toothache*) e infine una lista di variabili H non osservate (dette **nascoste**, nell'esempio c'è solo *Catch*). Allora, la query $P(X|e)$ può essere calcolata come segue

$$P(X|e) = \alpha P(X, e) = \alpha \sum_y P(X, e, y)$$

dove e è un valore di un'evidenza ($E \in \{e_1, \dots, e_n\}$) e la sommatoria va fatta su tutti i possibili valori y ($Y \in \{y_1, \dots, y_n\}$).

5.2.1 Il problema con la distribuzione di probabilità congiunta

Il problema principale è che la complessità temporale nel caso peggiore è $O(d^n)$, dove d indica la cardinalità dell'alfabeto più grande tra le variabili aleatorie presenti (2 nel caso di booleane) e n indica il numero di variabili. Inoltre memorizzare una tabella di d^n entrate può essere un problema.

Fortunatamente in molti casi è possibile **fattorizzare** tabelle di distribuzioni in tabelle più piccole sfruttando l'indipendenza tra le variabili.

5.3 La regola di Bayes e il suo utilizzo

Dalla definizione di probabilità condizionata possiamo scrivere la **regola del prodotto**:

$$P(a, b) = P(a|b)P(b).$$

Siccome $P(a, b) = P(b, a)$, possiamo scrivere la **regola di Bayes**

$$P(a|b)P(b) = P(b|a)P(a)$$

che può essere scritta in maniera equivalente come segue

$$P(a|b) = \frac{P(b|a)}{P(b)} P(a)$$

Spesso capita di voler conoscere la probabilità associata ad una causa sapendo un certo effetto. In questi casi la regola di Bayes ci viene in aiuto.

$$P(\text{cause}|\text{effect}) = \frac{P(\text{effect}|\text{cause})P(\text{cause})}{P(\text{effect})}$$

La probabilità condizionata $P(\text{effect}|\text{cause})$ quantifica la relazione nella *direzione causale* mentre la condizionata $P(\text{cause}|\text{effect})$ descrive la *direzione diagnostica*.

Esempio di applicazione della regola di Bayes Un dottore sa che se un paziente ha la meningite, allora ha il 70% di avere il torcicollo. Il dottore sa anche a priori (incondizionatamente) che la probabilità che un paziente abbia la meningite è di 1/50000 e sa anche che la probabilità di avere un torcicollo è dell'1%. Chiamiamo $s = \text{torcicollo}$ (effect) e $m = \text{meningite}$ (causa). Dalle informazioni del testo si ricava che:

$$P(s|m) = 0.7, P(m) = 1/50000, P(s) = 0.01.$$

Sfruttando la regola di Bayes possiamo calcolare $P(m|s)$:

$$P(m|s) = \frac{P(s|m)P(m)}{P(s)} = 0.0014.$$

Si ottiene un valore molto piccolo visto che $P(s) \gg P(m)$. Avendo le probabilità di un sintomo possiamo calcolare la probabilità che una causa si realizzi sapendo che si è verificato il sintomo. In questo modo si ottiene una nuova informazione che può essere aggiunta al Knowledge Base.

Capitolo 6

Probabilistic reasoning

6.1 Bayesian networks

Nella sezione precedente abbiamo visto che a livello teorico è possibile rispondere a qualsiasi query sapendo la distribuzione congiunta e applicando solamente le regole della probabilità. Per esempio, se conosciamo $P(\text{Malattia}, \text{Sintomo}_A, \text{Sintomo}_B) \equiv P(M, A, B)$, possiamo rispondere a delle query del tipo:

$$P(M|A, B) = \frac{P(M, A, B)}{P(A, B)} = \frac{P(M, A, B)}{\sum_D P(D, A, B)}$$
$$P(A|D) = \frac{P(D, A)}{P(D)} = \frac{\sum_B P(D, A, B)}{\sum_{A, B} P(D, A, B)}$$

Tuttavia abbiamo anche detto che la dimensione delle tabella della distribuzione congiunta cresce esponenzialmente con il numero di variabili (nell'esempio è 2^3). Come possiamo risolvere tale problema?

Possiamo iniziare decomponendo la distribuzione congiunta in un prodotto di probabilità condizionate, questo processo prende il nome di **chain rule**.

$$\begin{aligned} P(X_1, \dots, X_n) &= P(X_n|X_{n-1}, \dots, X_1)P(X_{n-1}, \dots, X_1) \\ &= P(X_n|X_{n-1}, \dots, X_1)P(X_{n-1}|X_{n-2}, \dots, X_1)P(X_{n-2}, \dots, X_1) \\ &\vdots \\ &= \prod_j P(X_j|X_1, \dots, X_{j-1}) \end{aligned}$$

Per esempio: $P(X_1, X_2, X_3) = P(X_3|X_1, X_2)P(X_2|X_1)P(X_1)$.

Se X_j è indipendente da tutte le variabili aleatorie, tranne che da un sotto insieme, allora possiamo sfruttare le proprietà della condizionata per eliminare tutte le variabili indipendenti da X_j . Si ottiene:

$$P(X_1, \dots, X_n) = \prod_j P(X_j|X_1, \dots, X_{j-1}) = \prod_j P(X_j|\text{Parents}(X_j))$$

dove $\text{Parents}(X_j)$ rappresenta il sottoinsieme dei predecessori di X_j (**parents**) cioè l'insieme delle variabili da cui X_j dipende.

$$\text{Parents}(X_j) \subseteq \{X_1, \dots, X_{j-1}\}.$$

In questo modo abbiamo applicato la **product decomposition**. Dall'esempio precedente, se abbiamo che X_3 è indipendente da X_1 sapendo X_2 , possiamo scrivere: $P(X_1, X_2, X_3) = P(X_3|\text{Parents}(X_3))P(X_2|\text{Parents}(X_2))P(X_1|\text{Parents}(X_1)) = P(X_3|X_2)P(X_2|X_1)P(X_1)$.

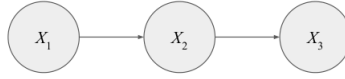


Figura 6.1: Rappresentazione grafica della dipendenza tra le variabili aleatorie X_1, X_2, X_3 . X_3 è indipendente da X_1 sapendo X_2 , quindi $P(X_1, X_2, X_3) = P(X_3|X_2)P(X_2|X_1)P(X_1)$. In questo caso X_2 è genitore di X_3 , X_1 è genitore di X_2 mentre X_1 non ha genitori.

6.1.1 Da distribuzione di probabilità congiunta a Bayesian network

Come mostrato in Figura 6.1, possiamo rappresentare la dipendenza tra le variabili aleatorie sfruttando le **Bayesian networks** che di fatto sono dei grafi: i nodi rappresentano le variabili aleatorie; gli archi rappresentano le dipendenze.

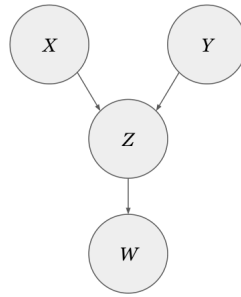


Figura 6.2: In questo caso abbiamo che $P(X, Y, Z, W) = P(W|Z)P(Z|X, Y)P(X)P(Y)$.

Le Bayesian networks possono essere scomposte in tre strutture principali; strutture più complesse vengono create componendo quelle di base. Analizziamo le strutture principali:

1. **Chain** o **head-to-tail** connection: abbia

$$P(X, Y, Z) = P(Z|Y)P(Y|X)P(X).$$

Conoscendo Y , le variabili X e Z diventano condizionatamente indipendenti. Infatti si ha che:

$$P(Z|X, Y) = \frac{P(Z|Y)P(Y|X)P(X)}{P(X, Y)} = \frac{P(Z|Y)P(X, Y)}{P(X, Y)} = P(Z|Y).$$

2. **Fork** o **tail-to-tail** connection:

$$P(X, Y, Z) = P(Y|X)P(Z|X)P(X).$$

Conoscendo X , le variabili Y e Z diventano condizionatamente indipendenti. Infatti si ha che:

$$P(Y, Z|X) = \frac{P(Y|X)P(Z|Y)P(X)}{P(X)} = P(Y|X)P(Z|X).$$

3. **Collider** o **head-to-head** connection:

$$P(X, Y, Z) = P(Z|X, Y)P(X)P(Y).$$

Se non si conosce Z , le variabili X e Y sono condizionatamente indipendenti. Infatti

$$P(X, Y) = \sum_Z P(Z)P(X)P(Y) = P(X)P(Y) \underbrace{\sum_Z P(Z)}_1 = P(X)P(Y).$$

Consideriamo un esempio in cui: X è il risultato di un lancio di un dado a 6 facce; Y è il risultato del secondo lancio; Z è la somma dei due risultati. Se sappiamo a priori che $Z = 7$, allora: se $X = 1$, $Y = 6$; se $X = 2$, $Y = 5$; etc. Notiamo quindi che la conoscenza di Z crea una dipendenza tra X e Y .

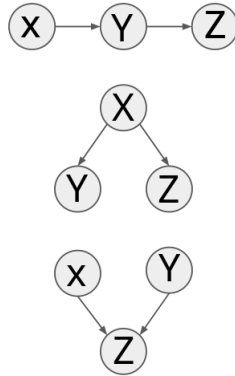


Figura 6.3: In ordine: *Chain*, *Fork* e *Collider*.

6.1.2 Conditional probability tables e i vantaggi della product decomposition

Nel caso di variabili discrete, nelle Bayesian networks, viene associata ad ogni nodo una tabella (**Conditional Probability Table (CPT)**) contenenti le probabilità condizionate.

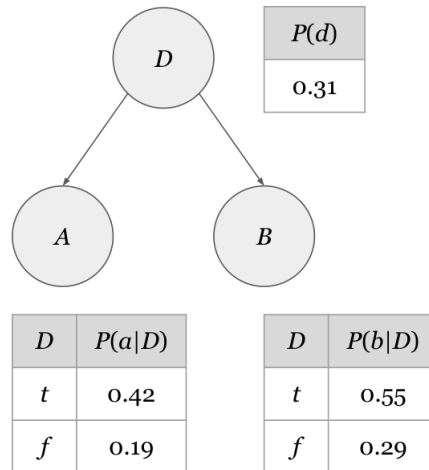


Figura 6.4: Bayesian network con le Conditional Probability tables associate ad ogni nodo. Le probabilità rimanenti si ottengono sfruttando la proprietà della probabilità: $P(\neg d) = 1 - P(d)$, $P(\neg a|D) = 1 - P(a|D)$, $P(\neg b|D) = 1 - P(b|D)$.

Abbiamo visto che per 3 variabili booleane, la tabella della distribuzione congiunta ha 2^3 ingressi. Sfruttando le Bayesian networks possiamo ridurre la dimensione della tabella: nell'esempio siamo passati da 8 a 5.

La product decomposition rappresentata da una Bayesian network, oltre a farci risparmiare memoria, permette di ottenere stime delle probabilità più accurate. Per esempio: poniamo di voler stimare la probabilità $P(A, B, C)$ (con tutte variabili aleatorie booleane) e poniamo di avere un dataset di 25 campioni. Siccome abbiamo 8 possibili tuple (A, B, C) , significa

che in media abbiamo 25/8 campioni per stimare la probabilità di ogni caso (anche se in realtà alcuni casi potrebbero avere più campioni, altri nessuno). Andando invece a stimare le probabilità $P(A|D)$, $P(B|D)$, $P(D)$, si hanno più possibilità di avere abbastanza campioni, infatti: 25/4 campioni in media per $P(A|D)$; 25/4 campioni in media per $P(B|D)$; 25/2 campioni in media per $P(D)$. Queste probabilità stimate sono più accurate in quanto si basano su più campioni.

6.1.3 Esempio: burglary alarm

L'allarme di una casa scatta solo se entra un ladro o se c'è un terremoto. I vicini *John* e *Mary* devono chiamare il proprietario della casa se sentono suonare l'allarme.

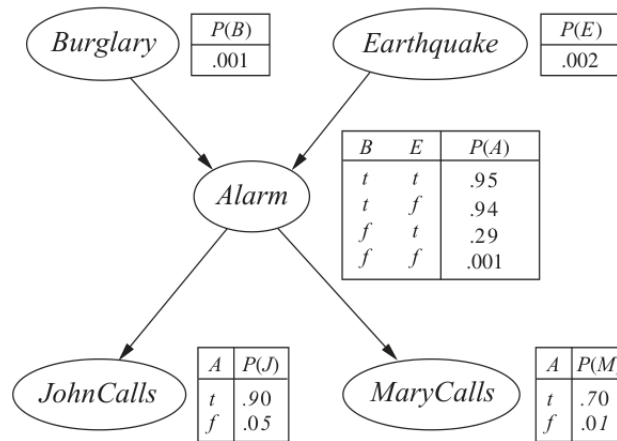


Figura 6.5: Bayesian network del burglary alarm problem. Le lettere minuscole indicano eventi veri, per esempio $j \equiv J = \text{true}$.

In questo caso (Figura 6.5) notiamo che il numero di entrate totali delle CPT è 10. La tabella della probabilità congiunta in questo caso avrebbe $2^5 = 32$ ingressi. Sfruttando la product decomposition possiamo ricavarci per esempio:

$$P(j, m, a, \neg b, \neg e) = P(j|a)P(m|a)P(a|\neg b, \neg e)P(\neg b)P(\neg e) = 0.00062.$$

Dalla Figura 6.6 notiamo che la conformazione delle reti Bayesiane dipende da modello utilizzato. In generale è vero che tutti i modelli seguiti producono una rete Bayesiana corretta, tuttavia il modello causale (quello in cui $Parents(X)$ sono le cause di X) è quello più efficiente. Infatti, usando la rete di Figura 6.6(a) si ha $P(E, B, A, J, M) = P(E|B, A)P(B|A)P(A|J, M)P(J|M)P(M)$ quindi in totale 13 ingressi delle CPT; mentre usando la rete di Figura 6.6(b) si ha $P(E, B, A, J, M) = P(A|B, E, J, M)P(B|E, J, M)P(E|J, M)P(J|M)P(M)$ e dunque in totale 31 ingressi delle CPT. Usando un modello causale (quello di Figura 6.5) si ottengono 10 ingressi totali delle CPT.

6.2 Inferenza esatta nelle Bayesian networks

6.2.1 Inference by enumeration

Consideriamo una generica query per calcolare la probabilità di X date delle evidenze $e = \{e_1, \dots, e_n\}$

$$P(X|e) = \frac{P(X, e)}{P(e)} = \frac{P(X, e)}{\sum_X P(X, e)} = \alpha P(X, e)$$

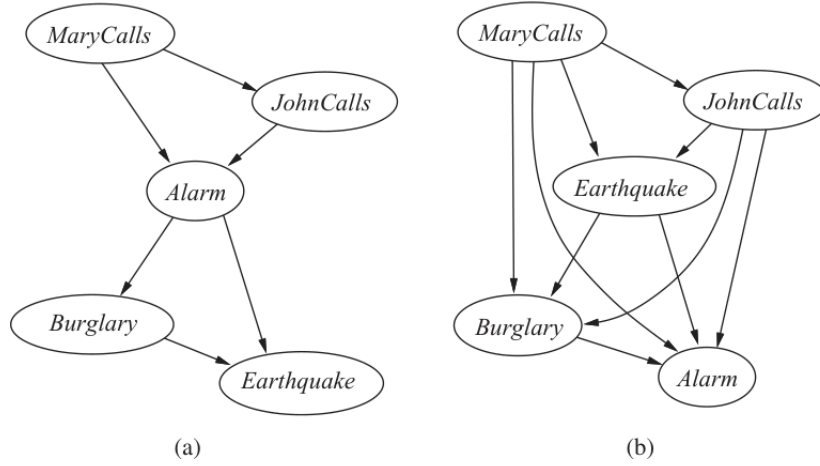


Figura 6.6: Bayesian network del burglary alarm problem: due Bayesian network corrette che non si basano sul modello causale e che di conseguenza sono meno efficienti.

dove α è la costante di normalizzazione. Sfruttando la distribuzione congiunta di una rete Bayesiana, possiamo rispondere ad qualsiasi query usando l'equazione

$$P(X|e) = \alpha P(X, e) = \alpha \sum_y P(X, e, y)$$

dove $y = \{y_1, \dots, y_k\}$ sono le variabili aleatorie nascoste (non osservabili) che non fanno parte della query. Per esempio, facendo riferimento all'esempio del burglary allarm problem (Figura 6.5), possiamo rispondere alla query

$$P(B|j, m) = \alpha P(B, j, m) = \alpha \sum_e \sum_a P(B, j, m, e, a)$$

dove E ed A sono le variabili nascoste. Poniamo $B = \text{true}$, otteniamo:

$$\begin{aligned} P(b|j, m) &= \alpha P(b, j, m) = \alpha \sum_e \sum_a P(b, j, m, e, a) \\ &= \alpha \sum_e \sum_a P(b)P(e)P(a|b, e)P(j|a)P(m|a) \\ &= \alpha \cdot P(b) \sum_e P(e) \sum_a P(a|b, e)P(j|a)P(m|a) \\ &= \alpha \cdot 0.0005922 \end{aligned}$$

Poniamo $B = \text{false}$, otteniamo analogamente:

$$\begin{aligned} P(\neg b|j, m) &= \alpha P(\neg b, j, m) = \alpha \cdot P(\neg b) \sum_e P(e) \sum_a P(a|\neg b, e)P(j|a)P(m|a) \\ &= \alpha \cdot 0.0014949 \end{aligned}$$

Infine possiamo ricavare la distribuzione della query:

$$P(B|j, m) = \alpha \langle 0.0005922, 0.0014949 \rangle = \frac{\langle 0.0005922, 0.0014949 \rangle}{0.0005922 + 0.0014949} = \langle 0.284, 0.716 \rangle.$$

Quindi la probabilità che ci sia un ladro sapendo che entrambi i vicini hanno chiamato è del 0.284, mentre la probabilità che non ci sia un ladro sapendo che hanno chiamato è del 0.716.

6.2.2 Variable elimination

Il problema dell'inferenza per enumerazione è la complessità dell'algoritmo che la realizza. Se abbiamo n variabili booleane la complessità al caso peggiore è $O(n \cdot 2^n)$. Tale algoritmo può essere migliorato andando a salvare i risultati di alcune operazioni intermedie, in questo modo si evita di ricalcolare conti già effettuati in precedenza. Esistono diversi algoritmi che si basano su questo approccio, noi consideriamo l'algoritmo **variable elimination**.

6.3 Inferenza approssimata nelle Bayesian networks

Nella precedente sotto-sezione abbiamo visto un metodo di inferenza esatta (inferenza per enumerazione). Il problema principale dell'inferenza esatta è la complessità computazionale. L'inferenza approssimata è una valida alternativa che permette di rispondere alle query di modelli probabilistici molto grandi in un tempo ragionevole.

L'idea che c'è alla base è quella di ricavare la distribuzione di probabilità (che non è fornita a priori) della rete Bayesiana basandosi su un certo numero di campioni. Ricavata un'approssimazione della distribuzione di probabilità, possiamo rispondere alle query $P(X| \text{evidence})$. Per esempio poniamo di avere una cesta di palline colorate di rosso, verde e blu: non avendo la distribuzione congiunta, possiamo pescare 20 palline in maniera casuale e ricavare una distribuzione approssimata. Più palline peschiamo e più accurata sarà l'approssimazione.

I metodi basati su campioni generati in maniera randomica vengono chiamati algoritmi di **Monte Carlo**. Questi algoritmi vengono divisi in due categorie principali: **direct sampling** e **Markov Chain Monte Carlo (MCM)**. Nei prossimi paragrafi andiamo ad analizzare alcuni algoritmi che appartengono a queste due categorie, considerando il classico esempio degli irrigatori.

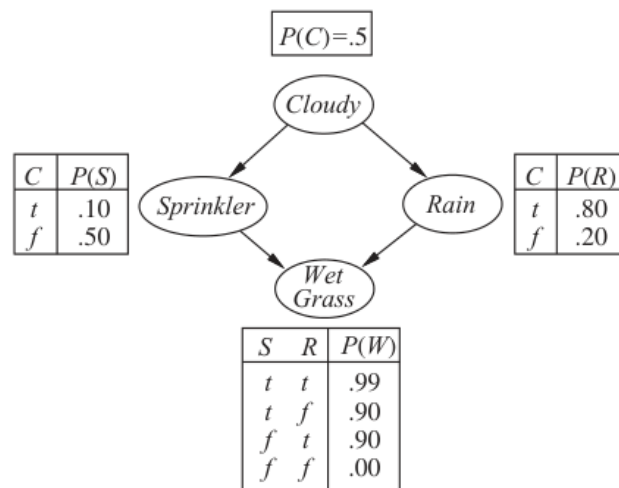


Figura 6.7: Il problema degli irrigatori, rete Bayesiana.

6.3.1 Direct sampling

Prior sampling L'idea alla base è quella di generare degli eventi senza aver a disposizione delle evidenze. Facendo riferimento alla Figura 6.7, consideriamo il seguente ordine topologico: $[Cloudy, Sprinkler, Rain, GrassWet]$. Procedendo per passi:

1. genero un campione (randomico) da $P(Cloudy)$, poniamo che esca *true*;

2. genero un campione da $P(\text{Sprinkler}|\text{Cloudy} = \text{true}) \rightarrow \text{false}$;
3. genero un campione da $P(\text{Rain}|\text{Cloudy} = \text{true}) \rightarrow \text{false}$;
4. genero un campione da $P(\text{GrassWet}|\text{Sprinkler} = \text{false}, \text{Rain} = \text{true}) \rightarrow \text{true}$.

L'output che abbiamo ottenuto in questa generazione dei campioni è $[\text{true}, \text{false}, \text{true}, \text{true}]$. Ripetendo il processo per un certo numero di volte N , si riesce ad approssimare la distribuzione congiunta $P(\text{Cloudy}, \text{Sprinkler}, \text{Rain}, \text{GrassWet})$ ad un valore molto vicino a quello reale. In generale quindi

$$P(x_1, \dots, x_n) \approx \frac{N_{PS}(x_1, \dots, x_n)}{N}$$

dove $[x_1, \dots, x_n]$ è l'evento generato e $N_{PS}(x_1, \dots, x_n)$ ritorna il numero di volte in cui compare l'evento nell'insieme degli eventi generati.

Cloudy	Sprinkler	Rain	GrassWet
true	false	true	true
true	false	false	false
false	true	true	false
true	false	true	true
false	false	false	false
true	true	true	true
true	true	false	false
true	true	true	false
false	false	false	true
false	true	false	true

Tabella 6.1: Campioni generate in $N = 10$ iterazioni dell'algoritmo prior sampling.

Dalla tabella possiamo ricavare per esempio:

$$P(\text{Cloudy} = \text{true}, \text{Rain} = \text{true}) = \frac{4}{10}.$$

Usando il metodo esatto (con i dati di Figura 6.7) troviamo $P(c, \neg s, r, g) = 0.324$; ci si avvicina allo stesso risultato sfruttando prior sampling e generando un buon numero di campioni.

```

1 def function Prior-Sample(bn) return an event sampled from the prior specified
   by bn
2
3 inputs: bn, a Bayesian network specifying joint distrubution  $P(X_1, \dots, X_n)$ 
4
5 x ← an event with n elements
6 for each variable  $X_i$  in  $X_1, \dots, X_n$  do
7     x[i] ← a random sample from  $P(X_i|\text{Parents}(X_i))$ 
8 return x
    
```

Rejection sampling Il funzionamento del rejection sampling è molto simile a quello del prior sampling, la differenza è che rejection sampling non considera tutti i campioni generati ma scarta quelli che non sono consistenti con le evidenze.

Poniamo di avere alcune evidenze e (ottenute mediante prior sampling) e di voler calcolare $P(X|e)$. Sfruttando rejection sampling otteniamo

$$P(X|e) \approx \hat{P}(X|e) = \frac{\hat{P}(X, e)}{\hat{P}(e)} = \alpha \frac{N_{PS}(X, e)}{N_{PS}(e)} = \beta N_{PS}(X, e)$$

dove $N_{PS}(X, e)$ indica il numero di volte in cui l'evento $[X, e]$ compare nel dataset generato con prior sampling, $N_{PS}(e)$ indica il numero volte in cui e compare nel dataset generato, mentre α, β sono delle costanti di normalizzazione.

Consideriamo l'esempio degli irrigatori di Figura 6.7. Poniamo di voler stimare $P(Rain|Sprinkler = true)$ basandoci su 100 campioni che abbiamo generato con prior sampling. Di questi 100 campioni: 73 hanno $Sprinkler = false$ e quindi vengono rifiutati in quanto non sono consistenti con la nostra evidenza ($Sprinkler = true$); dei 27 rimanenti con $Sprinkler = true$, 8 hanno $Rain = true$ e 19 hanno $Rain = false$. L'approssimazione della distribuzione condizionata fornita da rejection sampling è

$$\hat{P}(Rain|Sprinkler = true) = \beta \langle 8, 19 \rangle = \frac{\langle 8, 19 \rangle}{8 + 19} = \langle 0.296, 0.704 \rangle$$

Likelihood weighting Il problema principale di rejection sampling è lo spreco di campioni che vengono generati e non vengono utilizzati. Likelihood weighting previene questo spreco andando a generare campioni che sono consistenti con le evidenze e mantenendo fissi i valori e andando a campionare soltanto le variabili che non sono delle evidenze.

Per esempio, se la query è $P(Rain|c, g)$, allora vengono fissati i valori $Cloudy = true$ e $GrassWet = true$ e vengono campionati i valori di $Rain$ e $Sprinkler$. Assumiamo l'ordine $[Cloudy, Sprinkler, Rain, GrassWet]$, e procediamo per step:

1. il peso w viene inizializzato a 1;
2. $Cloudy$ è un'evidenza con valore $true$ quindi il peso viene settato a $w = w \cdot P(Cloudy = true) = 0.5$;
3. $Sprinkler$ non è un'evidenza e quindi si campiona da $P(Sprinkler|c)$, poniamo che ritorni $false$;
4. $Rain$ non è un'evidenza e quindi campioniamo da $P(Rain|c)$, poniamo che torni $true$;
5. $GrassWet$ è un'evidenza con valore $true$ dunque aggiorniamo il peso $w = w \cdot P(GrassWet = true|Sprinkler = false, Rain = true) = 0.45$;
6. ritorna l'evento $[true, false, true, true]$ con peso associato $w = 0.45$.

<i>Cloudy</i> (fixed)	<i>Sprinkler</i>	<i>Rain</i>	<i>GrassWet</i> (fixed)	<i>w</i>
true	false	true	true	0.45
true	true	true	true	0.475
true	false	false	true	0.05
true	false	true	true	0.45
true	false	true	true	0.45
true	false	true	true	0.45
true	true	true	true	0.475
true	false	true	true	0.45
true	false	false	true	0.05
true	false	true	true	0.45

Figura 6.8: Campioni generati in $N = 10$ iterazioni dell'algoritmo likelihood weighting.

Ora che abbiamo abbastanza campioni possiamo approssimare $P(Rain|c, g)$. Andiamo a sommare tutti i pesi in cui $Rain = true$ e normalizziamo per la somma di tutti i pesi:

$$P(r|c, g) = \frac{\sum_r w}{\sum w} = 0.9733$$

Sommiamo e normalizziamo tutti i pesi in cui $Rain = false$:

$$P(\neg r|c, g) = \frac{\sum_{\neg r} w}{\sum w} = 1 - P(r|c, g) = 0.0267$$

Quindi la risposta alla query è:

$$P(Rain|c, g) = \langle 0.9733, 0.0267 \rangle$$

6.3.2 Inference by Markov chain simulation

Il problema dell'algoritmo likelihood weighting è che le sue performance peggiorano molto all'aumentare del numero di variabili. L'idea del metodo Markov Chain Monte Carlo (metodo **MCMC**) è quella di generare ogni campione andando a modificare il campione precedente in maniera casuale. In questo modo evita di generare campioni partendo da zero.

Prima di vedere un algoritmo che sfrutta il metodo MCMC, definiamo la **Markov blanket** (proprietà d'indipendenza). La Markov blanket di un nodo X di una rete Bayesiana comprende tutti i genitori di X , tutti i figli di X e i genitori dei figli. Il nodo X è condizionatamente indipendente da tutti gli altri nodi della rete conoscendo il suo Markov blanket.

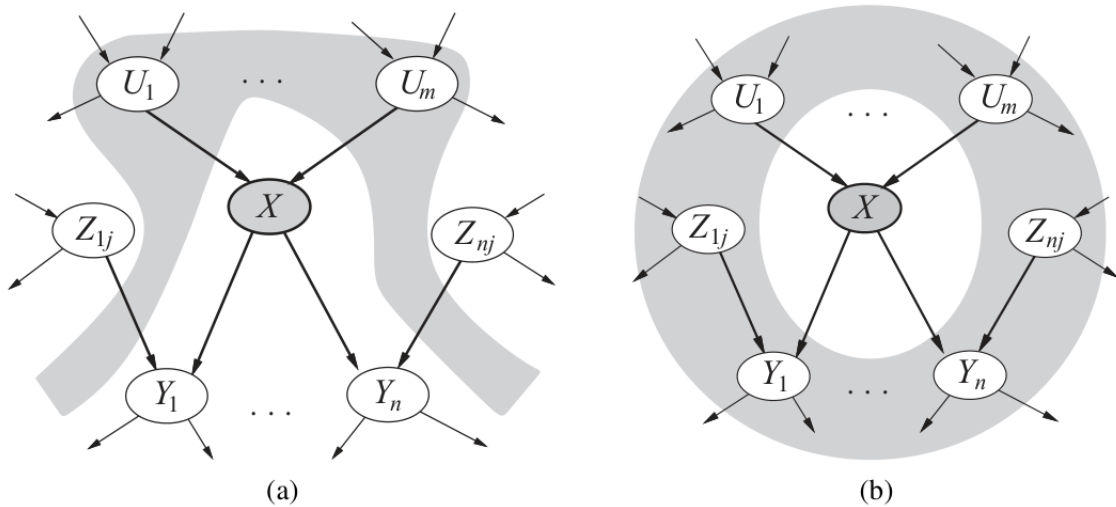


Figura 6.9: (a) Il nodo X è condizionatamente indipendente dai suoi non discententi (es: i nodi Z_{ij}), conoscendo i suoi genitori (i nodi U_i nell'area grigia). (b) Il nodo X è condizionatamente indipendente da tutti gli altri nodi della rete, conoscendo il suo Markov blanket (area grigia).

Gibbs sampling Gibbs sampling è un algoritmo che sfrutta il metodo MCMC. L'algoritmo inizia da uno stato arbitrario (in cui le evidenze vengono fissate ai loro valori osservati) e genera un nuovo stato andando a campionare un valore di una delle variabili Z_i che non sono delle evidenze. Il campionamento di Z_i viene calcolato condizionando Z_i rispetto ai valori assunti in quel momento dalle variabili appartenenti al suo Markov blanket $mb(Z_i)$ (ovvero si

calcola $P(Z_i|mb(Z_i))$. L'algoritmo quindi va a campionare una delle variabili non evidenze alla volta, mantenendo sempre fissi i valori delle evidenze.

Poniamo di voler rispondere a alla query $\mathbf{P}(Rain|s, g)$, dove s, g sono le evidenze (facendo riferimento alla Figura 6.7). Poniamo che lo stato iniziale sia $[c, s, \neg r, g]$. Procediamo per passi:

1. campiono *Cloudy* conoscendo il suo Markov blanket $\{Sprinkler, Rain\}$. Poniamo che $\mathbf{P}(Cloudy|s, \neg r)$ ritorni *false*: ottengo un nuovo campione $[\neg c, s, \neg r, g]$;
2. campiono *Rain* dato il suo Markov blanket $\{Cloudy, Sprinkler, Grass\}$. Poniamo che $\mathbf{P}(Rain|\neg c, s, g)$ ritorni *true*: ottengo un nuovo campione $[\neg c, s, r, g]$;
3. continua in maniera analoga scegliendo casualmente se campionare *Rain* o *Cloudy*.

Immaginiamo di aver fatto girare l'algoritmo 80 volte: 20 volte abbiamo trovato $Rain = true$; 60 $Rain = false$. Dunque possiamo rispondere alla query:

$$\mathbf{P}(R|s, g) = \alpha \langle 20, 60 \rangle = \frac{\langle 20, 60 \rangle}{20 + 60} = \langle 0.25, 0.75 \rangle$$

Capitolo 7

Probabilistic reasoning over time

7.1 Tempo ed incertezza

Nelle sezioni successive andremo a lavorare su modelli a tempo discreto in cui il mondo circostante può essere visto come una serie di snapshots. Indicheremo con Δ l'intervallo di tempo tra uno snapshot e l'altro; assumiamo che l'intervallo sia costante. Inoltre indicheremo con X_t l'insieme delle variabili di stato al tempo t , e assumeremo che queste siano non osservabili. Infine l'insieme di variabili osservabili (evidenze) verrà indicato con E_t .

7.1.1 Transition model e sensor model

Transition model In generale il modello transizionale permette di predire in che stato mi troverò, partendo dallo stato in cui mi trovo ora. Per esempio, il modello fisico è un modello transizionale. Più precisamente il modello transizionale specifica la distribuzione di probabilità delle variabili allo stato attuale, conoscendo i valori assunti negli stati precedenti:

$$P(X_t | X_0, X_1, \dots, X_{t-1}) \equiv P(X_t | X_{0:t-1}).$$

Dato che l'insieme delle variabili $X_{0:t-1}$ potrebbe essere potenzialmente infinito, si sfrutta la **Markov assumption**: uno stato corrente dipende solamente da un numero finito di stati precedenti. In particolare:

First-order Markov process: $P(X_t | X_{t-1})$

Second-order Markov process: $P(X_t | X_{t-1}, X_{t-2})$

$\vdots$

Aumentando di grado si migliora l'approssimazione. Comunemente si sfrutta la *first-order Markov assumption*.

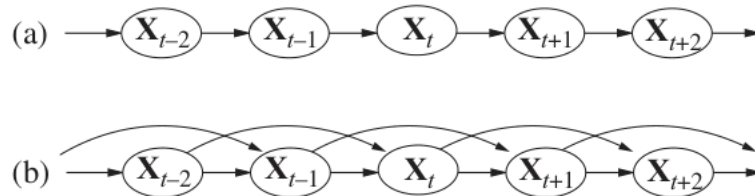


Figura 7.1: (a) Struttura della Bayesian network corrispondente al first order Markov process: lo stato X_t dipende solamente dallo stato precedente. (b) Struttura corrispondente al second-order Markov process: lo stato X_t dipende da due stati precedenti.

Sensor model Definiamo $P(E_t|X_t)$ il nostro sensor model. Il modello è caratterizzato dal **sensor Markov assumption**: conoscendo X_t , le evidenze osservate E_t sono indipendenti da tutto il resto. Quindi

$$P(E_t|X_{0:t}, E_{1:t-1}) = P(E_t|X_t)$$

in altre parole, l'evidenza all'istante t dipende solamente dallo stato allo stesso istante t .

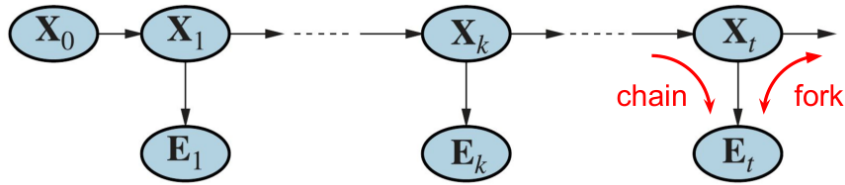


Figura 7.2: Struttura delle due assunzioni di Markov combinate.

Distribuzione congiunta con le assunzioni di primo ordine di Markov Conoscendo a priori la distribuzione probabilità $P(X_0)$ al tempo 0, la distribuzione di probabilità congiunta può essere ottenuta sfruttando le due assunzioni di Markov (e la product decomposition vista alla sezione 6.1):

$$P(X_{0:t}, E_{1:t}) = P(X_0) \prod_{i=1}^t \underbrace{P(X_i|X_{i-1})}_{1^\circ \text{ assunzione}} \underbrace{P(E_i|X_i)}_{2^\circ \text{ assunzione}} .$$

Per ottenere un'approssimazione più precisa bisogna aumentare il grado del Markov process.

La Figura 7.3 mostra un esempio in cui vengono rappresentate le strutture delle due assunzioni di Markov nel caso del umbrella word problem. **Descrizione del problema:** immaginiamo di essere rinchiusi in una cantina senza finestre e il nostro obiettivo è capire se fuori piove basandoci solamente sul fatto che la guardia che ci viene a far visita porta o meno con se un ombrello. Sfruttiamo la first-order Markov process e assumiamo quindi che la probabilità di pioggia dipenda solamente se ha piovuto o meno il giorno prima.

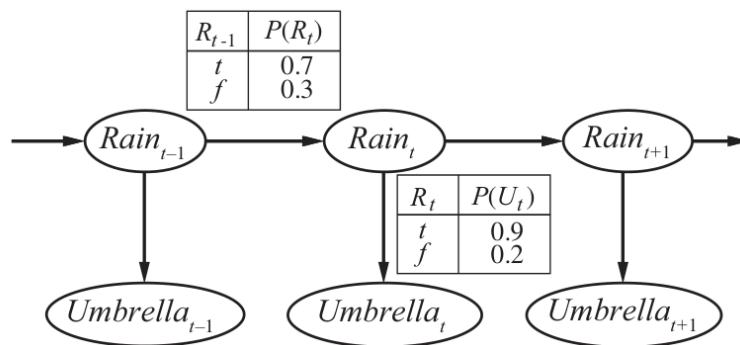


Figura 7.3: Bayesian network e distribuzioni condizionate dell'umbrella world problem. Il transition model è $P(Rain_t|Rain_{t-1})$; il sensor model è $P(Umbrella_t|Rain_t)$.

7.2 Inferenza nei temporal models

Ora che abbiamo definito la struttura di un temporal model generico, possiamo andare a definire i task basilari da risolvere mediante inferenza:

- **Filtering o state estimation:** calcolare il *belief state* $P(X_t|e_{1:t})$. Si tratta di ricavare la distribuzione dello stato al tempo t che non conosco, conoscendo tutte le evidenze disponibili. Nel caso dell'esempio del umbrella world problem di Figura 7.3, significherebbe calcolare la probabilità che oggi stia piovendo, conoscendo tutte le osservazioni fatte nei giorni precedenti sull'ombrello della guardia.
- **Prediction:** calcolare la distribuzione a posteriori dello stato futuro, date tutte le evidenze raccolte. Dunque si vuole ricavare $P(X_{t+k}|e_{1:t})$ per $k > 0$. Nell'esempio del umbrella world problem: calcolare la probabilità che tra k giorni piova, date tutte le osservazioni fatte sull'ombrello della guardia fino ad oggi.
- **Smoothing:** calcolare la distribuzione a posteriori di uno stato passato, date tutte le evidenze raccolte. Dunque si vuole ricavare $P(X_k|e_{1:t})$ con $0 \leq k < t$. Nell'esempio del umbrella world problem: calcolare la probabilità che k giorni fa abbia piovuto, conoscendo tutte le osservazioni fatte fino ad oggi sull'ombrello della guardia.
- **Most likely explanation:** data una sequenza di osservazioni, si vuole ricavare la sequenza di stati che più probabilmente ha generato tali osservazioni. In altre parole si vuole ricavare $\text{argmax}_{x_{1:t}} P(x_{1:t}|e_{1:t})$. Nell'esempio del umbrella world problem: se la guardia porta l'ombrello nei primi tre giorni e non nel quarto, allora è più probabile che abbia piovuto nei primi tre giorni e non nel quarto.

7.2.1 Filtraggio e predizione

Un algoritmo di filtraggio mantiene una stima dello stato corrente in memoria che aggiorna di volta in volta, ciò viene fatto per non dover risalire l'intera cronologia delle percezioni a ogni aggiornamento. In altre parole, dato il risultato del filtraggio fino al tempo t , l'agente deve calcolare il risultato per $t + 1$ a partire dalla nuova evidenza e_{t+1} . Dunque abbiamo:

$$P(X_{t+1}|e_{1:t+1}) = f(e_{t+1}, P(X_t|e_{1:t}))$$

per qualche funzione f . Questo processo viene chiamato **stima ricorsiva**. L'elaborazione viene divisa in due parti: nella prima si proietta in avanti la distribuzione dello stato corrente da t a $t + 1$; successivamente si aggiorna la distribuzione in base alla nuova evidenza e_{t+1} . Si ottiene:

$$\begin{aligned}
 P(X_{t+1}|e_{1:t+1}) &= P(X_{t+1}|e_{1:t}, e_{t+1}) \quad (\text{isolo l'evidenza a tempo } t + 1) \\
 &= \frac{P(X_{t+1}, e_{1:t}, e_{t+1})}{P(e_{1:t}, e_{t+1})} \\
 &= \frac{P(e_{t+1}|X_{t+1}, e_{1:t})P(X_{t+1}, e_{1:t})}{P(e_{1:t}, e_{t+1})} \\
 &= \frac{P(e_{t+1}|X_{t+1}, e_{1:t})P(X_{t+1}|e_{1:t})P(e_{1:t})}{P(e_{1:t}, e_{t+1})} \\
 &= \frac{P(e_{1:t})}{P(e_{1:t}, e_{t+1})} P(e_{t+1}|X_{t+1}, e_{1:t})P(X_{t+1}|e_{1:t}) \\
 &= \alpha P(e_{t+1}|X_{t+1}, e_{1:t})P(X_{t+1}|e_{1:t}) \\
 &= \underbrace{\alpha P(e_{t+1}|X_{t+1})}_{\text{aggiornamento}} \underbrace{P(X_{t+1}|e_{1:t})}_{\text{predizione}} \quad (\text{per la sensor Markov assumption}).
 \end{aligned}$$

Possiamo sviluppare ulteriormente il fattore della predizione. Si ottiene:

$$\begin{aligned}
 P(X_{t+1}|e_{1:t+1}) &= \alpha P(e_{t+1}|X_{t+1})P(X_{t+1}|e_{1:t}) \\
 &= \alpha P(e_{t+1}|X_{t+1}) \sum_{x_t} P(X_{t+1}|x_t, e_{1:t}) P(x_t|e_{1:t}) \\
 &= \alpha \underbrace{P(e_{t+1}|X_{t+1})}_{\text{sensor model}} \sum_{x_t} \underbrace{P(X_{t+1}|x_t)}_{\text{transition model}} \underbrace{P(x_t|e_{1:t})}_{\text{ricorsione}} \quad (\text{per la first-order Markov assumption}).
 \end{aligned}$$

Possiamo considerare la stima filtrata $P(X_t|e_{1:t})$ come un "messaggio" $f_{1:t}$ propagato in avanti lungo tutta la sequenza, modificato da ogni transizione e aggiornato da ogni nuova osservazione. Il processo è dato da:

$$f_{1:t+1} = \text{Forward}(f_{1:t}, e_{t+1})$$

dove *Forward* implementa l'aggiornamento descritto dall'equazione precedente e il processo inizia con $f_{1:0} = P(X_0)$.

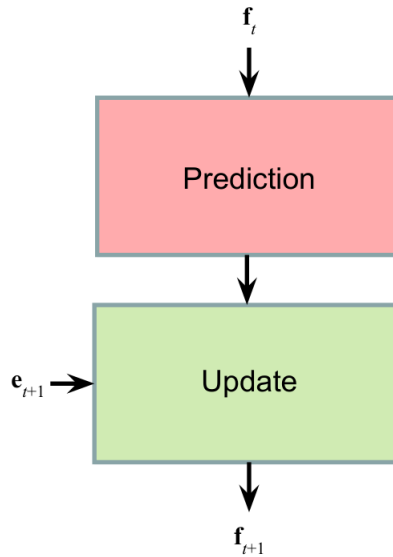


Figura 7.4: Schema operazione di filtraggio.

Vediamo alcuni passi del processo di filtraggio applicato all'esempio del umbrella world problem (Figura 7.3). Vogliamo calcolare la probabilità che oggi ($t = 2$) piova sapendo che ieri e oggi abbiamo osservato che la guardia ha portato l'ombrello. Ovvero vogliamo ricavare $P(r_2|u_1, u_2)$: lo facciamo calcolando $P(R_2|u_{1:2})$ tramite il filtering.

- Il giorno 0 (stato iniziale) non abbiamo osservazioni. Poniamo che $P(r_0) = P(\neg r_0) = 0.5$.
- Il giorno 1 sappiamo che $U_1 = \text{true}$. Calcoliamo $f_1 = \text{Forward}(P(R_0), u_1) \equiv P(R_1|u_1)$:

$$\begin{aligned}
 P(r_1|u_1) &= \alpha_1 P(u_1|r_1)[P(r_1|r_0)P(r_0) + P(r_1|\neg r_0)P(\neg r_0)] \\
 &= \alpha_1 \cdot 0.9 \cdot \underbrace{0.5}_{\text{prediction}} = \alpha_1 \cdot 0.45
 \end{aligned}$$

$$\begin{aligned}
 P(\neg r_1|u_1) &= \alpha_1 P(u_1|\neg r_1)[P(\neg r_1|r_0)P(r_0) + P(\neg r_1|\neg r_0)P(\neg r_0)] \\
 &= \alpha_1 \cdot 0.2 \cdot \underbrace{0.5}_{\text{prediction}} = \alpha_1 \cdot 0.1
 \end{aligned}$$

da cui infine:

$$f_1 = \alpha_1 \langle 0.45, 0.1 \rangle = \frac{\langle 0.45, 0.1 \rangle}{0.55} = \underbrace{\langle 0.818, 0.182 \rangle}_{\text{update}}$$

- Nel giorno 2 si ha che $U_2 = \text{true}$. Calcoliamo $f_2 = \text{Forward}(f_1, u_2) \equiv P(R_2|u_1, u_2)$:

$$\begin{aligned} P(r_2|u_1, u_2) &= \alpha_2 P(u_2|r_2) [P(r_2|r_1) \underbrace{P(r_1|u_1)}_{0.818} + P(r_2|\neg r_1) \underbrace{P(\neg r_1|u_1)}_{0.182}] \\ &= \alpha_2 \cdot 0.9 \cdot \underbrace{0.627}_{\text{prediction}} = \alpha_2 \cdot 0.564 \end{aligned}$$

$$\begin{aligned} P(\neg r_2|u_1, u_2) &= \alpha_2 P(u_2|\neg r_2) [P(\neg r_2|r_1) \underbrace{P(r_1|u_1)}_{0.818} + P(\neg r_2|\neg r_1) \underbrace{P(\neg r_1|u_1)}_{0.182}] \\ &= \alpha_2 \cdot 0.2 \cdot \underbrace{0.373}_{\text{prediction}} = \alpha_2 \cdot 0.075 \end{aligned}$$

da cui infine:

$$f_2 = \alpha_2 \langle 0.564, 0.075 \rangle = \frac{\langle 0.564, 0.075 \rangle}{0.639} = \underbrace{\langle 0.883, 0.117 \rangle}_{\text{update}}$$

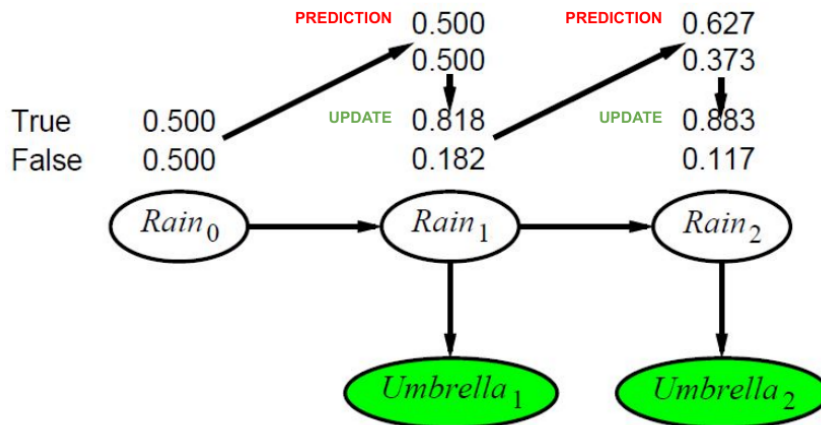


Figura 7.5: Rappresentazione grafica dell'esempio del filtraggio nel caso del problema umbrella world.

7.2.2 Smoothing

Lo smoothing permette di calcolare la distribuzione di uno stato passato X_k al tempo $0 \leq k < t$, conoscendo tutte evidenze $e_{1:t}$ fino al tempo t .

$$\begin{aligned}
 P(X_k | e_{1:t}) &= P(X_k | e_{1:k}, e_{k+1:t}) \quad (\text{split delle evidenze}) \\
 &= \frac{P(X_k, e_{1:k}, e_{k+1:t})}{P(e_{1:k}, e_{k+1:t})} \\
 &= \frac{P(e_{k+1:t} | X_k, e_{1:k}) P(X_k, e_{1:k})}{P(e_{1:k}, e_{k+1:t})} \\
 &= \frac{P(e_{k+1:t} | X_k, e_{1:k}) P(X_k | e_{1:k}) P(e_{1:k})}{P(e_{1:k}, e_{k+1:t})} \\
 &= \frac{P(e_{1:k})}{P(e_{1:k}, e_{k+1:t})} P(X_k | e_{1:k}) P(e_{k+1:t} | X_k, e_{1:k}) \\
 &= \alpha P(X_k | e_{1:k}) P(e_{k+1:t} | X_k, e_{1:k}) \\
 &= \alpha P(X_k | e_{1:k}) P(e_{k+1:t} | X_k) \quad (\text{per indipendenza condizionale}) \\
 &= \alpha f_{1:k} \times b_{k+1:t}
 \end{aligned}$$

dove: $\times$ rappresenta la moltiplicazione tra vettori; $f_{1:k}$ è un filtraggio calcolato in avanti da 1 a k ; $b_{k+1:t}$ è un messaggio all'indietro (al contrario di $f_{1:k}$ che invece è un messaggio in avanti). L'espressione di $b_{k+1:t}$ può essere sviluppata ulteriormente, si ottiene un processo ricorsivo che va all'indietro partendo dal tempo t :

$$\begin{aligned}
 P(e_{k+1:t} | X_k) &= \sum_{x_{k+1}} P(e_{k+1:t} | X_k, x_{k+1}) P(x_{k+1} | X_k) \quad (\text{condizionando a } X_{k+1}) \\
 &= \sum_{x_{k+1}} P(e_{k+1:t} | X_k, x_{k+1}) P(x_{k+1} | X_k) \quad (\text{per l'indipendenza condizionale}) \\
 &= \sum_{x_{k+1}} P(e_{k+1}, e_{k+2:t} | x_{k+1}) P(x_{k+1} | X_k) \\
 &= \sum_{x_{k+1}} \underbrace{P(e_{k+1} | x_{k+1})}_{\text{sensor model}} \underbrace{P(e_{k+2:t} | x_{k+1})}_{\text{ricorsione}} \underbrace{P(x_{k+1} | X_k)}_{\text{transition model}} \quad (\text{per l'indipendenza condizionale})
 \end{aligned}$$

Definiamo:

$$b_{k+1:t} = \text{Backword}(b_{k+2:t}, e_{k+1})$$

dove *Backword* implementa l'aggiornamento descritto dall'equazione precedente.

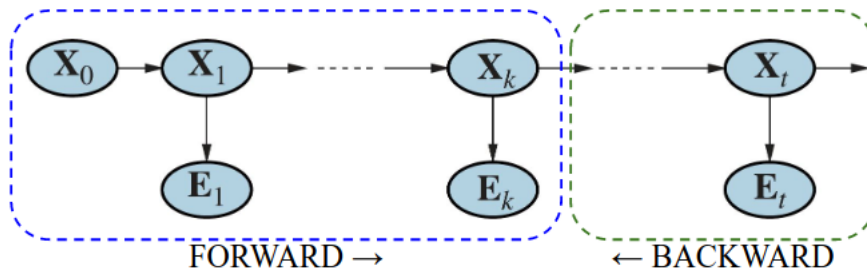


Figura 7.6: Smoothing

7.2.3 Trovare la sequenza più probabile

Data una sequenza di osservazioni, l'obiettivo è quello di trovare la sequenza di stati più probabile che l'ha generata. Si tratta di calcolare:

$$\operatorname{argmax}_{x_{1:t}} P(x_{1:t} | e_{1:t}).$$

Esiste un algoritmo che ha prestazioni lineari nel tempo che si basa sulle stesse proprietà di Markov sfruttate dagli algoritmi di filtraggio e smoothing. In pratica ogni sequenza viene vista come un percorso attraverso un grafo i cui nodi rappresentano tutti i possibili stati ad ogni istante temporale (esempio Figura 7.7(a)).

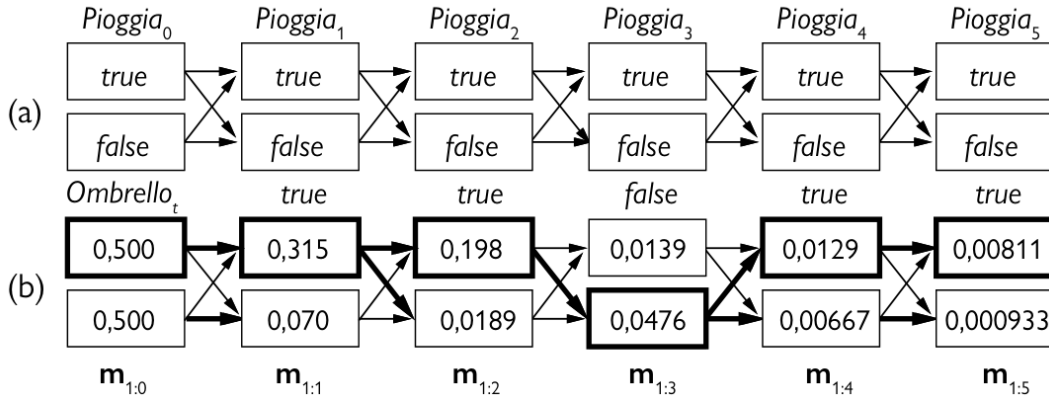


Figura 7.7: Trovare la sequenza di stati più probabile che ha generato la sequenza di osservazioni $[true, true, false, true, true]$ sull'ombrello della guardia nel umbrella world problem. (a) Tutte le possibili sequenze di stati per $Rain_t$ possono essere viste come dei percorsi in un grafo di stati possibili a ogni istante temporale. (b) Funzionamento dell'algoritmo Viterbi per la sequenza di osservazioni dell'ombrello $[true, true, false, true, true]$ (nota che le evidenze iniziano al tempo 1). La sequenza più probabile si ottiene seguendo all'indietro le frecce in grassetto partendo dallo stato più probabile in $m_{1:5}$.

Consideriamo l'esempio dell'umbrella world problem. Abbiamo osservato la seguente sequenza di osservazioni sull'ombrello della guardia: $[true, true, false, true, true]$. Il nostro obiettivo è quello di trovare la sequenza di stati più probabile che ha portato a quelle osservazioni. La Figura 7.7(a) mostra un grafo da cui possiamo ricavare tutte le possibili sequenze di stati di $Rain_t$. La probabilità di una particolare sequenza di stati è data dal prodotto delle probabilità di transizione del percorso per le probabilità delle osservazioni rilevate ad ogni stato.

Concentriamoci per un momento a tutte le sequenze che portano allo stato $Rain_5 = true$. Il percorso più probabile per lo stato $Rain_5 = true$ è il cammino più probabile verso un qualche stato all'istante 4 seguito da una transizione da lì a $Rain_5 = true$; e lo stato all'istante 4 che diventerà parte del cammino per $Rain_5 = true$ è quello che massimizza la probabilità di tale cammino. In altre parole, esiste una relazione ricorsiva tra la sequenza di stati più probabile che porta ad ogni stato x_{t+1} e la sequenza di stati più probabili che porta ad ogni stato x_t . Si sfrutta questa proprietà per costruire un algoritmo che ricavi la sequenza più probabile partendo dalle evidenze fornite.

Definiamo il messaggio¹

$$m_{1:t} = \max_{x_{1:t-1}} P(x_{1:t-1}, x_t, e_{1:t})$$

¹Non si considera $P(x_{1:t-1}, x_t | e_{1:t})$ perché i due vettori sono correlati per un fattore costante $P(e_{1:t})$.

il quale contiene le probabilità della sequenza più probabile che raggiunge ognuno degli stati finali. Sviluppiamo l'espressione di $m_{1:t+1}$ per ricavare la relazione ricorsiva:

$$\begin{aligned}
 m_{1:t+1} &= \max_{x_{1:t}} P(x_{1:t}, X_{t+1}, e_{1:t}) \\
 &= \max_{x_{1:t}} P(x_{1:t}, X_{t+1}, e_{1:t+1}) \\
 &= \max_{x_{1:t}} P(x_{1:t}, X_{t+1}, e_{1:t}, e_{t+1}) \\
 &= \max_{x_{1:t}} P(e_{t+1} | x_{1:t}, X_{t+1}, e_{1:t}) P(x_{1:t}, X_{t+1}, e_{1:t}) \\
 &= \max_{x_{1:t}} P(e_{t+1} | X_{t+1}) P(x_{1:t}, X_{t+1}, e_{1:t}) \quad (\text{Markov sensor assumption}) \\
 &= P(e_{t+1} | X_{t+1}) \max_{x_{1:t}} P(X_{t+1} | x_{1:t}, e_{1:t}) P(x_{1:t}, e_{1:t}) \\
 &= P(e_{t+1} | X_{t+1}) \max_{x_{1:t}} P(X_{t+1} | x_t) P(x_{1:t}, e_{1:t}) \quad (\text{Markov transition assumption}) \\
 &= P(e_{t+1} | X_{t+1}) \max_{x_t} P(X_{t+1} | x_t) \underbrace{\max_{x_{1:t-1}} P(x_{1:t-1}, x_t, e_{1:t})}_{\text{ricorsione}} \\
 &= P(e_{t+1} | X_{t+1}) \max_{x_t} P(X_{t+1} | x_t) m_{1:t}
 \end{aligned}$$

L'algoritmo per calcolare la sequenza più probabile è simile all'algoritmo di filtraggio: inizia all'istante di tempo 0 con $m_{1:0} = P(X_0)$ e poi procede in avanti lungo la sequenza, calcolando il messaggio m a ogni passo mediante l'equazione appena descritta. La Figura 7.7(b) mostra come avanza il calcolo. L'algoritmo che abbiamo appena descritto prende il nome di **algoritmo di Viterbi**.

Analizziamo le iterazioni dell'algoritmo di Viterbi applicato al caso dell'umbrella world problem e considerando la sequenza di osservazioni $[true, true, false, true, true]$ sull'ombrello della guardia. Calcoliamo i vari messaggi:

1. $m_{1:1}$:

$$\begin{aligned}
 m_{1:1} &= P(u_1 | R_1) \max_{r_0} P(R_1 | r_0) m_{1:0} \\
 &= \langle 0.9, 0.2 \rangle \max_{r_0} [\langle P(r_1 | r_0), P(\neg r_1 | r_0) \rangle \cdot P(r_0), \langle P(r_1 | \neg r_0), P(\neg r_1 | \neg r_0) \rangle \cdot P(\neg r_0)] \\
 &= \langle 0.9, 0.2 \rangle \max_{r_0} [\langle 0.7, 0.3 \rangle \cdot 0.5, \langle 0.3, 0.7 \rangle \cdot 0.5] \\
 &= \langle 0.9, 0.2 \rangle \max_{r_0} [\langle 0.35, 0.15 \rangle, \langle 0.15, 0.35 \rangle] \\
 &= \langle 0.9, 0.2 \rangle \langle 0.35, 0.35 \rangle \\
 &= \langle 0.315, 0.07 \rangle
 \end{aligned}$$

2. $m_{1:2}$:

$$\begin{aligned}
 m_{1:2} &= P(u_2 | R_2) \max_{r_1} P(R_2 | r_1) m_{1:1} \\
 &= \langle 0.9, 0.2 \rangle \max_{r_1} [\langle P(r_2 | r_1), P(\neg r_2 | r_1) \rangle \cdot 0.315, \langle P(r_2 | \neg r_1), P(\neg r_2 | \neg r_1) \rangle \cdot 0.07] \\
 &= \langle 0.9, 0.2 \rangle \max_{r_1} [\langle 0.7, 0.3 \rangle \cdot 0.315, \langle 0.3, 0.7 \rangle \cdot 0.07] \\
 &= \langle 0.9, 0.2 \rangle \max_{r_1} [\langle 0.221, 0.095 \rangle, \langle 0.021, 0.025 \rangle] \\
 &= \langle 0.9, 0.2 \rangle \langle 0.221, 0.095 \rangle \\
 &= \langle 0.198, 0.0189 \rangle
 \end{aligned}$$

3. $m_{1:3}$:

$$\begin{aligned}
 m_{1:3} &= P(\neg u_3 | R_3) \max_{r_2} P(R_3 | r_2) m_{1:2} \\
 &= \langle 0.1, 0.8 \rangle \max_{r_2} [\langle P(r_3 | r_2), P(\neg r_3 | r_2) \rangle \cdot 0.198, \langle P(r_3 | \neg r_2), P(\neg r_3 | \neg r_2) \rangle \cdot 0.0189] \\
 &= \langle 0.1, 0.8 \rangle \max_{r_2} [\langle 0.7, 0.3 \rangle \cdot 0.198, \langle 0.3, 0.7 \rangle \cdot 0.0189] \\
 &= \langle 0.1, 0.8 \rangle \max_{r_2} [\langle 0.139, 0.059 \rangle, \langle 0.006, 0.013 \rangle] \\
 &= \langle 0.1, 0.8 \rangle \langle 0.139, 0.059 \rangle \\
 &= \langle 0.0139, 0.0472 \rangle
 \end{aligned}$$

4. $m_{1:4}$:

$$\begin{aligned}
 m_{1:4} &= P(u_4 | R_4) \max_{r_3} P(R_4 | r_3) m_{1:3} \\
 &= \langle 0.9, 0.2 \rangle \max_{r_3} [\langle P(r_4 | r_3), P(\neg r_4 | r_3) \rangle \cdot 0.0139, \langle P(r_4 | \neg r_3), P(\neg r_4 | \neg r_3) \rangle \cdot 0.0472] \\
 &= \langle 0.9, 0.2 \rangle \max_{r_3} [\langle 0.7, 0.3 \rangle \cdot 0.0139, \langle 0.3, 0.7 \rangle \cdot 0.0472] \\
 &= \langle 0.9, 0.2 \rangle \max_{r_3} [\langle 0.0097, 0.0041 \rangle, \langle 0.014, 0.033 \rangle] \\
 &= \langle 0.9, 0.2 \rangle \langle 0.014, 0.033 \rangle \\
 &= \langle 0.0126, 0.0066 \rangle
 \end{aligned}$$

5. $m_{1:5}$:

$$\begin{aligned}
 m_{1:5} &= P(u_5 | R_5) \max_{r_4} P(R_5 | r_4) m_{1:4} \\
 &= \langle 0.9, 0.2 \rangle \max_{r_4} [\langle P(r_5 | r_4), P(\neg r_5 | r_4) \rangle \cdot 0.0126, \langle P(r_5 | \neg r_4), P(\neg r_5 | \neg r_4) \rangle \cdot 0.0066] \\
 &= \langle 0.9, 0.2 \rangle \max_{r_4} [\langle 0.7, 0.3 \rangle \cdot 0.0126, \langle 0.3, 0.7 \rangle \cdot 0.0066] \\
 &= \langle 0.9, 0.2 \rangle \max_{r_4} [\langle 0.0088, 0.0038 \rangle, \langle 0.00198, 0.0046 \rangle] \\
 &= \langle 0.9, 0.2 \rangle \langle 0.0088, 0.0046 \rangle \\
 &= \langle 0.00792, 0.00092 \rangle
 \end{aligned}$$

7.3 Hidden Markov models

Il **modello di Markov nascosto** o **HMM** è un modello temporale probabilistico in cui lo stato (nascosto) del processo è descritto da una singola variabile aleatoria discreta. In questo modello non vengono imposti vincoli sulle evidenze: possono essere quante si vogliono e di qualsiasi tipo (continue, discrete).

Il problema umbrella world che abbiamo considerato nei paragrafi precedenti è un HMM in quanto ha una sola variabile di stato $Rain_t$. Se il problema considerato presenta più di una variabile di stato, allora per farlo rientrare negli HMM basta considerare una singola variabile di stato composta da una coppia di variabili aleatorie.

7.3.1 Algoritmi semplificati basati su matrici

Sfruttando la notazione matriciale possiamo riscrivere gli algoritmi di filtering e smoothing in maniera più compatta. Avendo una singola variabile X_t di dimensione S , il modello di transizione $P(X_t | X_{t-1})$ diventa una matrice $S \times S$ chiamata T , dove

$$T_{ij} = P(X_t = j | X_{t-1} = i)$$

l'elemento T_{ij} indica la probabilità di transizione dallo stato i allo stato j .

Anche il sensor model diventa una matrice (diagonale) $S \times S$ chiamata O_t (una matrice per ogni istante temporale t). Gli elementi sulla diagonale indicano le probabilità dell'evidenza e_t dato ogni possibile stato X_t

$$P(e_t | X_t = i).$$

Definiamo le matrici nel caso del problema umbrella world (Figura 7.3):

$$T = P(X_t | X_{t-1}) = \begin{bmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{bmatrix}$$

$$O_t = P(e_t = \text{true} | X_t) = \begin{bmatrix} 0.9 & 0 \\ 0 & 0.2 \end{bmatrix} \quad \text{e} \quad O_t = P(e_t = \text{false} | X_t) = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.8 \end{bmatrix}$$

Possiamo rappresentare i messaggi in avanti e all'indietro (che compaiono negli algoritmi di filtering e smoothing) sfruttando la notazione matriciale. Per **filtering** otteniamo

$$f_{1:t+1} = \alpha_t O_{t+1} T^t f_{t-1}.$$

Applichiamo la formula al problema umbrella world. Ponendo $e_1 = \text{True}$:

$$f_1 = \alpha_1 O_1 T^0 f_0 = \alpha_1 \begin{bmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{bmatrix} \begin{bmatrix} 0.9 & 0 \\ 0 & 0.2 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \alpha_1 \begin{bmatrix} 0.45 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.818 \\ 0.182 \end{bmatrix}$$

Ponendo $e_2 = \text{True}$:

$$f_1 = \alpha_2 O_2 T^1 f_1 = \alpha_2 \begin{bmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{bmatrix} \begin{bmatrix} 0.9 & 0 \\ 0 & 0.2 \end{bmatrix} \begin{bmatrix} 0.818 \\ 0.182 \end{bmatrix} = \alpha_2 \begin{bmatrix} 0.565 \\ 0.075 \end{bmatrix} = \begin{bmatrix} 0.883 \\ 0.117 \end{bmatrix}$$

7.3.2 Esempio di HMM: robot localization

Consideriamo l'esempio di un robot avente 4 sensori di posizione (affetti da rumore) che si muove in maniera casuale con ugual probabilità in una cella vuota adiacente alla posizione in cui si trova. La variabile di stato X_t indica la posizione del robot sulla griglia discreta; il suo dominio è l'insieme dei riquadri vuoti che indichiamo con i numeri interi $\{1, \dots, S = 42\}$.

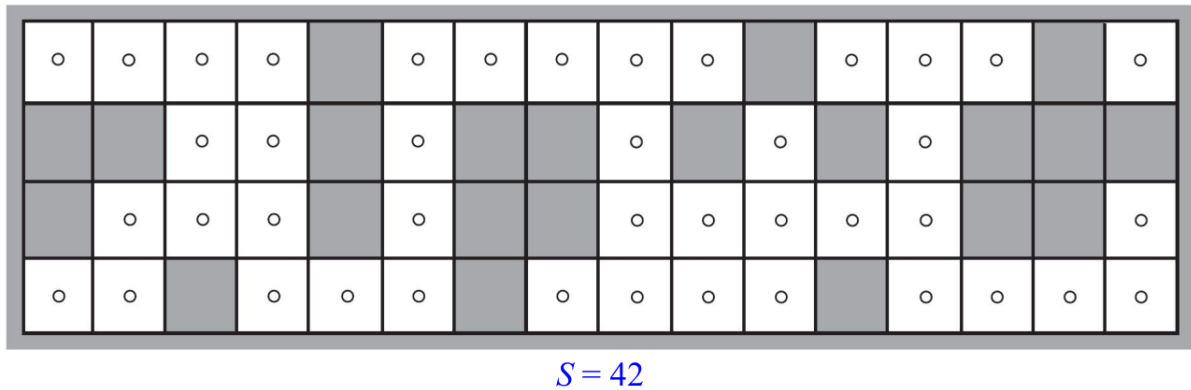


Figura 7.8: Griglia all'interno del quale il robot si può muovere. Le caselle più scure indicano la presenza di un muro.

Indichiamo con $N(i)$ il numero di celle libere adiacenti alla cella i , allora la probabilità di transizione può essere definita come segue:

$$P(X_{t+1} = j | X_t = i) = T_{ij} = \begin{cases} 1/N(i) & \text{se } j \in \text{CellaAdiacenteDi}(i) \\ 0 & \text{altrimenti} \end{cases}$$

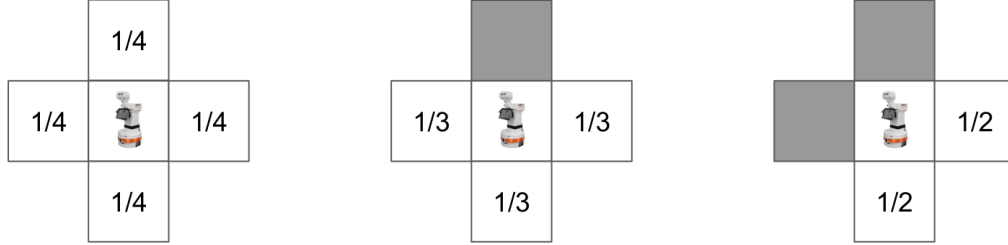
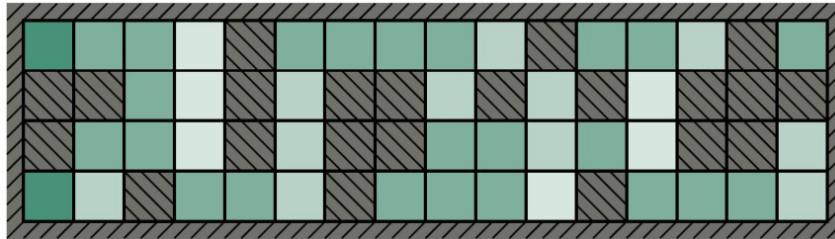


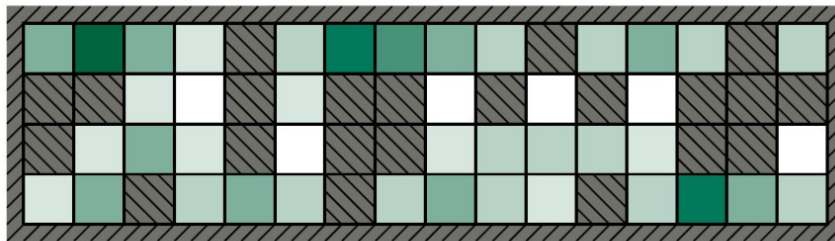
Figura 7.9: Esempi di probabilità di transizione nei diversi stati.

La posizione iniziale del robot è sconosciuta, quindi $P(X_0) = 1/S = 1/42$. La matrice di transizione T ha $42 \times 42 = 1764$ elementi. La variabile sensore E_t viene rappresentata su 4 bit, i quali indicano la presenza o l'assenza di un ostacolo nelle direzioni *N-E-S-W*, per esempio: 1010 indica che a *North* e a *Sud* è presente un ostacolo, mentre a *East* e a *West* la via è libera. Consideriamo che il tasso di errore di ogni sensore sia ε e che ognuno sia indipendente dagli altri. Ciò significa che la probabilità che tutti e 4 i bit siano sbagliati è ε^4 mentre la probabilità che siano tutti corretti è $(1 - \varepsilon)^4$. Chiamiamo d_{it} il numero di bit errati nella lettura di e_t in un certo istante t , allora la probabilità che un robot nella cella i riceva una lettura del sensore e_t è:

$$P(E_t = e_t | X_t = i) = (O_t)_{ii} = (1 - \varepsilon)^{4-d_{it}} \varepsilon^{d_{it}}$$



(a) Posterior distribution over robot location after $E_1 = 1011$



(b) Posterior distribution over robot location after $E_1 = 1011, E_2 = 1010$

Figura 7.10: Distribuzione a posteriori sulla posizione del robot. Più la cella è scura e più è alta la probabilità che il robot si trovi in quella.

Conoscendo T e O_t possiamo sfruttare il **filtering** per ricavare la distribuzione a posteriori dopo ogni nuova osservazione del robot. La Figura 7.10 mostra le distribuzioni $P(X_1|E_1 = 1011)$ e $P(X_2|E_1 = 1011, E_2 = 1010)$. Sfruttando il filtering, dopo aver ricavato un buon numero di osservazioni, si può individuare la posizione del robot con quasi assoluta certezza.

Sfruttando la tecnica dello **smoothing** possiamo risalire alla posizione del robot in un certo istante di tempo passato, per esempio possiamo ricavare la posizione all'istante di partenza X_0 . Infine tramite l'**algoritmo di Viterbi** possiamo calcolare il percorso che con più probabilità ha seguito il robot per raggiungere la posizione corrente.

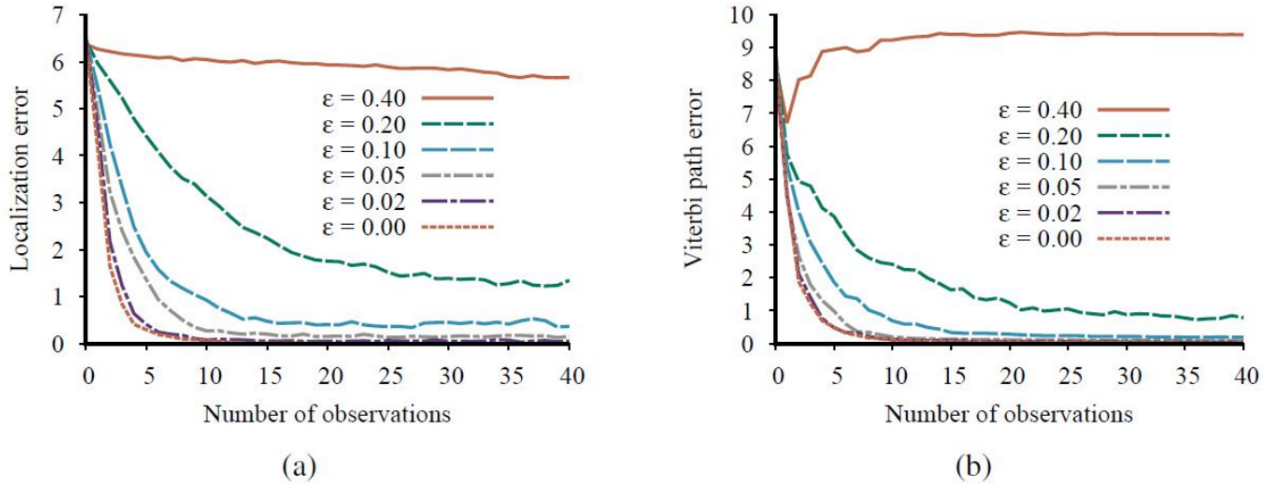


Figura 7.11: Errore di localizzazione ed errore dell'algoritmo di Viterbi nel trovare il percorso seguito, per diversi valori del tasso di errore sul bit ϵ . Si nota che per $\epsilon \geq 0.4$ non è possibile determinare la posizione corrente e neanche il percorso seguito nemmeno avendo a disposizione molte osservazioni.

7.4 Esempi

7.4.1 Markov model: Grades example

Consideriamo un corso di laurea triennale in cui alla fine di ogni anno ogni studente riceve un voto finale che può essere A , B o C . La probabilità che uno studente riceva lo stesso voto per due anni consecutivi è del 60% mentre la probabilità che ne riceva due diversi è del 20%. Ogni voto dipende solamente da quello ricevuto l'anno precedente. Vogliamo trovare la probabilità che uno studente prenda A il terzo anno sapendo che aveva B al primo anno.

Chiamiamo G_1, G_2, G_3 le variabili che indicano rispettivamente il voto preso nel primo, secondo e terzo anno. Vogliamo calcolare $P(G_3 = A | G_1 = B)$.



Figura 7.12: Dipendenza tra le variabili G_1, G_2, G_3 .

Calcoliamo più in generale $P(G_3|G_1 = B)$:

$$\begin{aligned}
 P(G_3|G_1 = B) &= \frac{P(G_3, G_1 = B)}{P(G_1 = B)} \\
 &= \frac{1}{P(G_1 = B)} \sum_{G_2} P(G_3, G_2, G_1 = B) \\
 &= \frac{1}{P(G_1 = B)} \sum_{G_2} P(G_3|G_2)P(G_2|G_1 = B)P(G_1 = B) \\
 &= \sum_{G_2} P(G_3|G_2)P(G_2|G_1 = B)
 \end{aligned}$$

da cui ricaviamo la risposta alla domanda che cerchiamo:

$$\begin{aligned}
 P(G_3 = A|G_1 = B) &= P(G_3 = A|G_2 = A)P(G_2 = A|G_1 = B) + \\
 &\quad P(G_3 = A|G_2 = B)P(G_2 = B|G_1 = B) + \\
 &\quad P(G_3 = A|G_2 = C)P(G_2 = C|G_1 = B) + \\
 &= 0.6 \cdot 0.2 + 0.2 \cdot 0.6 + 0.2 \cdot 0.2 = 0.28
 \end{aligned}$$

7.4.2 HMM: baby example (sequence of observations)

Descrizione del problema Un neonato indossa un pannolino che con la stessa probabilità può essere *clean* o *dirty* (stato nascosto). Il neonato utilizza il seguente vocabolario per comunicare $\{Mama, Papa, poo-poo, pee-pee\}$. Nel momento in cui il pannolino deve essere cambiato, se era pulito, allora la probabilità che la prossima volta sia pulito o sporco è la stessa; se il pannolino era sporco, allora la probabilità che sia pulito la prossima volta è 80% e 20% che sia sporco di nuovo. Inoltre, se il pannolino è pulito allora la probabilità che dica "Mama" o "Papa" è del 40% in entrambi i casi, mentre la probabilità che dica "poo-poo" o "pee-pee" è del 10% per entrambi i casi. Viceversa se il pannolino è sporco allora le probabilità che il neonato dica "Mama" o "Papa" è del 10% in entrambi i casi, mentre la probabilità che dica "poo-poo" o "pee-pee" è del 40% per entrambi i casi.

Vogliamo calcolare la probabilità che il neonato dica la seguente sequenza di parole: $[Mama, Mama, pee-pee, pee-pee]$.

Formalizzare il problema Poniamo:

- $X_t \in \{clean, dirty\}$ variabile di stato nascosta;
- e_t parole dette dal neonato.

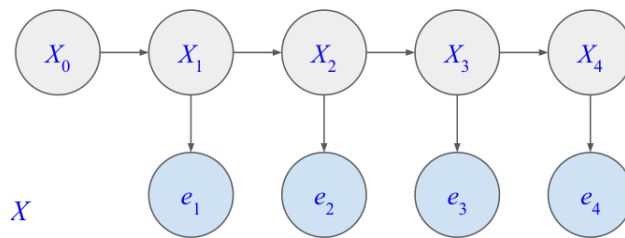


Figura 7.13: Dipendenze tra stati ed evidenze (transition model e sensor model)

Sfruttando i dati del problema possiamo definire il transition model:

$X_{t-1} \backslash X_t$	clean	dirty
clean	0.5	0.5
dirty	0.8	0.2

$$T = P(X_t|X_{t-1}) = \begin{bmatrix} 0.5 & 0.5 \\ 0.8 & 0.2 \end{bmatrix}$$

Definiamo il sensor model ($P(e_t|X_t)$):

$X_t \backslash e_t$	Mama	Papa	poo-poo	pee-pee
clean	0.4	0.4	0.1	0.1
dirty	0.1	0.1	0.4	0.4

Si vuole ricavare $P(e_1 = \text{Mama}, e_2 = \text{Mama}, e_3 = \text{pee-pee}, e_4 = \text{pee-pee})$. Nota che non vogliamo ricavare la sequenza più probabile di stati, quindi non si sfrutta l'algoritmo di Viterbi.

Risposta alla query La probabilità che si verifichi la sequenza di parole $\{e_1, e_2, e_3, e_4\}$ è uguale alla somma delle probabilità della sequenza per ogni possibile valore dello stato X_4 . Quindi:

$$\begin{aligned} P(e_1, e_2, e_3, e_4) &= \sum_{x_4} P(e_1, e_2, e_3, e_4, x_4) \\ &= P(e_1, e_2, e_3, e_4, x_4 = \text{clean}) + P(e_1, e_2, e_3, e_4, x_4 = \text{dirty}). \end{aligned}$$

In generale vale che:

$$P(e_{1:t}) = \sum_{x_t} P(e_{1:t}, x_t).$$

Chiamiamo $s_t = P(e_{1:t}, X_t)$, si ha che

$$\begin{aligned} s_t &= \sum_{x_{t-1}} P(e_{1:t}, x_{t-1}, X_t) \\ &= \sum_{x_{t-1}} P(e_{1:t-1}, e_t, x_{t-1}, X_t) \\ &= \sum_{x_{t-1}} P(e_t|e_{1:t-1}, x_{t-1}, X_t) P(e_{1:t-1}, x_{t-1}, X_t) \\ &= \sum_{x_{t-1}} P(e_t|X_t) P(e_{1:t-1}, x_{t-1}, X_t) \quad (\text{Markov sensor assumption}) \\ &= P(e_t|X_t) \sum_{x_{t-1}} P(X_t|e_{1:t-1}, x_{t-1}) P(e_{1:t-1}, x_{t-1}) \\ &= P(e_t|X_t) \sum_{x_{t-1}} P(X_t|x_{t-1}) P(e_{1:t-1}, x_{t-1}) \quad (\text{Markov transition assumption}) \\ &= P(e_t|X_t) \sum_{x_{t-1}} P(X_t|x_{t-1}) s_{t-1} \\ &= O_t T^t s_{t-1}. \end{aligned}$$

Abbiamo ottenuto un algoritmo ricorsivo (come il filtering ma in questo caso senza il fattore di normalizzazione α). Applichiamo l'algoritmo, otteniamo:

- caso $e_1 = \text{Mama}$:

$$s_1 = O_1 T^t s_0 = \begin{bmatrix} 0.4 & 0 \\ 0 & 0.1 \end{bmatrix} \begin{bmatrix} 0.5 & 0.8 \\ 0.5 & 0.2 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} 0.26 \\ 0.035 \end{bmatrix}$$

- caso $e_2 = \text{Mama}$:

$$s_2 = O_2 T^t s_1 = \begin{bmatrix} 0.4 & 0 \\ 0 & 0.1 \end{bmatrix} \begin{bmatrix} 0.5 & 0.8 \\ 0.5 & 0.2 \end{bmatrix} \begin{bmatrix} 0.26 \\ 0.035 \end{bmatrix} = \begin{bmatrix} 0.063 \\ 0.014 \end{bmatrix}$$

- caso $e_3 = \text{pee-pee}$:

$$s_3 = O_3 T^t s_2 = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.4 \end{bmatrix} \begin{bmatrix} 0.5 & 0.8 \\ 0.5 & 0.2 \end{bmatrix} \begin{bmatrix} 0.063 \\ 0.014 \end{bmatrix} = \begin{bmatrix} 0.004 \\ 0.014 \end{bmatrix}$$

- caso $e_4 = \text{pee-pee}$:

$$s_4 = O_4 T^t s_3 = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.4 \end{bmatrix} \begin{bmatrix} 0.5 & 0.8 \\ 0.5 & 0.2 \end{bmatrix} \begin{bmatrix} 0.004 \\ 0.014 \end{bmatrix} = \begin{bmatrix} 0.001 \\ 0.002 \end{bmatrix}$$

Infine, ponendo $[e_1, e_2, e_3, e_4] = [\text{Mama}, \text{Mama}, \text{pee-pee}, \text{pee-pee}]$, ricaviamo la risposta cercata

$$\begin{aligned} P(e_1, e_2, e_3, e_4) &= \sum_{x_4} P(e_1, e_2, e_3, e_4, x_4) \\ &= P(e_1, e_2, e_3, e_4, x_4 = \text{clean}) + P(e_1, e_2, e_3, e_4, x_4 = \text{dirty}) \\ &= 0.001 + 0.002 = 0.003 \end{aligned}$$

Applicare l'algoritmo di Viterbi Poniamo ora che la query sia quella di trovare la sequenza di stati più probabile che ha generato le osservazioni $[\text{Mama}, \text{Mama}, \text{pee-pee}, \text{pee-pee}]$. Per rispondere utilizziamo l'algoritmo di Viterbi. Procediamo per passi:

- caso $e_1 = \text{Mama}$:

$$\begin{aligned} m_{1:1} &= P(e_1|X_1) \max_{x_0} P(X_1|x_0) m_{1:0} \\ &= \langle 0.4, 0.1 \rangle \max_{x_0} [\langle 0.5, 0.5 \rangle \cdot 0.5, \langle 0.8, 0.2 \rangle \cdot 0.5] \\ &= \langle 0.4, 0.1 \rangle \max_{x_0} [\langle 0.25, 0.25 \rangle, \langle 0.4, 0.1 \rangle] \\ &= \langle 0.4, 0.1 \rangle \langle 0.4, 0.25 \rangle \\ &= \langle 0.16, 0.025 \rangle \end{aligned}$$

- caso $e_2 = \text{Mama}$:

$$\begin{aligned} m_{1:2} &= P(e_2|X_2) \max_{x_1} P(X_2|x_1) m_{1:1} \\ &= \langle 0.4, 0.1 \rangle \max_{x_1} [\langle 0.5, 0.5 \rangle \cdot 0.16, \langle 0.8, 0.2 \rangle \cdot 0.025] \\ &= \langle 0.4, 0.1 \rangle \max_{x_1} [\langle 0.08, 0.08 \rangle, \langle 0.02, 0.005 \rangle] \\ &= \langle 0.4, 0.1 \rangle \langle 0.08, 0.08 \rangle \\ &= \langle 0.32, 0.008 \rangle \end{aligned}$$

- caso $e_3 = \text{pee-pee}$:

$$\begin{aligned}
 m_{1:3} &= P(e_3|X_3) \max_{x_2} P(X_3|x_2) m_{1:2} \\
 &= \langle 0.1, 0.4 \rangle \max_{x_2} [\langle 0.5, 0.5 \rangle \cdot 0.32, \langle 0.8, 0.2 \rangle \cdot 0.008] \\
 &= \langle 0.1, 0.4 \rangle \max_{x_2} [\langle 0.16, 0.16 \rangle, \langle 0.0064, 0.0016 \rangle] \\
 &= \langle 0.1, 0.4 \rangle \langle 0.16, 0.16 \rangle \\
 &= \langle 0.016, 0.064 \rangle
 \end{aligned}$$

- caso $e_4 = \text{pee-pee}$:

$$\begin{aligned}
 m_{1:4} &= P(e_4|X_4) \max_{x_3} P(X_4|x_3) m_{1:3} \\
 &= \langle 0.1, 0.4 \rangle \max_{x_3} [\langle 0.5, 0.5 \rangle \cdot 0.016, \langle 0.8, 0.2 \rangle \cdot 0.064] \\
 &= \langle 0.1, 0.4 \rangle \max_{x_3} [\langle 0.008, 0.008 \rangle, \langle 0.0512, 0.0128 \rangle] \\
 &= \langle 0.1, 0.4 \rangle \langle 0.0512, 0.0128 \rangle \\
 &= \langle 0.00512, 0.00512 \rangle
 \end{aligned}$$

Quindi la sequenza di stati più probabile che ha generato la sequenza di osservazioni $[Mama, Mama, pee-pee, pee-pee]$ è $[clean, clean, dirty, clean]$ oppure $[clean, clean, dirty, dirty]$

Capitolo 8

Inferenza causale

8.1 Introduzione

L'inferenza causale applicata all'intelligenza artificiale permette di rispondere alla domanda "perché ciò è accaduto?". I modelli probabilistici che abbiamo studiato nei precedenti capitoli non riuscivano a rispondere a tale domanda: le loro risposte si basano solo su dati i quali non possono spiegare relazioni di tipo causa-effetto. L'inferenza causale può essere utilizzata per esempio per verificare l'efficacia di una nuova medicina, per predire l'effetto di una nuova politica economica, per capire il comportamento delle persone, e molto altro ancora.

8.1.1 Correlazione e causalità

Per farla breve: "Correlazione non implica causalità". Se correlazione implicasse causalità, allora dalla frase "quando il gallo canta, il sole sta sorgendo" si potrebbe dedurre che "se il sole non sorge è perché il gallo non ha cantato". C'è sicuramente una forte correlazione tra i due eventi ma non possiamo dire che uno causa l'altro basandoci solo su questi dati. Un altro esempio: "gli annegamenti incrementano con l'aumento di consumo di gelati". Ci potrebbe essere una causa comune (es: la stagione) tra i due eventi, non possiamo concludere che il gelato è la causa degli annegamenti.

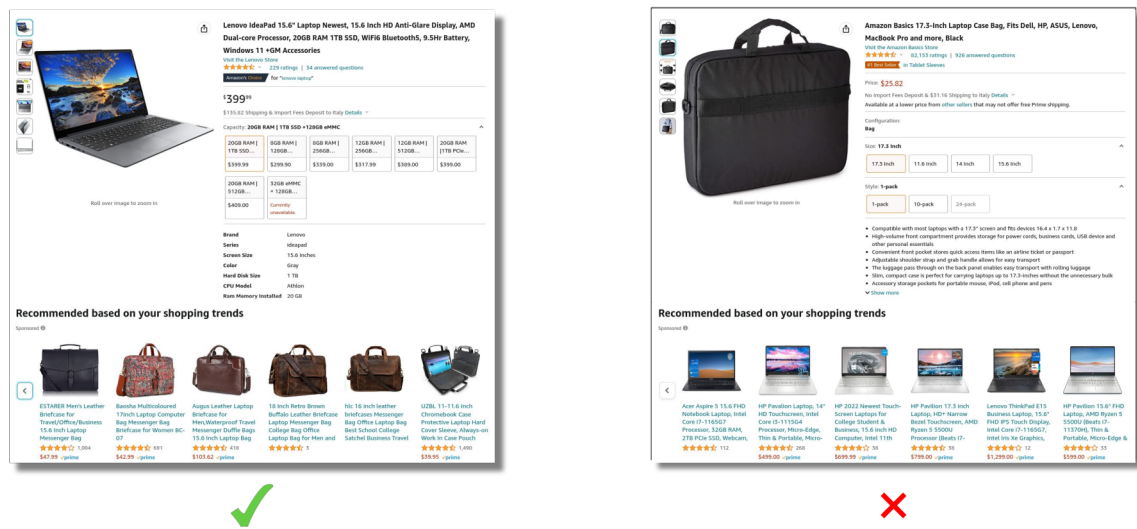


Figura 8.1: Sistemi di raccomandazione. Nella prima immagine: se compro un computer potrei essere interessato ad una borsa. Nella seconda immagine: computer e borsa sono correlati ma se sto cercando una borsa è probabile che il computer ce l'abbia già.

8.1.2 Il paradosso di Simpson

Una nuova medicina è stata offerta a 700 pazienti: 350 di questi hanno deciso di prenderla; 350 no.

	Medicine	No medicine
Men	81 out of 87 recovered (93%)	234 out of 270 recovered (87%)
Women	192 out of 263 recovered (73%)	55 out of 80 recovered (69%)
Combined data	273 out of 350 recovered (78%)	289 out of 350 recovered (83%)

Figura 8.2: Paradosso di Simpson: dati relativi alle guarigioni dalla malattia.

Dalla Figura 8.2 notiamo che la medicina ha dato degli esiti positivi se si analizzano solamente i sottogruppi di uomini e donne che l'hanno assunta. Tuttavia, se si mette a confronto l'insieme dei pazienti che ha assunto la medicina con quelli che non l'hanno assunta, non sembra che la medicina abbia avuto un esito positivo visto che ci sono più persone che sono guarite senza averla presa. Come è possibile che si verifichi questa situazione? La medicina effettivamente favorisce la guarigione o no?

Il paradosso di Simpson mostra che non è possibile rispondere a tutte le domande basandosi solamente sulle osservazioni raccolte: i dati non riescono a rispondere ai perché.

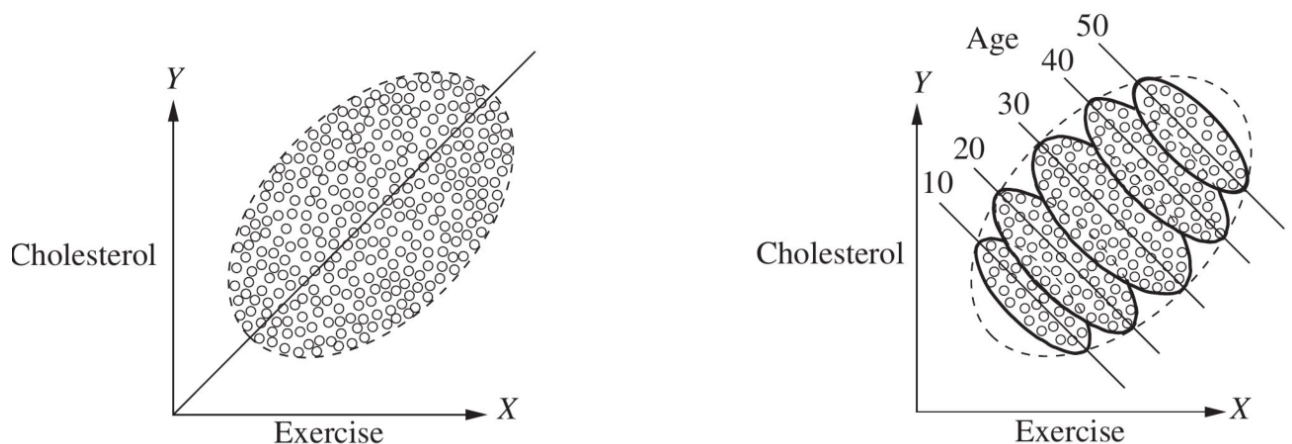


Figura 8.3: Il problema con i dati: il primo grafico mostra che l'esercizio fisico fa aumentare il colesterolo: si nota una tendenza lineare. Il secondo grafico mostra invece che se si considerano i dati divisi per fasce d'età, allora la tendenza è totalmente diversa.

8.1.3 Il problema delle probabilità congiunte

Nel precedente capitolo abbiamo visto che la distribuzione di probabilità congiunta $P(X_1, \dots, X_n)$ permette di descrivere le relazioni tra un insieme di variabili aleatorie $X_1, \dots, X_n$. Abbiamo visto che è possibile rappresentare la dipendenza tra le variabili sfruttando le Bayesian networks. Tuttavia non esiste una rappresentazione unica: dalla Figura 8.4

$$P(A, B) = P(A|B)P(B) = P(B|A)P(A)$$

$$\begin{aligned}
 P(A, B) &= P(A|B, C)P(B|C)P(C) \\
 &= P(C|B, A)P(B|A)P(A) \\
 &= \dots
 \end{aligned}$$

sono tutte relazioni matematicamente corrette ma non è detto che esprimano tutte un rapporto causa-effetto. La probabilità congiunta da sola non riesce a rispondere a tutte le domande.

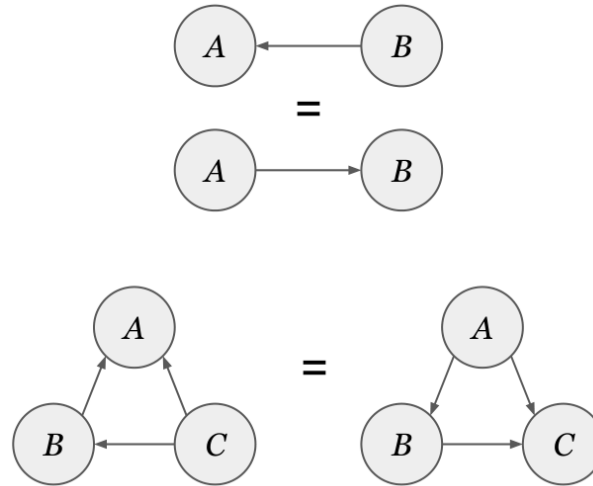


Figura 8.4: Il problema delle probabilità congiunte: diverse rappresentazioni delle dipendenze, tutte matematicamente equivalenti.

8.2 Il modello causale

Dati e probabilità congiunte non sono sufficienti a rispondere a tutte le domande: ci serve un modello causale che spieghi come i dati sono stati generati. Da questo modello (e con l'aiuto dei dati se disponibili) possiamo rispondere a domande del tipo "perché il gallo canta?", "per quale ragione aumenta il numero degli annegamenti?" o ancora "la medicina aiuta alla guarigione?" (in riferimento alle sezioni precedenti). Una definizione intuitiva che possiamo dare di **causalità** è: X è una causa di Y se i valori di Y "ascoltano" X .

8.2.1 Modello causale strutturale (SCM)

Uno **Structural Causal Model (SCM)** consiste in un insieme di **variabili esogene** U e un insieme di **variabili endogene** V , più un insieme di funzioni F (e un grafo) che descrive le relazioni causali tra di queste. Le variabili esogene, a differenza di quelle endogene, non dipendono da nessun'altra variabile. Per esempio: SCM del salario C basato sull'educazione A e sull'esperienza B

$$\begin{aligned}
 U &= \{A, B\}, \quad V = \{C\}, \quad F = \{f_C\} \\
 f_C : \underbrace{C}_{\text{effetto}} &= \underbrace{2A + 3B}_{\text{cause dirette}}
 \end{aligned}$$

Una variabile X è una **causa diretta** di Y se appare nella funzione f_Y ; una variabile X è una **causa** se è una causa diretta di Y , o di una qualsiasi causa di Y (definizione ricorsiva). Nell'esempio sopra: A, B sono entrambe cause dirette di C .

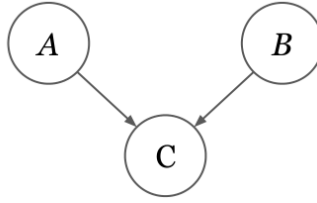


Figura 8.5: Grafo abbinato al modello SCM del salario.

8.2.2 Causal networks

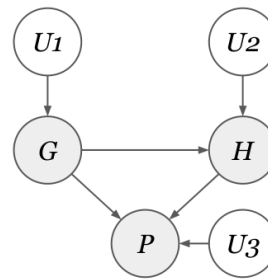
In molti casi, le relazioni causali e/o le variabili non sono spesso accessibili. Per esempio: definiamo P le abilità nel gioco del basket, H l'altezza e G il gender

$$V = \{H, G, P\}, \quad U = \{U_1, U_2, U_3\}, \quad F = \{f_1, f_2\}$$

$$G = U_1$$

$$H = f_1(G, U_2)$$

$$P = f_2(G, H, U_3)$$



Le variabili U rappresentano dei termini correttivi (come delle costanti nelle equazioni) che sono indipendenti da tutto il resto e che generalmente (se sono mutuamente indipendenti) vengono omessi. Spesso anche senza le equazioni del SCM, possiamo sfruttare

le informazioni qualitative del modello grafico associato (la rete causale) per inferire delle probabilità causa-effetto.

8.2.3 Esempio: sprinkler problem

Facciamo riferimento all'esempio del problema degli irrigatori presentato nella sezione 6.3. Scriviamo il modello causale strutturale:

$$V = \{C, R, S, W\}, \quad U = \{U_1, U_2, U_3, U_4\}$$

$$F = \{f_C, f_R, f_S, f_W\}$$

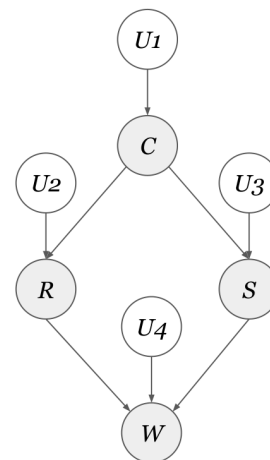
$$f_C : C = U_1$$

$$f_R : R = C \wedge U_2$$

$$f_S : S = \neg C \vee U_3$$

$$f_W : W = R \vee S \vee U_4$$

le variabili esogene che appartengono a U rappresentano delle relazioni causali con le variabili endogene appartenenti a V . Per esempio: gli irrigatori possono essersi accesi non solo perché oggi non piove ma perché magari sono difettosi: la variabile U_3 permette di modellare questa relazione causale tra il difetto e l'attivazione non voluta dell'irrigatore. Tuttavia



dato che $U_1, \dots, U_4$ sono mutuamente indipendenti, possiamo rimuoverle dal grafico.

Consideriamo una versione semplificata dell'esempio dell'irrigatore. Se noi osserviamo dei casi specifici della variabile R (es: $R = \text{true}$) allora W è condizionatamente indipendente da C dato R . Ciò deriva dal fatto che se si considerano solamente i nodi C, R, W della rete causale, si ottiene una *chain* (vedi sezione 6.1.1).

$$\begin{aligned} f_C : C &= U_1 \\ f_R : R &= C \wedge U_2 \\ f_W : W &= R \vee S \vee U_4 \end{aligned}$$

possiamo anche scrivere

$$W \perp\!\!\!\perp C \mid R$$

(ovvero W è condizionatamente indipendente da C dato R). Se osserviamo solamente dei casi specifici di C (es: $C = \text{true}$) allora R è condizionatamente indipendente da S dato C (*fork* composto dai nodi C, R, S).

$$\begin{aligned} f_C : C &= U_1 \\ f_R : R &= C \wedge U_2 \\ f_S : S &= \neg C \vee U_3 \end{aligned}$$

possiamo anche scrivere

$$R \perp\!\!\!\perp S \mid C.$$

Se osserviamo solamente dei casi specifici di W (es: $W = \text{true}$) allora R è condizionatamente dipendente da S dato W (*collider* composto dai nodi W, R, S).

$$\begin{aligned} f_R : R &= C \wedge U_2 \\ f_S : S &= \neg C \vee U_3 \\ f_W : W &= R \vee S \vee U_4 \end{aligned}$$

possiamo anche scrivere

$$R \not\perp\!\!\!\perp S \mid W$$

(ovvero R e S sono condizionatamente dipendenti dato W).

8.2.4 D-separation

Basandoci su tutte i concetti che abbiamo ricavato nella sottosezione precedente, possiamo definire un criterio generale per l'indipendenza di due variabili in una rete, chiamato **d-separation**.

Definizione 8.1 (D-separation). Chiamiamo Z l'insieme delle variabili condizionanti. Due variabili X e Y sono **d-separated** da Z se tutti i percorsi p che le collegano soddisfano almeno una delle seguenti condizioni:

- p contiene una catena $A \rightarrow B \rightarrow C$ o un fork $A \leftarrow B \rightarrow C$ tale che B è nell'insieme Z ;
- p contiene un collider $A \rightarrow B \leftarrow C$, e B e i suoi discendenti non sono in Z .

In tal caso si dice che X e Y sono condizionatamente indipendenti:

$$X \perp\!\!\!\perp Y \mid Z.$$

Facendo riferimento ai capitoli precedenti, possiamo dire che d-separation è un concetto che permette di determinare un'indipendenza tra le variabili simile al concetto del Markov blanket (sezione 6.3.2) ma di applicazione più generale.

8.3 Interventi e osservazioni

Per poter studiare la relazione causa-effetto, dobbiamo considerare le differenze che ci sono tra osservazioni e interventi. Utilizziamo l'operatore $do(X = x)$ per indicare che forziamo il valore $X = x$, il che è diverso da dire che osserviamo il valore di X . Vale che

$$\underbrace{P(Y = y|X = x)}_{\text{probabilità di } Y = y \text{ osservando che } X = x} \neq \underbrace{P(Y = y|do(X = x))}_{\text{probabilità che } Y = y \text{ forzando } X = x}$$

Si tratta di due concetti completamente diversi: da una parte ho un'osservazione passiva (osservazione); dall'altra un intervento attivo (la forzatura). Facendo riferimento all'esempio dell'irrigatore: la probabilità che l'erba sia bagnata dato che ho osservato che l'irrigatore è acceso, è diversa dalla probabilità che l'erba sia bagnata dato che io volontariamente accendo l'irrigatore.

In un dataset, quando consideriamo il risultato $Y = y$ data l'osservazione $X = x$, andiamo a considerare il sottoinsieme di Y in cui osserviamo che X vale x . Quando andiamo a condizionare $Y = y$ sull'intervento $do(X = x)$, forziamo il valore di X sull'intero insieme di Y .

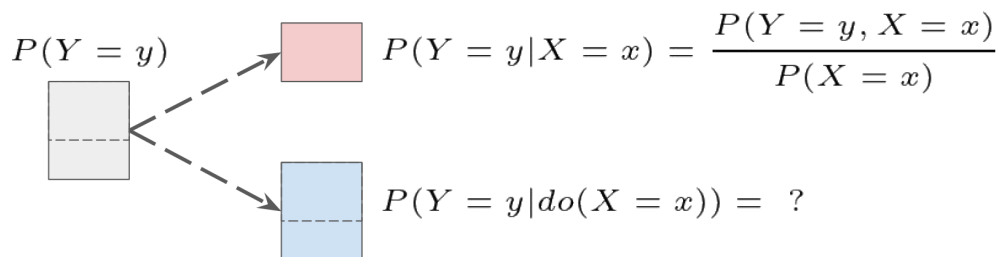


Figura 8.6: $P(Y = y|X = x)$ considero un sottoinsieme di Y , nel caso di $P(Y = y|do(X = x))$ considero tutto l'insieme di Y .

<i>Cloudy</i>	<i>Sprinkler</i>	<i>Rain</i>	<i>GrassWet</i>
TRUE	FALSE	TRUE	TRUE
TRUE	FALSE	FALSE	FALSE
FALSE	TRUE	TRUE	FALSE
TRUE	FALSE	TRUE	TRUE
FALSE	FALSE	FALSE	FALSE
TRUE	TRUE	TRUE	TRUE
TRUE	TRUE	FALSE	FALSE
TRUE	TRUE	TRUE	FALSE
FALSE	FALSE	FALSE	TRUE
FALSE	TRUE	FALSE	TRUE

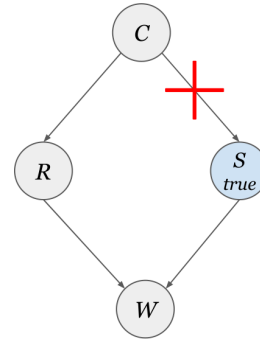
Figura 8.7: $P(\text{GrassWet}=\text{true}|\text{Sprinkler} = \text{true})$ corrisponde a considerare tutte le righe della tabella in cui $\text{Sprinkler} = \text{true}$ (5 nella figura) e contare, tra di queste, quante hanno $\text{GrassWet} = \text{true}$ (2 nella figura). Si ottiene in questo modo che la probabilità cercata è $2/5$. Nel caso di $do(\text{Sprinkler} = \text{true})$ significa che stiamo forzando tutta la colonna di Sprinkler a true e dunque il calcolo è diverso.

8.3.1 Intervento tramite chirurgia

Un intervento sulla variabile X corrisponde ad una modifica alla struttura del modello causale (SCM) originale in modo che la variabile X sia fissata e tutti gli archi entranti nel nodo X vengono rimossi (chirurgia sul grafo).

Considerando l'esempio dell'irrigatore, se forziamo $\text{do}(\text{Sprinkler} = \text{true})$ otteniamo:

$$\begin{aligned} f_C : C &= U_1 \\ f_R : R &= C \wedge U_2 \\ f_S : S &= \text{true} \\ f_W : W &= R \vee S \vee U_4 \end{aligned}$$



8.4 Calcolare gli interventi basandosi sulle osservazioni: adjustment formula

Come facciamo a liberarci dell'operatore $\text{do}(\cdot)$? Consideriamo l'esempio del paradosso di Simpson, definiamo: Z il gender dei pazienti che influenza il trattamento X (le donne sono più propense a prendere la medicina rispetto agli uomini) e il risultato Y (la guarigione).

$$\begin{aligned} Z &= U_Z \\ X &= f_X(Z, U_X) \\ Y &= f_Y(X, Z, U_Y) \end{aligned}$$

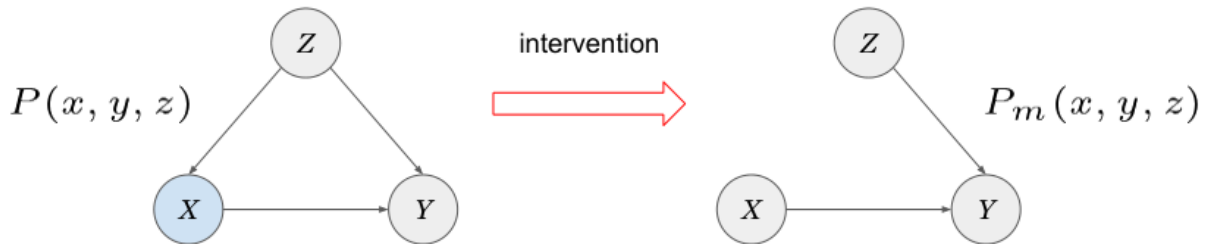


Figura 8.8: Rete causale prima e dopo l'intervento.

L'effetto causale $P(Y = y | \text{do}(X = x))$ equivale alla probabilità $P_m(Y = y | X = x)$ del modello manipolato m dopo l'intervento di chirurgia sul grafo. Quindi possiamo scrivere

$$P(Y = y | \text{do}(X = x)) = P_m(Y = y | X = x)$$

e in questo modo ci riconduciamo alle osservazioni. Notiamo inoltre che possiamo ricavare altre due uguaglianze

$$P(Z = z) = P_m(Z = z)$$

$$P(Y = y | Z = z, X = z) = P_m(Y = y | Z = z, X = z).$$

Infine, ricaviamo la risposta alla query $P(Y = y|do(X = x))$:

$$\begin{aligned}
 P(Y = y|do(X = x)) &= P_m(Y = y|X = x) \quad (\text{dalla definizione di intervento}) \\
 &= \sum_z P_m(y|x, z)P_m(z|x) \quad (\text{dalla legge di probabilità totale}) \\
 &= \sum_z P_m(y|x, z)P_m(z) \quad (\text{dall'impedenza di } X \text{ e } Z \text{ (collider)}) \\
 &= \sum_z P(y|x, z)P(z)
 \end{aligned}$$

In generale possiamo definire l'**adjustment formula** o **causal effect rule**:

$$P(Y = y|do(X = x)) = \sum_{z \in \Lambda} P(y|x, z)P(z)$$

dove Λ è l'insieme dei genitori di X .

8.4.1 Effetto causale medio

Supponiamo di voler conoscere il risultato di un esperimento ($Y = 1$) dipende dal fatto che si effettui il trattamento ($X = 1$) o meno ($X = 0$). In questi casi possiamo calcolare l'**effetto causale medio (ACE)** dato dall'applicazione dell'adjustment formula:

$$ACE = P(Y = 1|do(X = 1)) - P(Y = 1|do(X = 0))$$

se $ACE > 0$ allora il trattamento X ha un effetto positivo su Y ; se $ACE < 0$ o $ACE = 0$, X ha avuto un effetto negativo su Y .

Nel caso del paradosso di Simpson possiamo calcolare se la medicina ha avuto o meno un effetto positivo sulla guarigione. Abbiamo $P(Z = 1) = 0.51$ (pazienti maschi) e $P(Z = 0) = 0.49$ (pazienti femmine). Troviamo ACE :

$$\begin{aligned}
 P(Y = 1|do(X = 1)) &= P(Y = 1|X = 1, Z = 1)P(Z = 1) + P(Y = 1|X = 1, Z = 0)P(Z = 0) = 0.832 \\
 P(Y = 1|do(X = 0)) &= P(Y = 1|X = 0, Z = 1)P(Z = 1) + P(Y = 1|X = 0, Z = 0)P(Z = 0) = 0.7818 \\
 ACE &= 0.832 - 0.7818 = 0.0502.
 \end{aligned}$$

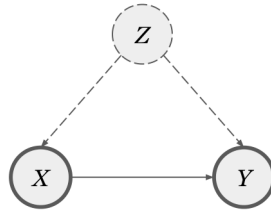
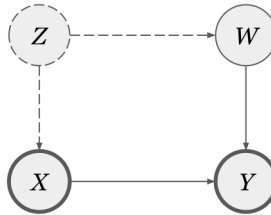
Risulta che la medicina favorisce la guarigione.

8.5 Unobserved confounders

Nella sezione precedente, abbiamo derivato un'espressione per calcolare l'effetto causale di X su Y basandoci su un comune **confounder** Z . Diciamo che Z è un confounder in quanto influisce sulla probabilità $P(Y = y|do(X = x))$ (vedi adjustment formula); in altre parole influisce sull'effetto causale di X su Y .

Tuttavia può capitare che il confounder Z non sia osservabile nel problema che stiamo considerando e non possiamo sfruttare l'adjustment formula. In questi casi nel grafo possiamo indicare che tra le variabili X e Y esiste una correlazione spuria che influenza l'effetto causale, andando a tratteggiare il nodo Z e i suoi archi Figura 8.9.

Consideriamo il caso (Figura 8.10) in cui è possibile ricavare una variabile osservabile da aggiungere al nostro problema. In questo modo possiamo ricavare $P(Y = y|do(X = x))$ sfruttando la nuova variabile introdotta W . Prima di vedere come dobbiamo introdurre un nuovo concetto.

Figura 8.9: Z è un unobserved confounder.Figura 8.10: Introduciamo una nuova variabile osservabile nel modello considerato. Z e W sono confounders, Z non è osservabile mentre W lo è.

8.5.1 Backdoor path

Un **backdoor path**¹ è un qualsiasi percorso che parte da X e arriva in Y che parte da un arco che entra in X (ovvero che include il suo genitore). Un percorso può essere "**bloccato**" andando a condizionare sulle configurazioni di chains/forks presenti oppure se sono presenti dei colliders.

Un insieme S di variabili soddisfa il **criterio backdoor** se sono verificate le seguenti due condizioni:

1. blocca tutti i percorsi backdoor tra X e Y ;
2. non contiene nessun discendente di X .

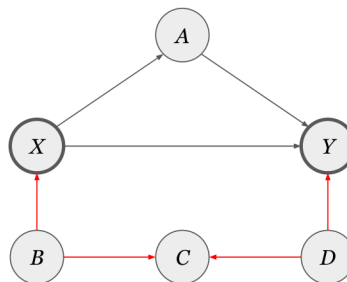


Figura 8.11: I set che verificano il criterio di backdoor sono: $\{\}$, $\{B\}$, $\{D\}$, $\{B, D\}$, $\{B, C\}$, $\{C, D\}$, $\{B, C, D\}$. C forma un collider: può essere inclusa nell'insieme S se anche B o D oppure sia B che D appartengono all'insieme altrimenti il percorso non è bloccato.

¹Nota che il percorso deve partire con un arco entrante in X (per includere un suo genitore), gli archi successivi possono essere percorsi senza considerare l'orientamento.

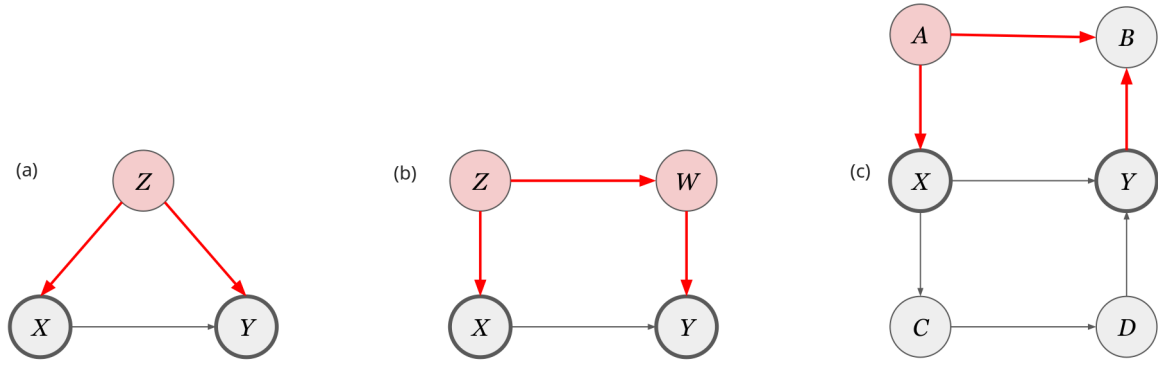


Figura 8.12: Esempi di backdoor path: (a) condizionando su Z (fork) blocchiamo il percorso (creiamo un'indipendenza) quindi $S = \{Z\}$; (b) condizionando su Z (fork) blocchiamo il percorso quindi $S = \{Z\}$, ma anche condizionando su W (chain) blocchiamo il percorso quindi $S = \{W\}$, possiamo condizionare sia su Z che su W e quindi $S = \{Z, W\}$; (c) condizionando su A (fork) blocchiamo il percorso, quindi $S = \{A\}$, oppure possiamo anche porre $S = \{\}$ visto che B forma un collider.

8.5.2 Backdoor adjustment

Se nella rete causale troviamo un insieme S di variabili che soddisfa il criterio backdoor, allora si ha che:

$$P(Y = y | do(X = x)) = \sum_s P(Y = y | X = x, s) P(s)$$

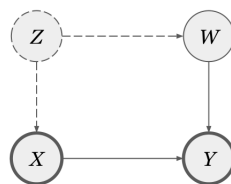
dove s sono tutti i possibili valori assunti dagli elementi di S . L'adjustment formula ricavata nella sezione 8.4 è solo un caso particolare in cui l'insieme delle variabili che soddisfa il criterio di backdoor sono i genitori di X .

8.5.3 Esempi backdoor path

Assumiamo che X, Y, Z, W siano delle variabili booleane.

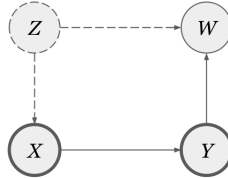
Esempio 1 Vogliamo trovare l'effetto causale di X su Y . Possiamo considerare il seguente percorso backdoor: $X \leftarrow Z \rightarrow W \rightarrow Y$. I possibili insiemi validi sono: $\{Z\}, \{W\}, \{Z, W\}$. Notiamo che solo W è osservabile quindi consideriamo $S = \{W\}$.

$$\begin{aligned} P(Y = y | do(X = x)) &= \sum_{s \in W} P(Y = y | X = x, s) P(s) \\ &= P(y | x, w) P(w) + P(y | x, \neg w) P(\neg w) \end{aligned}$$



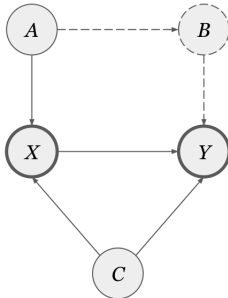
Esempio 2 In questo caso abbiamo il percorso backdoor: $X \leftarrow Z \rightarrow W \leftarrow W$. Abbiamo un collider in W quindi possiamo considerare $S = \{Z\}$.

$$\begin{aligned} P(Y = y | do(X = x)) &= \sum_{s \in W} P(Y = y | X = x, s) P(s) \\ &= P(y | x) \end{aligned}$$



Esempio 3 Assumiamo che anche A, B, C siano booleane. Vogliamo trovare l'effetto causale di X su Y . In questo caso abbiamo due percorsi backdoor da considerare: $X \leftarrow C \rightarrow Y$; $X \leftarrow A \rightarrow B \rightarrow Y$. Gli insiemi validi possibili devono tutti includere C visto che è l'unico modo di bloccare il primo percorso. Insiemi validi: $\{A, C\}$; $\{B, C\}$; $\{A, B, C\}$. Consideriamo $S = \{A, C\}$ visto che B non è osservabile.

$$P(Y = y | do(X = x)) = \sum_{i \in A} \sum_{j \in C} P(y | x, i, j) P(i, j)$$



8.5.4 Verificare la correttezza dei modelli usando il criterio di backdoor

Consideriamo il caso in cui esiste più di una variabile che può bloccare l'unico percorso backdoor tra X e Y . Poniamo che gli insiemi possibili sono $\{W\}$ e $\{Z\}$. In questo caso è possibile calcolare $P(Y = y | do(X = x))$ utilizzando la formula di adjustment con backdoor (sezione 8.5.2) considerando prima $S = \{W\}$ e successivamente $S = \{Z\}$: se otteniamo due risultati diversi allora il modello causale costruito non è corretto. Possiamo utilizzare questo metodo per verificare la correttezza delle ipotesi fatte per costruire il modello causale del problema che stiamo considerando.

8.5.5 Frontdoor path e frontdoor adjustment

Nelle precedenti sottosezioni abbiamo sempre considerato che almeno un confounder era osservabile. Ora consideriamo il caso in cui nessun confounder è osservabile: come si possiamo calcolare l'effetto causale di X su Y in questo caso?

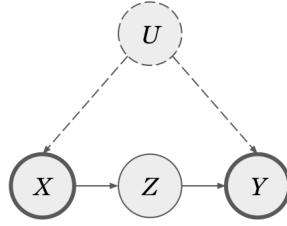


Figura 8.13: Esempio in cui tutti gli confounder sono non osservabili e in cui esiste un variabile tra il trattamento X e il risultato Y .

Consideriamo l'esempio di Figura 8.13. Possiamo ricavarci $P(Y = y|do(X = x))$ applicando il criterio di backdoor a $X \rightarrow Z$ e a $Z \rightarrow Y$ e combinare i due risultati invece di applicarlo a $X \rightarrow Y$. Appliciamo il criterio backdoor a $X \rightarrow Z$:

percorso backdoor: $X \leftarrow U \rightarrow Y \leftarrow Z$
insieme valido: $S = \{ \}$ (collider con Y)

$$P(Z = z|do(X = x)) = P(z|x)$$

Applichiamo il criterio backdoor a $Z \rightarrow Y$:

percorso backdoor: $Z \leftarrow X \leftarrow U \rightarrow Y$
insieme valido: $S = \{X\}$ (chain in X)

$$P(Y = y|do(Z = z)) = \sum_x P(y|z, x)P(x)$$

Combiniamo le due espressioni e otteniamo:

$$\begin{aligned}
 P(Y = y|do(X = x)) &= \sum_z P(y|do(z))P(z|do(x)) \\
 &= \sum_z \left[\sum_{x'} P(y|z, x')P(x') \right] P(z|x) \\
 &= \sum_z P(z|x) \sum_{x'} P(y|z, x')P(x')
 \end{aligned}$$

Si ottiene un'espressione che non dipende da U .

La formula ottenuta può essere generalizzata: introduciamo il **criterio frontdoor**. Un insieme S di variabili soddisfa il criterio frontdoor se vengono verificate le seguenti due condizioni:

1. se blocca ogni percorso diretto (**frontdoor path**) da X a Y è bloccato (i percorsi diretti partono da un arco uscente da X , ovvero includono un figlio di X);
2. se non ci sono percorsi backdoor da X a S ;
3. se tutti i percorsi backdoor da S a Y sono bloccati da X .

Se S soddisfa il criterio frontdoor, allora possiamo usare la seguente formula per calcolare l'effetto causale di X su Y (**frontdoor adjustment**):

$$P(Y = y|do(X = x)) = \sum_s P(s|x) \sum_{x'} P(y|x', s)P(x')$$

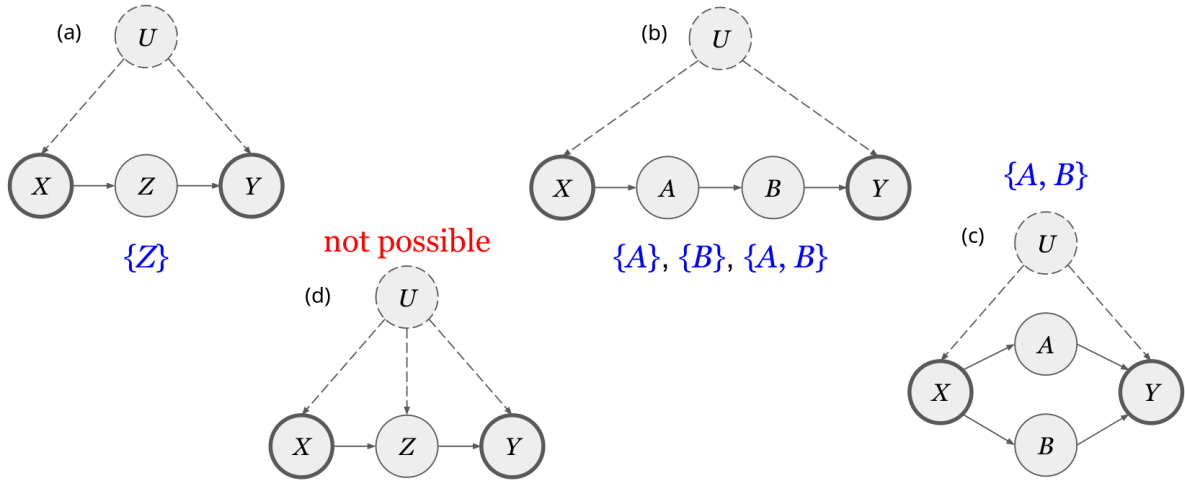


Figura 8.14: Esempi di percorsi frontdoor. (a) L'unico percorso frontdoor è $X \rightarrow Z \rightarrow Y$; l'unico insieme che soddisfa tutte e tre le condizioni è $S = \{Z\}$. (b) L'unico percorso frontdoor è $X \rightarrow A \rightarrow B \rightarrow Y$; gli insiemi validi sono $\{A\}$, $\{B\}$ oppure $\{A, B\}$. (c) Dobbiamo considerare due percorsi frontdoor: $X \rightarrow A \rightarrow Y$ e $X \rightarrow B \rightarrow Y$. L'unico insieme che soddisfa tutte le condizioni è $S = \{A, B\}$. (d) In questo caso non esiste nessun insieme che soddisfa tutte le condizioni: se prendessimo $\{Z\}$, non sarebbe soddisfatta la seconda condizione del criterio frontdoor; esiste infatti il percorso backdoor $X \leftarrow U \rightarrow Z$.

8.6 Causal discovery

Nelle sezioni precedenti abbiamo assunto che il modello causale del problema fosse sempre ben definito. In questa sezione vediamo come si ricava un modello causale. L'idea di base è quella di ricavare il modello partendo dai dati osservati (generati o ricavati da esperimenti). In questo modo si cerca di ricavare un modello causale che spieghi i dati ricavati.

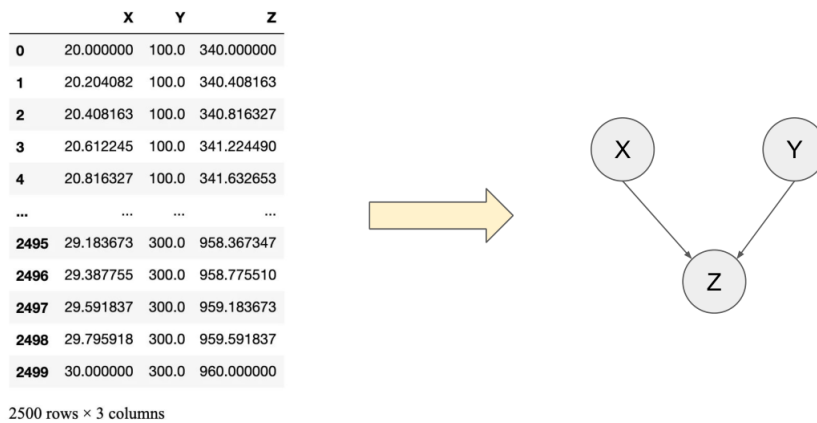


Figura 8.15: L'idea di causal discovery: dai dati ricavo il modello casuale.

I due algoritmi che andremo ad analizzare utilizzano entrambi metodi basati sui vincoli (**constraint-based method**). In pratica effettuano dei test di indipendenza condizionale (vincoli) per misurare l'indipendenza tra le variabili: in questo modo ricavano la rete causale. Gli algoritmi che consideriamo sono: **PC** (*Peter Spirtes and Clark Glymour*) e **FCI** (*Fast Causal Inference*). Sia PC che FCI si basano sulle seguenti assunzioni:

- **Acyclicity**: la rete causale non deve contenere cicli.

- **Sufficiency**: tutte le variabili importanti del problema devono essere osservabili.
- **Markov**: la presenza di *d-separation* nel grafo implica indipendenza condizionata nella distribuzione.

$$\underbrace{X \perp\!\!\!\perp_{\text{Graph}} Y \mid Z}_{\text{dal grafo teorico}} \Rightarrow \underbrace{X \perp\!\!\!\perp_{\text{Data}} Y \mid Z}_{\text{dalla tabella dei dati}}$$

(dove Z è l'insieme delle variabili che rende condizionatamente indipendenti X e Y). L'implicazione può non essere verificata per esempio nel caso in cui abbiamo raccolto pochi dati nella tabella delle osservazioni. In tal caso, non abbiamo dati a sufficienza per determinare delle distribuzioni che si avvicinano al modello teorico.

- **Faithfulness**: per applicare la causal discovery deve essere valida anche l'inverso dell'assunzione di Markov.

$$\underbrace{X \perp\!\!\!\perp_{\text{Graph}} Y \mid Z}_{\text{dal grafo teorico}} \Leftarrow \underbrace{X \perp\!\!\!\perp_{\text{Data}} Y \mid Z}_{\text{dalla tabella dei dati}}$$

Se X e Y sono indipendenti condizionando su Z nella distribuzione, allora X e Y sono *d-separated* da Z nella rete causale.

Nei casi reali può capitare che non sia possibile soddisfare completamente tutte le quattro assunzioni, in questi casi potrebbe non essere un problema (caso fortunato), oppure consideriamo che siano tutte verificate e generiamo un modello che è una buona approssimazione.

Prima di passare all'analisi degli algoritmi, introduciamo il concetto di **Markov equivalence**. Se abbiamo che $A \perp\!\!\!\perp C \mid B$ può essere rappresentato da tre configurazioni differenti (Figura 8.16): tutte e tre implicano la stessa indipendenza condizionata. In tal caso si dice che le configurazioni appartengono tutte alla stessa **Markov Equivalence Class**.

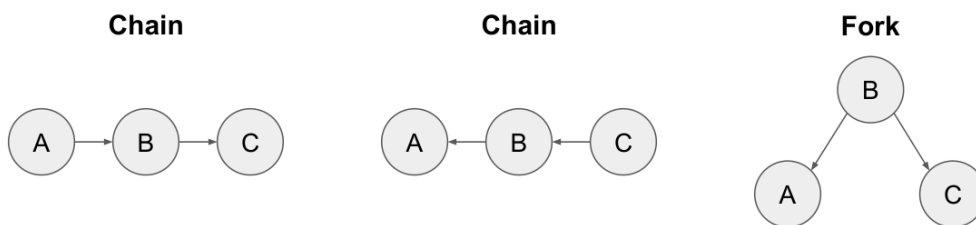


Figura 8.16: Markov equivalence: tutte e tre le configurazioni implicano la stessa indipendenza condizionata, quindi appartengono alla stessa Markov Equivalence Class.

Ricavando la rete causale dai dati può essere complesso dedurre quale delle configurazioni appartenenti alla stessa Markov Equivalence Class è quella più corretta. Ci vengono in aiuto i collider: la presenza di un collider nella rete può aiutare a identificare una delle possibili configurazioni equivalenti.

8.6.1 Algoritmo PC

Consideriamo corretto il modello di Passi dell'algoritmo:

1. Parto da un **grafo completo² non orientato**.
2. **Identifico lo scheletro** (Figura 8.17): in questa fase vengono rimossi alcuni archi della rete e viene verificata la presenza di indipendenza condizionata tra le variabili attraverso

²Un grafo completo è un grafo in cui esiste un arco per ogni coppia di vertici.

dei test statistici. In altre parole: si condiziona su una variabile e si effettuano dei test statistici per verificare se si creano delle indipendenze.

$$X \perp\!\!\!\perp Y \mid Z$$

Si parte condizionando su Z insieme vuoto e man mano si incrementa la sua dimensione aggiungendo altre variabili (viene fatto per tutte le combinazioni possibili). Nel nostro esempio, l'output di questo passo è il seguente:

$$A \perp\!\!\!\perp C \mid \{\}, A \perp\!\!\!\perp D \mid \{B\}, D \perp\!\!\!\perp E \mid \{B\}, C \perp\!\!\!\perp E \mid \{B\}, A \perp\!\!\!\perp E \mid \{B\}, C \perp\!\!\!\perp D \mid \{B\}$$

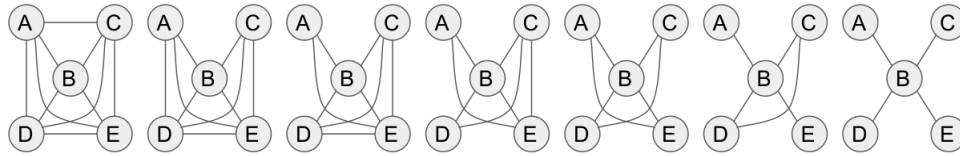


Figura 8.17: Fase di identificazione dello scheletro nell'algoritmo PC: il grafo più a destra rappresenta l'output.

3. **Individuo i collider:** per ogni percorso $X - Z - Y$ in cui non c'è nessun arco che collega X a Y e in cui Z non è mai inclusa nell'insieme delle variabili su cui condizioniamo, allora si ha che $X \rightarrow Z \leftarrow Y$, ovvero abbiamo individuato un collider. Nel nostro esempio abbiamo ricavato dal passo precedente che

$$A \perp\!\!\!\perp C \mid \{\} \text{ da cui si ricava che } A \rightarrow B \leftarrow C \text{ forma un collider.}$$

Le rimanenti indipendenze condizionate che abbiamo trovato appartengono tutte alla stessa Markov equivalence class (chain o fork).

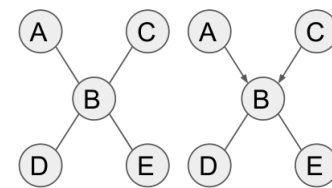


Figura 8.18: Fase di identificazione dei collider nell'algoritmo PC.

4. **Definire l'orientamento degli archi che non formano collider:** ogni percorso $X - Z - Y$ in cui ogni arco $Z - Y$ del percorso parzialmente diretto $X \rightarrow Z - Y$ non presenta un arco tra X e Y , può essere orientato come $Z \rightarrow Y$. Nel nostro esempio abbiamo che:

$$A \rightarrow B - D, \text{ non c'è un arco tra } A \text{ e } D \text{ quindi } \Rightarrow A \rightarrow B \rightarrow D$$

$$A \rightarrow B - E, \text{ non c'è un arco tra } A \text{ e } E \text{ quindi } \Rightarrow A \rightarrow B \rightarrow E$$

Nota che nel nostro esempio queste erano le uniche direzioni che potevamo dare agli archi rimanenti senza andare a formare dei collider.

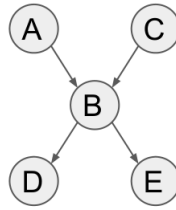


Figura 8.19: Output dell'algoritmo PC.

8.6.2 Algoritmo FCI

L'algoritmo FCI è una variante dell'algoritmo PC in cui sono ammessi i confounders non osservabili (dunque non viene rispettata l'assunzione di sufficienza). L'algoritmo esegue gli stessi del PC; se alla fine dell'esecuzione è presente un arco bidirezionale, questo indica la presenza di almeno un confounder nascosto.

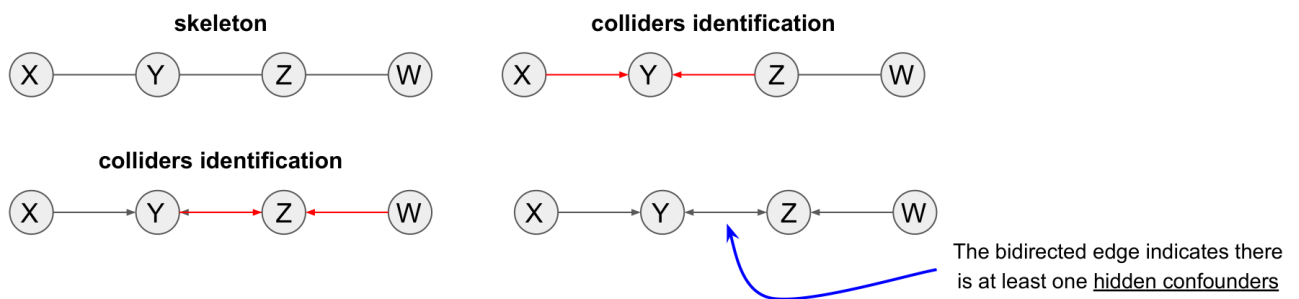


Figura 8.20: Esecuzione dell'algoritmo FCI.

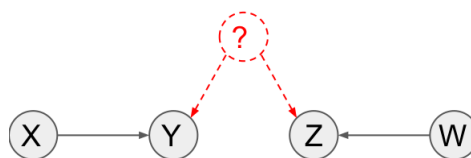


Figura 8.21: Rete compatibile con l'output dell'algoritmo FCI

8.7 Mediation

Alcune volte siamo interessati a differenziare la **causa diretta (direct effect)** (es: $X \rightarrow Y$) dalla **causa indiretta (mediated effect)** (es: $X \rightarrow Z \rightarrow Y$).

Per esempio (Figura 8.22(a)), consideriamo che X rappresenti il gender di un candidato, Z la sua qualifica e Y l'assunzione. Siamo interessati a determinare il direct effect del gender X sul risultato dell'assunzione Y , per farlo andiamo a condizionare su Z (andando a chiudere il percorso (chain)). Dunque si ottiene:

$$P(y|\text{do}(X = x), z) - P(y|\text{do}(X = x^*), z)$$

per un qualche valore di $Y = y$ e $Z = z$. Se il risultato della differenza è diverso da zero, allora significa che il gender ha un'influenza sull'esito dell'assunzione.

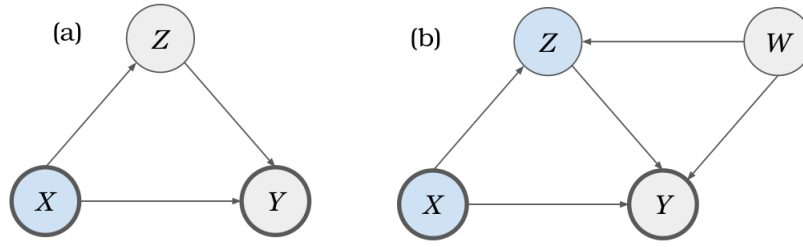


Figura 8.22

Tuttavia non è sempre possibile condizionare su Z . Introduciamo nel nostro modello causale una variabile W che modella il reddito del candidato (Figura 8.22(b)). In questo caso, condizionando su Z , non si elimina la causa indiretta visto che Z forma un collider con W e X (si elimina la causa indiretta creata da Z ma se ne forma una nuova con W). La soluzione in questo caso è quella di intervenire sul **mediator** Z , ciò ha l'effetto di rimuovere gli archi entranti in Z (vedi sezione 8.4). A questo punto possiamo calcolare la **controlled direct effect (CDE)**:

$$\text{CDE} = P(y|\text{do}(x), \text{do}(z)) - P(y|\text{do}(x^*), \text{do}(z))$$

per un qualche valore di $Z = z$.

La domanda che sorge a questo punto è: come facciamo a risolvere il doppio $\text{do}(\cdot)$. Si ha che:

$$P(y|\text{do}(x), \text{do}(z)) = P(y|x, \text{do}(z)) = \sum_w P(y|x, z, w)P(w)$$

dove al primo passaggio abbiamo rimosso il $\text{do}(x)$ in quando non ci sono backdoor path da X a Y , mentre per rimuovere il $\text{do}(z)$ abbiamo utilizzato la backdoor adjustment (sotto-sezione 8.5.2) formula considerando il backdoor path $Z \leftarrow W \rightarrow Y$ (nota che il percorso $Z \leftarrow X \rightarrow Y$ è bloccato dato che conosciamo $X = x$). Infine possiamo riscrivere il CDE come segue:

$$\text{CDE} = \sum_w [P(y|x, z, w) - P(y|x^*, z, w)]P(w)$$

Applicazione al caso generale La formula che abbiamo ricavato può essere applicata se:

1. esiste almeno un set S_1 di variabili che bloccano tutti i backdoor paths da Z a Y ;
2. esiste almeno un set S_2 di variabili che bloccano tutti i backdoor paths da X a Y dopo aver rimosso gli archi entranti in Z .

Se entrambe le condizioni sono verificate, allora ponendo $S = \{S_1, S_2\}$ si ottiene:

$$P(y|\text{do}(x), \text{do}(z)) = \sum_{s \in S} P(y|x, z, s)P(s)$$

8.8 Linear SCM

Nei structural causal model lineari, le relazioni che legano le variabili sono del tipo (Figura 8.23):

$$\begin{aligned} X &= U_X \\ Z &= aX + U_Z \\ W &= bX + cZ + U_W \\ Y &= dZ + eW + U_Y \end{aligned}$$

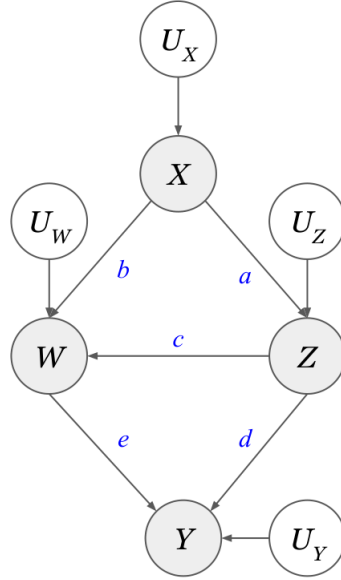


Figura 8.23: Linear SCM.

Nei SCM lineari, i **coefficienti strutturali** (a, b, c, d, e) rappresentano i direct effects dei nodi genitori sui loro figli. Nella sezione 8.6 abbiamo visto degli algoritmi di causal discovery che riescono a ricavare dei modelli causali come quello in Figura 8.23 avendo a disposizione solamente dei dati. Ma come facciamo a ricavare anche i coefficienti strutturali?

8.8.1 Linear Gaussian models

Consideriamo il caso in cui le funzioni che legano le variabili del modello causale siano lineari. Consideriamo inoltre che le variabili aleatorie coinvolte nel modello siano gaussiane (vedremo più avanti come ci tornerà utile questa assunzione).

Fino ad ora abbiamo sempre analizzato SCMs e grafi aventi solamente variabili aleatorie discrete, tuttavia esistono molte classi di problemi che considerano variabili aleatorie continue. Nel nostro caso consideriamo $X \sim N(\mu, \sigma)$, dove i valori x che può assumere X sono descritti dalla seguente funzione di distribuzione di probabilità

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

dove μ è l'aspettazione $E[X]$ e σ è la deviazione standard o la radice quadrata della varianza $\sigma^2 = E[(X - \mu)^2]$.

Le distribuzioni gaussiane hanno le seguenti proprietà:

- a) $X \perp\!\!\!\perp Y | Z \Rightarrow P(Y|X, Z) = P(Y|Z) \Rightarrow E[Y|X, Z] = E[Y|Z]$
- b) $\hat{Y} = E[Y|X_1, X_2, \dots, X_n] = r_0 + r_1X_1 + r_2X_2 + \dots + r_nX_n$

dove i coefficienti $r_1, r_2, \dots, r_n$ sono i **partial regression coefficients**, ovvero i coefficienti che individuano una retta/iperpiano che minimizza la somma degli errori quadratici medi di tutti i punti. Nel caso $n = 1$, si parla di regressione lineare: si tratta di trovare i valori di r_1, r_2 in modo da individuare la retta che approssima meglio i punti del piano.

$$\operatorname{argmin}_r \sum_{i=1:S} (y_i - \hat{y}_i)^2 \quad \text{per } S \text{ campioni}$$

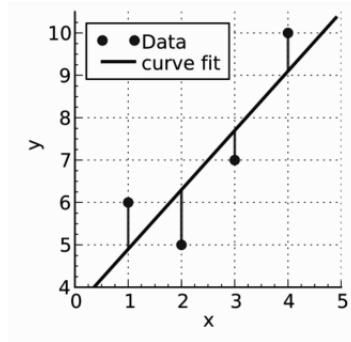


Figura 8.24: Caso in cui $n = 1$ e quindi $\hat{Y} = r_0 + r_1 X_1$ è una retta.

Tornando alle equazioni che descrivono il nostro modello causale, possiamo ricondurre il calcolo dei coefficienti strutturali al calcolo dei coefficienti di regressione parziale. Considerando il nostro esempio si ha che:

$$\begin{aligned}
 X &= U_X && \rightarrow E[Z|X] = r_{0Z} + aX \\
 Z &= aX + U_Z && \rightarrow E[W|X, Z] = r_{0W} + bX + cZ \\
 W &= bX + cZ + U_W && \rightarrow E[Y|Z, W] = r_{0Y} + dZ + eW \\
 Y &= dZ + eW + U_Y &&
 \end{aligned}$$

In generale si tratta di effettuare una regressione lineare sulla variabile figlia dati i genitori (es: regressione lineare su $W|X, Z$).

8.8.2 Total causal effect

I structural coefficients (a, b, c, d, e) che abbiamo ricavato, rappresentano il direct effect della variabile genitore sulla sua variabile figlia. L'**effetto causale totale** tra X e Y può essere calcolato andando a sommare tutti i prodotti dei coefficienti strutturali su ogni percorso non backdoor da X a Y . Nell'esempio di Figura 8.23, l'effetto causale totale di Z su Y è $d + c \cdot e$.

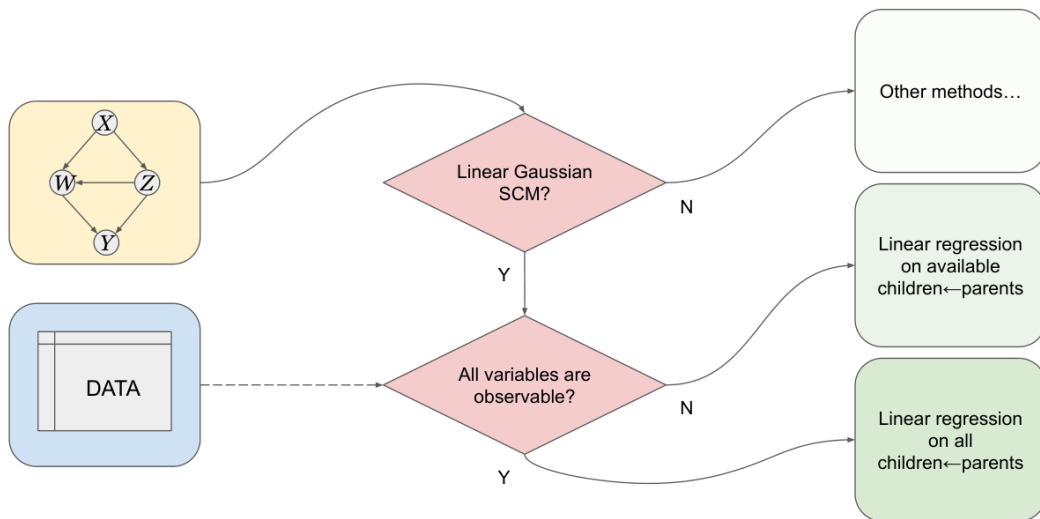


Figura 8.25: Procedura generale per ricavare i coefficienti strutturali avendo a disposizione il SCM (ricavato mediante causal discovery).

Capitolo 9

Making complex decisions

9.1 Introduzione

Partiamo considerando un esempio che ci permette di introdurre la classe di problemi che consideriamo in questo capitolo. La Figura 9.1 rappresenta un classico quiz game: ad ogni domanda si ha la possibilità di non rispondere ed incassare il denaro accumulato, oppure si può scegliere di andare avanti alla prossima rischiando di perdere tutto (se questa viene sbagliata). Le domande sono ordinate in maniera crescente rispetto alla difficoltà.

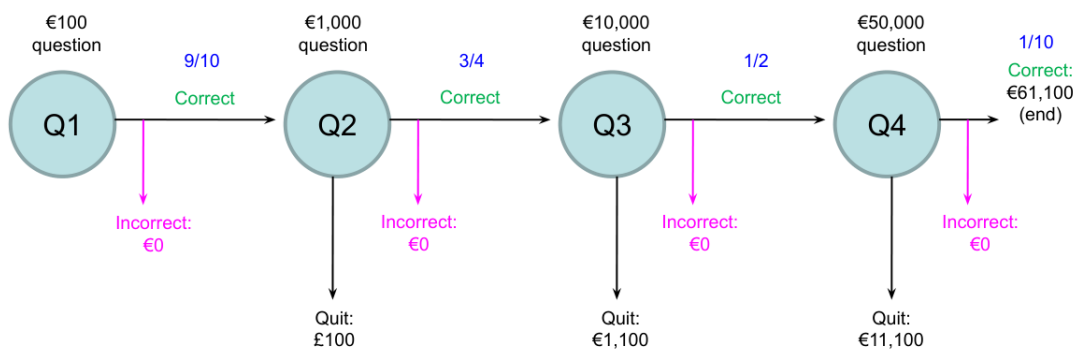
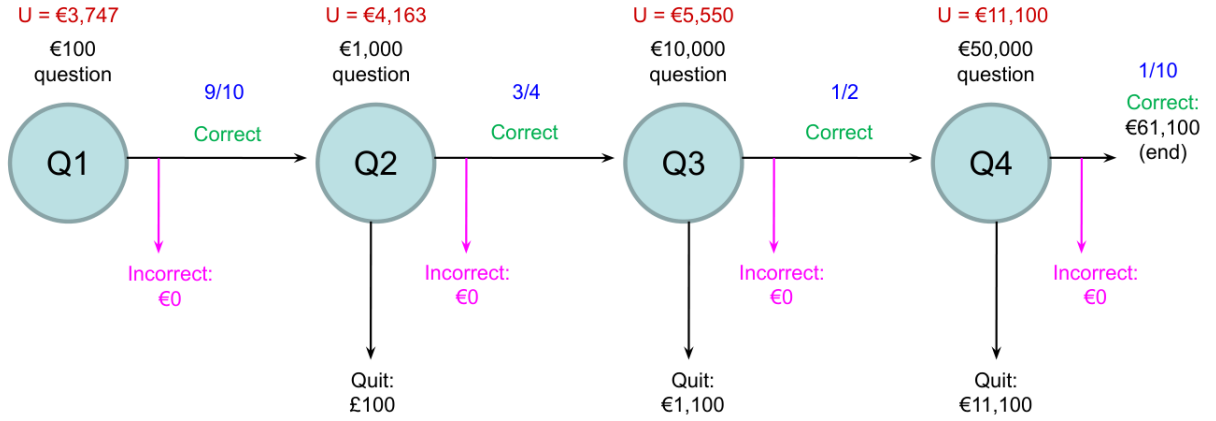


Figura 9.1: Esempio di problema di decisione sequenziale. Troviamo: il premio per ogni domanda (sopra al nodo); il guadagno se si decide di non rispondere (freccia sotto al nodo) e la probabilità di azzeccare la risposta (in blu). Se si tenta la risposta e si sbaglia, allora si perde il guadagno accumulato.

Poniamo che sia un agente intelligente a partecipare al quiz. Quali sarebbero le sue decisioni? Considerando di essere alla domanda Q4, conviene rischiare o uscire ed incassare il denaro accumulato? Il guadagno che si ottiene rispondendo giusto anche a questa domanda è di \$61,100. Possiamo calcolare l'aspettazione del guadagno se decidiamo di continuare: $0.1 \cdot 61,100 + 0.9 \cdot 0 = \$6,110$. Siccome l'aspettazione è minore del guadagno che si ottiene decidendo di non rispondere (\$11,000), conviene non rischiare e non rispondere. Ripetiamo lo stesso ragionamento per le domande precedenti: in Q3 l'aspettazione è $0.5 \cdot 11,000 = 5,550 > 1,100$ quindi conviene rispondere; in Q2 l'aspettazione è $0.75 \cdot 5,550 = 4,163 > 100$ quindi conviene rispondere; infine in Q1 conviene ovviamente rispondere.

9.1.1 Expected utility

Il guadagno potenziale ad ogni domanda (stato) indica l'**utilità** di quella domanda. Indichiamo con $U(s)$ l'utilità di essere in un certo stato s . L'utilità $U(s)$ è una sorta di punteggio asse-


 Figura 9.2: Utilità U di ogni stato.

gnato allo stato s ; in pratica indica quanto è desiderabile trovarsi in quello stato. Definiamo l'**utilità attesa** di un'azione $EU(a)$ come la media dell'utilità dei risultati di quell'azione, pesata rispetto la probabilità che il risultato si verifichi:

$$EU(a) = \sum_{s'} P(\text{Result}(a) = s') U(s').$$

L'agente intelligente che abbiamo descritto precedentemente, decideva basandosi sull'utilità attesa: sceglieva l'azione che massimizzava l'utilità attesa (**MEU principle**, *Maximize Expected Utility*).

L'utilità è un numero reale che viene associato ad ogni stato possibile del problema che stiamo considerando. Ma come scegliamo il numero da assegnare? Non esiste una formula generica, dipende dal problema che stiamo considerando, solitamente si fissa l'utilità di due particolari stati e da questi si ricava una scala. Tipicamente si fissano i seguenti due valori di utilità:

- *best possible prize* $U(s) = u_{\top}$;
- *worst possible catastrophe* $U(s) = u_{\perp}$.

Normalizzando l'utilità si ottiene una scala che va da un minimo di $u_{\perp} = 0$ ad un massimo di $u_{\top} = 1$.

9.1.2 Modello di transizione

L'agente associa una probabilità $P(s)$ per ogni possibile stato corrente s . Il **modello di transizione** è dato dalla probabilità $P(s'|s, a)$. Quindi, la probabilità di raggiungere lo stato s' eseguendo l'azione a da un qualsiasi stato s è data da

$$P(\text{Result}(a) = s') = \sum_s P(s) P(s'|s, a).$$

Notiamo che la formula utilizzata è molto simile all'adjustment formula utilizzata per calcolare $P(s'|\text{do}(a))$ nei modelli causali.

Consideriamo l'esempio di un utility-based agent che deve raggiungere un goal evitando degli ostacoli (Figura 9.3). Potremmo aver dato all'agente il comando *GO UP* ma, per qualche motivo (es: un malfunzionamento) svolta a sinistra o a destra. C'è dunque dell'incertezza sul risultato di un'azione, non è deterministica: poniamo per esempio che la probabilità di

ottenere l'esito desiderato sia di 0.8 mentre 0.2 è la probabilità che l'agente si muova in una direzione ortogonale a quella data. Il modello di transizione tiene conto anche dell'incertezza del risultato delle azioni.

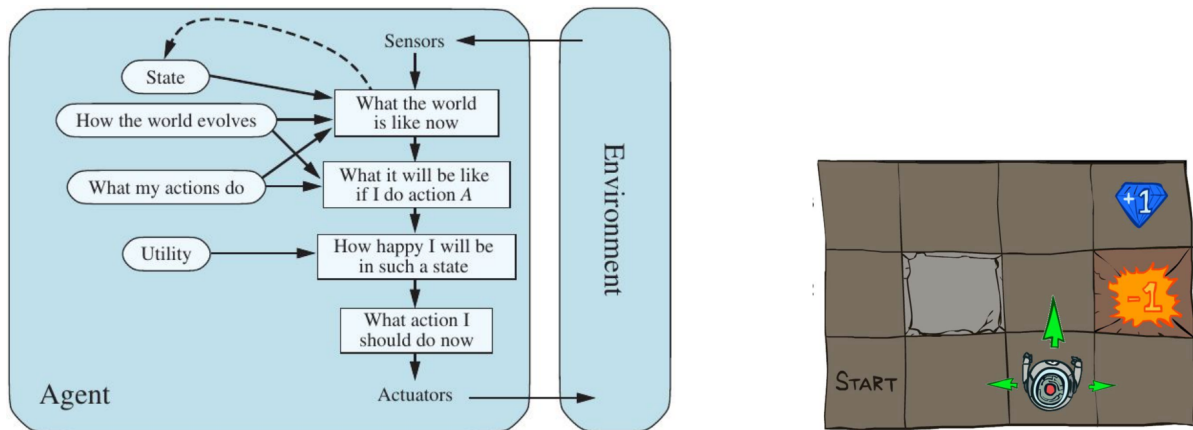


Figura 9.3: Utility-based agent. L'obiettivo è quello di raggiungere la casella (stato) con il diamante schivando quella con il fuoco.

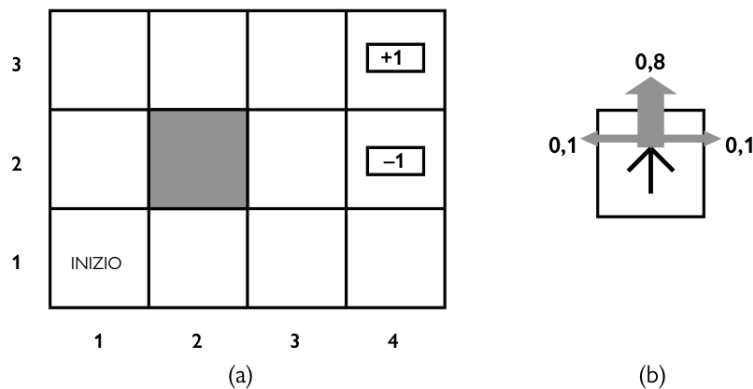


Figura 9.4: (a) Rappresentazione dell'ambiente (stocastico) in cui opera l'agente. (b) Il modello di transizione: 0.8 è la probabilità di ottenere l'esito desiderato; 0.2 è la probabilità che l'agente si muova in una direzione ortogonale a quella voluta. La collisione con un muro non provoca nessuno spostamento. La ricompensa ottenuta raggiungendo gli stati terminali è +1 e -1; tutte le altre transizioni hanno ricompensa $r = -0.04$.

9.1.3 Problemi decisionali sequenziali (MDP)

La classe di problemi che andiamo a considerare sono i problemi decisionali sequenziali. In particolare consideriamo problemi decisionali sequenziali in ambienti stocastici completamente osservabili. Questa tipologia di problemi prende il nome di **processo decisionale di Markov** o **MDP** (*Markov Decision Process*). Più precisamente un MDP consiste in:

- un insieme di stati (con stato di partenza indicato da s_0);
- un insieme di azioni $ACTIONS(s)$ per ogni stato;
- un modello di transizione $P(s'|s, a)$;

- una **funzione di ricompensa (reward function)** $R(s, a, s')$.

La funzione di ricompensa $R(s, a, s')$ ritorna il valore (positivo o negativo) associato alla transizione da s a s' mediante l'azione a . Risolvere un MDP significa individuare una **policy**, ovvero una funzione che indica l'azione che l'agente deve intraprendere se si trova in uno qualunque degli stati possibili. Una policy può essere vista come una mappa che indica l'azione da intraprendere per avvicinarsi al goal. La qualità di una policy viene misurata dall'aspettazione dell'utilità di tutte le sequenze di stati e azioni che genera. La policy con la più alta utilità attesa è detta **optimal policy**.

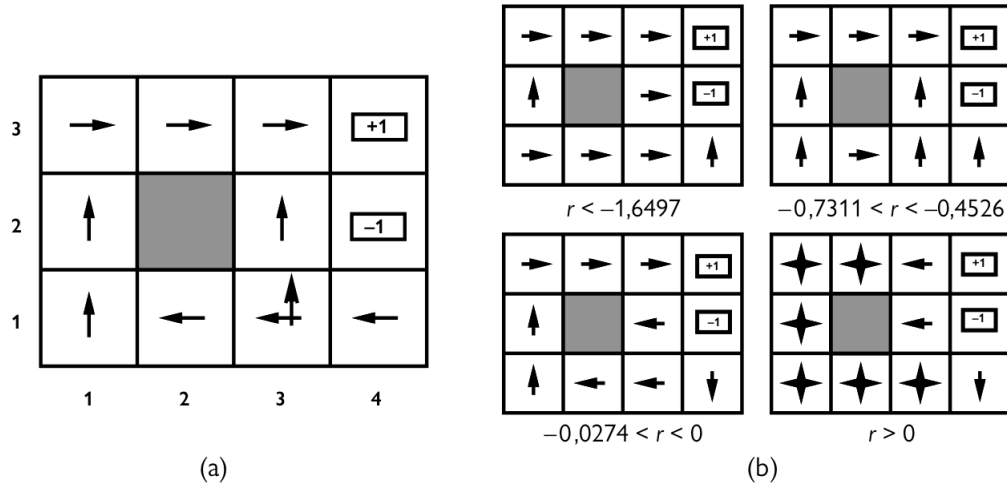


Figura 9.5: (a) Politica ottima per l'ambiente stocastico con $r = -0.4$. (b) Politiche ottime al variare del valore di ricompensa.

La Figura 9.5 mostra come il valore associato alla ricompensa influenza la politica ottima (ricavata tramite degli algoritmi che analizzeremo nel seguito). In tutti gli esempi si suppone che ogni transizione abbia la stessa ricompensa. Si nota che se la ricompensa è molto negativa ($r < -1.6497$) allora la policy che si ottiene tende a far sì che l'agente raggiunga uno degli stati terminali con il minor numero di transizioni possibili. Diciamo che ogni transizione è talmente costosa che l'agente vuole raggiungere un'uscita il più velocemente possibile. Per $-0.7311 < r < -0.4526$ si hanno valori di ricompense abbastanza negative; la policy ricavata punta a fare raggiungere all'agente lo stato +1 il seguendo la strada più breve, anche a costo di rischiare di finire in -1 passando per la casella (3,2). Tuttavia notiamo che partendo da (4,1) è più conveniente terminare in -1 piuttosto che fare tutto il giro per arrivare a +1. Per $-0.0274 < r < 0$ la policy punta a far raggiungere lo stato +1 seguendo il percorso più sicuro. Infine per $r > 0$, di fatto si viene a creare un loop infinito in quanto non conviene mai raggiungere uno dei due stati terminali.

9.1.4 Utilità nel tempo

Esistono diversi modi di definire la funzione di utilità $U_h([s_0, a_0, s_1, a_1, \dots, s_n])$ per sequenze temporali. Se il processo di decisione prevede un **orizzonte finito**, allora esiste un tempo prefissato N dopo il quale nulla importa più: diciamo che oltre quel tempo, il problema è terminato. In tal caso:

$$U_h([s_0, a_0, s_1, a_1, \dots, s_{N+k}]) = U_h([s_0, a_0, s_1, a_1, \dots, s_N])$$

per ogni $k > 0$. In questi casi l'azione ottimale da eseguire trovandosi in un certo stato, può dipendere dal tempo rimanente. Una politica che dipende dal tempo è detta **non stazionaria**.

Il caso opposto è quando si ha un **orizzonte infinito**. In questi casi non c'è nessun limite di tempo e dunque la decisione presa trovandomi in un certo stato al tempo t_0 , sarà esattamente la stessa che prenderò se mi dovessi trovare nello stesso stato al tempo $t_1 > t_0$. In altre parole, la decisione ottima dipende solamente dallo stato corrente e la politica ricavata si dice **stazionaria**.

Un altro punto da considerare è il come si valuta l'utilità di una sequenza di stati-azioni. Una scelta comune è quella di utilizzare la **additive discounted reward**:

$$\begin{aligned} U_h([s_0, a_0, s_1, a_1, \dots]) &= R(s_0, a_0, s_1) + \gamma R(s_1, a_1, s_2) + \gamma^2 R(s_2, a_2, s_3) + \dots \\ &= R(s_0, a_0, s_1) + \gamma U_h([s_0, a_0, s_1, a_1, \dots]) \end{aligned}$$

dove $0 < \gamma < 1$ viene detto **discount factor** (*fattore di sconto*). Per valori di γ vicini a 0, le azioni future sono associate ad una ricompensa molto più piccola e quindi diciamo che l'agente preferisce una ricompensa immediata rispetto a quelle future. Per valori di γ vicini a 1 l'agente è maggiormente disponibile ad attendere ricompense nel lungo termine. Nel caso speciale in cui $\gamma = 1$ si parla di **ricompense puramente additive** (come nell'esempio di Figura 9.5).

Il vantaggio delle additive discounted reward è che l'utilità di una sequenza infinita risulta finita. Infatti, per $\gamma < 1$ e per valori di ricompense compresi tra $\pm R_{\max}$, l'utilità è anch'essa limitata:

$$U_h([s_0, a_0, s_1, \dots]) = \sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \leq \sum_{t=0}^{\infty} \gamma^t R_{\max} = \frac{R_{\max}}{1 - \gamma}$$

9.2 Equazione di Bellman

L'equazione di Bellman si basa sul fatto che l'utilità $U(s)$ di uno stato s è data dalla ricompensa attesa per la successiva transizione sommata all'utilità scontata dello stato successivo, assumendo che l'agente scelga l'azione ottima. L'utilità è quindi data da

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$$

e quella appena definita è l'**equazione di Bellman**. La quantità che deve essere massimizzata rappresenta l'aspettazione dell'utilità data nell'effettuare una certa azione in un certo stato. Chiamiamo questo termine **action-utility function** o **Q-function** $Q(s, a)$. L'equazione può essere riscritta come segue:

$$U(s) = \max_a Q(s, a).$$

Sfruttando la Q-function possiamo ricavare

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

ovvero la policy ottima da applicare nello stato s . Gli algoritmi che risolvono problemi MDP che analizzeremo nelle sezioni successive, sfruttando tutti la Q-function per estrarre una politica ottima.

Applichiamo l'equazione di Bellman all'esempio di Figura 9.4. Nell'esempio abbiamo posto come ricompensa $r = -0.04$ per ogni transizione verso uno stato non terminale e abbiamo posto il discount factor a $\gamma = 1$. Applicando l'equazione di Bellman nello stato $(1, 1)$ otteniamo (sfruttando i dati di Figura 9.6):

$$\begin{aligned} U(1, 1) &= \max\{[0.8 \cdot (-0.04 + \gamma U(1, 2)) + 0.1 \cdot (-0.04 + \gamma U(2, 1)) + 0.1 \cdot (-0.04 + \gamma U(1, 1))] \text{ (} a = \text{UP)} \\ &\quad [0.9 \cdot (-0.04 + \gamma U(1, 1)) + 0.1 \cdot (-0.04 + \gamma U(1, 2))] \text{ (} a = \text{LEFT)} \\ &\quad [0.8 \cdot (-0.04 + \gamma U(2, 1)) + 0.1 \cdot (-0.04 + \gamma U(1, 1)) + 0.1 \cdot (-0.04 + \gamma U(1, 2))] \text{ (} a = \text{RIGHT)} \\ &\quad [0.9 \cdot (-0.04 + \gamma U(1, 1)) + 0.1 \cdot (-0.04 + \gamma U(2, 1))] \text{ (} a = \text{DOWN)} \\ &= \max\{0.74534, 0.71093, 0.67093, 0.7003\} = 0.74534 \end{aligned}$$

da cui si ricava che la migliore azione da applicare nello stato $(1, 1)$ è UP .

3	0,8516	0,9078	0,9578	+1
2	0,8016		0,7003	-1
1	0,7453	0,6953	0,6514	0,4279
	1	2	3	4

Figura 9.6: Utilità degli stati del mondo 4×3 con $\gamma = 1$ e $r = -0.04$ per transizioni verso stati non terminali.

9.2.1 Value iteration algorithm

L'equazione di Bellman è alla base dell'algoritmo iteration value per la risoluzione di MDP. Sappiamo che:

- se ci sono n possibili stati, allora ci sono esattamente n equazioni di Bellman;
- le n equazioni contengono n incognite, che sarebbero le utilities U degli stati;
- le n equazioni sono non lineari a causa della presenza dell'operatore max.

Un approccio per risolvere il sistema di equazioni non lineare, è quello iterativo. Si parte fissando dei valori arbitrari delle utilità, si calcola successivamente la parte destra dell'equazione di Bellman e la si sostituisce nella parte sinistra. In questo modo si va ad aggiornare l'utilità di ogni stato in base a quelle dei suoi vicini. Questo processo viene ripetuto finché non si raggiunge un equilibrio.

Chiamiamo $U_i(s)$ il valore di utilità dello stato s alla i -esima iterazione. Il passo di iterazione, chiamato **aggiornamento di Bellman**, è dato da

$$U_{i+1}(s) \leftarrow \max_{a \in A(s)} \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U_i(s')],$$

ipotizzando che l'aggiornamento sia applicato simultaneamente a tutti gli stati in ogni iterazione.

```

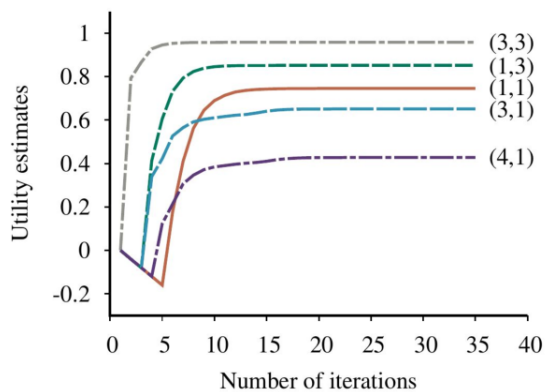
1 def function Q-VALUE(mdp, s, a, U) returns a utility value {
2   return  $\sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U[s']]$ 
3 }
4
5 def function VALUE-ITERATION(mdp,  $\epsilon$ ) returns a utility function {
6   inputs: mdp, an MDP with states  $S$ , action  $A(s)$ , transition model  $P(s'|s, a)$ ,
7         rewards  $R(s, a, s')$ , discount  $\gamma$ 
8
9   local variable:  $U, U'$ , vectors of utilities for states in  $S$ , initially zero
10                   $\delta$ , the maximum relative change in the utility of any state
11
```

```

12 repeat
13   U ← U'; δ ← 0
14   for each state s in S do {
15     U'[s] ← maxa ∈ A(s) Q-VALUE(mdp, s, a, U)
16     if |U'[s] - U[s]| > δ then
17       δ ← |U'[s] - U[s]|
18   }
19   until δ ≤ ε(1 - γ)/δ
20   return U
21 }

```

L'algoritmo termina quando la differenza tra le vecchie e le nuove utilità è al di sotto di una certa soglia. L'utilità restituita in output, può essere utilizzata per ricavare la politica ottima per mezzo dell'equazione $\pi^*(s) = \operatorname{argmax}_a Q(s, a)$.



3	0.8516	0.9078	0.9578	+1
2	0.8016		0.7003	-1
1	0.7453	0.6953	0.6514	0.4279
	1	2	3	4

Figura 9.7: Evoluzione dell'utilità di alcuni stati al crescere del numero di iterazioni dell'algoritmo value iteration.

9.2.2 Policy iteration algorithm

L'algoritmo policy iteration permette di fornire un metodo alternativo alla risoluzione dei MDPs. L'algoritmo alterna i due seguenti passi partendo da una politica iniziale π_0 .

- **Valutazione della policy:** data una policy π_i , si calcola $U_i = U^{\pi_i}$, l'utilità di ogni stato qualora si dovesse eseguire π_i .
- **Miglioramento della policy:** si calcola una nuova policy MEU π_{i+1} , scegliendo gli stati successivi in base a U_i mediante $\pi_{i+1}^*(s) = \operatorname{argmax}_a Q(s, a)$.

L'algoritmo termina quando il passo di miglioramento della policy non apporta alcun cambiamento alle utilità. In tal caso π_i è una policy ottima.

```

1 def function POLICY-ITERATION(mdp) returns a policy {
2   inputs: mdp, an MDP with states S, actions A(s), transition model P(s'|s,a)
3   local variables: U, a vector of utilities for states in S, initially zero
4                     π, a policy vector indexed by state, initially random
5
6   repeat
7     U ← POLICY-EVALUATION(π, U, mdp)
8     changed ← false
9     for each state s in S do {

```

```

10      $a^* \leftarrow \operatorname{argmax}_{a \in A(s)} Q\text{-VALUE}(\text{mdp}, s, a, U)$ 
11     if  $Q\text{-VALUE}(\text{mdp}, s, a^*, U) > Q\text{-VALUE}(\text{mdp}, s, \pi[s], U)$  then
12          $\pi[s] \leftarrow a^*$ ;  $\text{changed} \leftarrow \text{true}$ 
13     }
14 until changed
15 return  $\pi$ 
16 }

```

Come può essere implementata la procedura POLICY-EVALUATION? All' i -esima iterazione, la politica π_i nello stato s specifica l'azione $\pi_i(s)$. Quindi possiamo considerare una versione semplificata dell'equazione di Bellman che collega l'utilità di s (data π_i) a quella dei suoi vicini

$$U_i(s) = \sum_{s'} P(s'|s, \pi_i(s)) [R(s, \pi_i(s), s') + \gamma U_i(s')].$$

Notiamo che non è presente l'operatore max e dunque si ottiene un sistema di n equazioni lineari che può essere risolto con algoritmi standard di algebra lineare.

Consideriamo per esempio che π_i sia la policy mostrata in Figura 9.5(a). Dunque abbiamo che $\pi_i(1, 1) = UP$, $\pi_i(1, 2) = UP$, $\pi_i(1, 3) = RIGHT$, etc. Le equazioni di Bellman semplificate che si ottengono sono (considerando sempre $r = -0.04$ e $\gamma = 1$):

$$U_i(1, 1) = 0.8 \cdot [-0.04 + \gamma U_i(1, 2)] + 0.1 \cdot [-0.04 + \gamma U_i(2, 1)] + 0.1 \cdot [-0.04 + \gamma U_i(1, 1)]$$

$$U_i(1, 2) = 0.8 \cdot [-0.04 + \gamma U_i(1, 3)] + 0.2 \cdot [-0.04 + \gamma U_i(1, 2)]$$

$$U_i(1, 3) = 0.8 \cdot [-0.04 + \gamma U_i(2, 3)] + 0.1 \cdot [-0.04 + \gamma U_i(1, 3)] + 0.1 \cdot [-0.04 + \gamma U_i(1, 2)]$$

⋮ per tutti gli stati possibili

9.3 MDP parzialmente osservabili (POMDP)

Nelle precedenti sezioni abbiamo considerato processi decisionali di Markov che presupponessero che l'ambiente fosse **completamente osservabile**. In questi casi l'agente conosce sempre lo stato in cui si trova e abbiamo visto che la policy ottima dipende solamente dallo stato corrente ($\pi^*(s)$).

In molti dei problemi reali però, l'agente non sa esattamente in che stato si trova. In questi casi si parla di ambiente **parzialmente osservabile**. Questa classe di problemi prende il nome di **POMDP** (*Partially Observable MDP*).

Markov models	Passive (observation only)	Active (actions possible)
State fully observable	Markov chain	MDPs
State partially observable	HMM	POMDP

Tabella 9.1: Class of Markov models.

9.3.1 Definizione di un POMDP

Abbiamo visto che un MDP è definito da: un modello di transizione $P(s'|s, a)$; dalle azioni possibili sullo stato s $A(s)$; da una funzione di ricompensa $R(s, a, s')$. Un POMDP è essenzialmente un MDP con in aggiunta un sensor model $P(e|s)$, dove e è l'osservazione. Come abbiamo detto, l'agente non sa esattamente in che stato si trova e per questo andiamo a definire il **belief state** b . Nei POMDP il belief state b è una distribuzione di probabilità su tutti gli stati possibili. Utilizziamo la notazione $b(s)$ per indicare la probabilità assegnata allo stato reale s dal belief state b .

1/9	1/9	1/9	0
1/9		1/9	0
1/9	1/9	1/9	1/9

$t = 1$

1/9	1/9	1/9	0
1/18		1/9	0
2/9	1/18	1/9	1/9

$t = 2$

Figura 9.8: I possibili stati sono 11. All'istante $t = 1$, il belief state è $b_1 = \langle \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, 0, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, 0 \rangle$. La probabilità di trovarsi negli stati terminali è 0, mentre $b_1([1, 1]) = 1/9$, $b_1([2, 1]) = 1/9$, etc. All'istante $t = 2$ il belief state diventa: $b_2 = \langle \frac{2}{9}, \frac{1}{18}, \frac{1}{9}, \frac{1}{9}, \frac{1}{18}, \frac{1}{9}, 0, \frac{1}{9}, \frac{1}{9}, \frac{1}{9}, 0 \rangle$, dunque abbiamo che $b_2([1, 1]) = 2/9$, $b_2([2, 1]) = 1/18$, etc.

9.3.2 Stima del belief state

Ad ogni nuova osservazione l'agente può aggiornare il suo belief state. A questo scopo viene utilizzato l'algoritmo di **filtraggio** (vedi sottosezione 7.3.2): tramite filtraggio applicato dopo ogni nuova osservazione, possiamo stimare con sempre maggiore precisione lo stato in cui ci troviamo. In aggiunta al filtraggio, in un POMDP, dobbiamo anche prendere una decisione sull'azione da intraprendere.

Considerando che siano valide le Markov assumptions (vedi sottosezione 7.1.1), possiamo ricavare una stima del nuovo belief state sfruttando la sequenza di osservazioni e azioni fatte fino all'istante corrente. Similmente all'espressione utilizzata per il filtraggio, si ottiene che il belief state aggiornato b' è dato da:

$$b'(s') = \alpha P(e|s') \sum_s P(s'|s, a) b(s)$$

dove s è lo stato precedente, a è l'azione, s' è lo stato corrente, e è l'osservazione corrente e α è un fattore di normalizzazione. Possiamo riscrivere l'espressione come un'operazione ricorsiva:

$$b' = \text{FORWARD}(b, a, e).$$

se b, a ed e sono noti, allora questo costituisce un aggiornamento deterministico del belief state.

9.3.3 Il ciclo di decisione in POMDP

Nei POMDP, l'azione ottima dipende solamente dal belief state corrente dell'agente. Il ciclo di decisione di un agente POMDP viene descritto dai seguenti passi:

1. dato il belief state corrente b , esegui l'azione $a = \pi^*(b)$ (ovvero la migliore azione possibile dato lo stato (supposto) corrente);
2. osserva la percezione e ;
3. setta come belief state corrente $b' = \text{FORWARD}(b, a, e)$ e torna al passo 1.

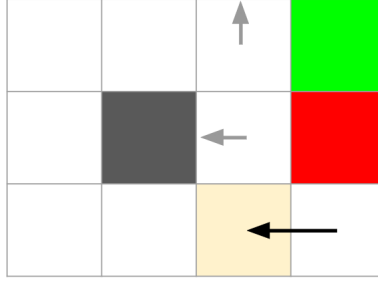


Figura 9.9: Stati con più alta probabilità dopo l'azione *Left* e sapendo che il sensore percepisce la presenza di un muro adiacente. (3, 1), (3, 2) e (3, 3) sono gli stati con la più alta probabilità; tutti gli altri stati hanno due muri adiacenti oppure sono gli stati terminali.

9.3.4 Modello di transizione e funzione di ricompensa nei POMDP

Per poter ricavare una policy ottima ci serve un modello di transizione e una funzione di ricompensa. Definiamo il **belief transition model** $P(b'|b, a)$ come la probabilità che l'agente si trovi nel belief state b' partendo dal belief state b mediante l'azione a . Per ottenere l'espressione del belief transition model dobbiamo prima calcolare la probabilità $P(e|a, b)$, ovvero la probabilità di percepire e , avendo eseguito a partendo dal belief state b . Si ha che:

$$\begin{aligned}
 P(e|a, b) &= \sum_{s'} P(e|a, s', b) P(s'|a, b) \\
 &= \sum_{s'} P(e|s') P(s'|a, b) \quad (\text{sensor Markov assumption}) \\
 &= \sum_{s'} P(e|s') \sum_s P(s'|a, b, s) P(s|a, b) \\
 &= \sum_{s'} P(e|s') \sum_s P(s'|s, a) b(s) \quad (\text{first order Markov assumption} + \text{action})
 \end{aligned}$$

Infine il belief transition model è dato dalla seguente espressione:

$$\begin{aligned}
 P(b'|b, a) &= \sum_e P(b'|e, a, b) P(e|a, b) \\
 &= \sum_e P(b'|e, a, b) \sum_{s'} P(e|s') \sum_s P(s'|s, a) b(s)
 \end{aligned}$$

dove $P(b'|e, a, b) = 1$ se $b' = \text{FORWARD}(b, a, e)$ e 0 altrimenti (in quanto FORWARD è deterministica).

La **reward function** per la transizione da un belief state ad un altro deriva dalla ricompensa attesa per le transizioni tra stati reali che potrebbero verificarsi. La ricompensa attesa se l'agente effettua l'azione a nel belief state b è

$$\rho(b, a) = \sum_s b(s) \sum_{s'} P(s'|s, a) R(s, a, s')$$

dove nella seconda sommatoria troviamo probabilità e ricompense calcolate su stati reali.

9.3.5 Policy ottima

Mettendo insieme $P(b'|b, a)$ e $\rho(b, a)$, possiamo **definire un MDP osservabile nello spazio dei belief state**. Si può dimostrare che una policy ottima $\pi^*(b)$ per questo MDP, lo è

anche per il POMDP originale. In altre parole, *la soluzione di un POMDP nello spazio degli stati reali può essere ridotta a quella di un MDP nel corrispondente spazio dei belief state*. Tuttavia, siccome lo spazio dei belief state non è finito (dato che la probabilità è continua ci sono infatti infiniti belief state possibili), non possiamo utilizzare direttamente gli algoritmi *value iteration* e *policy iteration*, in quanto una delle ipotesi era che lo spazio degli stati fosse finito. Una possibile soluzione è quella di discretizzare lo spazio dei belief state; soluzione non applicabile in generale. Un'alternativa più ragionevole (usata in pratica) è quella di risolvere i POMDP considerando delle soluzioni approssimate. Tali soluzioni si ottengono con opportune modifiche degli algoritmi per MDP già studiati.

Capitolo 10

Machine learning

10.1 Forme di apprendimento

Nei capitoli precedenti (facendo eccezione per *causal discovery* della sezione 8.6) abbiamo sempre considerato problemi il cui modello era già dato. L'agente (di qualsiasi tipologia) rispondeva a delle domande poste basandosi sul modello fornito. In questo capitolo vedremo come costruire un modello basandosi sui dati raccolti (**data set**). In generale qualsiasi componente di un programma agente (vedi sottosezione 2.3) può essere migliorata con l'apprendimento automatico; tali miglioramenti si ottengono addestrando l'agente.

Consideriamo tre tipologie di apprendimento:

- **Supervised learning**: all'agente vengono forniti degli esempi di input ed output associati; l'agente impara dagli esempi: il suo obiettivo è quello di ricavare una funzione che mappa gli input in output che sia applicabile in casi generali (cioè non solo se si presenta uno dei casi forniti come esempio).
- **Unsupervised learning**: l'obiettivo dell'agente è quello di riconoscere dei pattern tra gli input forniti senza aver ricevuto informazioni esplicite. Un esempio di applicazione è il *clustering*: riuscire ad individuare dei gruppi in cui suddividere un insieme di dati fornito come input.
- **Reinforcement learning**: l'agente viene addestrato mediante l'utilizzo di *reward* e *punishment* conferiti in base alle azioni intraprese.
- **Classification**: l'output è un valore appartenente ad un insieme finito di valori.
- **Regression**: l'output è un numero.

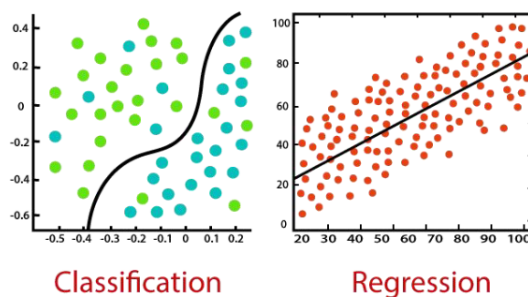


Figura 10.1: Classification e regression.

10.2 Supervised learning

Si tratta di una forma di apprendimento basata su un insieme di dati; più precisamente un insieme di coppie di input-output. Definiamo il **training set** di N esempi input-output

$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$$

dove ogni coppia è stata generata da una funzione ignota $f(x)$. I valori y_i vengono chiamati **ground truth**.

L'obiettivo dell'agente è quello di ricavare una funzione h (detta **ipotesi**) che si avvicini alla vera funzione f . In altre parole h rappresenta un modello dei dati appartenente ad una certa classe di modelli H , o una funzione appartenente ad una certa classe di funzioni.

Tra tutte le ipotesi possibili, siamo interessati alle **ipotesi consistenti**, ovvero alle ipotesi h tali per cui ogni valore x_i del training set verifica l'equazione $h(x_i) = y_i$. Quando l'output è definito su un dominio continuo, allora non possiamo aspettarci una corrispondenza perfetta, in questi casi si cerca la **funzione con il miglior adattamento (best-fit function)** per cui $h(x_i) \approx y_i$.

La qualità dell'ipotesi non viene misurata sul training set, bensì sul **test set**: un insieme di input-output che non sono presenti nel training set. L'obiettivo è quello di scoprire come si comporta l'ipotesi h su dei valori non ancora incontrati. Diciamo che h **generalizza** bene se predice in modo accurato gli output dell'insieme di test.

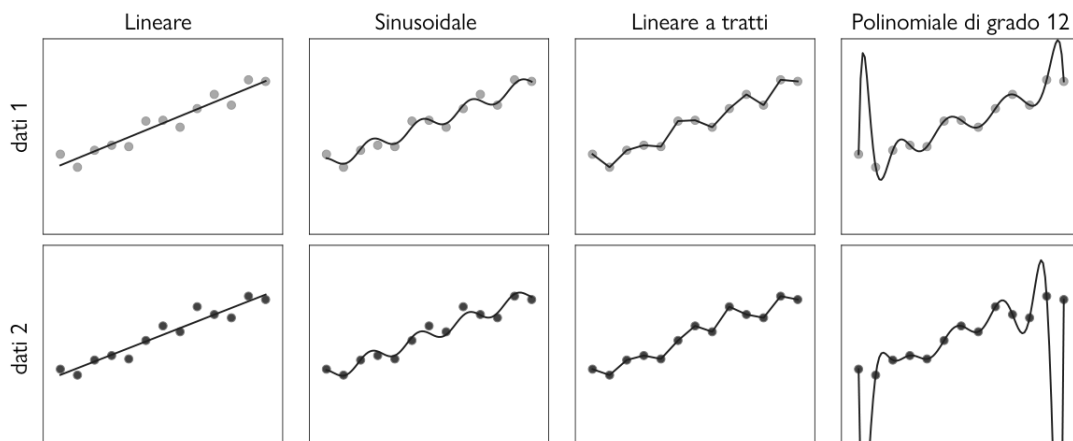


Figura 10.2: Trovare un'ipotesi per il fitting dei dati. Nelle due colonne troviamo due data set leggermente differenti: campioni ottenuti da una stessa funzione $f(x)$ ad istanti diversi.

10.2.1 Bias e underfitting

Con **bias** (**distorsione** in italiano) si intende la tendenza di un'ipotesi predittiva a deviare dal valore atteso quando si calcola la media su diversi training set. Ipotesi aventi un bias alto non si adattano molto ai dati. Un esempio di ipotesi aventi un bias alto sono le funzioni lineari (Figura 10.2 nella prima colonna): non riescono a rappresentare pattern non lineari. Viceversa le funzioni a bias basso sono le lineari a tratti: la forma della funzione è determinata dai dati. Quando un'ipotesi non riesce a trovare un pattern nei dati diciamo che presenta **sottoadattamento (underfitting)**.

10.2.2 Varianza e overfitting

Con **varianza** indichiamo la media della variazione nell'ipotesi dovuta alla fluttuazione nei dati di addestramento. In Figura 10.2 nelle prime tre colonne, notiamo che piccole variazioni

nei data set producono piccole differenze nelle ipotesi: si parla di bassa varianza. Le funzioni polinomiali della quarta colonna presentano un'alta varianza: piccole variazioni dei dati comportano grandi variazioni nelle ipotesi. Diciamo che una funzione presenta **sovradattamento (overfitting)** dei dati quando presta troppa attenzione al particolare data set su cui è addestrata, ciò causa scarse prestazioni su dati mai visti prima.

10.2.3 Compromesso bias-varianza e scelta dell'ipotesi ottima

Spesso si considera un compromesso bias-varianza che comporta una scelta fra ipotesi più complesse e a bassa distorsione che si adattano bene ai dati e ipotesi più semplici, a bassa varianza, che potrebbero fornire una migliore generalizzazione. Come si sceglie l'ipotesi migliore? L'idea è quella di scegliere l'ipotesi h^* più probabile a partire dai dati:

$$h^* = \operatorname{argmax}_{h \in H} P(h|\text{dati}).$$

Per la regola di Bayes possiamo riscrivere l'espressione come:

$$h^* = \operatorname{argmax}_{h \in H} P(\text{dati}|h)P(h).$$

Ipotesi h più semplici (es: funzione lineare) hanno una probabilità a priori $P(h)$ più alta rispetto a ipotesi più complesse (es: funzione polinomiale di grado 12); la $P(\text{data}|h)$ quantifica la probabilità di ottenere i dati del data set dall'ipotesi h . Massimizzando questo prodotto otteniamo l'ipotesi ottima.

10.2.4 Caso di studio: restaurant example

Il problema consiste nel decidere se aspettare o meno che sia disponibile un tavolo al ristorante. Nel seguito andremo a studiare diverse classi di modelli applicate a questo problema. Definiamo come output del problema y una variabile booleana chiamata *WillWait*. L'input x è un vettore di valori per 10 attributi, ognuno avente valori discreti.

1. *Alt*: se c'è un ristorante alternativo accettabile nei paraggi.
2. *Bar*: se il ristorante ha un'area bar confortevole per l'attesa.
3. *Fri/Sat*: vero di venerdì e sabato.
4. *Hun*: se siamo affamati o meno.
5. *Pat*: quante persone sono presenti nel ristorante (i valori possibili sono *None*, *Some*, *Full*).
6. *Price*: la categoria di costo del ristorante (\$, \$\$, \$\$\$).
7. *Rain*: se fuori sta piovendo.
8. *Res*: se abbiamo fatto una prenotazione.
9. *Type*: tipologia di ristorante (*French*, *Italian*, *Thai*, *Burger*).
10. *Est*: stima del tempo di attesa da parte del cameriere (0-10 min, 10-30 min, 30-60 min o > 60 min).

Il data set che abbiamo a disposizione è piccolo: conosciamo solamente 12 output corretti su un totale di $2^6 \cdot 3^2 \cdot 4^2 = 9216$ possibili valori di input. Per le 9204 possibilità non sappiamo quale siano gli output corretti, e questi possono essere ricavati basandosi solamente su 12 esempi.

Example	Input Attributes										Output
	Alt	Bar	Fri	Hun	Pat	Price	Rain	Res	Type	Est	WillWait
x ₁	Yes	No	No	Yes	Some	\$\$\$	No	Yes	French	0-10	y ₁ = Yes
x ₂	Yes	No	No	Yes	Full	\$	No	No	Thai	30-60	y ₂ = No
x ₃	No	Yes	No	No	Some	\$	No	No	Burger	0-10	y ₃ = Yes
x ₄	Yes	No	Yes	Yes	Full	\$	Yes	No	Thai	10-30	y ₄ = Yes
x ₅	Yes	No	Yes	No	Full	\$\$\$	No	Yes	French	>60	y ₅ = No
x ₆	No	Yes	No	Yes	Some	\$\$	Yes	Yes	Italian	0-10	y ₆ = Yes
x ₇	No	Yes	No	No	None	\$	Yes	No	Burger	0-10	y ₇ = No
x ₈	No	No	No	Yes	Some	\$\$	Yes	Yes	Thai	0-10	y ₈ = Yes
x ₉	No	Yes	Yes	No	Full	\$	Yes	No	Burger	>60	y ₉ = No
x ₁₀	Yes	Yes	Yes	Yes	Full	\$\$\$	No	Yes	Italian	10-30	y ₁₀ = No
x ₁₁	No	No	No	No	None	\$	No	No	Thai	0-10	y ₁₁ = No
x ₁₂	Yes	Yes	Yes	Yes	Full	\$	No	No	Burger	30-60	y ₁₂ = Yes

Figura 10.3: Dataset del restaurant example.

10.3 Decision tree

Un albero di decisione (**decision tree**) è la rappresentazione di una funzione che mappa un vettore di valori di attributi in un singolo valore in output (una decisione). La decisione viene presa effettuando una sequenza di test, partendo dalla **radice** e seguendo il ramo appropriato fino a raggiungere un nodo foglia. Ogni **nodo interno** dell'albero corrisponde a un test del valore di uno degli attributi di input; ogni **ramo** che parte da un nodo indica i possibili valori dell'attributo; le **foglie** rappresentano i valori ritornati dalla funzione (le decisioni). L'obiettivo è quello di ricavare un decision tree partendo dal data set fornito.

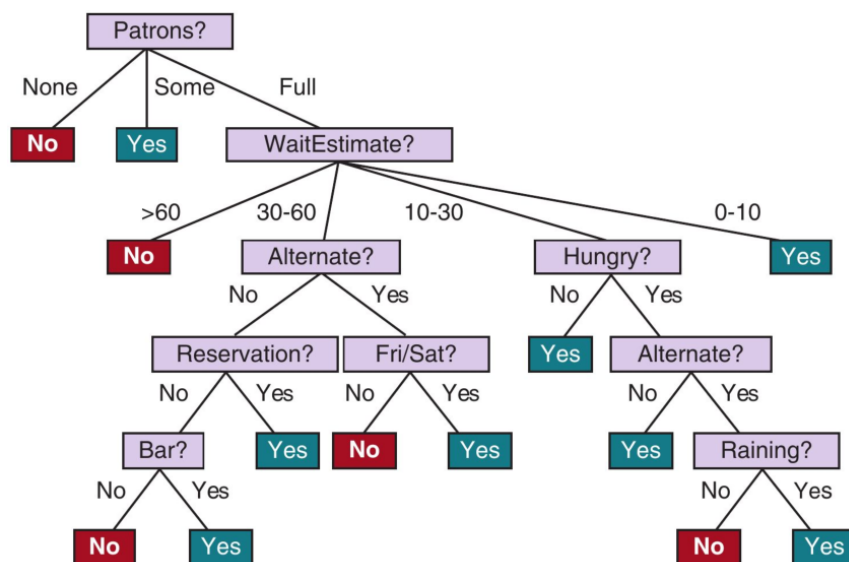


Figura 10.4: L'albero di decisione per decidere se attendere che si liberi un tavolo (ground truth).

10.3.1 Importanza degli attributi

Decision trees ordinati secondo l'importanza degli attributi sono più piccoli e quindi più efficienti. Ma come definiamo l'importanza degli attributi?

Dalla Figura 10.5 notiamo che l'attributo *Type* è poco importante: offre quattro possibili risultati, ognuno dei quali ha lo stesso numero di esempi positivi e negativi. L'attributo *Patrons* invece è più importante in quanto notiamo che se il valore è *None* o *Some*, allora abbiamo un singolo esito (*No* e *Yes* rispettivamente). Se invece il valore è *Full* abbiamo un insieme di esempi misti, in questi casi conviene considerare anche altri attributi, per esempio possiamo considerare *Hungry* e dividere ulteriormente gli esempi e vedere se i gruppi ottenuti sono tutti dello stesso tipo o meno. Procedendo ricorsivamente in questo modo si viene a formare un albero. Questa è l'idea alla base dell'algoritmo che andremo ad analizzare nella successiva sottosezione.

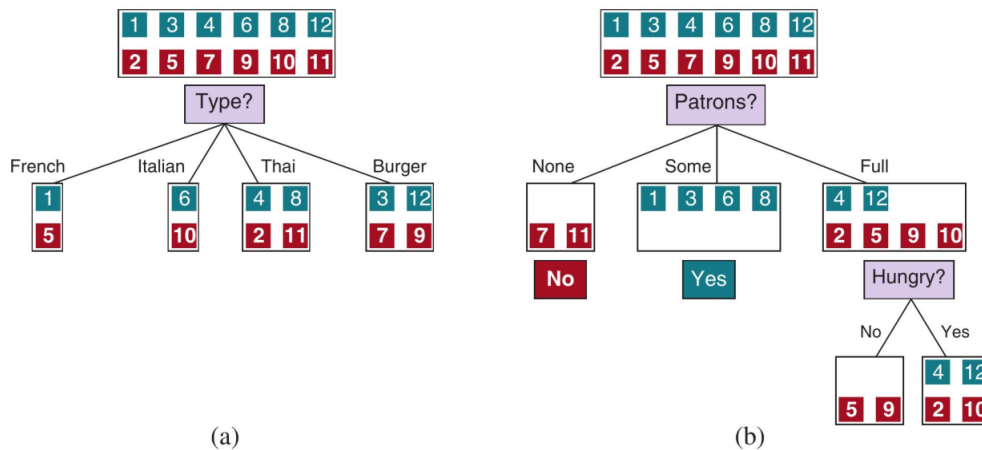


Figura 10.5: Suddivisione degli esempi mediante test sugli attributi. Le caselle verdi indicano le volte che abbiamo deciso di aspettare che si liberi un tavolo; le caselle rosse indicano che abbiamo scelto di non attendere.

10.3.2 Learning decision trees

L'algoritmo learning decision tree permette di costruire un albero di decisione. L'obiettivo è quello di costruire un albero che sia il più piccolo possibile e che sia consistente con gli esempi di training forniti. L'idea è quella di considerare (ad ogni chiamata ricorsiva) l'attributo più importante come radice di un sottoalbero. Ad ogni sottoproblema considerato si separano gli esempi (y_i) rispetto ai valori che può assumere l'attributo considerato, successivamente si controlla in quale dei seguenti quattro casi ci si trova:

1. Se gli esempi divisi per gruppi sono tutti positivi o tutti negativi (come per i valori *None* e *Some* dell'attributo *Patrons* di Figura 10.5(b)), allora abbiamo finito (abbiamo individuato una foglia): possiamo rispondere *Yes* o *No*.
2. Se ci sono esempi sia positivi che negativi, occorre scegliere il miglior attributo per suddividerli (in Figura 10.5(b) abbiamo scelto *Hungry* per dividere ulteriormente gli esempi).
3. Se non sono rimasti esempi, significa che non è stato osservato alcun esempio per questa combinazione di valori di attributi, e restituiamo il valore di output più comune dall'insieme di esempi usati nella costruzione del genitore del nodo.
4. Se non sono rimasti attributi, ma sono rimasti esempi positivi e negativi, significa che questi esempi hanno esattamente la stessa descrizione, ma classificazioni diverse. In questo caso viene restituito il valore di output più comune degli esempi rimanenti.


```

1 def LEARN-DECISION-TREE(examples, attributes, parent_examples) returns a tree {
2   if examples is empty then return PLURALITY-VALUE(parent_examples)
3   else if all examples have the same classification the return the
       classification
4   else if attributes is empty then return PLURALITY-VALUE(examples)
5   else
6     A ← argmaxa ∈ attributes IMPORTANCE(a, examples)
7     tree ← a new decision tree with root test A
8     for each value v of A do
9       exs ← {e : e ∈ examples and e.A = v }
10      subtree ← LEARN-DECISION-TREE(exs, attributes-A, examples)
11      add a branch to tree with label (A=v) and subtree
12   return tree
13 }

```

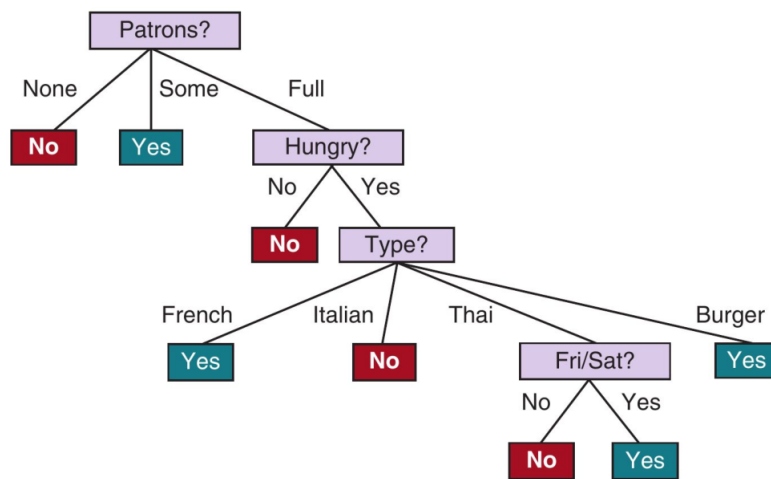


Figura 10.6: Decision tree ricavato dal training set di 12 esempi.

La Figura 10.6 mostra l'output dell'algoritmo eseguito sul training set di 12 esempi di Figura 10.3. Notiamo che l'albero indotto non corrisponde a quello di Figura 10.4: l'ipotesi h ottenuta non coincide esattamente con la funzione f , tuttavia è consistente ed è molto più semplice. Ciò è dovuto al fatto che il numero di esempi del training set è molto piccolo: se aggiungessimo ulteriori esempi otterremmo un albero diverso ma la funzione rappresentata sarebbe simile a quella ricavata.

10.3.3 La scelta dell'attributo e definizione di importanza

Abbiamo detto che l'algoritmo che ci permette di ricavare un decision tree, considera in maniera ricorsiva l'attributo più importante. Definiamo formalmente l'importanza associata agli attributi di un problema sfruttando il concetto di guadagno informativo, definito in termini di **entropia**.

L'entropia B di una variabile aleatoria booleana che assume il valore *True* con probabilità q è data da:

$$B(q) = -(q \log_2 q + (1 - q) \log_2 (1 - q)).$$

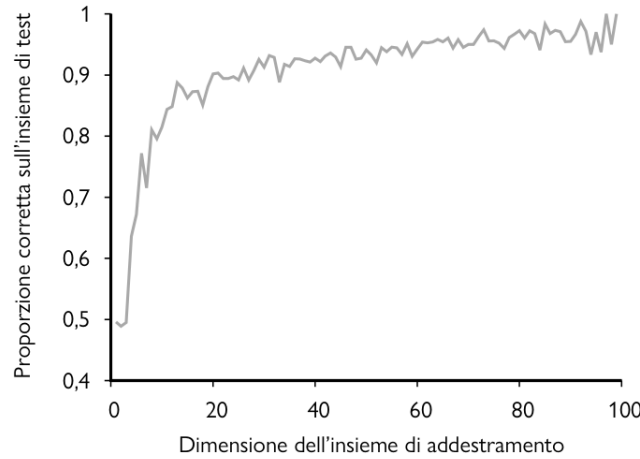


Figura 10.7: Curva di apprendimento per l'algoritmo di apprendimento degli alberi. Notiamo che la correttezza dell'ipotesi ricavata (verificata tramite test set) aumenta all'aumentare della dimensione del training set considerato.

Se un training set contiene p esempi positivi e n negativi, allora

$$q = p/(n + p) \Rightarrow B(q) = B\left(\frac{p}{p + n}\right).$$

Il training set di Figura 10.3 ha $p = n = 6$, dunque $B(q) = B(0.5) = 1 \text{ bit}$.

Un attributo A con d valori possibili, divide il training set E in sottoinsiemi $E_1, \dots, E_d$. Ogni sottoinsieme E_k ha p_k esempi positivi e n_k esempi negativi. Possiamo definire l'**entropia attesa** dopo il test sull'attributo A come:

$$\text{Remainder}(A) = \sum_{k=1}^d \frac{p_k + n_k}{p + n} B\left(\frac{p_k}{p_k + n_k}\right)$$

dove $\frac{p_k + n_k}{p + n}$ è la probabilità che un esempio scelto a caso dal training set appartenga al sottoinsieme E_k ; $B(q_k)$ è l'entropia calcolata considerando il sottoinsieme E_k . Infine calcoliamo il **guadagno informativo (information gain)** come la differenza tra l'entropia calcolata considerando il training set e l'entropia attesa data dal test sull'attributo A :

$$\text{Gain}(A) = B\left(\frac{p}{p + n}\right) - \text{Remainder}(A).$$

Più è **alto il Gain** e più è **alta l'importanza dell'attributo**.

Consideriamo l'esempio di Figura 10.5 e calcoliamoci il guadagno informativo:

$$\begin{aligned} \text{Gain}(\text{Type}) &= B(0.5) - \left[\frac{2}{12} B(0.5) + \frac{2}{12} B(0.5) + \frac{4}{12} B(0.5) + \frac{4}{12} B(0.5) \right] \\ &= 1 - \left[\frac{2}{12} + \frac{2}{12} + \frac{4}{12} + \frac{4}{12} \right] = 0 \end{aligned}$$

$$\text{Gain}(\text{Patrons}) = B(0.5) - \left[\frac{2}{12} B(0/2) + \frac{4}{12} B(1) + \frac{6}{12} B(2/6) \right] \approx 0.541$$

Dunque, come avevamo intuito, l'attributo *Patrons* è più importante di *Type*, più precisamente si tratta dell'attributo con il maggior *Gain* di tutti e quindi quello che viene scelto come radice dell'algoritmo LEARN-DECISION-TREE.

10.3.4 Generalizzazione e sovradattamento

La parte fondamentale degli algoritmi di apprendimento non prevede soltanto di trovare un'ipotesi che si adatti agli esempi forniti; si richiede inoltre di trovare una generalizzazione che abbia prestazioni buone anche con gli esempi del test dataset (diversi dagli esempi del training set). Come abbiamo visto in precedenza (Figura 10.2) una funzione polinomiale di grado 12 si adatta bene a tutti i dati, ma drastici cambiamenti si osservano se questi vengono leggermente modificati. In questi casi si parla di **sovradattamento (overfitting)**: l'ipotesi trovata si adatta molto bene ai dati del training set ma non altrettanto nel test set. Il sovradattamento diventa più probabile all'aumentare del numero di attributi e meno probabile al crescere del numero di esempi. Per contrastare il sovradattamento esistono diverse tecniche:

- *Pruning.*
- *Bagging.*
- *Random forest.*

Pruning Il pruning (*potatura*) dell'albero di decisione permette contrasta l'overfitting andando a eliminare i nodi che non sono particolarmente rilevanti. I nodi che sono meno rilevanti sono quelli che hanno un *information gain* basso. In pratica si utilizza un **test di significatività (significance test)** statistici: si parte con una **ipotesi nulla (null hypothesis)** (si pone che ϵ il suo information gain tenda a 0 per infiniti esempi); se il test ci dice che l'ipotesi nulla è statisticamente improbabile (solitamente significa avere una probabilità inferiore al 0.05), allora l'ipotesi nulla viene scartata e l'attributo si considera utile; altrimenti l'attributo viene considerato irrilevante. Le probabilità vengono calcolate basandosi sulla deviazione totale.

Bagging Il bagging permette di ridurre la varianza dell'output. Si tratta di dividere il training set in K distinti training sets generati randomicamente prendendo N esempi (possono essere scelti più di una volta) dall'training set originale. Successivamente viene fatto girare l'algoritmo LEARN-DECISION-TREE sui set individuati generando K decision trees differenti (ipotesi h_i). Per predire il valore di un input x , si vanno ad aggregare le predizioni di tutte le K ipotesi.

Random forests Si tratta del metodo più utilizzato dei tre. Il bagging produce K alberi che sono molto correlati: se uno o più attributi hanno un information gain molto alto, allora si troveranno nei livelli più bassi¹ degli alberi sempre gli stessi nodi. Random forests è simile al metodo bagging, ma invece di andare a randomizzare la scelta dei K training sets, va a randomizzare la scelta degli attributi: non prende tutti gli attributi disponibili ma prende dei sottoinsiemi. Successivamente va ad applicare l'algoritmo LEARN-DECISION-TREE considerando i K sottoinsiemi di attributi e, come per bagging, combina le ipotesi ottenute per predire l'output per un qualche valore di input x .

10.4 Selezione del modello e ottimizzazioni

Trovare una buona ipotesi significa selezionare un modello (es: decision tree oppure una funzione polinomiale, etc) e definire una fase di ottimizzazione (che dipende dai parametri del modello). Un modo per valutare la qualità di un modello è quello di andare a calcolare l'**error rate**: con quale frequenza si ha che $h(x) \neq y$ per la coppia (x, y) . Per effettuare lo studio della qualità ci servono tre tipologie di dataset:

¹La radice si trova al livello zero, i suoi figli al livello 1, e così via.

1. un **training set** per creare i modelli candidati;
2. un **validation set** per valutare e scegliere il modello migliore (fine tuning dei parametri);
3. un **test set** per effettuare una valutazione imparziale finale sul modello migliore.

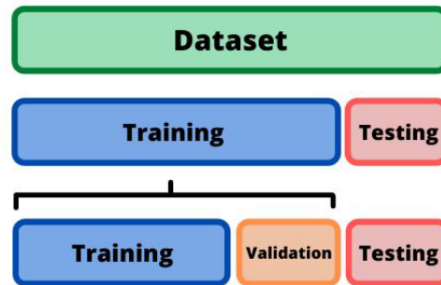


Figura 10.8

Se non abbiamo abbastanza dati per poter creare i 3 dataset necessari, allora possiamo sfruttare il **k-fold cross-validation**. Partendo dal dataset che abbiamo a disposizione, mettiamo da parte dei dati per formare il testing set. Successivamente, con i rimanenti dati, si procede in questo modo:

- si divide il dataset che si ha a disposizione in k uguali subsets s_i con $i = 1, \dots, k$;
- si effettuano k addestramenti sui k subsets in maniera iterativa;
- ad ogni iterazione, un subset s_i viene considerato come validation set mentre i rimanenti subsets formano il training set.

Così facendo il validation set cambia ad ogni iterazione. Alla fine il modello ricavato sarà dato combinando i risultati delle iterazioni. Se k coincide con la dimensione del dataset di partenza, allora si parla di **leave-one-out cross-validation**.

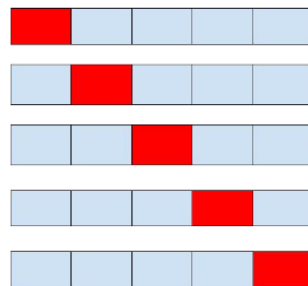


Figura 10.9: Cross validation.

10.4.1 Da tasso di errore a loss function

Gli agenti che apprendono non puntano a massimizzare la funzione di utilità ma piuttosto a minimizzare la **loss function**. La loss function $L(x, y, \hat{y})$ è definita come l'ammontare di utilità persa andando a predire $h(x) = \hat{y}$ quando invece la risposta corretta sarebbe $f(x) = y$.

$$L(x, y, \hat{y}) = \text{Utility}(\text{risultato di usare } y \text{ dato in input } x) \\ - \text{Utility}(\text{risultato di usare } \hat{y} \text{ dato in input } x)$$

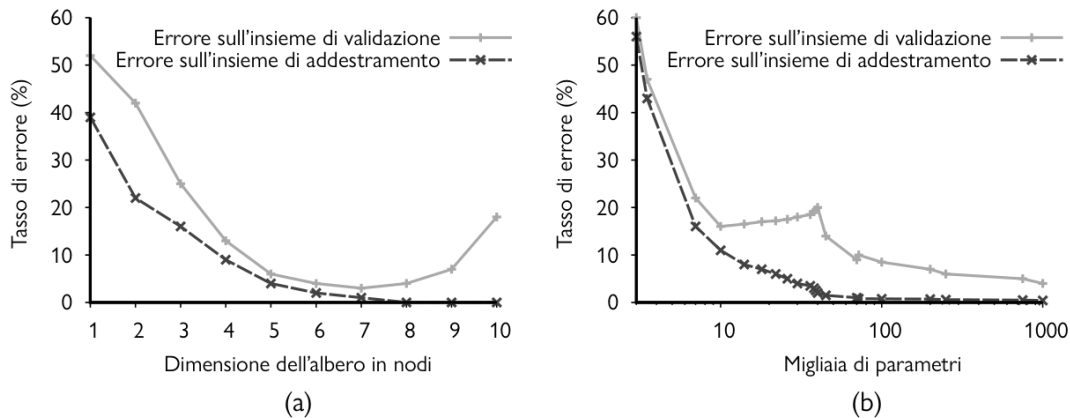


Figura 10.10: Tassi di errore su dati di addestramento e dati di validazione per modelli di diversa complessità su due problemi differenti. Viene scelto il valore dell'iperparametro con il minimo errore sull'insieme di validazione. (a) La classe dei problemi è quella degli alberi di decisione, l'iperparametro è il numero di nodi. Notiamo che la dimensione ottima è 7: oltre a tale valore, l'errore sull'insieme di validazione inizia a crescere. Ciò è dovuto al fatto che il modello che se ne ricava non riesce a generalizzare; è molto preciso solo sul training set (problema dell'overfitting). (b) La classe dei modelli è quella delle reti neurali convoluzionali e l'iperparametro è il numero di parametri ordinari nella rete. Il numero ottimo di parametri è 10^6 . Per questa classe di problemi si nota la presenza di uno scalino, passato questo si raggiunge un punto comune tra il tasso di errore nel validation set e nel training set.

Spesso si utilizza la versione semplificata $L(y, \hat{y})$ in cui non si considera x .

Consideriamo un esempio per capire meglio come interpretare la loss function. Classificare una email importante come spam è generalmente peggio di classificare una email spam come importante. L'errore in entrambi i casi è lo stesso (abbiamo sbagliato a classificare l'email) ma l'utilità non è la stessa:

$$L(\text{good}, \text{spam}) \gg L(\text{spam}, \text{good}).$$

Esistono due function loss che vengono comunemente utilizzate:

- **Absolute-value loss** indicata con L_1 : $L_1(y, \hat{y}) = |y - \hat{y}|$;
- **Squared-error loss** indicata con L_2 : $L_2(y, \hat{y}) = (y - \hat{y})^2$.

Ogni algoritmo definisce la function loss che utilizza.

10.4.2 La migliore ipotesi

Il learning agent massimizza l'aspettazione dell'utilità andando a scegliere le ipotesi che minimizzano la perdita attesa per tutte le coppie input-output che vede. Per calcolare questo valore atteso, si sfrutta la distribuzione di probabilità a priori $P(X, Y)$ sugli esempi. Definiamo ϵ l'insieme di tutti i possibili esempi di input-output (x, y) , allora la **predita di generalizzazione** attesa per una ipotesi h (rispetto alla funzione di perdita L) è data da:

$$\text{GenLoss}_L(h) = \sum_{(x, y) \in \epsilon} L(y, h(x))P(x, y)$$

la migliore ipotesi h^* è quella con la minima perdita di generalizzazione attesa:

$$h^* = \underset{h \in H}{\operatorname{argmin}} \text{GenLoss}_L(h).$$

Tuttavia solitamente $P(x, y)$ non è nota a priori e quindi l'agente può soltanto stimare la predita di generalizzazione mediante la **perdita empirica** su un insieme di esempi E di dimensione N :

$$\text{EmpLoss}_{L,E}(h) = \sum_{(x,y) \in E} L(y, h(x)) \frac{1}{N}$$

ovvero tutte le coppie hanno la stessa probabilità. In questo caso la migliore ipotesi stimata è quella con la minima perdita empirica:

$$h^* = \underset{h \in H}{\operatorname{argmin}} \text{EmpLoss}_{L,E}(h).$$

10.4.3 Gestire la complessità: regolarizzazione

La **regolarizzazione** permette di penalizzare le ipotesi più complesse a favore di quelle più semplici. Se per esempio l'ipotesi è un polinomio, allora più il grado di questo è alto e più aumenta la sua complessità. Definiamo il costo totale di una ipotesi come la somma tra la predita empirica e la complessità dell'ipotesi:

$$\text{Cost}(h) = \text{EmpLoss}(h) + \lambda \text{Complexity}(h)$$

l'ipotesi che cerchiamo è quella che minimizza la perdita empirica e che ha la minore complessità, quindi

$$\hat{h}^* = \underset{h \in H}{\operatorname{argmin}} \text{Cost}(h).$$

Il valore di λ ci permette di fissare l'importanza che diamo alla complessità (0 se non ha nessuna rilevanza). La scelta della funzione di regolarizzazione $\lambda \text{Complexity}(h)$ dipende dallo spazio delle ipotesi.

10.5 Regressione lineare

Una funzione lineare univariata (una retta) con input x e output y ha la forma

$$y = w_1 x + w_0$$

dove w_1, w_0 sono valori a coefficienti reali da apprendere. Vengono chiamati **pesi**. L'ipotesi che si ottiene è del tipo

$$h_w(x) = w_1 x + w_0.$$

Il compito di trovare un ipotesi h_w che si adatta meglio ai dati è detto **regressione lineare**. In pratica si tratta di trovare i valore dei pesi (w_0, w_1) che minimizzano la perdita empirica. Solitamente, nella regressione lineare, si utilizza come function loss la L_2 , dunque si ottiene:

$$\text{Loss}(h_w) = \sum_{j=1}^N L_2(y_j, h_w(x_j)) = \sum_{j=1}^N (y_j - h_w(x_j))^2 = \sum_{j=1}^N (y_j - (w_1 x_j + w_0))^2.$$

Siamo interessati a cercare:

$$w^* = \underset{w}{\operatorname{argmin}} \text{Loss}(h_w)$$

dove $w^* = (w_0^*, w_1^*)$. In generale, esistono due algoritmi che permettono di ricavare i pesi ottimi in maniera tale da minimizzare $\text{Loss}(h_w)$: **gradient descent** e **batch gradient descent**.

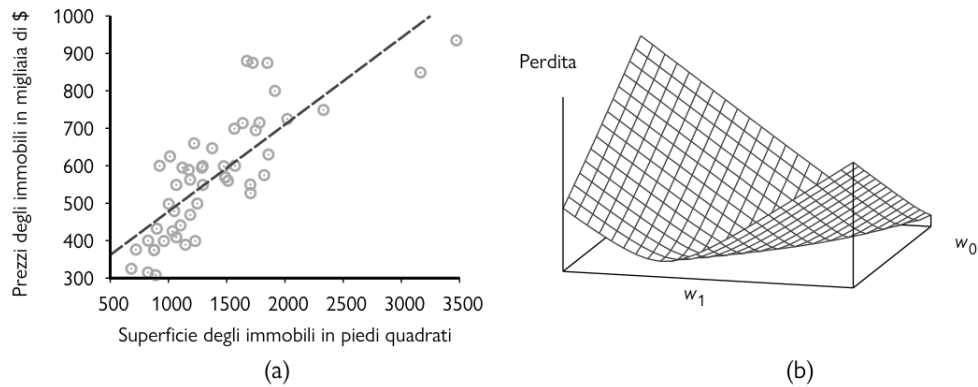


Figura 10.11: (a) Esempio di regressione lineare. (b) Grafico della funzione di perdita per vari valori di w_0, w_1 .

L'algoritmo **gradient descent**:

```

1   $\mathbf{w} \leftarrow$  any point in the parameter space
2  while not converged do
3      for each  $w_i$  in  $\mathbf{w}$  do
4           $w_i \leftarrow w_i - \alpha \frac{\partial}{\partial w_i} \text{Loss}(\mathbf{w})$ 
    
```

dove α viene chiamata **step size** o **learning rate** e può essere un valore fisso oppure può decrementare iterativamente. In pratica: si fissa un punto di partenza qualsiasi nello spazio dei pesi, che nel caso della regressione lineare significa fissare un punto nel piano (w_0, w_1) ; successivamente applichiamo una stima del gradiente e ci spostiamo di α nella direzione della discesa più ripida; si ripete il processo fino a convergere su un punto nello spazio dei pesi con perdita minima (localmente).

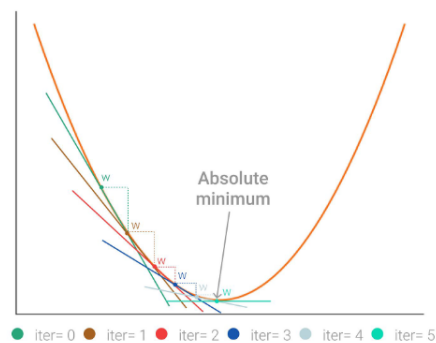


Figura 10.12: Trovare i pesi che minimizzano la loss function con gradient descent.

L'aggiornamento del gradient descent copre il caso in cui c'è un solo esempio nel nostro training set. Nel caso in cui il training set abbia N esempi, si vuole minimizzare la somma delle singole perdite per ciascun esempio. Siccome la derivata di una somma è la somma delle derivate, si ha che:

$$w_i \leftarrow w_i - \alpha \sum_j^N \frac{\partial}{\partial w_i} \text{Loss}_j$$

dove $\text{Loss}_j = L_2(y_j, h_w(x_j))$ per $j = 1, \dots, N$. Questi aggiornamenti costituiscono la regola di apprendimento del **batch gradient descent**. La convergenza al minimo locale è garantita

se α non è troppo grande (con α grande potremmo andare oltre, potremmo "sorpassarlo"), il tempo necessario per raggiungerlo potrebbe essere elevato.

Una variante più veloce è il **stochastic gradient descent (SGD)**: seleziona a caso un esempio casuale dal training set ad ogni passo. Esiste anche il **minibatch SGD**, il quale considera dei minibatch composti da m esempi su N scelti in maniera casuale. In questo modo l'aggiornamento dei pesi può essere effettuato sfruttando il calcolo in parallelo.

10.5.1 Regressione lineare multipla

Precedentemente abbiamo considerato il caso in cui l'input era una singola variabile x , ma le stesse considerazioni possiamo estenderle al caso in cui l'input sia un vettore $\vec{x}$. In questo caso l'ipotesi che si ottiene è data da:

$$h_w(\vec{x}) = \vec{w} \cdot \vec{x} = \sum_i w_i x_i.$$

Il peso ottimo è un vettore $\vec{w}^*$ che minimizza la function loss:

$$\vec{w}^* = \operatorname{argmin}_w \sum_j L_2(y_j, \vec{w} \cdot x_j).$$

10.6 Classificazione lineare

Le funzioni lineari possono essere utilizzate anche per effettuare delle classificazioni sui dati. La classificazione lineare prevede di individuare un **decision boundar**: una linea (o una superficie nel caso di più dimensioni) che separi i dati in due classi distinte.

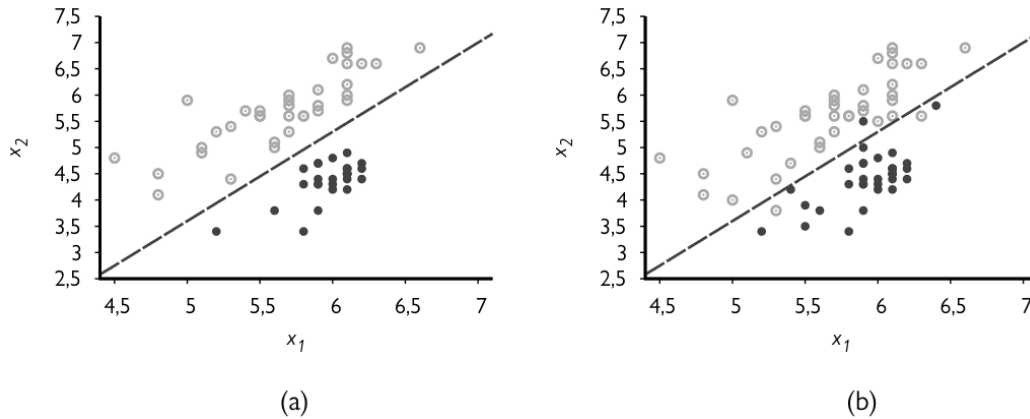


Figura 10.13: Confine di decisione lineare che individua due classi. (a) I dati sono linearmente separabili, in (b) no.

I dati che ammettono una separazione lineare sono detti **linearmente separabili**. L'ipotesi h in questo caso può essere vista come il risultato ottenuto passando la funzione lineare $\vec{w} \cdot \vec{x}$ attraverso una **funzione di soglia**:

$$h_w(\vec{x}) = \text{Threshold}(\vec{w} \cdot \vec{x}) \quad \text{dove } \text{Threshold}(z) = 1 \text{ se } z \geq 0 \text{ e } 0 \text{ altrimenti.}$$

10.6.1 Support vector machines

Le SVM hanno tre proprietà interessanti:

1. Permettono di costruire un **separatore con massimo margine**: costruiscono un confine di decisione con la massima distanza possibile dai punti di esempio. Questo permette una buona generalizzazione.
2. Creano iperpiani di separazione lineare.
3. Sono non parametriche: l'iperpiano di separazione è definito da un insieme di punti di esempio e non da una raccolta di valori di parametri.

Le SVM, invece di minimizzare la perdita empirica attesa sui dati di addestramento, cercano di minimizzare la perdita di generalizzazione attesa. Si può dimostrare che la minima perdita di generalizzazione attesa si ottiene con un separatore il più possibile lontano dagli esempi.

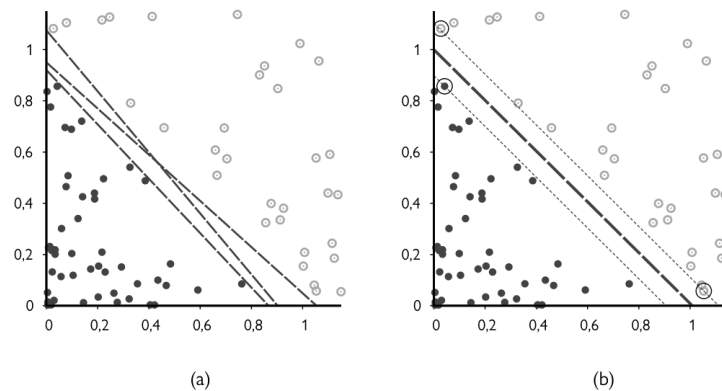


Figura 10.14: Classificazione con support vector machines. (a) Due classi di punti e tre separatori lineari candidati. Alcuni separatori passano molto vicini agli esempi e quindi può capitare che altri esempi neri vadano a finire sul lato sbagliato della retta. (b) Il separatore con massimo margine (linea più spessa) è situato a metà del margine (area compresa tra le linee sottili tratteggiate). I vettori di supporto (punti cerchiati in nero) sono gli esempi più vicini al separatore. In questo modo è molto meno probabile che altri punti neri (o bianchi) vadano a finire nel lato sbagliato della retta.

Spesso capita che dati non linearmente separabili nello spazio di input di origine, siano facilmente separabili in uno spazio con più dimensioni. Sfruttando il **kernel trick** non è necessario convertire gli input in modo tale che appartengano al nuovo spazio di dimensione maggiore.

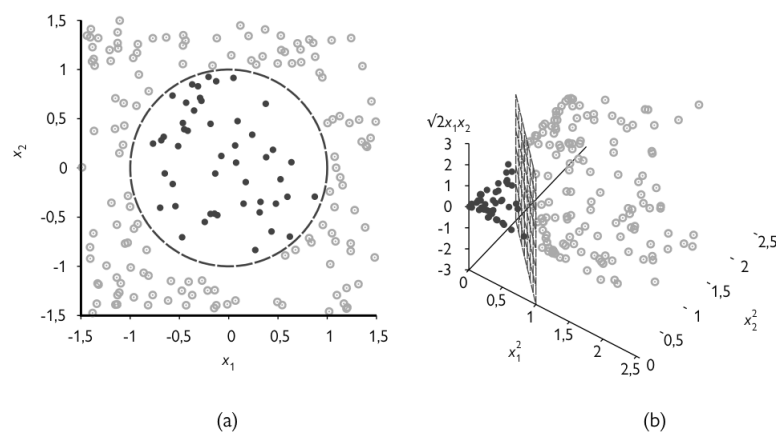


Figura 10.15: (a) Insieme di addestramento bidimensionale: i dati non sono linearmente separabili. (b) Gli stessi dati dopo la mappatura in uno spazio di input tridimensionale. Il confine di decisione che ha forma di circonferenza in (a) diventa lineare in tre dimensioni. Ciò si ottiene sfruttando il **kernel trick**.

Capitolo 11

Deep learning

11.1 Principi base

Le origini del deep learning risalgono ai primi lavori che tentarono di modellare le reti di neuroni nel cervello con circuiti computazionali. Per questo motivo, le reti addestrate mediante metodi di deep learning vengono spesso chiamate **reti neurali**, anche se la somiglianza con le vere celle e strutture neurali è soltanto superficiale.

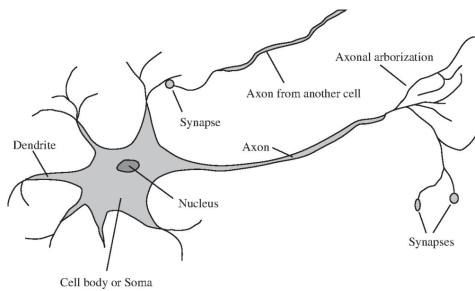


Figura 11.1: Un neurone: reagisce se stimolato dagli input forniti dalle celle adiacenti. Reagisce in modo differente a seconda degli input ricevuti.

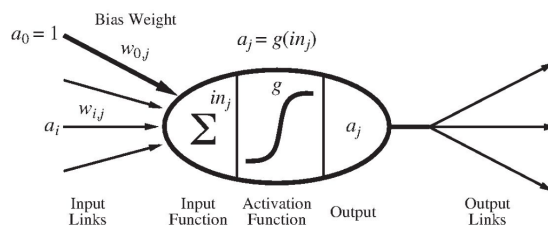


Figura 11.2: Un percettore (nel gergo viene chiamato unità **(unit)**).

Il percettore è in sostanza un neurone artificiale. Descriviamo le parti che lo compongono:

a_0 = valore fisso che funge da offset

a_i = valori in input

$w_{i,j}$ = **Bias Weight**: pesi assegnati al link che collega la unit i a alla unit j

$in_j = \sum_{i=0}^n w_{i,j} a_i$ = **Input Function**: somma tutti i valori in input

g = **Activation Function**: prende in input il valore della sommatoria

$a_j = g(in_j) = g\left(\sum_{i=0}^n w_{i,j} a_i\right)$ = **Output** dell'unit j

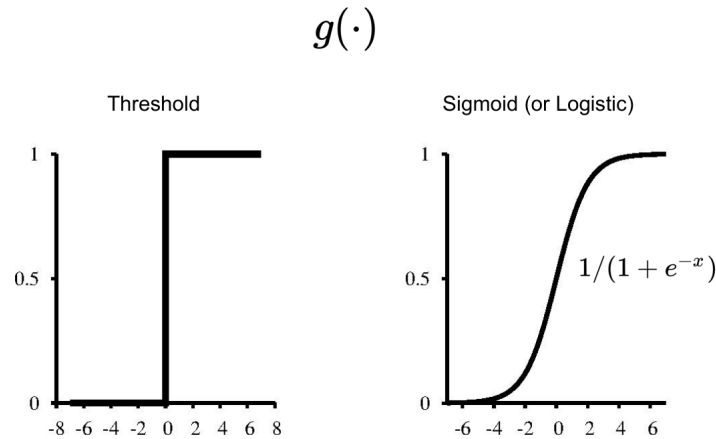


Figura 11.3: Due esempi di activation function

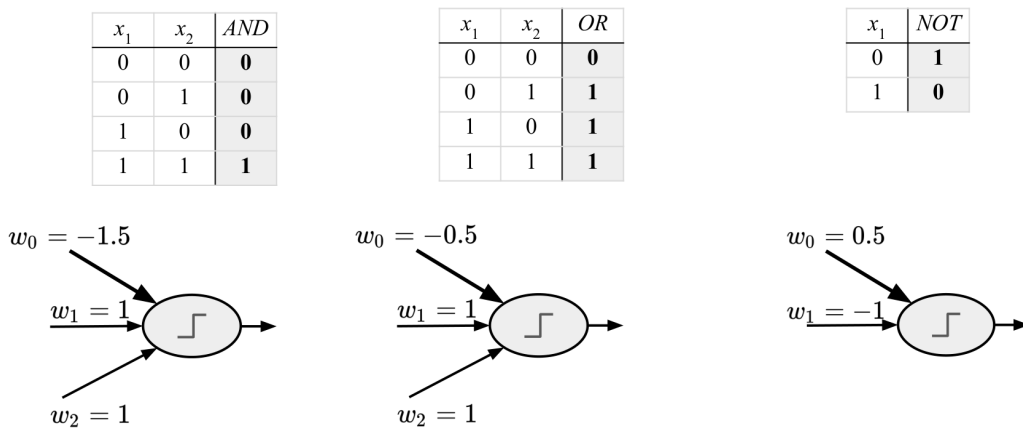


Figura 11.4: Realizzare con un perceptrone delle funzioni booleane, avendo il threshold come activation function. Nel caso della AND: solo se ho entrambi gli input a 1 ho che la sommatoria mi torna $1+1-1.5 = 0.5$ e quindi $g(0.5) > 0 \Rightarrow$ l'output è 1.

11.1.1 Separabilità lineare e non

Utilizzando come activation function il threshold, possiamo risolvere solamente problemi che sono linearmente separabili.

Notiamo in Figura 11.5 notiamo come la disequazione $w_0 + w_1x_1 + w_2x_2 > 0$ può essere riscritta rispetto alla variabile x_1 come $x_1 > -\frac{w_2}{w_1}x_2 - \frac{w_0}{w_1}$ e rappresentata sul piano come una retta che divide il piano individuando le regioni appartenenti ai due differenti output. Notiamo inoltre che il valore di w_0 permette di cambiare il valore dell'intercetta orizzontale della retta.

Per gestire i problemi che non sono linearmente separabili (come per il caso della xor di Figura 11.6) si sfruttano le **multilayer (feed-forward) neural network** (Figura 11.7).

L'output della rete si ricava come segue:

$$\begin{aligned}
 a_5 &= g(w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\
 &= g(w_{0,5} + w_{3,5}g(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2) + w_{4,5}g(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2)) \\
 &= h_w(\mathbf{x})
 \end{aligned}$$

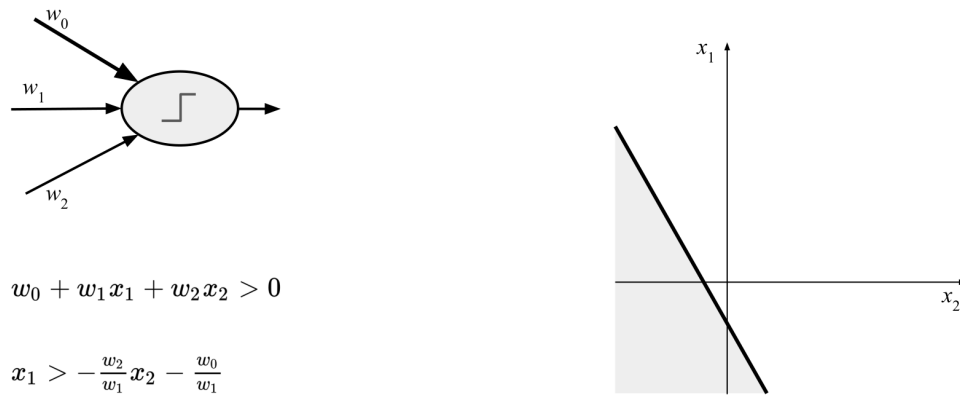


Figura 11.5: Threshold perceptron.

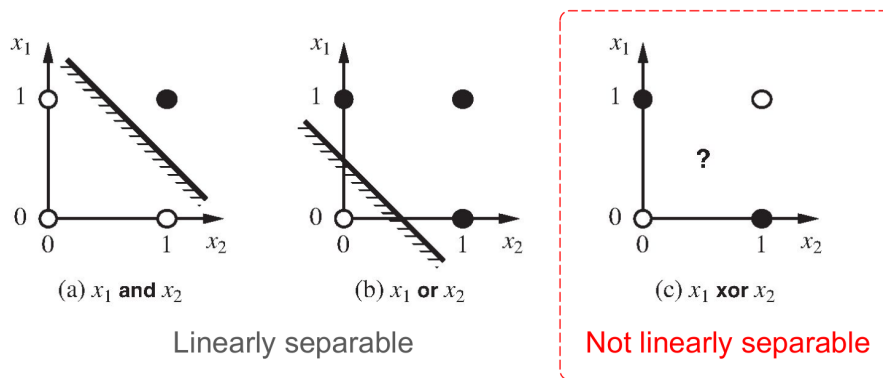


Figura 11.6: Non tutti i problemi sono linearmente separabili: x_1 xor x_2 non lo è.

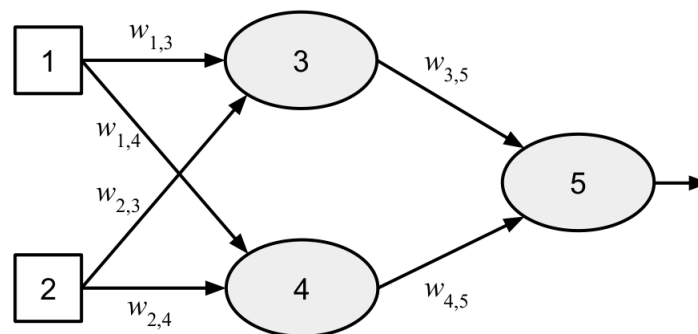


Figura 11.7: Multilayer (feed-forward) neural network. I percettroni 3 e 4 sono delle **hidden unit** che formano il cosiddetto **layer nascosto (hidden layer)**.

In generale si preferisce l'utilizzo di activation function continue e non lineari, come per esempio il **soft threshold** (Figura 11.8). Per quanto riguarda la non linearità, questa preferenza è dovuta al semplice fatto che combinazioni di funzioni lineari sono a loro volta funzioni lineari e quindi non sarebbe possibile risolvere problemi che non sono lineari. Combinando due soft threshold hidden units si può ricavare una **ridge**; combinando 2 ridges, usando 4 hidden units, possiamo ricavare un **bump**. Più in generale, l'**universal approximation theorem** ci

dice che ogni funzione continua può essere approssimata con solamente due layers.

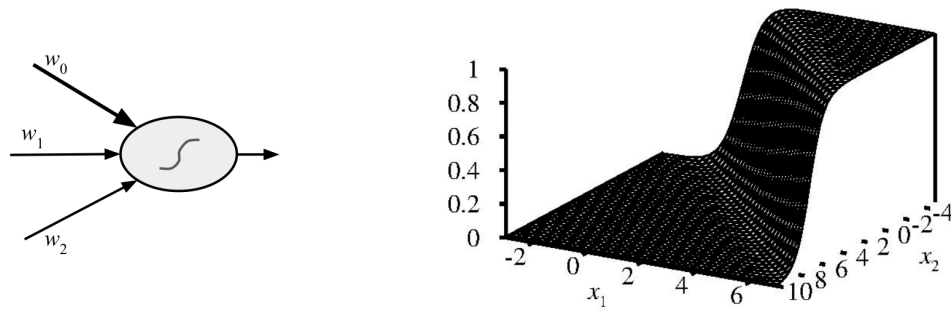


Figura 11.8: Soft threshold perceptron.

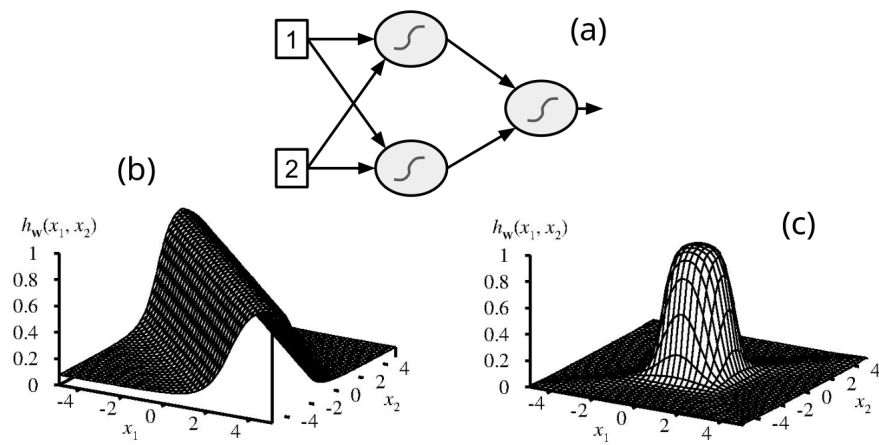


Figura 11.9: (a) Rete avente due hidden units; (b) ridge function; (c) bump function.

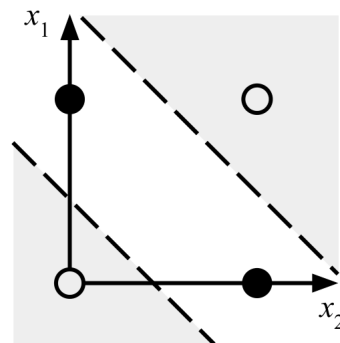


Figura 11.10: Xor function: separazione ottenuta utilizzando una multilayer neural network che realizza una ridge function.

11.2 Reti feedforward semplici

Con **feedforward** network si intendono le reti in cui "il flusso va solo in avanti", ovvero le connessioni vanno in un'unica direzione e quindi si viene a formare un grafo aciclico orientato

con nodi di input e di output definiti. Ogni nodo calcola una funzione dei suoi input e passa il risultato ai suoi successori. All'interno della rete possiamo distinguere tre tipi di layer differenti: l'**input layer**, l'**output layer** e uno o più **hidden layer** che si trovano tra input ed output. Successivamente andremo ad analizzarli uno per uno.

Esistono anche le **recurrent network (RNN)** nelle quali sono presenti dei loop: gli output intermedi o finali tornano in input. Questo meccanismo permette di creare uno **stato interno/memoria**.

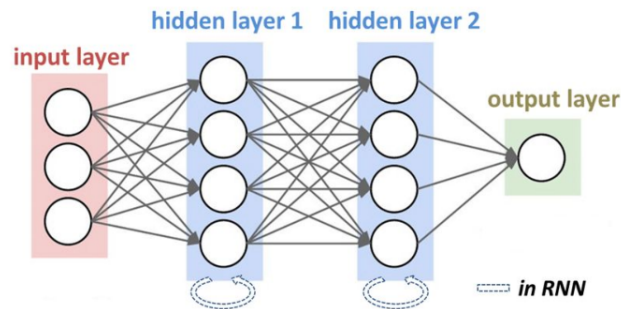


Figura 11.11: Simple feedforward network e recurrent network.

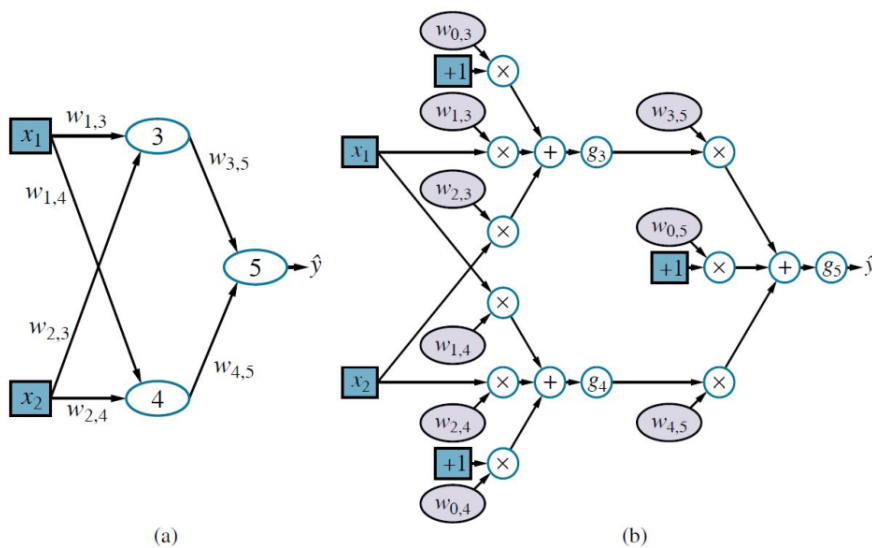


Figura 11.12: Rappresentazione delle reti neurali. (a) Rappresentazione senza input, con uno hidden layer con due units e un output (non vengono mostrati i bias, vengono sottointesi). Si tratta della rappresentazione più utilizzata. (b) La stessa rete neurale del punto (a) rappresentata da un full **computation graph**. Si tratta di una rappresentazione molto più dettagliata (ma meno utilizzata).

11.2.1 Activation functions più utilizzate

Le activation functions più popolari sono le seguenti:

- **Logistic o sigmoid:** $\sigma(x) = 1/(1 + e^{-x})$.

- **Rectifier Linear Unit (ReLU)**: $\text{ReLU}(x) = \max(0, x)$.
- **Softplus** (versione smooth di ReLU): $\text{softplus}(x) = \log(1 + e^x)$.
- **Tanh** (sigmoid scalata e traslata in $(-1, 1)$): $\text{tanh}(x) = \frac{e^{2x}-1}{e^{2x}+1}$.

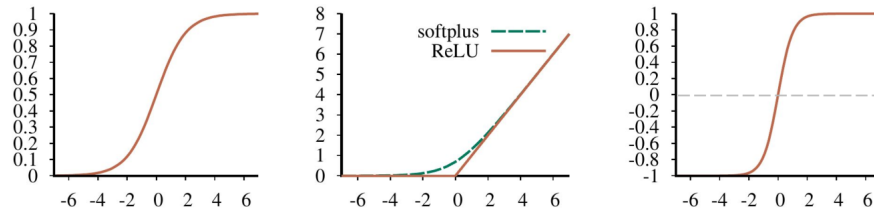


Figura 11.13: Activation functions più utilizzate. Da sinistra a destra: sigmoid; ReLU e softplus; tanh.

11.2.2 Apprendere i pesi della rete

Per ricavare i pesi della rete (fissati nella fase di training) si sfrutta una loss function $L(y, \hat{y})$ (vedi sottosezione 10.4.1) dove $\hat{y} = h_w(x)$ è l'output della predizione e y è l'output desiderato. In maniera simile alla regressione lineare, possiamo utilizzare l'algoritmo **gradient descent** per ricavare i pesi w che minimizzano l'errore. Nell'apprendimento dei pesi della rete, risulta fondamentale il concetto di **back propagation**: si tratta di un processo tramite il quale l'errore $(y - \hat{y})$ relativo all'output, viene propagato all'indietro nella rete. L'idea è la seguente:

1. si considera il layer m (che inizialmente sarà l'output layer);
2. si calcolano gli errori delle unità appartenenti al layer m ;
3. si trova quanto hanno contribuito a tale errore le units del layer $m - 1$;
4. modifico i pesi relativi alle units (per esempio abbassando il peso della unit che influisce maggiormente sull'errore);
5. ripeto dal passo (1) con $m \leftarrow m - 1$ finché non raggiungo l'input layer.

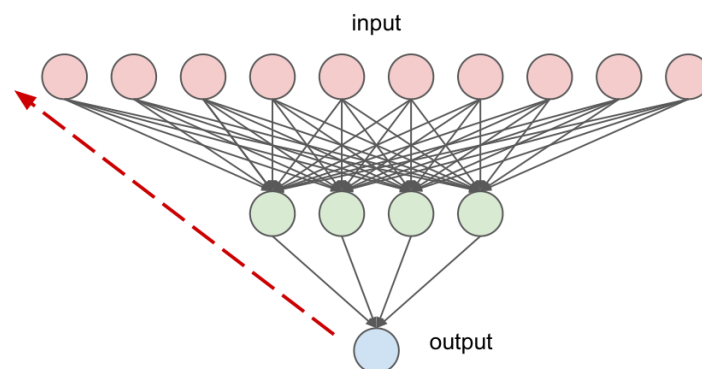


Figura 11.14: Back propagation.

11.3 Codifica degli input

I nodi di input ed output di un grafo computazionale sono quelli che sono collegati direttamente ai dati di input x e ai dati di output y . La codifica dei dati in input dipende dal tipo di dato:

- Input **booleani**: *False* viene mappato con il valore di input 0 mentre *True* viene mappato a 1. Altri valori utilizzati sono $(-1, 1)$.
- Input **numerici**: vengono utilizzati così come sono, non necessitano nessuna codifica.
- Input **categorici**: se le categorie sono più di due, si utilizza la **one-hot encoding**. In pratica, se un attributo ha d possibili valori allora viene rappresentato da d bit di input: un bit viene fissato a 1, tutti gli altri a zero. Per esempio poniamo le seguenti categorie

[French, Italian, Thai, Burger]

se l'input è la categoria *Italian*, allora setto il bit corrispondente

[0, 1, 0, 0].

11.4 Output layers e loss functions

Una scelta molto comune per i valori degli output $\hat{y}$ sono le probabilità: ci viene restituita la probabilità che l'uscita assuma un certo valore; successivamente si utilizza un qualche componente aggiuntivo che prende una decisione in base alle probabilità fornite. Rappresentare l'output come una probabilità permette di renderlo indipendente dal tipo di dato in input.

Per ricavare i pesi della rete viene minimizzata la loss function che in questo caso è la **negative log likelihood**:

$$w^* = \underset{w}{\operatorname{argmin}} \left\{ - \sum_{j=1}^N \log P_w(y_j | x_j) \right\}$$

Equivalentemente può essere riscritta come segue:

$$w^* = \underset{w}{\operatorname{argmax}} \left\{ \sum_j \log P_w(y_j | x_j) \right\} = \underset{w}{\operatorname{argmax}} \left\{ \prod_j P_w(y_j | x_j) \right\}$$

in altre parole si tratta di trovare il valore di w che massimizza la probabilità dei dati osservati.

11.4.1 Output layers per classificazione e regressione

Nel caso in cui la rete neurale viene utilizzata per risolvere problemi di **classificazione**, gli output vengono presi in ingresso dal **softmax layer**, il quale ritorna in uscita delle probabilità. Poniamo che gli output siano in totale d , allora possiamo definire il vettore (di valori) $\mathbf{in} = \langle in_1, \dots, in_d \rangle$ il quale viene fornito in input al softmax:

$$\operatorname{softmax}(\mathbf{in})_k = \frac{e^{in_k}}{\sum_{k'=1}^d e^{in_{k'}}}$$

in uscita ci viene restituito un vettore di valori non negativi che sommano a 1. Per esempio se abbiamo che $\mathbf{in} = \langle 5, 2, 0, -2 \rangle$, il softmax ci potrebbe restituire $\langle 0.946, 0.047, 0.006, 0.001 \rangle$.

Mentre nel caso di problemi di **regressione** si utilizza il **linear output layer**: l'output viene preso così com'è:

$$\hat{y}_j = in_j = \sum_i w_{i,j} a_i.$$

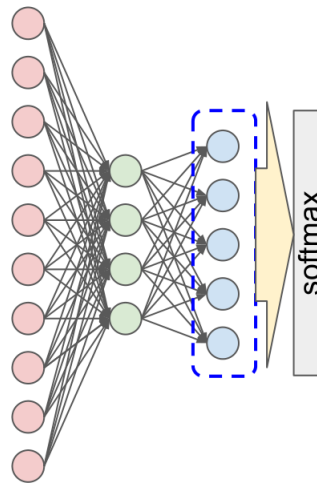


Figura 11.15: Softmax layer per problemi di classificazione.

11.5 Hidden layers

I layer nascosti (uno o più) effettuano delle computazioni intermedie che influenzano l'output y della rete. In generale, più è grande il numero di hidden layers, più è *profonda* la rete. In pratica ogni layer effettua una **trasformazione** dell'input, ottenendo in questo modo una nuova **rappresentazione** che viene passata in ingresso al layer successivo. Effettuando queste trasformazioni interne, le reti neurali spesso riescono a trovare delle rappresentazioni intermedie significative per il tipo di dati su cui lavorano.

Nota che le varie rappresentazioni non vengono definite da noi ma vengono imparate dalla rete. Prima delle reti neurali, si faceva ricerca per trovare delle rappresentazioni intelligenti dei dati da fornire in input ad algoritmi (es: SVM) per risolvere problemi di classificazione. Ad oggi non esiste questa necessità in quanto le reti neurali sono in grado imparare rappresentazioni molto più efficienti (che spesso sono controintuitive).

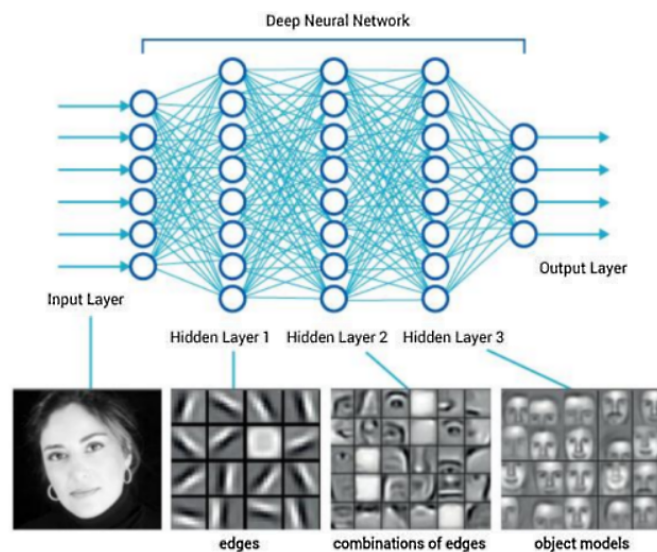


Figura 11.16: Esempio di deep neural network per la classificazione di immagini. Ogni hidden layer mappa l'input ricevuto in un nuovo spazio e ottiene in questo modo una diversa rappresentazione dei dati.

11.6 Reti convoluzionali

Le reti convoluzionali (**Convolutional Neural Networks, CNN**) sono una tipologia di deep neural network molto utilizzata.

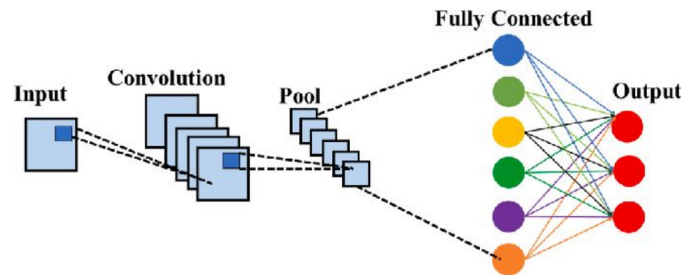


Figura 11.17: Convolutional Neural Network.

Le reti convoluzionali sfruttano il concetto di località dei dati in input. Invece di considerare i singoli pixel dell'immagine in input, consideriamo un blocco, in questo modo indichiamo che tra questi c'è una relazione: punti vicini sono più influenti di punti lontani. Per implementare una CNN si utilizzano i **kernel**: un pattern di K pesi replicato tra più regioni locali. Il processo di applicazione del kernel ai pixel dell'immagine viene chiamato **convoluzione**.

Consideriamo un vettore di input $\mathbf{x}$ di dimensione n , che corrisponde a n pixels di un'immagine monodimensionale, e un vettore kernel $\mathbf{k}$ di dimensione l (per semplicità assumiamo che l sia dispari). Definiamo l'operazione di convoluzione $z = \mathbf{x} * \mathbf{k}$ come segue:

$$z_i = \sum_{j=1}^l k_j x_{j+i-(l+1)/2}.$$

In altre parole, per ogni posizione di output i , si effettua il prodotto scalare tra il kernel $\mathbf{k}$ e una porzione di $\mathbf{x}$ centrata su x_i con ampiezza l .

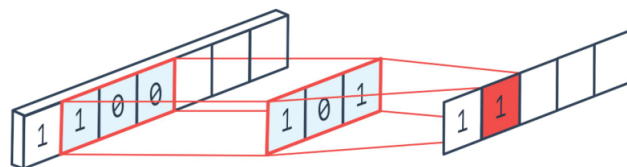


Figura 11.18: Convoluzione (applicazione del kernel).

La convoluzione può essere vista come una moltiplicazione tra matrici. I centri dei kernel sono separati da una distanza chiamata **stride** (*passo*) s . Nella Figura 11.19 notiamo che i pixel sui quali sono centrati i kernel sono separati da una distanza di 2 pixel; in questo caso il kernel è applicato con uno stride $s = 2$ (i pesi si ripetono con distanza 2 uno dall'altro). Notiamo inoltre che nella matrice dei pesi i kernel appaiono in ogni riga shiftati verso destra dello stesso valore dello stride. Lo strato di output prodotto ha meno pixel dell'ingresso: a causa del passo, il numero di pixel si riduce a circa n/s . Diciamo "circa" n/s in quanto la convoluzione si interrompe ai bordi dell'immagine. Tale problematica si può risolvere aggiungendo dei pixel extra (**padding**) all'input (con valore 0 oppure copie dei pixel più esterni), così facendo il kernel può essere applicato esattamente $\lfloor n/s \rfloor$ volte. Spesso, per kernel piccoli si utilizza $s = 1$ e quindi si ottiene un output che ha le stesse dimensioni dell'immagine in input (Figura 11.20).

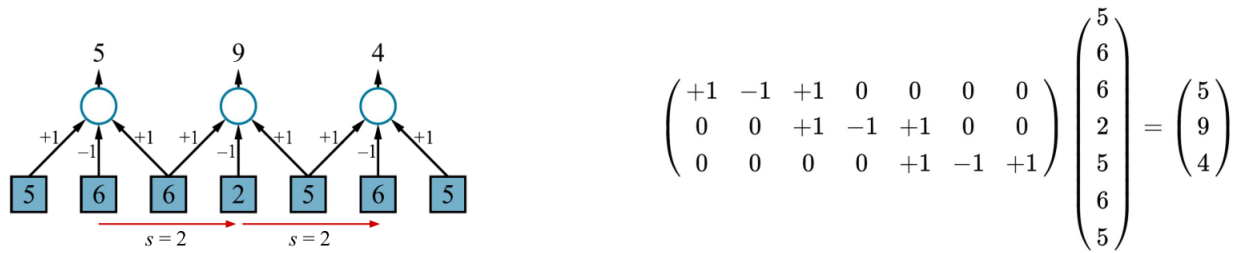


Figura 11.19: Esempio di convoluzione monodimensionale con un kernel di dimensione $l = 3$ e stride $s = 2$. I risultati in output solitamente vengono passati a una activation function non lineare (non mostrata nell'immagine) prima di passare allo strato successivo.

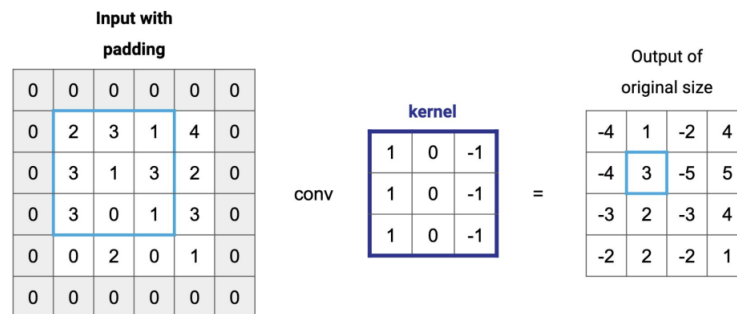


Figura 11.20: Convoluzione per un'immagine bidimensionale con kernel di dimensione $l = 3 \times 3$ e stride $s = 1$. Per risolvere il problema dei bordi è stata aggiunta all'immagine una cornice di padding: in questo modo è possibile applicare il kernel su tutti i pixel. Notiamo che i pixel in output sono $\lfloor n/s \rfloor = 16/1 = 16$.

11.6.1 Receptive field

Le CNN sono state ispirate a modelli della corteccia visiva. In questi modelli, il **campo recettivo** (**receptive field**) di un neurone è la porzione dell'input sensoriale che può determinare l'attivazione del neurone. In una CNN, il campo recettivo di un'unità nel primo strato nascosto è piccolo, è pari alla dimensione del kernel, ovvero l pixel. Negli strati più profondi può essere molto più grande.

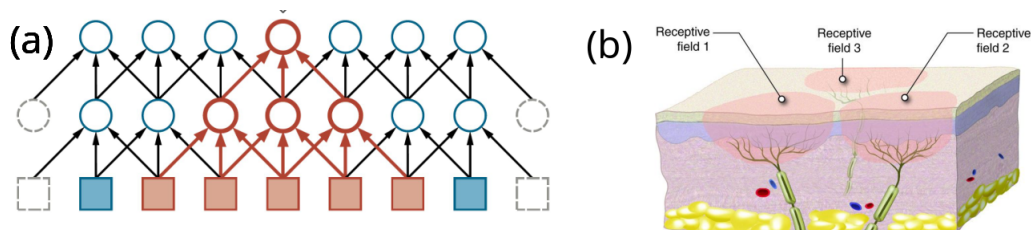


Figura 11.21: (a) I primi due strati di una rete neurale convoluzionale per un'immagine monodimensionale (1D) con kernel di dimensione $l = 3$ e stride $s = 1$. A sinistra e a destra sono stati aggiunti pixel di completamento (padding) per fare in modo che gli strati nascosti abbiano la stessa dimensione dell'input. In rosso è evidenziato il campo recettivo di una unità nel secondo strato nascosto. In generale, **più profonda è l'unità, più ampio è il campo recettivo**. (b) Modello del campo recettivo in neuroscienze.

11.6.2 Pooling e downsampling

Uno strato di **pooling** in una rete neurale riassume un insieme di unità adiacenti dello strato precedente, utilizzando un singolo valore. Può essere considerato come un caso speciale della convoluzione in cui i pesi sono fissati e non imparati. Esistono due forme comuni di pooling:

- **Average-pooling** calcola la media dei I valori di input. In pratica è come una convoluzione con un vettore kernel uniforme $\mathbf{k} = [1/I, \dots, 1/I]$. Ponendo $I = s$, l'effetto è quello di ridurre la risoluzione dell'immagine, con un **sottocampionamento** (**downsampling**) di un fattore s . L'average-pooling facilita il riconoscimento multiscala.
- **Max-pooling** calcola il massimo valore dei suoi I input. Può essere utilizzato per puro scopo di sottocampionamento.

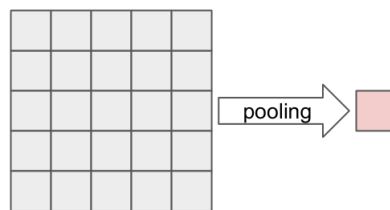


Figura 11.22: Pooling.

11.6.3 Operazioni tensoriali in CNN

I tensori sono semplicemente vettori multidimensionali di qualsiasi dimensione. Nelle CNN, i tensori permettono di tenere traccia della "forma" dei dati mentre avanzano attraverso gli strati della rete.

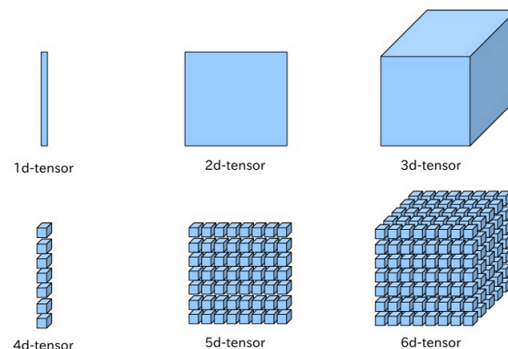


Figura 11.23: Tensori.

Consideriamo un esempio per vedere come cambia la forma dei dati in input rispetto alla forma assunta dai dati in output del primo layer nascosto. Poniamo di dover effettuare l'addestramento di una rete neurale su 64 immagini RGB di dimensione 256×256 (Figura 11.24). L'input in questo caso sarà un tensore quadridimensionale di dimensione $256 \times 256 \times \underbrace{3}_{\text{canali}} \times 64$.

Applichiamo ai nostri dati 96 kernel di dimensione $I = 5 \times 5 \times 3$ (ognuno dei quali elabora

una sottoimmagine di 5×5 pixel) con uno stride $s = 2$ in entrambe le direzioni x e y nell'immagine. Applicare un kernel di dimensione l ad una immagine di 256×256 pixel, mi produce in uscita un'immagine di $\frac{n}{s} \times \frac{n}{s} = \frac{256}{2} \times \frac{256}{2} = 128 \times 128$ pixel. Applicando tutti i 96 kernel a tutte le 64 immagini, ottengo in output un tensore di dimensione $128 \times 128 \times 96 \times 64$. Un tale tensore viene spesso chiamato **feature map** (mappa delle caratteristiche): mostra come ogni caratteristica estratta da un kernel appare attraverso l'intera immagine; in questo caso è composto da 96 canali, ognuno dei quali contiene informazioni da una sola caratteristica.



Figura 11.24: 64 immagini RGB di 256×256 pixel. Per applicare i kernel che processano ognuno sottoimmagini di 5×5 pixel, è necessario aggiungere un padding di 2 pixel.

11.7 Algoritmi di apprendimento

Addestrare una rete neurale consiste nel modificare i parametri della rete in modo da minimizzare la funzione di perdita sull'insieme di addestramento. Quasi tutte le reti moderne utilizzano un algoritmo di addestramento che è una qualche variante del **stochastic gradient descent (SGD)** (vedi sezione 10.5). Lo scopo è quello di minimizzare $L(\mathbf{w})$, dove $\mathbf{w}$ rappresenta tutti i parametri della rete. Ogni passo di aggiornamento nel processo di discesa del gradiente (*gradient descent*) appare come segue:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} L(\mathbf{w})$$

dove α è il tasso di apprendimento.

11.7.1 Calcolo di gradienti nei grafi computazionali

Per applicare il stochastic gradient descent, dobbiamo calcolare il gradiente $\frac{\partial L}{\partial w}$ per ogni peso w . Tale calcolo viene effettuato tramite **back-propagation** in due fasi distinte:

- Per prima cosa viene effettuata una **forward-pass**, nella quale ogni nodo calcola una qualche funzione arbitraria (prodotto, somma, activation function) dei dati in input che gli arrivano da altri nodi. Si parte dagli input della rete e si arriva a ricavare l'output.
- Il calcolo del gradiente viene effettuato durante il **backward-pass**, propagando l'errore dall'output layer della rete verso gli hidden layers. Quindi si parte dall'output della rete, si calcola l'errore (comparando l'output ottenuto con ciò che ci si aspettava) e si effettua un aggiornamento dei pesi di tutti gli strati nascosti.

Il back-propagation può essere visto (a livello algoritmico) come un passaggio all'indietro di messaggi attraverso ogni arco del grafo computazionale. Questi messaggi non sono altro che le derivate parziali della loss L (Figura 11.26). Per esempio, il messaggio all'indietro $\partial L / \partial h_j$ è la derivata parziale di L rispetto al primo input di j , che è il messaggio passato in avanti da h a j . Ora h influisce su L tramite j e k , perciò si ha che

$$\frac{\partial L}{\partial h} = \frac{\partial L}{\partial h_j} + \frac{\partial L}{\partial h_k}.$$

Ovvero il nodo h calcola la derivata di L rispetto ad h sommando i messaggi in arrivo da j e k . In generale, ogni nodo effettua la somma dei messaggi backward che gli arrivano. Ora, per calcolare i messaggi in uscita $\partial L / \partial f_h$ e $\partial L / \partial g_h$, utilizziamo le equazioni seguenti:

$$\frac{\partial L}{\partial f_h} = \frac{\partial L}{\partial h} \frac{\partial h}{\partial f_h}, \quad \frac{\partial L}{\partial g_h} = \frac{\partial L}{\partial h} \frac{\partial h}{\partial g_h}$$

dove $\partial L / \partial h$ è già stata calcolata in precedenza e $\frac{\partial h}{\partial f_h}$, $\frac{\partial h}{\partial g_h}$ sono semplicemente le derivate parziali di h rispetto ai suoi due argomenti. Per esempio, se h fosse un nodo di moltiplicazione (ovvero $h(f, g) = f \cdot g$), allora $\frac{\partial h}{\partial f_h} = g$, $\frac{\partial h}{\partial g_h} = f$. Il processo di retropropagazione inizia dal nodo di output, dove il messaggio iniziale è $\partial L / \partial \hat{y}$, e termina in ogni nodo del grafo computazionale che rappresenta un peso w (nodi in grigio di Figura 11.25).

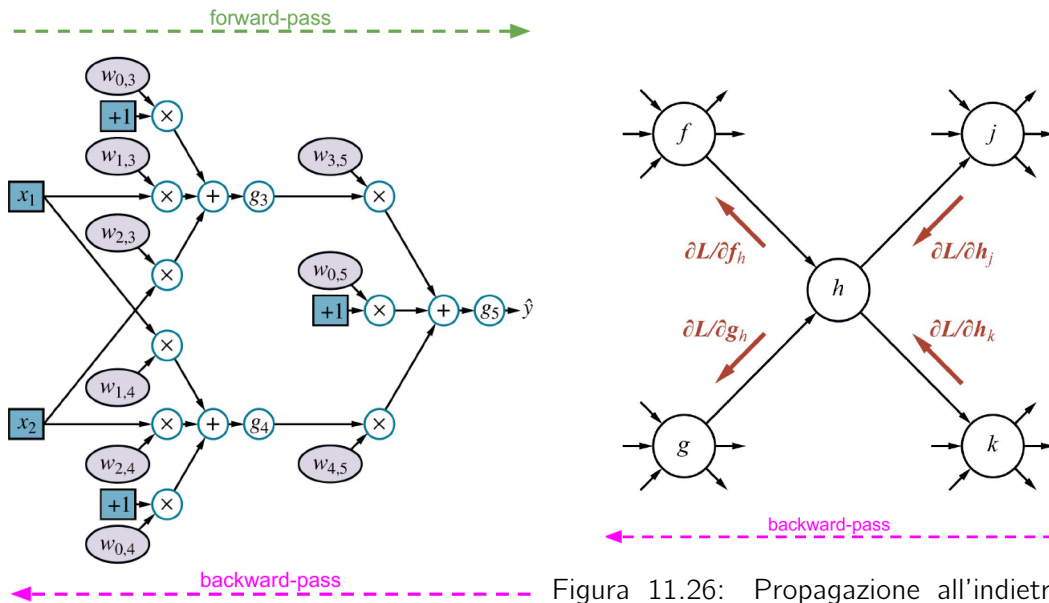


Figura 11.26: Propagazione all'indietro dei messaggi attraverso gli archi della rete.

Figura 11.25: Forward-pass e backward-pass.

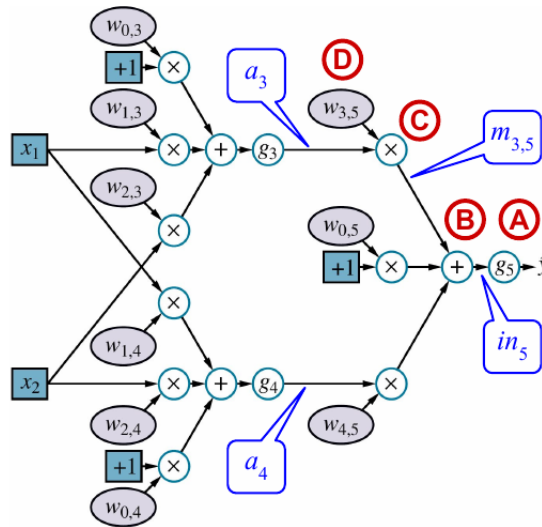


Figura 11.27: Esempio: calcolo di $\partial L / \partial w$ con L funzione squared loss.

Consideriamo l'esempio di Figura 11.27: vediamo come in effetti il back-propagation riesce a calcolare l'equazione corretta per un particolare gradiente. Poniamo che $L = (y - \hat{y})^2$ (squared loss). Calcoliamo analiticamente la derivata parziale della loss per il peso $w_{3,5}$:

$$\begin{aligned}
 \frac{\partial}{\partial w_{3,5}} \text{Loss}(h_w) &= \frac{\partial}{\partial w_{3,5}} (y - \hat{y})^2 \\
 &= -2(y - \hat{y}) \frac{\partial \hat{y}}{\partial w_{3,5}} \\
 &= -2(y - \hat{y}) \frac{\partial}{\partial w_{3,5}} g_5(in_5) \\
 &= -2(y - \hat{y}) g'_5(in_5) \frac{\partial}{\partial w_{3,5}} in_5 \\
 &= -2(y - \hat{y}) g'_5(in_5) \frac{\partial}{\partial w_{3,5}} (w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4) \\
 &= -2(y - \hat{y}) g'_5(in_5) a_3
 \end{aligned}$$

Calcoliamo la stessa derivata parziale ma utilizzando il metodo algoritmico:

$$\begin{aligned}
 A) \quad \frac{\partial L}{\partial g_5} &= \frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} (y - \hat{y})^2 = -2(y - \hat{y}) \\
 B) \quad \frac{\partial L}{\partial in_5} &= \frac{\partial L}{\partial g_5} \frac{\partial g_5}{\partial in_5} = \frac{\partial L}{\partial g_5} g'_5(in_5) \\
 C) \quad \text{Poniamo } m_{3,5} &= w_{3,5}a_3 \Rightarrow in_5 = m_{3,5} + w_{0,5} + w_{4,5}a_4 \\
 \frac{\partial L}{\partial m_{3,5}} &= \frac{\partial L}{\partial in_5} \frac{\partial in_5}{\partial m_{3,5}} = \frac{\partial L}{\partial in_5} \\
 D) \quad \frac{\partial L}{\partial m_{3,5}} \frac{\partial m_{3,5}}{\partial w_{3,5}} &= \frac{\partial L}{\partial m_{3,5}} a_3 = -2(y - \hat{y}) g'_5(in_5) a_3 \quad \text{Siccome } \frac{\partial m_{3,5}}{\partial w_{3,5}} = a_3
 \end{aligned}$$

come volevasi dimostrare, abbiamo ottenuto lo stesso risultato anche con il metodo proposto dall'algoritmo.

Lo **svantaggio** principale dell'algoritmo back-propagation è che richiede la memorizzazione di molti valori intermedi (come i valori di output a di ogni unit) che sono stati calcolati durante il forward-pass e che sono necessari per il calcolo del gradiente durante il backward-pass.

11.8 Generalizzazione

Alcune architetture di reti neurali sono state progettate appositamente per generalizzare bene su particolari tipi di dati. Uno dei risultati più importanti nel campo del deep learning è che, quando si confrontano due reti con un numero di pesi simile, la rete più profonda solitamente generalizza meglio (Figura 11.28).

11.8.1 Dropout

Anche se le reti neurali generalizzano bene, non sono tuttavia immuni da errori. Potrebbero verificarsi casi in cui una rete che prende in input un'immagine di un autobus contenente qualche pixel errato (*noise*), la classifichi come un'immagine di un tacchino.

Per risolvere questi ed altre tipologie di problemi, si può sfruttare la tecnica del **dropout**. L'idea è quella di disabilitare alcune units durante la fase di addestramento, in questo modo rendiamo le rimanenti units più resistenti alla presenza del rumore. Utilizzare la tecnica del dropout su layer che sono vicini all'output rende quest'ultimo dipendente da molti più receptive fields.

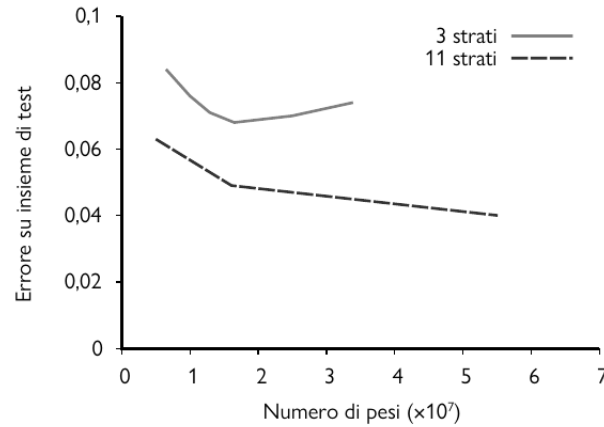


Figura 11.28: Errore su insieme di test in funzione dell'ampiezza degli strati per reti convoluzionali a tre strati e a undici strati. Notiamo come la rete a 3 strati soffre del problema dell'overfitting.

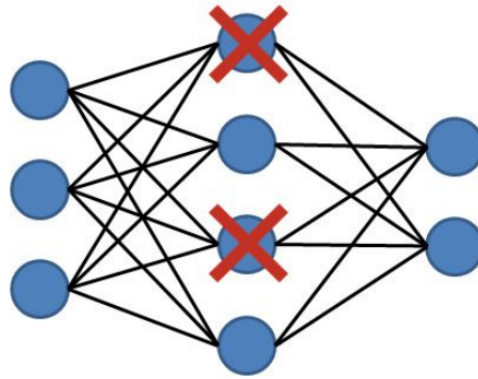


Figura 11.29: Dropout.

11.9 Reti neurali ricorrenti

Le **Recurrent Neural Networks (RNN)** vengono rappresentate come mostrato in Figura 11.30.

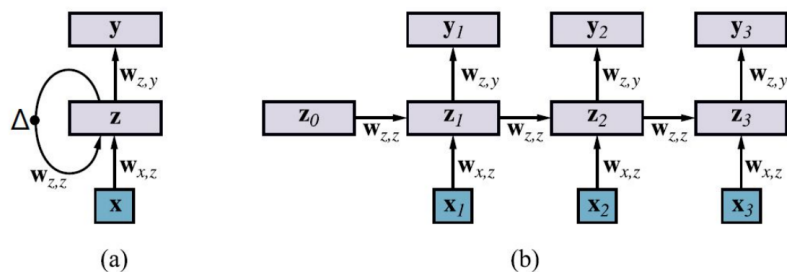


Figura 11.30: (a) Diagramma schematico di una RNN di base, in cui lo strato nascosto z ha connessioni ricorrenti; il simbolo Δ indica un ritardo. (b) La stessa rete ma srotolata su tre passi temporali per creare una rete feedforward. Nota che i pesi sono condivisi su tutti i passi temporali.

Le RNN permettono cicli all'interno del grafo computazionale. In questa tipologia di reti neurali, l'input di alcune unità può essere il loro stesso output calcolato allo step precedente. Come abbiamo anticipato in precedenza, i cicli all'interno delle reti neurali permettono di creare uno **stato interno/memoria**.

Le reti neurali ricorrenti, come strumenti per l'analisi di dati sequenziali, possono essere comparate agli Hidden Markov Models. Come per gli HMM, queste reti neurali fanno una **Markov assumption**: lo stato nascosto z_t della rete è sufficiente a catturare le informazioni fornite da tutti gli input precedenti. In altre parole, lo stato all'istante z_{t+1} dipende solamente dallo stato all'istante precedente z_t e dall'input x_{t+1} . In generale:

$$z_t = f_w(z_{t-1}, x_t).$$

11.9.1 Addestramento di una RNN

Consideriamo il caso in cui abbiamo un layer di input $\mathbf{x}$, un hidden layer $\mathbf{z}$ avente delle connessioni ricorsive e un output layer $\mathbf{y}$ (Figura 11.30). Le equazioni che descrivono il modello fanno riferimento ai valori delle variabili indicizzati al passo temporale t :

$$\begin{aligned} \mathbf{z}_t &= f_w(\mathbf{z}_{t-1}, \mathbf{x}_t) = \mathbf{g}_z(\mathbf{W}_{z,z}\mathbf{z}_{t-1} + \mathbf{W}_{x,z}\mathbf{x}_t) \equiv \mathbf{g}_z(\text{in}_{z,t}) \\ \hat{\mathbf{y}}_t &= \mathbf{g}_y(\mathbf{W}_{z,y}\mathbf{z}_t) \equiv \mathbf{g}_y(\text{in}_{y,t}) \end{aligned}$$

dove g_z e g_y sono le activation functions rispettivamente per gli hidden e output layers. Per semplificare i conti consideriamo un'unica unità per ogni layer (per non dover lavorare con i vettori). Possiamo calcolare il gradiente per l'unità nascosta z_t sfruttando il passo temporale precedente:

$$\frac{\partial z_t}{\partial w_{z,z}} = g'_z(\text{in}_{z,t}) \left(z_{t-1} + w_{z,z} \frac{\partial z_{t-1}}{\partial w_{z,z}} \right).$$

Notiamo che $\partial z_{t-1} / \partial w_{z,z}$ individua una ricorrenza, ciò significa che il contributo del gradiente al passo temporale t viene calcolato utilizzando il contributo del passo temporale $t - 1$. Il tempo totale necessario per calcolare il gradiente è lineare nella dimensione della rete.

Nelle RNN, si dimostra che:

- se $w_{z,z} < 1$, allora una semplice RNN può soffrire del problema del **vanishing gradient problem**: mancanza di aggiornamenti significativi dei pesi nella fase di training;
- se $w_{z,z} > 1$, può soffrire del problema del **exploding gradient problem**.

11.9.2 RNN con LSTM

Esistono diverse architetture di RNN specializzate con l'obiettivo di consentire di preservare le informazioni attraverso più passi temporali. Una delle più popolari è chiamata **LSTM** (*Long Short-Term Memory*). In una LSTM, il componente di memoria a lungo termine, chiamato **cella di memoria** e denotato da c , è in sostanza copiato da un passo temporale all'altro (in una RNN di base, invece, la memoria viene moltiplicata per una matrice di pesi a ogni passo temporale)

Le LSTM includono inoltre un **unità di porta (gating units)**: dei vettori che controllano il flusso di informazioni nella LSTM attraverso una moltiplicazione elemento per elemento dei corrispondenti vettori di informazioni. In sostanza i vettori permettono di regolare come la memory cell viene aggiornata nel tempo.

- **Forget gate f**: determina se ogni elemento della memory cell deve essere ricordato (copiato nel successivo passo temporale) o dimenticato (riportato a zero).

- **Input gate i**: determina se ogni elemento della cella di memoria è aggiornato in modo additivo da nuove informazioni provenienti dal vettore di input al passo temporale corrente.
- **Output gate o**: determina se ogni elemento della cella di memoria è trasferito nella memoria a breve termine $\mathbf{z}$, che svolge un ruolo simile a quello dello stato nascosto delle RNN di base.

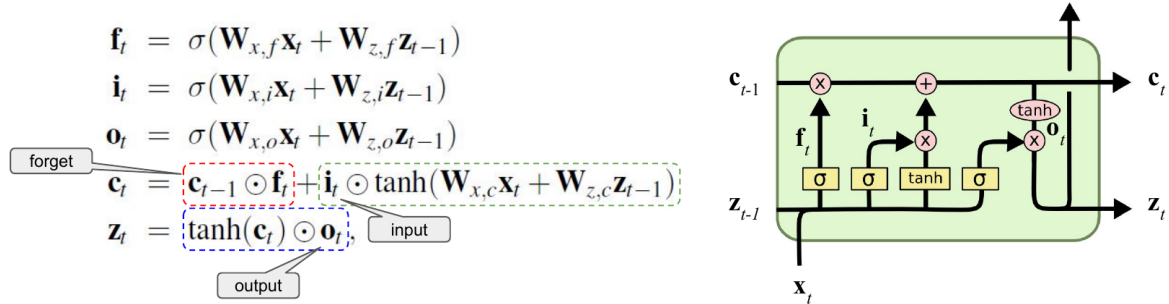


Figura 11.31: Equazioni di aggiornamento per LSTM.

11.10 Apprendimento non supervisionato

Fino ad adesso (compreso il capitolo precedente) abbiamo sempre lavorato con l'apprendimento supervisionato in cui ad ogni input $\mathbf{x}$ era associato un output $\mathbf{y}$ e l'obiettivo era quello di apprendere una funzione che mappasse gli input in output.

Nel caso dell'**unsupervised learning**, il training set è composto da esempi $\mathbf{x}$ privi di etichetta e gli output $\mathbf{y}$ non sono presenti. Non è necessario che gli esempi forniti siano classificati e non serve nemmeno specificare le categorie per la classificazione: il modello che andiamo ad addestrare si occuperà lui di individuare le categorie ed assegnerà a ciascuna un'etichetta arbitraria.

L'apprendimento non supervisionato viene utilizzato per due scopi principali:

1. per apprendere **nuove rappresentazioni**: caratteristiche chiave di un'immagine che aiutano la classificazione;
2. per apprendere **modelli generativi (generative models)**: modelli che ricavano distribuzioni $P(\mathbf{x})$ e utilizzano questa informazione per generare nuovi esempi.

11.10.1 Autoencoders

Gli **autoencoders (AE)** sono un particolare modello non supervisionato che comprende:

- un **encoder** f che mappa gli input $\mathbf{x}$ in una rappresentazione $\hat{\mathbf{z}}$ (*bottleneck*);
- un **decoder** g che mappa $\hat{\mathbf{z}}$ a dati osservati $\mathbf{x}$

$$\mathbf{x} \approx g(f(\mathbf{x})).$$

Un esempio di autoencoder è il **linear autoencoder**, nel quale sia f che g sono funzioni lineari che condividono una matrice dei pesi $\mathbf{W}$:

$$\begin{aligned} \hat{\mathbf{z}} &= f(\mathbf{x}) = \mathbf{W}\mathbf{x} \\ \mathbf{x} &= g(\hat{\mathbf{z}}) = \mathbf{W}^T \hat{\mathbf{z}} \end{aligned}$$

Una possibile loss function per addestrare un autoencoder è la seguente:

$$\sum_j \|x_j - g(f(x_j))\|^2$$

che sarebbe l'equivalente della funzione di loss L_2 che abbiamo utilizzato nel caso supervisionato.

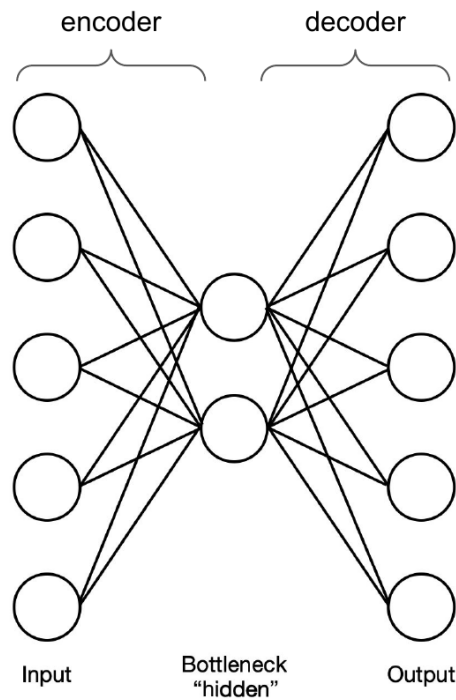


Figura 11.32: Autoencoder (AE).

Esistono applicazioni in cui, una volta addestrata la rete, viene rimosso il layer dell'decoder e viene sostituito con una rete neurale differente. L'idea è quella di addestrare l'encoder per poi utilizzarlo per altre applicazioni, per esempio per la compressione dei dati. Altre applicazioni invece sfruttano il decoder per generare nuovi dati, un esempio di utilizzo è quello relativo al rilevamento di anomalie.

11.10.2 Variational autoencoder

I **Variational Autoencoder (VAE)** sono degli autoencoder più complessi che permettono di realizzare dei modelli generativi. In sostanza sono la versione probabilistica degli AE che permette la generazione di nuovi esempi.

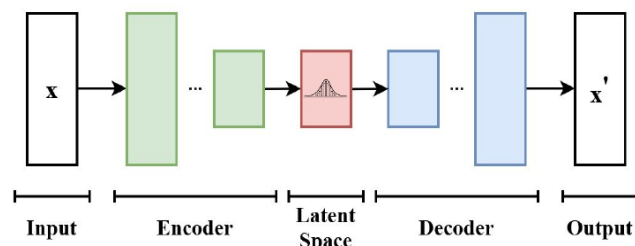


Figura 11.33: Variational autoencoder (VAE).

Mentre negli AE gli input vengono codificati in punti $\mathbf{z}$ dello spazio latente, nei VAE vengono codificate le distribuzioni di probabilità $P(\mathbf{z}|\mathbf{x})$ (per esempio gaussiane). Addestrare questa tipologia di reti consiste nel massimizzare una **evidence lower bound (ELBO)**:

$$L(\mathbf{x}, Q) = \log P(\mathbf{x}) - D_{KL}(Q(\mathbf{z})||P(\mathbf{z}|\mathbf{x}))$$

dove $Q(\mathbf{z})$ è la distribuzione a posteriori variazionale che approssima $P(\mathbf{z}|\mathbf{x})$; D_{LK} è una distanza statistica calcolata su due distribuzioni di probabilità. Se $D_{LK} \approx 0$ allora significa che $Q(\mathbf{z})$ è una buona approssimazione.

11.10.3 VAEs vs AEs

Mettiamo a confronto i due modelli: quale dei due ha prestazioni migliori?

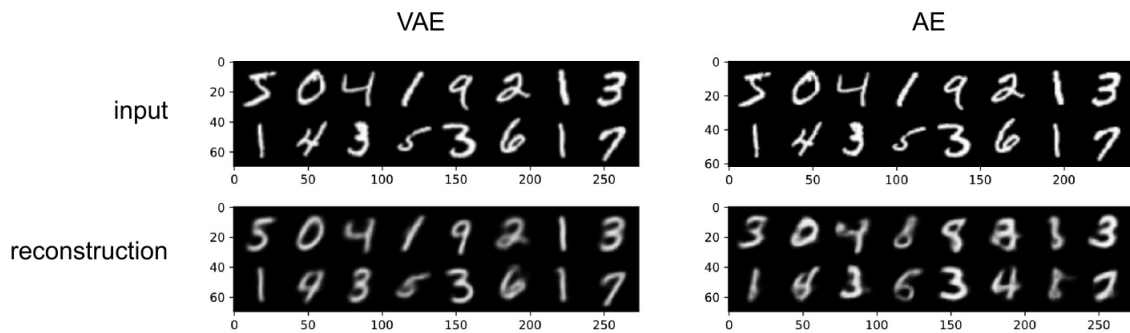


Figura 11.34: Ricostruzione: VAE ha prestazioni migliori. AE sbaglia facilmente se l'input è molto diverso dagli esempi forniti nel training set; VAE è molto più robusto alle variazioni e ciò è dovuto alle distribuzioni di probabilità imparate.

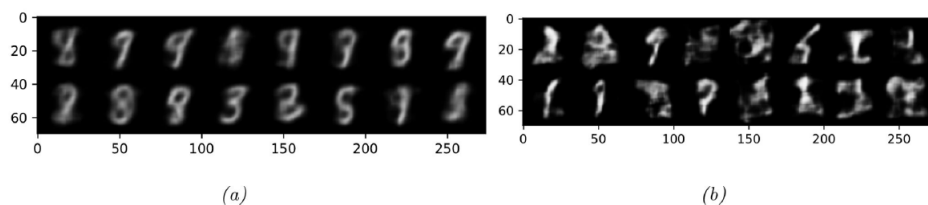


Figura 11.35: Data generation: VAE riesce a generare nuovi dati partendo da rumori casuali come input. (a) VAE: riesce a generare delle immagini che sono delle cifre anche se in input riceve del rumore casuale; (b) AE: ricevendo del rumore non riesce a generare cifre ma immagini incomprensibili.

11.10.4 Generative adversarial networks

Una **Generative Adversarial Networks (GAN)** consiste in un paio di reti combinate per formare un sistema generativo. Dalla Figura 11.36: il **generator** mappa i valori $\mathbf{z}$ in $\mathbf{x}$; il **discriminator** è un classificatore addestrato per classificare gli input $\mathbf{x}$ come *real* o *fake* (generati).

Le GAN rappresentano modelli impliciti, nel senso che si possono generare campioni, ma le loro probabilità $P(\mathbf{z}|\mathbf{x})$ non sono prontamente disponibili. Generator e discriminator vengono addestrati insieme: il gneerator impara a ingannare il discriminator; il discriminator impara a distinguere i dati reali da quelli fittizi.

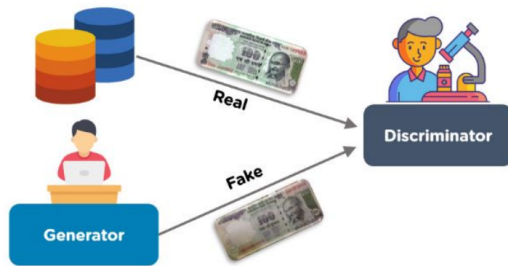


Figura 11.36: Generative adversarial network (GAN).

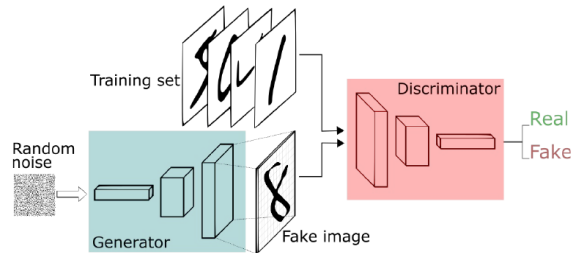


Figura 11.37



Figura 11.38: Esempio di applicazione. Le GAN funzionano particolarmente bene per task che riguardano la generazione di immagini. Per esempio possono creare immagini ad alta risoluzione di persone che non sono mai esistite.

11.10.5 Transfer learning

L'idea del **transfer learning** è che l'esperienza acquisita per uno specifico task può aiutare l'addestramento riguardante un altro task di un agente. In questo modo il suddetto agente sfrutta in qualche modo il lavoro di addestramento già fatto da un altro. L'approccio generale per le reti neurali è il seguente:

- I pesi appresi per il task A vengono copiati in una rete che verrà utilizzata per l'addestramento riguardante il task B;
- L'aggiornamento dei pesi della rete avviene come al solito (per esempio con il gradient descent) utilizzando i dati per il task B.

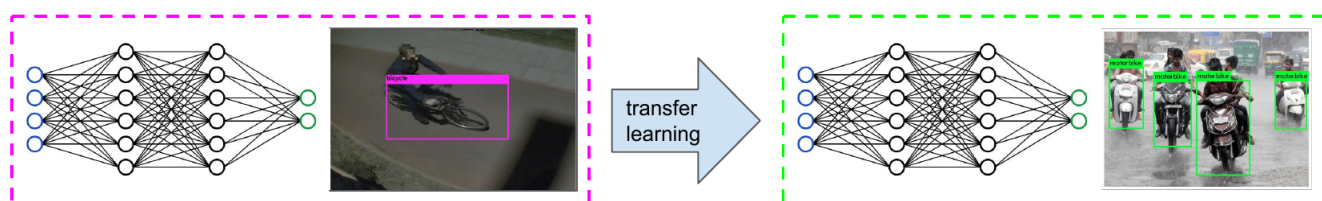


Figura 11.39: Il concetto di transfer learning: utilizzo i pesi appresi nell'addestramento di una rete che riconosce le biciclette in un'altra rete che riconosce moto.

Le pratiche che si seguono per attuare il transfer learning:

1. Vengono congelati i primi strati del modello preaddestrato che servono come rilevatori di caratteristiche e saranno utili per il nuovo modello.
2. Il nuovo modello potrà modificare i parametri soltanto per i livelli di profondità più elevata, ovvero quelli che individuano caratteristiche specifiche del problema ed eseguono la classificazione.

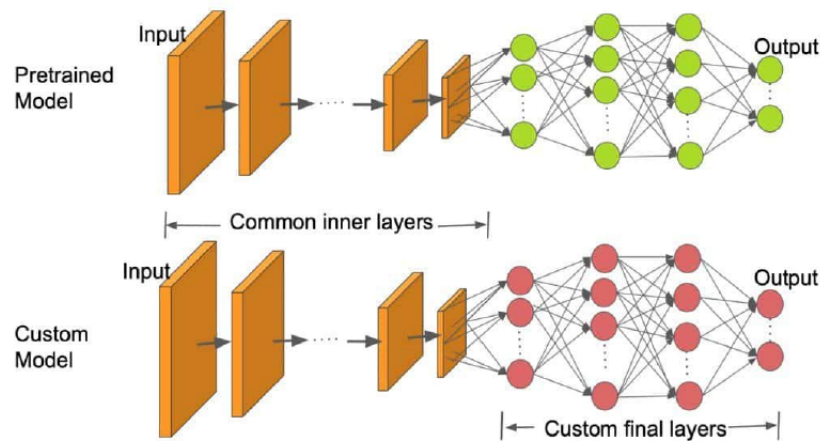


Figura 11.40

Un tipo molto importante di apprendimento per trasferimento riguarda il trasferimento tra le simulazioni e il mondo reale. Per esempio, il controllore per un'auto a guida autonoma può essere addestrato su miliardi di chilometri di guida simulata, che sarebbe impossibile percorrere nel mondo reale. Poi, una volta che il controllore è inserito nel veicolo reale, esso si adatterà rapidamente al nuovo ambiente.

11.11 Language models

I language models sono dei modelli probabilistici del linguaggio naturale. Con **Large Language Models (LLM)** si intende una rete neurale che ottiene una comprensione e generazione del linguaggio per scopi generici. In generale, una forma di generative AI, prende in input un testo e predice la prossima parola ripetutamente.

11.11.1 Word encoding and embedding

Le parole vengono rappresentate mediante dei numeri detti **token**, in si tratta di una codifica che non tiene conto della vicinanza. L'intero insieme di token di un LLM costituisce il suo **vocabolario** (può essere estremamente grande).

D'altro canto, il **word embeddings** va oltre la codifica: raggruppa parole correlate in uno spazio dei token di piccola dimensione.

11.11.2 Token prediction

Data una sequenza di token, il LLM predice la probabilità del prossimo token. Per esempio, dato in input *[To, be, or, not, to, be]*, l'output sarà una predizione sulle probabilità associate ad ognuna delle parole presenti del vocabolario del LLM. La scelta della prossima parola può

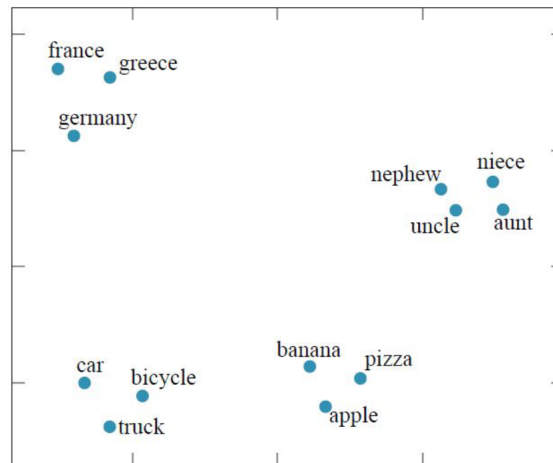


Figura 11.41: Word encoding and word embedding.

essere fatta sfruttando una **greedy selection** (es: probabilità più alta), oppure utilizzando una **random selection** basata sulle distribuzioni di probabilità dei token (generate dalla predizione). La random selection introduce maggiore varietà nella risposta.

Per predire una frase basta predire iterativamente una parola dopo l'altra:

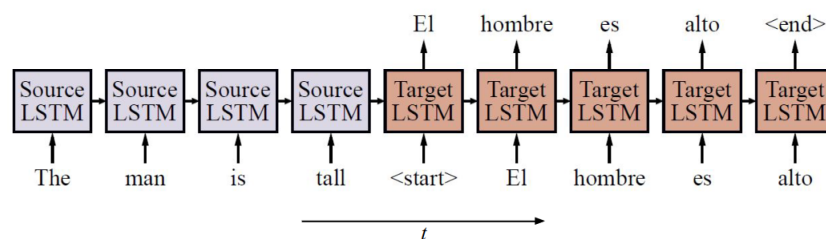
```

1 sentence = prompt
2 token = predict from sentence
3 sentence = append token to sentence
4 if token == period '.' then return
5 go to line (2)

```

11.11.3 Sequence-to-sequence models

Un modo per predire i token è quello di utilizzare il **modello sequence-to-sequence**. Vengono utilizzate due LSTM-based Recursive Neural Network, una per il **source** e una per il **target**. La RNN source mantiene alcune informazioni riguardanti i token vicini. Successivamente, l'informazione, che dipende dal contesto della parola, viene passata alla RNN target per la predizione della nuova sequenza di token.

Figura 11.42: Sequence-to-sequence model. *<start>* identifica l'inizio della risposta.

Il problema principale del modello di Figura 11.42 è che la RNN tende a dimenticare le informazioni meno recenti dando più importanza ai token più recenti (anche con LSTM). Non possiamo considerare tutte le unità della sorgente in quanto sarebbe computazionalmente impossibile, ma possiamo introdurre dei collegamenti che ci permettono di catturare meglio il contesto delle parole. Questi collegamenti vengono appresi dalla rete nella fase di training.

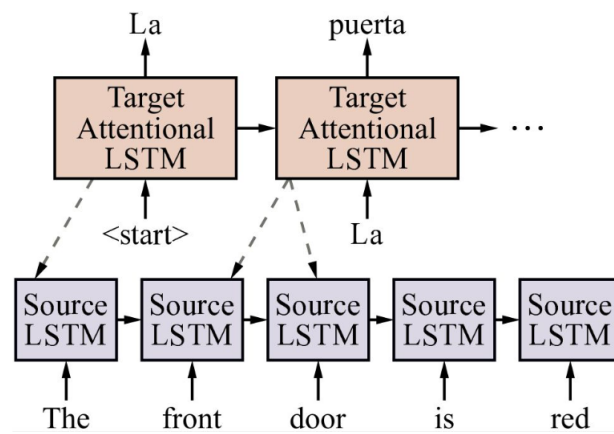


Figura 11.43: Sequence-to-sequence model con collegamenti aggiuntivi per poter catturare il contesto delle parole.